
BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY

ANALYSIS OF ALGORITHMS

Correctness, Cost, and the Limits of Efficiency

UNIFIED LECTURE HANDOUT

Complete Course in One Volume • Fall 2026



Two questions about every algorithm: is it right, and what does it cost?

And a third the first two do not answer: is there a better one, and how would you know?

What this volume covers. Every clause of the HEC 2023 outline for *Analysis of Algorithms*, in the order the outline gives them, with the published learning outcomes traced chapter by chapter in the Course Specification overleaf.

Under HEC 2023 this is a **computing core** course — compulsory in all ten BS computing degrees, this one included — taken in Semester 7, worth 3 (3–0) credit hours, with *Data Structures* as its prerequisite. The 2023 outline is word for word the 2017 outline, under which the course was BSCS-only and absent from BSIT.

About a third of that outline you have already met in Data Structures. Appendix F draws the line: that course owns the structures, this one owns the *analysis* and the design paradigms.

Six Parts • Twenty-Three Chapters • Seven Appendices

How to Use This Handout

Most of what this course teaches is not an algorithm. It is a way of arguing about one.

By Semester 7 you can already write a program that sorts. What you cannot yet do reliably is say *why* it is correct for every input rather than the ones you tried, *what* it will cost when the input is a thousand times larger, and *whether* anything could do better. Those three questions — correctness, cost, and the limit — are the whole subject, and each has a technique behind it: the loop invariant, the asymptotic bound, and the lower bound.

The six parts

Part I — Analysing an Algorithm (Chapters 1–4) builds the vocabulary. What “efficient” means and against what model; best, expected and worst case; the five asymptotic notations defined properly, not just O ; and the loop invariant, which is how a correctness claim is proved rather than asserted.

Part II — Recursion and Divide-and-Conquer (Chapters 5–9) is the machinery. Recurrences solved three ways, the master theorem and where it does not apply, brute force taken seriously as a baseline, then merge sort and quicksort analysed in full.

Part III — Sorting, Selection and the Structures Beneath (Chapters 10–13) establishes the first lower bound in the book — no comparison sort can beat $n \lg n$ — and then shows what buying extra assumptions gets you. Chapters 12 and 13 revisit heaps, search trees and hashing from Data Structures, for their *analysis* rather than their implementation.

Part IV — The Design Paradigms (Chapters 14–17) is the part you will use most after the examination: greedy, dynamic programming, and the amortised analysis without which several earlier claims cannot be stated honestly.

Part V — Graph Algorithms (Chapters 18–20) applies all of it, and is where sparsity starts changing every bound you write.

Part VI — Strings, Limits and Practice (Chapters 21–23) closes with string matching, the complexity classes, and your project.



Why the lower bound in Chapter 10 matters more than the algorithms around it. Everything before it answers “how fast is this?”. Chapter 10 answers “how fast could *anything* be?”, and that is a different kind of statement — it is about the problem, not about any program. Once you have seen one proved, Chapter 22’s claim that we do not know whether $P = NP$ stops being a slogan and becomes a recognisable open question of the same shape.

The furniture of every chapter

Box	What it is for
🎯 Outcomes	What you will be able to <i>do</i> . Bloom verbs, not topics.
🔗 Before you start	Which earlier chapters this one stands on.
📖 Key terms	The vocabulary introduced; all of it is in the glossary of Appendix D.

Box	What it is for
 Definition	The canonical statement. In this subject the definitions are load-bearing: most wrong answers in the examination are a misremembered definition, not a failed calculation.
 Theorem ,  Lemma ,  Corollary	A result, numbered within the chapter so the text can say “by Lemma 6.2”.
<i>Proof.</i>	Deliberately not in a box — it is prose and should be read as prose. Every proof here is one you could reproduce; the ones that are not are cited instead.
 Algorithm	Pseudocode, numbered by line, so the analysis can point at line 4.
 Concept	The judgement call — what an experienced practitioner would decide, and why.
 Warning	Something that will bite you, usually in the examination.
 Worked example	Real arithmetic, every step shown. Do them with a pen.
 Common pitfalls	The mistakes that are actually made.
 Try it yourself	The laboratory: C++ you compile, run and <i>time</i> .
 Checkpoint	Three questions. Attempt them before reading Appendix C.
 Chapter summary	What to retain.
 Review questions	Examination practice: multiple choice, short answer, applied.

The measured strand, and why it is here

Every chapter also carries at least one *measurement*: the algorithm compiled, run, and timed, with the figures printed in the book.

This is not decoration. Asymptotic analysis deliberately discards constant factors, and a student who only ever sees the asymptotics draws two wrong conclusions — that an $O(n \lg n)$ algorithm always beats an $O(n^2)$ one, and that the analysis is therefore unreliable. Both are cured by seeing where the crossover actually falls. Insertion sort really does beat merge sort on small inputs, and the asymptotics really do win by the time it matters.

Every figure in those boxes was produced by the file named beside it, in `code/`, with the compiler and flags recorded in Appendix A. Rerun them. Your numbers will differ from the book's; the *shape* should not.

Course Specification

This page is the course file. It states what HEC requires of a course called *Analysis of Algorithms* and shows where each requirement is met. Three editions are in force in different programmes, so all three are traced.

Course particulars

Item	Value
Course title	Analysis of Algorithms (HEC 2023). HEC 2017 and HEC 2025 both call it <i>Design and Analysis of Algorithms</i>
Programme	BS Information Technology. Under HEC 2023 this is a computing core course — compulsory in all ten BS computing degrees — in Semester 7. Under HEC 2017 it was a BSCS core course and <i>absent from BSIT entirely</i> ; under HEC 2025 it is a BSCS major core in Semester IV
Credit hours	3 (3–0): three credit hours of lectures, no separate laboratory credit. The laboratory work in this handout is therefore done in lecture time and as homework
Prerequisite	<i>Data Structures</i> (HEC 2023). HEC 2017 required <i>Data Structures and Algorithms</i> ; HEC 2025 publishes no prerequisites at all
Duration	Sixteen teaching weeks, midterm in Week 8, project due Week 16 (Appendix G)
Course introduction	The HEC 2023 statement, verbatim: “Detailed study of the basic notions of the design of algorithms and the underlying data structures. Several measures of complexity are introduced. Emphasis on the structure, complexity, and efficiency of algorithms.”
Set texts	As published: Cormen, Leiserson, Rivest and Stein, <i>Introduction to Algorithms</i> ; Kleinberg and Tardos, <i>Algorithm Design</i> ; Sedgewick and Wayne, <i>Algorithms</i> . See the note below on the edition, and Appendix E

Four defects in the published documents

Recorded here so that a reader comparing this handout with the HEC booklet is not confused, and so that the department’s own course file does not inherit them.

1. The 2023 CLO labels are mis-aligned with their own text. In the 2023 booklet CLO-1 sits against the first half of outcome 1 and CLO-2 against its continuation, leaving 3, 5 and 7 dangling. The 2017 listing of the same eight outcomes is correctly numbered, and *that* is the numbering used below.

2. The Bloom’s Taxonomy column is empty in both editions, for all eight outcomes — although the neighbouring Data Structures table fills it in. The levels in the table below are therefore *assigned by this department*, and are marked as such.

3. “Corman” is the booklets’ misspelling of **Cormen**, in both 2017 and 2023.

4. The 3rd edition (2009) is cited; the 4th edition (2022) is current. Appendix E cites the 4th and keeps the 3rd beside it, because chapter numbers moved between them.

Coverage of the HEC 2023 course outline

The published outline is a single run-on paragraph. Each of its clauses is traced below, in the order HEC writes them. Nothing is omitted.

HEC 2023 clause	Chapter	Where exactly
“Introduction; role of algorithms in computing”	1	What an algorithm is, why the question is asymptotic, and the model of computation the whole book assumes
“Analysis on nature of input and size of input”	2	What “size” means for an array, a graph, a matrix and a number; best, expected and worst case
“Asymptotic notations; Big-O, Big Ω , Big Θ , little-o, little- ω ”	3	All five defined formally and proved from the definitions
“Sorting Algorithm analysis”	8, 9, 10	Merge sort, quicksort, and the lower bound that bounds them both
“loop invariants”	4	Initialisation, maintenance, termination; insertion sort proved correct
“Recursion and recurrence relations”	5, 6	Substitution, recursion trees, and the master theorem
“Algorithm Design Techniques”	7–9, 14–16	The four the outline names, each with the property that makes it applicable
“Brute Force Approach”	7	Taken seriously: the baseline every other technique must beat, and the cases where it wins
“Divide-and-conquer approach; Merge, Quick Sort”	8, 9	Both, in full, with the average-case analysis of quicksort
“Greedy approach”	14	Greedy-choice and optimal substructure, proved by exchange argument
“Dynamic programming; Elements of Dynamic Programming”	15, 16	The two elements as their own chapter, then the classic problems
“Search trees”	12	Revisited for analysis: why the height is what it is, and what that costs
“Heaps”	12	Likewise, with the priority-queue bounds Chapters 19 and 20 depend on
“Hashing”	13	Expected versus worst case, load factor, and what “constant time” actually promises
“Graph algorithms, shortest paths, sparse graphs”	18, 20	Representation and sparsity first, because sparsity changes every bound in the part; then single-source and all-pairs

HEC 2023 clause	Chapter	Where exactly
“String matching”	21	Naive, Rabin–Karp and Knuth–Morris–Pratt
“Introduction to complexity classes”	22	P, NP, NP-hard and NP-complete, with one reduction worked in full

Course learning outcomes

The eight outcomes HEC publishes, in the 2017 numbering (see defect 1 above). Bloom levels are assigned by this department, because the published tables leave the column blank.

CLO	By the end of the course the student will be able to	Bloom	Chapters
CLO-1	Explain what is meant by “best”, “expected” and “worst” case behaviour of an algorithm	C2 Understand	2, 9
CLO-2	Identify the characteristics of data, or other conditions or assumptions, that lead to different behaviours	C4 Analyse	2, 9, 13, 18
CLO-3	Determine informally the time and space complexity of simple algorithms	C3 Apply	1, 3, 17, and every chapter’s worked example
CLO-4	List and contrast standard complexity classes	C2 Understand	22
CLO-5	Use big O , Ω and Θ notation formally to give asymptotic bounds on time and space complexity	C3 Apply	3, 5, 6
CLO-6	Use the strategies — brute force, greedy, divide-and-conquer and dynamic programming — to solve an appropriate problem	C3 Apply	7, 8, 9, 14, 15, 16
CLO-7	Solve problems using graph algorithms, including single-source and all-pairs shortest paths and at least one minimum spanning tree algorithm	C3 Apply	18, 19, 20
CLO-8	Trace and/or implement a string-matching algorithm	C3 Apply	21

💡 One chapter the outline does not name, and why it is here

Chapter 19, Minimum Spanning Trees, appears nowhere in the outline paragraph — and **CLO-7 requires it by name**: “at least one minimum spanning tree algorithm”. The outline and the outcomes disagree, and this handout follows the outcome, because that is what a student is assessed against.

Two further chapters go beyond the letter of the outline and are justified rather than smuggled in. **Chapter 11 (Selection)** is the natural companion to Chapter 10’s lower bound and costs one lecture. **Chapter 17 (Amortised Analysis)** is needed before Chapter 13’s hash-table claim or Chapter 19’s union-find can be stated honestly; without it, “constant

time” in those chapters would have to be asserted rather than proved, and CLO-3 asks for complexity to be *determined*.

Coverage of the HEC 2025 outcomes

HEC 2025 publishes no course contents for any course — only outcomes. Its five bullets for this course map onto the same chapters, so that a course file written against either edition has something to check.

HEC 2025 outcome	Chapters	Met by
Design efficient algorithms for common computational problems	7–9, 14–16, 18–21	CLO-6 and CLO-7
Analyze algorithm complexity using Big O notation	3, 5, 6	CLO-5
Solve problems involving recursion, divide-and-conquer and dynamic programming	5–9, 15, 16	CLO-6
Compare different algorithmic approaches to problem-solving	7, 14–16, 22	CLO-6; the comparison tables that close Parts II and IV
Demonstrate correctness and optimality of algorithms	4, 10, 14, 20	Loop invariants in 4; the lower bound in 10; exchange arguments in 14; Dijkstra’s correctness proof in 20

⚠ Where this course sits in the degree, and the one awkwardness

HEC 2023 places this course in **Semester 7** and *Artificial Intelligence* in Semester 3 — so a student meets heuristic search four semesters before meeting the analysis that would let them reason about its cost. HEC 2025 restores the sensible order, putting algorithms in Semester IV and AI in Semester V.

If your department follows the 2023 scheme, this handout assumes nothing from the later courses and can be taught as published. If it follows 2025, this course comes earlier and Chapters 12 and 13 may need more time, because Data Structures will be fresher but less practised.

The Course at a Glance

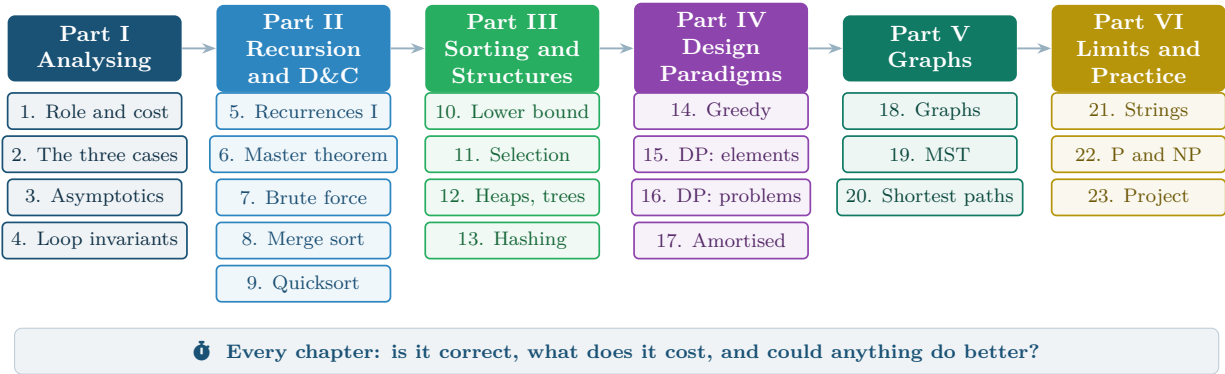


Figure F.1: The six parts and the chapters in each. Chapter 10 is the hinge: it is the first result in the book about a *problem* rather than about an algorithm.

This course is the published continuation of Data Structures and sits beside Theory of Automata. Appendix F states what belongs to each.

Notation

Symbol	Meaning
n	The size of the input. Chapter 2 is about what that sentence hides
$T(n)$	The running time on an input of size n , in the model of Chapter 1
$\lg n$	$\log_2 n$. Used throughout; $\ln$ is natural and $\log$ without a base means the base does not matter
$O(f(n))$	An asymptotic <i>upper</i> bound: T grows no faster than f
$\Omega(f(n))$	An asymptotic <i>lower</i> bound
$\Theta(f(n))$	Both: T grows exactly as fast as f
$o(f(n)), \omega(f(n))$	Strict versions of O and Ω — strictly slower, strictly faster
$A[1..n]$	An array of n elements, indexed from 1 , as the pseudocode does. The C++ in <code>code/</code> indexes from 0, and says so where it matters
$ V , E $	The number of vertices and edges of a graph. Chapter 18 explains why bounds are written in both
$\hat{c}_i$	The amortised cost of the i th operation (Chapter 17)
Φ	A potential function (Chapter 17)
P, NP	The complexity classes of Chapter 22
$\leq_p$	Polynomial-time reducibility (Chapter 22)
$\lceil x \rceil, \lfloor x \rfloor$	Ceiling and floor

⚠ Three notational habits worth breaking now

O is not Θ . “Merge sort is $O(n^2)$ ” is *true* and nearly useless. If you mean the bound is tight, write Θ . Chapter 3 is blunt about this because the examination is.

= in “ $T(n) = O(n^2)$ ” is not equality. It is set membership by long-standing abuse: $O(n^2)$ is a *set of functions* and T belongs to it. You may never write $O(n^2) = T(n)$, and $O(n) = O(n^2)$ is meaningless.

Constants are discarded, not absent. $\Theta(n)$ with a constant of 10^6 loses to $\Theta(n^2)$ with a constant of 1 until n exceeds a million. The measured strand in every chapter exists to keep this in view.

Prerequisite Self-Check

Twelve questions. If you can answer ten, begin at Chapter 1. If you cannot, Appendix B covers the mathematics and your Data Structures notes cover the rest — and the gap is worth closing in Week 1 rather than Week 9.

1. Write the sum $1 + 2 + \dots + n$ in closed form, and say why. (Appendix B)
2. Evaluate $\sum_{i=0}^k 2^i$. What is it approximately, for large k ? (Appendix B)
3. Simplify $\log_2(n^3)$ and $2^{\log_2 n}$. (Appendix B)
4. State the principle of mathematical induction, and use it to show $\sum_{i=1}^n i = n(n+1)/2$. (Appendix B)
5. A binary tree of height h has at most how many leaves? Why? (Data Structures)
6. What is the height of a binary heap containing n elements? (Chapter 12)
7. Give the worst-case height of an unbalanced binary search tree on n keys, and the input that produces it. (Chapter 12)
8. Describe how a hash table resolves collisions by chaining, and what happens as the load factor rises. (Chapter 13)
9. What is the difference between an adjacency matrix and an adjacency list, in space? (Chapter 18)
10. If a fair coin is tossed n times, what is the expected number of heads? (Appendix B)
11. In C++, what is the difference between `std::vector` and a raw array, and which one has $O(1)$ indexing? (Appendix A)
12. Write a function that reverses an array in place. What does it cost? (Programming Fundamentals)



Notice what is *not* on that list: calculus, linear algebra, and any particular cleverness. This course needs fluency with sums, logarithms and induction, comfort with the data structures you already built, and a willingness to finish a proof rather than stop when the idea is clear. It also needs you to be able to compile and run a C++ program. Appendix A sets that up in one sitting; do it before Week 2, because from Chapter 1 onward every chapter asks you to measure something.

Contents

How to Use This Handout	ii
Course Specification	iv
The Course at a Glance	viii
Notation	ix
Prerequisite Self-Check	x
I Analysing an Algorithm	1
1 The Role of Algorithms, and What Efficient Means	3
1.1 Two Programs That Agree on Every Answer	3
1.2 The Model: What Counts as One Step	4
1.3 Why the Question Is Asymptotic	5
1.4 The Claim, Measured	6
1.5 Space, and the Trade Against Time	7
2 Input Size, Input Nature, and the Three Cases	11
2.1 The Size of the Input	11
2.2 The Nature of the Input: Three Cases	13
2.3 Which Case Should You Report?	15
3 Asymptotic Notation	19
3.1 Three Bounds, Defined	19
3.2 Proving a Bound From the Definition	20
3.3 The Strict Bounds: o and ω	21
3.4 Properties, and the Analogy With Inequality	22
3.5 The Limit Test	23
3.6 What Θ Asserts, Measured	24
3.7 Four Abuses, and What Each Conceals	26
4 Correctness: Loop Invariants	30
4.1 Why Testing Is Not Enough	30
4.2 The Loop Invariant	31
4.3 Insertion Sort, Proved	32
4.4 Binary Search, Where the Invariant Is About Exclusion	33
4.5 Termination: The Obligation People Skip	34
4.6 A Loop Whose Invariant Holds and Which Never Finishes	35

II	Recursion and Divide-and-Conquer	39
5	Recurrences I: Substitution and Recursion Trees	41
5.1	What a Recurrence Is	41
5.2	The Substitution Method	42
5.3	Recursion Trees: a Way to Guess	43
5.4	An Unbalanced Split	44
5.5	Floors, Ceilings and Boundaries	45
6	Recurrences II: The Master Theorem	49
6.1	One Theorem for a Whole Family	49
6.2	The Theorem, Verified Exactly	51
6.3	The Gap: When the Theorem Does Not Apply	52
7	Brute Force, and When It Is the Right Answer	56
7.1	The Strategy With No Ideas In It	56
7.2	A Problem With Three Answers	57
7.3	An Idea About the Problem	58
7.4	The Three, Raced	59
7.5	When Brute Force Is the Right Answer	60
8	Divide and Conquer: Merge Sort and the Pattern	64
8.1	The Pattern	64
8.2	Merge Sort	65
8.3	Measured: the Bound Holds, the Clock Does Not	66
8.4	Where the Missing Factor of Two Went	68
9	Quicksort, Randomisation and the Average Case	72
9.1	Hard Split, Easy Join	72
9.2	Two Running Times, Both True	73
9.3	Measured: the Gap, and Closing It	75
9.4	What Is Actually Shipped	76
III	Sorting, Selection and the Structures Beneath	80
10	Lower Bounds and Sorting in Linear Time	82
10.1	A Different Kind of Question	82
10.2	The Decision Tree	83
10.3	How Close Does a Real Sort Get?	84
10.4	Sorting Without Comparing	85
11	Selection and Order Statistics	90
11.1	Asking for Less	90
11.2	Quickselect	91
11.3	Measured	92
11.4	Linear in the Worst Case	93

12 Heaps and Search Trees, Revisited for Analysis	97
12.1 A Heap's Height Is a Guarantee	97
12.2 Build-Heap Is Linear, and the Obvious Analysis Says Otherwise	98
12.3 A Search Tree's Height Is a Hope	99
12.4 Choosing	101
13 Hashing, and What Constant Time Actually Promises	104
13.1 The Claim, and Its Fine Print	104
13.2 Measured: the Assumption, and the Prediction	105
13.3 When the Assumption Quietly Fails	106
13.4 An Adversary	107
IV The Design Paradigms	111
14 Greedy Algorithms	113
14.1 Commit and Never Reconsider	113
14.2 Activity Selection	114
14.3 When Greedy Fails: Coin Change	115
14.4 Huffman Coding	116
15 Dynamic Programming I: The Elements	121
15.1 The Two Conditions	121
15.2 Rod Cutting	122
15.3 Two Ways to Collect the Saving	123
15.4 Measured	124
15.5 Greedy or Dynamic Programming?	125
16 Dynamic Programming II: Classic Problems	129
16.1 Longest Common Subsequence	129
16.2 Edit Distance	131
16.3 Saving Space, and What It Costs	131
16.4 0/1 Knapsack, and a Word That Matters	132
17 Amortised Analysis	137
17.1 A Loose Bound	137
17.2 Three Methods, One Answer	138
17.3 Measured	140
17.4 Where Else This Pattern Appears	142
V Graph Algorithms	145
18 Graphs: Representation, Traversal and Sparsity	147
18.1 Why Every Bound Here Has Two Parameters	147
18.2 Two Representations	148

18.3 Breadth-First Search	148
18.4 Measured: Where the Textbook Prediction Fails	149
19 Minimum Spanning Trees	154
19.1 The Cut Property	154
19.2 Kruskal, and the Structure It Needs	155
19.3 Prim	157
19.4 Measured	157
20 Shortest Paths: Single-Source and All-Pairs	162
20.1 Relaxation, the Common Operation	162
20.2 Dijkstra	163
20.3 Bellman–Ford	164
20.4 One Negative Edge	166
20.5 All Pairs	166
VI Strings, Limits and Practice	171
21 String Matching	173
21.1 The Problem, and the Obvious Algorithm	173
21.2 Rabin–Karp	174
21.3 Knuth–Morris–Pratt	174
21.4 Measured	176
22 Complexity Classes: P, NP and NP-Completeness	181
22.1 Decision Problems, and Two Classes	181
22.2 Measured: the Gap Between Checking and Finding	182
22.3 Reductions, and NP-Completeness	184
22.4 Why Knapsack Does Not Settle It	185
23 The Semester Project	189
23.1 What the Project Is	189
23.2 Choosing	190
23.3 The Method, Collected	190
23.4 The Report	192
23.5 The Defence	192
Appendices	196
A The C++ Toolkit	197
A.1 The Machine These Numbers Came From	197
A.2 Installing the Compiler	197
A.3 Why the Flags Matter	198
A.4 Timing Correctly	198

A.5	Pseudocode and C++: the Index Translation	199
A.6	Reproducing the Handout's Tables	200
B	The Mathematics This Course Assumes	201
B.1	Logarithms	201
B.2	Summations	201
B.3	Induction	202
B.4	Probability	203
B.5	Stirling's Approximation	203
B.6	Floors, Ceilings and Useful Inequalities	204
B.7	Where Each Topic Is Needed	204
C	Answers to the Checkpoints	205
D	Glossary	213
E	Further Reading	216
E.1	The Books the Outline Names	216
E.2	Beyond the List	216
E.3	The Original Papers	217
E.4	Online	218
E.5	A Reading Path	218
F	The Boundary With Other Courses	219
F.1	Data Structures	219
F.2	Theory of Automata	220
F.3	Discrete Structures	220
F.4	Programming Fundamentals and OOP	221
F.5	A Note on the 2017 Postgraduate Course	221
F.6	Summary: What to Assume on Day One	221
G	The Sixteen-Week Schedule	222
G.1	The Schedule	222
G.2	How the Weeks Map to the HEC Outline	223

List of Figures

F.1	The six parts and the chapters in each. Chapter 10 is the hinge: it is the first result in the book about a <i>problem</i> rather than about an algorithm.	viii
P1.1	Reading path through Part I. Chapters 1–3 build the language of cost; Chapter 4 is the other half of the subject, and the one students most often skip.	2
1.1	The six orders of growth this book is mostly about, on a logarithmic vertical axis — note that each gridline is a factor of a hundred. At $n = 100$ the gap between $n \lg n$ and n^2 is already a factor of fifteen, and the gap between n^2 and 2^n does not fit on the page.	5
4.1	INSERTION-SORT. The outer loop’s invariant concerns the prefix $A[1..j-1]$; the inner loop’s concerns where <i>key</i> has got to. Lines are numbered so the proof can cite them.	32
4.2	BINARY-SEARCH on a sorted array. The +1 on line 7 and the −1 on line 9 are what make the search range strictly smaller every iteration — Section 4.5 is about what happens when one of them is missing.	33
4.3	EUCLID-GCD. Here the invariant is an equality — $\gcd(a, b)$ never changes — and the variant is simply b , whose decrease is what makes the loop finite.	34
P2.1	Reading path through Part II. Chapter 6 is the tool you will reach for most; Chapter 7 is the baseline that makes the rest worth having.	40
5.1	The recursion tree for $T(n) = 2T(n/2) + n$. Each level holds twice as many nodes, each doing half as much work, so every level totals n . The depth is $\lg n$ because the argument halves, and the product is the bound.	43
7.1	MAX-SUBARRAY-BRUTE: every block, summed from scratch. Three nested loops, so $\Theta(n^3)$ — and lines 4 to 7 are doing work that lines 4 to 7 of the previous iteration already did.	57
7.2	MAX-SUBARRAY-QUADRATIC. The search is unchanged — still every pair (i, j) — but each candidate now costs $O(1)$ instead of $O(n)$. A better accountant, not a better idea.	58
7.3	MAX-SUBARRAY-KADANE. One pass, two variables. The loop invariant is what makes it believable, and Theorem 7.2 is what makes it true.	58
8.1	MERGE-SORT. The divide is one line of arithmetic; all the work is in MERGE. . . .	65
8.2	MERGE: two sorted runs become one in a single pass. Line 3’s $\leq$ rather than $<$ is what makes merge sort stable, and it is the only difference.	65
9.1	QUICKSORT. There is no line after the two recursive calls: once the pieces are sorted, the array is sorted, because PARTITION already put them in the right places. . . .	72
9.2	PARTITION (Lomuto). Everything $\leq$ the pivot is swept to the left; the pivot is then dropped into the gap. Linear, in place, and the only place quicksort compares anything.	73
P3.1	Reading path through Part III. Chapter 10 is the hinge of the whole book; Chapters 12 and 13 revisit Data Structures for their analysis, not their implementation. . . .	81

10.1	The decision tree of a comparison sort on three elements. Every one of the $3! = 6$ orderings must appear as a leaf, or some input would be sorted wrongly — and a binary tree with 6 leaves must have height at least $\lceil \lg 6 \rceil = 3$	83
10.2	COUNTING-SORT. No element is ever compared with another: line 3 uses a key as an <i>index</i> , which the decision-tree model does not permit and which is why the bound does not apply. Running time $\Theta(n + K)$	85
11.1	QUICKSELECT. Identical to QUICKSORT except that one of the two recursive calls is <i>deleted</i> rather than made. Written as a loop because the remaining call is in tail position — which also removes Chapter 9’s stack-overflow failure.	91
12.1	BUILD-HEAP. The loop runs $n/2$ times and SIFT-DOWN costs $O(\lg n)$, so the obvious bound is $O(n \lg n)$. The obvious bound is loose, and Theorem 12.2 says by how much.	98
P4.1	Reading path through Part IV. Chapter 15 is the one to understand rather than memorise: the two properties in it decide whether greedy or dynamic programming applies at all.	112
14.1	ACTIVITY-SELECT, greedy by earliest finishing time. After the sort it is a single pass, so the whole algorithm is $\Theta(n \lg n)$	114
14.2	HUFFMAN. The greedy choice is on line 5: the two <i>least</i> frequent symbols are merged first, which puts them deepest. With a binary heap the running time is $\Theta(n \lg n)$	116
15.1	CUT-ROD-NAIVE: a direct transcription of the recurrence. It is correct, and it takes $\Theta(2^n)$ time, because line 5 re-solves subproblems that other branches have already solved.	122
15.2	CUT-ROD-DP, bottom-up. Two nested loops, so $\Theta(n^2)$ — and the only change from the recurrence is that subproblems are visited in increasing order of size, so each is solved once.	124
16.1	LCS-LENGTH. mn subproblems, $O(1)$ work each, so $\Theta(mn)$ time — and $\Theta(mn)$ space, which Section 16.3 reduces.	130
16.2	KNAPSACK, one row at a time. The inner loop counts <i>down</i> , which is what enforces 0/1: counting up would let item i be used more than once, solving the unbounded knapsack problem instead.	132
17.1	APPEND with table doubling. Line 4 runs rarely, and when it does it costs $\Theta(n)$. The three methods below are three ways of saying that the rarity exactly pays for the cost.	138
P5.1	Reading path through Part V. Chapter 18 comes first because the representation decides the bounds: the same algorithm is a different cost on a sparse graph and a dense one.	146
18.1	BFS. Line 7 — “for each v adjacent to u ” — is the only line whose cost depends on the representation, and it is the whole of the asymptotic difference.	149
19.1	KRUSKAL. The sort is $\Theta(E \lg E)$ and dominates; the $O(E)$ union–find operations cost $O(E \alpha(V))$, which is why the amortised bound matters.	156

19.2	PRIM. With a binary heap this is $O((V + E) \lg V)$; the cut is always “tree so far” against “everything else”.	157
20.1	DIJKSTRA. Line 7 is the greedy commitment: once a vertex is extracted its distance is declared final and never reconsidered. Theorem 20.1 is what justifies that, and Section 20.4 is what happens when its hypothesis fails.	163
20.2	BELLMAN-FORD. No priority queue and no assumption about signs: $O(V E)$. The final pass is not decoration — it is the only way to detect a negative cycle, which Dijkstra cannot do at all.	164
20.3	FLOYD-WARSHALL. Five lines, $\Theta(n^3)$ regardless of density, and the smallest constant of any algorithm in this chapter — three nested loops over contiguous memory.	166
P6.1	Reading path through Part VI. Chapter 22 is the only chapter whose central claim is that nobody knows; Chapter 23 is where you use the rest.	172
21.1	NAIVE-MATCH. After a mismatch at shift s it begins again at $s + 1$, re-reading characters it has already examined. That re-reading is the whole of the $O(nm)$. . .	173
21.2	PREFIX-FUNCTION. It is KMP run against the pattern itself, and it is $\Theta(m)$ by the same amortised argument.	175
21.3	KMP-MATCH. The text index i advances on every iteration and never retreats, which is what makes the whole thing linear.	175

PART I

Analysing an Algorithm

Three questions are asked of every algorithm in this book, and Part I is the vocabulary for asking them: what does it cost, how does the input shape that cost, and how would you prove the thing is correct rather than merely believe it?

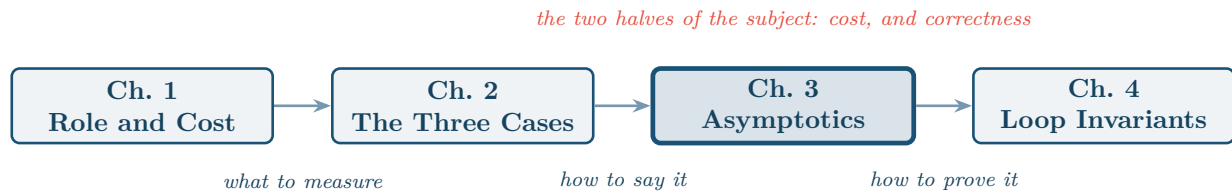


Figure P1.1: Reading path through Part I. Chapters 1–3 build the language of cost; Chapter 4 is the other half of the subject, and the one students most often skip.

The Role of Algorithms, and What Efficient Means

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part I — Analysing an Algorithm

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **State** what an algorithm is, and **distinguish** it from a program that implements it.
- **Describe** the random-access machine model this course measures against, and **name** three things it deliberately ignores.
- **Explain** why the interesting question about an algorithm is asymptotic, and **compute** how long a given algorithm takes at a given input size.
- **Predict** the ratio by which a running time grows when the input doubles, from the algorithm's order of growth, and **check** the prediction by measurement.
- **Explain** why a faster computer running a worse algorithm loses, and **find** the input size at which it starts losing.

🔗 Before You Start

Nothing in this book. You need to be able to read a loop and to have written programs that sort or search — *Data Structures* is the published prerequisite and this chapter assumes only its first fortnight. Appendix A sets up the compiler, and you will want it from Section 1.4 onward.

📖 Key Terms in This Chapter

algorithm • problem • instance • correctness • random-access machine • RAM model • primitive operation • running time • input size • order of growth • asymptotic analysis • constant factor • space complexity • time-space trade-off

1.1 Two Programs That Agree on Every Answer

Here are two ways to decide whether a sorted array of a million names contains one particular name.

The first walks the array from the start, comparing as it goes, and stops when it finds the name or runs off the end. The second looks at the middle element, discards the half that cannot contain the name, and repeats.

Both are correct. Both return the same answer for every input. Run them on the same machine with the same compiler and the first takes, on a million names, about half a million comparisons; the second takes twenty. Not twenty thousand — twenty.

That gap is the subject of this course, and the first thing to notice about it is that it has nothing to do with programming. It is not a faster loop, a better compiler or a tighter data structure. It is a different *idea*, and the difference between the two ideas survives every change of language, machine and decade.

Problem, Instance, Algorithm

A **problem** is a specification: a description of the input it accepts and the output it must produce. “Sorting” is a problem — given a sequence of n numbers, produce a permutation of them in non-decreasing order.

An **instance** is one particular input: $\langle 31, 41, 59, 26 \rangle$.

An **algorithm** is a finite, unambiguous procedure that produces the correct output for *every* instance. It is **correct** if it halts on every instance with the right answer — and “every” is doing real work in that sentence, which is why Chapter 4 exists.

A *program* is an algorithm written in a language, for a machine. This book analyses algorithms and measures programs, and is careful about which it is doing.

1.2 The Model: What Counts as One Step

To say an algorithm takes “half a million comparisons” we need to agree what a step is. Real machines make that awkward: an addition and a division cost differently, memory access costs differently depending on the cache, and the processor may be executing four instructions at once. So we measure against an idealisation.

The Random-Access Machine

The **RAM model** assumes:

- instructions execute one at a time, with no concurrency;
- each **primitive operation** — arithmetic, comparison, assignment, reading or writing one memory cell, following one array index — costs one unit of time;
- memory is unlimited and every cell costs the same to reach.

The **running time** $T(n)$ of an algorithm is then the number of primitive operations it performs on an input of size n .

Lightbulb Three Lies in That Model, and Why They Are Worth Telling

Every clause of the definition is false of a real machine, deliberately.

Not all operations cost the same. On a current processor an integer addition takes about one cycle and an integer division about twenty. The model prices them equally.

Not all memory costs the same. A read served by the level-1 cache takes about four cycles; one that goes to main memory takes a few hundred — a factor of fifty, which the model ignores entirely. Chapter 18 is where this starts to be visible in the measurements.

Instructions do not execute one at a time. Processors reorder, pipeline and predict branches.

The model is nevertheless the right one, for the reason the next section gives: every one of those lies is a *constant factor*, and the differences this course is about are not. A fifty-fold cache penalty is swamped by the difference between n and n^2 before n reaches a thousand. Where the constant factors do matter — and they sometimes do — this book measures rather than models, which is what the *Try it yourself* boxes are for.

1.3 Why the Question Is Asymptotic

We do not ask how long an algorithm takes. We ask how the time *grows* with the input, and we answer with the dominant term, discarding constants. That sounds like giving up information. It is the opposite: it is the only way to say something that stays true.

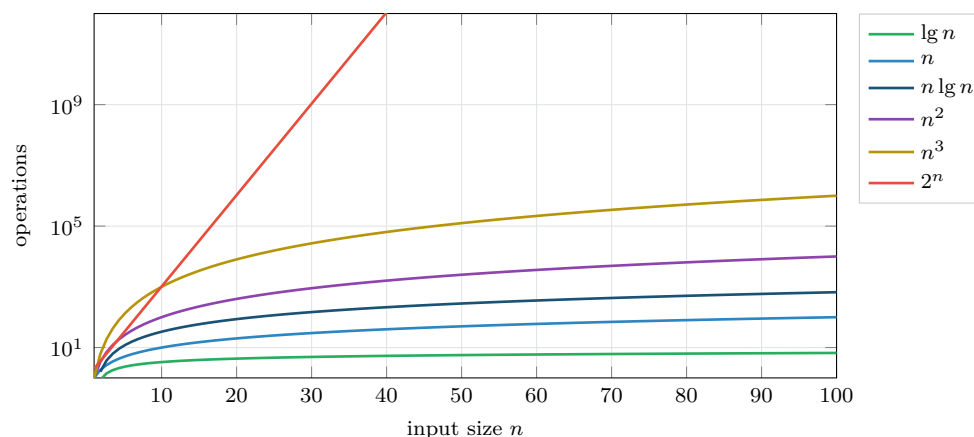


Figure 1.1: The six orders of growth this book is mostly about, on a logarithmic vertical axis — note that each gridline is a factor of a hundred. At $n = 100$ the gap between $n \lg n$ and n^2 is already a factor of fifteen, and the gap between n^2 and 2^n does not fit on the page.

⚠ What an exponential actually means

2^n is not “slow”. It is a wall.

At $n = 50$, an algorithm doing 2^n operations on a machine executing 10^{10} of them a second takes

$$\frac{2^{50}}{10^{10}} = \frac{1.13 \times 10^{15}}{10^{10}} \approx 1.1 \times 10^5 \text{ s} \approx 31 \text{ hours.}$$

At $n = 60$ it takes about **36 days**; at $n = 80$, about a hundred thousand years. Buying a computer a thousand times faster moves $n = 50$ to $n \approx 60$ and no further.

That is why Chapter 22 matters, and why “we will just use a faster machine” is not an answer to it.

📊 Worked Example: The Faster Computer Loses

Computer A executes 10^{10} primitive operations a second and runs insertion sort, which takes about $2n^2$ operations. Computer B executes 10^7 a second — a thousand times slower — and runs merge sort, which takes about $50n \lg n$. Sort ten million numbers on each.

Step 1 — Computer A.

$$\frac{2 \times (10^7)^2}{10^{10}} = \frac{2 \times 10^{14}}{10^{10}} = 2 \times 10^4 \text{ s} = \mathbf{5 \text{ hours } 33 \text{ minutes}}$$

Step 2 — Computer B. First $\lg(10^7) = 7 \lg 10 = 7 \times 3.322 = 23.25$.

$$\frac{50 \times 10^7 \times 23.25}{10^7} = 50 \times 23.25 = 1,163 \text{ s} = \mathbf{19 \text{ minutes } 23 \text{ seconds}}$$

Step 3 — the comparison. The machine that is *a thousand times slower* finishes **17 times sooner**, because its algorithm is better. The hardware advantage is not small and it is not enough.

Step 4 — where does B overtake A? Set the two times equal:

$$\frac{2n^2}{10^{10}} = \frac{50n \lg n}{10^7} \implies 2n = 5 \times 10^4 \lg n \implies n = 2.5 \times 10^4 \lg n.$$

There is no closed form, so iterate from $n = 10^5$:

$$2.5 \times 10^4 \times 16.6 = 4.15 \times 10^5 \rightarrow 4.66 \times 10^5 \rightarrow 4.71 \times 10^5 \rightarrow 4.71 \times 10^5.$$

So A wins below about $n = 471,000$ and loses above it — and loses by more, the larger the input gets.

Step 5 — what the arithmetic is really saying. Not that constant factors are unimportant. A constant of 25 between the two algorithms bought A nearly half a million elements' worth of advantage, which for many real problems is the whole range of interest. It is saying that constants buy a *bounded* advantage and a better order of growth buys an unbounded one, and that you should know which regime you are in.

1.4 The Claim, Measured

The worked example is arithmetic about a hypothetical machine. The claim it rests on — that a quadratic algorithm's time quadruples when the input doubles, and an $n \lg n$ one's roughly doubles — is testable on the machine in front of you, and ought to be tested.

💡 What the ratios should be

For $T(n) = cn^2$,

$$\frac{T(2n)}{T(n)} = \frac{c(2n)^2}{cn^2} = 4,$$

independent of c and of n . The constant cancels, which is exactly why the ratio is worth measuring: it tests the *order of growth* without needing to know anything about the machine.

For $T(n) = cn \lg n$,

$$\frac{T(2n)}{T(n)} = \frac{2n(\lg n + 1)}{n \lg n} = 2 + \frac{2}{\lg n},$$

which is a little above 2 and drifts *down* towards 2 as n grows. At $n = 1000$ it predicts $2 + 2/9.97 = 2.20$.

📊 Worked Example: The Prediction Against the Measurement

Running `code/ch01_growth.cpp` on the machine described in Appendix A (MSVC 19.44, /O2), median of five runs, random permutations:

n	insertion (ms)	ratio	merge (ms)	ratio
1,000	0.15	—	0.06	—
2,000	0.58	3.87	0.14	2.22
4,000	2.41	4.13	0.30	2.20
8,000	9.35	3.88	0.65	2.14
16,000	37.08	3.97	1.51	2.32
32,000	145.86	3.93	2.94	1.96

Insertion sort. Predicted 4; measured 3.87, 4.13, 3.88, 3.97, 3.93. Mean 3.96.

Merge sort. Predicted $2 + 2/\lg n$ — 2.20 at $n = 1000$, falling to 2.14 by $n = 8000$. Measured 2.22, 2.20, 2.14, 2.32, 1.96. The first three agree to within a per cent; the last two straddle the prediction.

Read the last column honestly. The 2.32 and the 1.96 are not the theory failing. They are two measurements of a quantity near 2.15 with a few per cent of noise, and a ratio magnifies noise in both terms. The way to tell noise from a real effect is to look at the *pattern*: insertion’s five ratios scatter about 4 and merge’s about 2.2, and neither drifts.

And notice what the measurement cost. At $n = 32,000$ insertion sort took 146 ms and merge sort 2.9 ms — a factor of 50. At $n = 1,000$ the factor was 2.5. The gap is not fixed; it widens with every doubling, and that is the whole content of “asymptotically faster”.

⚠ The asymptotics are not the whole story, and this book says so twice

The same program, run on small inputs, gives this:

n	insertion (μs)	merge (μs)	
20	0.20	0.30	insertion wins
40	0.50	0.60	insertion wins
60	0.70	1.00	insertion wins
70	1.20	1.10	merge wins
100	2.20	1.60	merge wins
200	7.40	4.60	merge wins

The crossover is at about $n = 65$ on this machine. Below it the “worse” algorithm is faster, every time, because merge sort’s recursion and its auxiliary array cost more than insertion sort’s whole quadratic loop on sixty elements.

This is not a curiosity. It is why every production sort in the world — the C++ standard library’s included — switches to insertion sort below a threshold of roughly this size. Chapter 9 returns to it.

1.5 Space, and the Trade Against Time

Time is not the only resource. **Space complexity** counts the memory an algorithm needs beyond its input, and the two are often tradeable: a lookup table turns a computation into a memory access, and Chapter 15’s whole technique is spending memory to avoid recomputation.

💡 Which Resource Binds?

For most of the twentieth century, memory was the scarce one. It is now usually not: a machine with 32 GB can hold an array of four billion integers, and most problems that fit in a lecture do not come close.

What has replaced it is *memory locality*. An algorithm using twice the memory but touching it in order can comfortably beat one using half as much in a scattered pattern, because of the cache the RAM model promised to ignore. Chapter 18 measures exactly this.

So: analyse space because the exam asks and because occasionally it binds; but when two algorithms have the same order of growth and different speeds, look at the access pattern before the byte count.

! Common Pitfalls

- **Confusing the algorithm with the program.** “My sort is slow” is usually a statement about an implementation; “insertion sort is quadratic” is a statement about an idea. Fixing the first will not change the second.
- **Timing an unoptimised build.** A debug build can be five to ten times slower and — worse — slower by *different* factors for different code, so the comparison is meaningless. Appendix A insists on /O2.
- **Timing once.** A single run contains whatever else the machine was doing. Every measurement in this book is the median of several.
- **Believing the asymptotics at small n .** The crossover above is at $n = 65$. If your inputs are always smaller than that, the better order of growth is buying you nothing.
- **Believing the constants matter at large n .** The symmetric error. At $n = 32,000$ the “unimportant” factor of 50 was entirely due to the order of growth.
- **Reporting a mean of timings.** One scheduler interruption drags a mean a long way. Use the median, and say how many runs.

🔧 Try It Yourself: Growth, Measured

Reproduces both tables above on your own machine. Appendix A sets up the compiler. Expect twenty minutes, most of it reading the output.

```
// ch01_growth.cpp -- what "the question is asymptotic" looks like.
// Build: cl /O2 /EHsc /std:c++17 ch01_growth.cpp      (see Appendix A)

// --- 1. The two algorithms, written plainly -----
static void insertion_sort(std::vector<int>& a) {
    for (size_t j = 1; j < a.size(); ++j) {
        int key = a[j];
        size_t i = j;
        while (i > 0 && a[i - 1] > key) { a[i] = a[i - 1]; --i; }
        a[i] = key;
    }
}

static void merge_sort(std::vector<int>& a, std::vector<int>& buf,
                      size_t lo, size_t hi) {
    if (hi - lo < 2) return;
    size_t mid = lo + (hi - lo) / 2;
    merge_sort(a, buf, lo, mid);
    merge_sort(a, buf, mid, hi);
    size_t i = lo, j = mid, k = lo;
    while (i < mid && j < hi) buf[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
    while (i < mid) buf[k++] = a[i++];
    while (j < hi)  buf[k++] = a[j++];
    std::copy(buf.begin() + lo, buf.begin() + hi, a.begin() + lo);
}

// --- 2. Timing that is honest -----
// The MEDIAN of several runs: a mean is dragged by one scheduler
// hiccup, and the minimum flatters the cache.
template <class F>
static double time_ms(F&& f, int reps) {
    std::vector<double> t;
    for (int r = 0; r < reps; ++r) {
        auto t0 = std::chrono::steady_clock::now();
        f();
        auto t1 = std::chrono::steady_clock::now();
        t.push_back(
```

```

        std::chrono::duration<double, std::milli>(t1 - t0).count());
    }
    std::sort(t.begin(), t.end());
    return t[t.size() / 2];
}

// --- 3. Double n each time, and watch the ratios -----
// Predicted: 4 for insertion sort, 2 + 2/lg(n) for merge sort.
for (size_t n = 1000; n <= 32000; n *= 2) {
    auto base = random_input(n, 12345);
    std::vector<int> buf(n);
    double ti = time_ms([&]{ auto a = base; insertion_sort(a); }, 5);
    double tm = time_ms([&]{ auto a = base;
        merge_sort(a, buf, 0, n); }, 5);
    printf("%8zu %12.2f %7.2f %12.2f %7.2f\n",
        n, ti, ti / prev_i, tm, tm / prev_m);
    prev_i = ti; prev_m = tm;
}

// --- 4. Now find where they cross -----
// The other half of the point: below the crossover the "worse"
// algorithm is faster, and every production sort knows it.
for (size_t n = 10; n <= 200; n += 10) {
    /* same two timings, many more repetitions because the times
       are microseconds; print both and mark which one won */
}

```

What to notice. Your absolute times will differ from the book's — a different processor, a different compiler, a different afternoon. The *ratios* should not. If your insertion-sort column is not near 4, something is wrong with the measurement, not with the theory: check that you built with optimisation on, and that nothing else was running.

? Checkpoint — Test Yourself

1. An algorithm takes 3 seconds on an input of size 1,000 and 48 seconds on an input of size 4,000. What is its order of growth, and how did you decide?
2. Computer A runs n^3 operations at 10^9 per second; computer B runs $100n^2$ at 10^7 per second. At $n = 10^4$, which finishes first, and by what factor?
3. The RAM model prices a cache hit and a main-memory access the same, although they differ by a factor of about fifty. Give one situation in which that lie is harmless and one in which it is not.

✓ Chapter Summary

- A problem is a specification, an instance is one input, an algorithm is a procedure correct on *every* instance. A program implements an algorithm; this book analyses the second and measures the first.
- The RAM model costs every primitive operation at one unit. All three of its assumptions are false of real hardware, and all three are constant factors — which is why the model is still the right one.
- We ask how running time *grows*, not what it is, because the growth is the part that survives a change of machine.
- Constants buy a bounded advantage; a better order of growth buys an unbounded one. A computer 1,000 times faster still lost by a factor of 17, and the crossover was at $n \approx 471,000$.
- Doubling the input multiplies a quadratic time by 4 and an $n \lg n$ time by $2 + 2/\lg n$. Measured on this machine: 3.96 and 2.17.

- Below $n \approx 65$ insertion sort beat merge sort on the same machine. Asymptotics describe the limit, and the limit is not everywhere.
- Measure with optimisation on, several times, and report the median.

Review Questions

1. **(MCQ)** The RAM model assumes: (a) memory access is free; (b) every primitive operation costs one unit of time; (c) instructions execute in parallel; (d) arithmetic is exact to arbitrary precision.
2. **(MCQ)** If $T(n) = cn^2$, then $T(2n)/T(n)$ equals: (a) 2; (b) $2 + 2/\lg n$; (c) 4; (d) it depends on c .
3. **(Short)** Explain why discarding constant factors makes an analysis *more* useful rather than less.
4. **(Short)** Give the three assumptions of the RAM model and, for each, one way real hardware violates it.
5. **(Short)** A colleague says “we do not need a better algorithm, we will buy a faster server”. Give the shortest argument that settles the question, and say what information you would need first.
6. **(Applied)** Algorithm P takes $0.5n^2$ operations and algorithm Q takes $200n \lg n$. Find the input size at which Q overtakes P, showing the iteration. Then say what you would recommend to a team whose inputs never exceed 2,000.
7. **(Applied)** Design a measurement that would distinguish an $O(n \lg n)$ algorithm from an $O(n)$ one, given only a compiled binary you can time. State the sizes you would use, how many repetitions, what you would compute, and what result would make you say the evidence is inconclusive.

Input Size, Input Nature, and the Three Cases

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part I — Analysing an Algorithm

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Define** the size of an input for an array, a matrix, a graph and a number, and **justify** why a number's size is its bit count.
- **Explain** what is meant by the best, expected and worst case behaviour of an algorithm (CLO-1).
- **Identify** the characteristics of the data, or the other conditions and assumptions, that lead an algorithm to different behaviours (CLO-2).
- **Derive** the best, worst and average case running time of insertion sort and linear search, and **say** which to report and why.
- **Distinguish** an average taken over inputs from an expectation taken over an algorithm's own random choices.

🔗 Before You Start

Chapter 1 — the RAM model and the idea of an order of growth. You will also need to be comfortable that $\lg$ means $\log_2$, and that $\sqrt{2^b} = 2^{b/2}$.

📖 Key Terms in This Chapter

input size • bit length • sparse • dense • best case • worst case • average case • expected running time • input distribution • uniform random permutation • randomised algorithm • adversary • pseudo-polynomial

2.1 The Size of the Input

Everything in Chapter 1 was written as a function of n without ever saying what n counts. For an array that is obvious enough. For the other things algorithms take as input it is a decision, and a decision that changes the answer.

📖 Input Size

The **input size** is whatever parameter the running time is most naturally expressed in — but it must be a measure of *how much input there is*, not of what the input says.

Input	Size, by convention
array, list	n , the number of elements
string	n , the number of characters
$n \times n$ matrix	n , the side — so an $O(n^3)$ algorithm is <i>linear</i> in the n^2 numbers it reads, which is worth saying out loud
graph	two parameters: $ V $ vertices and $ E $ edges

number	b , the number of <i>bits</i> needed to write it
---------------	--

Two of those rows need an argument.

Graphs take two parameters, and that is not a technicality

A graph has $|V|$ vertices and $|E|$ edges, and $|E|$ can be anywhere from 0 to $|V|(|V| - 1)/2$. A bound of $O(|V| + |E|)$ and a bound of $O(|V|^2)$ are the same thing on a **dense** graph and wildly different on a **sparse** one.

Worked Example: The Same Bound, Two Graphs

Breadth-first search is $O(|V| + |E|)$ from an adjacency list.

A road network: 2 million junctions, and each junction meets about three roads, so $|E| \approx 3 \times 10^6$.

$$|V| + |E| \approx 2 \times 10^6 + 3 \times 10^6 = 5 \times 10^6.$$

A social graph of the same vertex count, each person connected to 500 others, so $|E| \approx 2 \times 10^6 \times 500/2 = 5 \times 10^8$.

$$|V| + |E| \approx 5 \times 10^8.$$

A factor of 100, on the same algorithm, at the same $|V|$. This is why Chapter 18 comes before the graph algorithms rather than after them, and why every bound in Part V is quoted in both parameters.

A number's size is its length, not its value

This one students get wrong, and it is the root of a famous confusion that Chapter 16 and Chapter 22 both depend on.

Why not just use N itself?

Because the input to a program is *written down*, and a number N is written in about $b = \lfloor \lg N \rfloor + 1$ bits. Typing one more digit multiplies N by ten while adding three and a bit bits of input.

If we measured size by value, then an algorithm that loops N times would be “linear” — and we would be calling an algorithm efficient when doubling the *length* of its input multiplies its running time by 2^b .

So: **size** = $b \approx \lg N$, and an algorithm that loops N times is $O(2^b)$ — exponential. An algorithm whose cost is polynomial in N but exponential in b is called **pseudo-polynomial**; the 0/1 knapsack algorithm of Chapter 16 is the standard example, and Chapter 22 explains why that distinction decides whether a problem counts as solved.

Worked Example: The Claim, Measured

`code/ch02_cases.cpp` tests a prime by trial division — a loop running about $\sqrt{N}/2$ times. Since $\sqrt{N} = \sqrt{2^b} = 2^{b/2}$, adding *two* bits should exactly double the work. Each row below is a prime two bits longer than the row above; median of three runs.

bits	N	time (ms)	ratio
32	4,294,967,291	0.22	—
34	17,179,869,143	0.44	2.01
36	68,719,476,731	0.86	1.94
38	274,877,906,899	1.76	2.06
40	1,099,511,627,689	3.63	2.07
42	4,398,046,511,093	6.93	1.91
44	17,592,186,044,399	15.75	2.27
46	70,368,744,177,643	29.83	1.89
48	281,474,976,710,597	58.81	1.97

Eight ratios, mean 2.02. The prediction is exact and the measurement meets it.

Now extrapolate, which is the point. From 32 bits to 48 the time went from 0.22 ms to 58.81 ms — a factor of 267, against a predicted $2^8 = 256$. Carry that on:

- **64 bits:** $2^8 \times 58.81 \text{ ms} \approx 15 \text{ seconds}$.
- **128 bits:** $2^{40} \times 58.81 \text{ ms} \approx 6.5 \times 10^{10} \text{ s} \approx \mathbf{2,000 \text{ years}}$.
- **2048 bits,** which is what an RSA key is: there is no useful way to write the number.

And notice what did *not* happen. Going from 32 bits to 128 bits is going from a four-byte input to a sixteen-byte input — twelve extra bytes. The input grew by a factor of four and the running time by a factor of 4×10^{11} . That is what “exponential in the input size” means, and it is why measuring the size by the value would have hidden it completely.

2.2 The Nature of the Input: Three Cases

Fix the size. Two inputs of the same size can still cost an algorithm wildly different amounts, because an algorithm’s control flow depends on what the data *says*, not only on how much of it there is.

Best, Worst and Average Case

Let $T(x)$ be the number of primitive operations the algorithm performs on input x , and let X_n be the set of all inputs of size n .

Case	What it promises
best: $\min_{x \in X_n} T(x)$	Nothing useful — a lower bound on an optimist’s hopes.
worst: $\max_{x \in X_n} T(x)$	A <i>guarantee</i> : no input of size n costs more than this.
average: $\mathbb{E}_{x \sim D} [T(x)]$	What to expect <i>if</i> inputs really are drawn from D — and D must be stated.

⚠ “Average case” is meaningless until you name a distribution

There is no such thing as a random input. There is only an input drawn from a distribution you have chosen, and the choice is a modelling assumption that can be wrong.

The conventional assumption for sorting is a **uniform random permutation**: every ordering of the n elements equally likely. It is a reasonable default and it is often false —

real data arrives partly sorted, or sorted by the wrong key, or in a few long runs. When you quote an average case, say what distribution it averages over. When you read one, check.

Insertion sort, in all three cases

Insertion sort's inner loop shifts elements right until it finds the place for **key**. The number of shifts for element j is how many of the j elements before it are larger — the number of *inversions* involving j .

Worked Example: The Three Cases, Derived

Best case — already sorted. For every j , $a[j-1] \leq \text{key}$, so the **while** test fails at once and does zero shifts. The outer loop still runs $n-1$ times:

$$T(n) = c(n-1) = \Theta(n).$$

Worst case — strictly decreasing. Every element must travel to the front, so element j does j shifts:

$$\sum_{j=1}^{n-1} j = \frac{(n-1)n}{2} \approx \frac{n^2}{2} = \Theta(n^2).$$

Average case — uniform random permutation. For any pair (i, j) with $i < j$, the two orderings are equally likely, so the pair is inverted with probability $\frac{1}{2}$. There are $\binom{n}{2}$ pairs, so by linearity of expectation the expected number of inversions is

$$\mathbb{E}[\text{inversions}] = \frac{1}{2} \binom{n}{2} = \frac{n(n-1)}{4} \approx \frac{n^2}{4} = \Theta(n^2).$$

The conclusion, and it is the one that matters. The average case is *exactly half* the worst case. A factor of two is a constant, and $\Theta(\cdot)$ discards constants — so insertion sort's average and worst cases have the *same order of growth*, and the only case that differs is the best one.

Worked Example: One Algorithm, Three Orders of Growth, Measured

`code/ch02_cases.cpp` runs the same insertion sort on three shapes of input at each size; median of five runs, times in milliseconds.

n	sorted	ratio	random	ratio	reversed	ratio
2,000	0.002	—	0.340	—	0.634	—
4,000	0.004	1.90	1.430	4.21	2.551	4.03
8,000	0.008	2.05	5.288	3.70	9.736	3.82
16,000	0.021	2.73	21.357	4.04	42.714	4.39
32,000	0.028	1.33	89.790	4.20	185.054	4.33

The sorted column doubles — 1.90, 2.05, 2.73, 1.33. The ratios are ragged because the absolute times are two to twenty-eight *microseconds*, where the clock's own resolution and one stray interrupt are a large fraction of the measurement. The honest reading is: these are five numbers consistent with linear growth and far too small to be consistent with quadratic, which is all the experiment is being asked to show.

The random and reversed columns quadruple, mean 4.04 and 4.14, as $\Theta(n^2)$ requires.

And now the prediction the derivation made. Random should be half of reversed, because $n^2/4$ is half of $n^2/2$. Measured:

0.54, 0.56, 0.54, 0.50, 0.49.

Five independent confirmations of a factor the theory produced from counting inversions. This is what it looks like when an analysis is right about more than the exponent.

Finally, the spread. At $n = 32,000$: 0.028 ms on sorted input, 185 ms on reversed. Same algorithm, same compiler, same machine, same input size — a factor of **6,600**. “How big is the input” does not come close to determining how long a program takes.

2.3 Which Case Should You Report?

 The convention, and the reasons for it

Report the worst case unless you say otherwise. Three reasons:

1. It is a *guarantee*. An average tells you about a population of runs; a worst case tells you about the run you are having now, which is the one with the deadline.
2. It is often what happens. For searching an absent key, or any algorithm that must examine everything to be sure, the worst case is the typical case.
3. It is frequently the same order as the average anyway — insertion sort above, where the factor between them is two.

Report the best case almost never. “This sort is $O(n)$ ” with the best case quietly intended is the most common dishonest sentence in the subject.

Report the average case when the worst case is both rare and pessimistic, as with quicksort (Chapter 9) and hashing (Chapter 13) — and then state the distribution, because in both of those chapters there is an input that makes the worst case happen on purpose.

 Expected Running Time

A **randomised algorithm** makes random choices of its own. Its **expected running time** on an input x is the average over *those choices*, with x held fixed:

$$\mathbb{E}[T(x)], \quad \text{expectation over the algorithm's coin flips.}$$

 Two different averages, and the difference is the whole trick

Average-case analysis averages over inputs. You must assume a distribution on the input, and an adversary who knows your algorithm can choose an input outside it.

Expected-time analysis of a randomised algorithm averages over the algorithm's own coins, *for every input*. There is no assumption about the data at all. An adversary who knows your code still cannot pick a bad input, because he does not know your coins.

That is why Chapter 9 randomises: it converts an assumption about the world, which can be violated, into an assumption about a random number generator, which cannot. The bound goes from “ $\Theta(n \lg n)$ if your data is unsorted” to “ $\Theta(n \lg n)$ expected, on *every* input”.

 Common Pitfalls

- **Quoting one case and meaning another.** Always name it: “linear search is $\Theta(n)$ in the worst case and $\Theta(n)$ on average” is a real statement; “linear search is $O(n)$ ” is a weaker one that people read as the worst case, correctly.

- **Measuring the size of a number by its value.** Then trial division looks linear, and the table in Section 2.1 looks impossible.
- **Quoting a graph bound in one parameter.** $O(|V|^2)$ hides whether you have exploited sparsity; $O(|V| + |E|)$ says you have.
- **Assuming the average case without saying so.** Especially for sorting, where real inputs are very often partly sorted and the uniform permutation assumption is simply false.
- **Thinking the best case tells you the algorithm is fast.** Insertion sort's best case is $\Theta(n)$ and it is still a quadratic sort.
- **Confusing “average case” with “expected time”.** The first assumes something about the data; the second assumes nothing. The second is much stronger and is almost always what a randomised algorithm's bound means.

Try It Yourself: Three Shapes, and the Cost of a Bit

Reproduces both tables in this chapter. Expect twenty-five minutes — part B takes about two minutes of CPU on its own.

```
// ch02_cases.cpp -- the nature of the input, and the size of the input.
// Build: cl /O2 /EHsc /std:c++17 ch02_cases.cpp      (see Appendix A)

// --- 1. Three shapes of input, all of the same size -----
// Best case: already sorted, so the inner while never executes.
// Worst case: reversed, so it runs to the front every time.
// Average: a random permutation, which moves each element about half
// way -- and half of a quadratic is still a quadratic.
static std::vector<int> sorted_input(size_t n) {
    std::vector<int> a(n);
    std::iota(a.begin(), a.end(), 0);
    return a;
}
static std::vector<int> reversed_input(size_t n) {
    std::vector<int> a = sorted_input(n);
    std::reverse(a.begin(), a.end());
    return a;
}

// --- 2. Trial division, whose cost is governed by sqrt(N) -----
// The loop runs about sqrt(N)/2 times when n is prime, which is the
// worst case for this algorithm -- so a prime is what we time.
static bool is_prime_trial(uint64_t n) {
    if (n < 2) return false;
    if (n % 2 == 0) return n == 2;
    for (uint64_t d = 3; d * d <= n; d += 2)
        if (n % d == 0) return false;
    return true;
}

// --- 3. The same algorithm, three orders of growth -----
for (size_t n = 2000; n <= 32000; n *= 2) {
    auto s = sorted_input(n);
    auto r = random_input(n, 12345);
    auto v = reversed_input(n);
    double ts = time_ms([&]{ auto a = s; insertion_sort(a); }, 5);
    double tr = time_ms([&]{ auto a = r; insertion_sort(a); }, 5);
    double tv = time_ms([&]{ auto a = v; insertion_sort(a); }, 5);
    /* print the three times and the three ratios */
}
```

```
// --- 4. Why the size of a number is its number of bits -----
// Each row adds TWO bits -- four more characters of input, if you
// measure input by its value. Watch the time double anyway.
const uint64_t primes[] = {
    4294967291ULL,    17179869143ULL,    68719476731ULL,
    274877906899ULL,  1099511627689ULL,    4398046511093ULL,
    17592186044399ULL, 70368744177643ULL,  281474976710597ULL};
int bits = 32;
for (uint64_t p : primes) {
    bool r = false;
    double t = time_ms([&]{ r = is_prime_trial(p); }, 3);
    if (!r) { printf("  BUG: %llu is not prime\n", p); return 1; }
    printf("%4d %22llu %12.2f %7.2f\n", bits, p, t, t / prev);
    prev = t; bits += 2;
}
```

What to notice. In part A, look at the *ratio* of the random column to the reversed column rather than at either alone. It should be close to 0.5, and it is the one number in the experiment that tests the inversion count rather than just the exponent.

Then try to break it. Replace the random permutation with an array that is sorted except for the last 100 elements, which are shuffled. Predict the running time before you measure it. This is the input shape real data actually has, and it is the one both the best-case and the average-case analyses describe badly.

🔍 Checkpoint — Test Yourself

1. An algorithm takes as input an $n \times n$ matrix and runs in $\Theta(n^3)$. Is it polynomial in the size of its input? Answer in terms of the number of numbers actually read.
2. Linear search for a key that is present at a uniformly random position examines $(n+1)/2$ elements on average and n in the worst case. Why does the course nevertheless describe both cases with the same $\Theta(\cdot)$?
3. A colleague reports that their sorting routine is “ $O(n \lg n)$ on average”. What single question would you ask before believing the number means anything for your data?

📋 Chapter Summary

- Input size is a measure of how much input there is: elements for an array, the side for a matrix, *two* parameters for a graph, and **bits** for a number.
- A number’s size is its bit length, so a loop running N times is $O(2^b)$ — exponential. Measured: trial division doubled for every two bits added, eight times running, mean ratio 2.02.
- Best case promises nothing, worst case promises a guarantee, average case promises something only once you have named the distribution.
- Insertion sort: $\Theta(n)$ best, $\Theta(n^2)$ worst, $\Theta(n^2)$ average — the average being exactly half the worst, which measurement confirmed at 0.54, 0.56, 0.54, 0.50, 0.49.
- At $n = 32,000$ the same sort took 0.028 ms or 185 ms depending only on the *shape* of its input: a factor of 6,600.
- Report the worst case by default. Report an average only with its distribution attached.
- An average over inputs assumes something about the world; the expected time of a randomised algorithm assumes nothing, which is why Chapter 9 randomises.

 Review Questions

1. **(MCQ)** The input size of the integer N is taken to be: (a) N ; (b) $\sqrt{N}$; (c) about $\lg N$; (d) N^2 .
2. **(MCQ)** Insertion sort's average-case running time on a uniform random permutation is: (a) $\Theta(n)$; (b) $\Theta(n \lg n)$; (c) $\Theta(n^2)$; (d) $\Theta(n^2 \lg n)$.
3. **(Short)** Explain what is meant by the best, expected and worst case behaviour of an algorithm, and give one algorithm whose three cases have three different orders of growth (*CLO-1*).
4. **(Short)** Identify two characteristics of input data that change which case an algorithm exhibits, and name an algorithm affected by each (*CLO-2*).
5. **(Short)** Why is a bound of $O(|V| + |E|)$ more informative than one of $O(|V|^2)$, even when the two are equal for the graph in front of you?
6. **(Applied)** Trial division on a 48-bit prime took 58.81 ms on the book's machine. Estimate the time for a 70-bit prime, state the assumption your estimate rests on, and say why the estimate is more trustworthy than the absolute time it is derived from.
7. **(Applied)** You are given a sort you cannot read the source of. Design an experiment that decides whether its worst case is quadratic, using only the ability to supply inputs and time it. Say what inputs you would supply, what you would compute, and what result would leave you unable to tell.

Asymptotic Notation

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part I — Analysing an Algorithm

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **State** the formal definitions of O , Ω , Θ , o and ω , and **explain** what each one quantifies over (CLO-5).
- **Prove** that a given function belongs to a given asymptotic class by exhibiting the witnesses c and n_0 , and **disprove** membership by contradiction.
- **Use** the limit test, and **recognise** the case in which it does not apply.
- **Order** the standard functions by growth, and **apply** transitivity and the sum rule to simplify a bound.
- **Identify** the four common abuses of the notation, and **state** what each one conceals.

🔗 Before You Start

Chapter 1 and Chapter 2. From Appendix B you need the logarithm identities and, for Section 3.5 only, L'Hôpital's rule.

📖 Key Terms in This Chapter

asymptotic upper bound • asymptotic lower bound • tight bound • witness • O • Ω • Θ • o • ω • transitivity • transpose symmetry • sum rule • limit test • trichotomy

3.1 Three Bounds, Defined

Chapter 1 said we discard constants and keep the dominant term. That is a practice, not a definition, and the practice can be abused. This chapter gives it a definition precise enough to prove things with.

Throughout, f and g map the positive integers to the non-negative reals — running times, so never negative.

📑 O , Ω and Θ

Upper bound. $O(g(n))$ is the *set* of functions

$$O(g(n)) = \{ f(n) : \exists c > 0, \exists n_0 > 0, \forall n \geq n_0, 0 \leq f(n) \leq c g(n) \}.$$

Lower bound.

$$\Omega(g(n)) = \{ f(n) : \exists c > 0, \exists n_0 > 0, \forall n \geq n_0, 0 \leq c g(n) \leq f(n) \}.$$

Tight bound.

$$\Theta(g(n)) = \{ f(n) : \exists c_1, c_2 > 0, \exists n_0 > 0, \forall n \geq n_0, 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \}.$$

The constants c (or c_1, c_2) and n_0 are the **witnesses**. To prove a bound you produce them; to disprove one you show none can exist.

⚠ “ $f(n) = O(g(n))$ ” is an abuse of the equals sign, and you should know it

$O(g)$ is a set, so the honest statement is $f \in O(g)$. Everyone writes $f(n) = O(g(n))$ anyway, including this book, and the convention is harmless as long as you remember that it is **one-directional**:

- $n = O(n^2)$ is true.
- $O(n^2) = n$ is meaningless.
- From $f = O(n^2)$ and $h = O(n^2)$ you *cannot* conclude $f = h$.

Read the “=” as “is” — as in “Rawalpindi is a city”, which does not make every city Rawalpindi.

3.2 Proving a Bound From the Definition

The definitions are existential, so a proof is a construction: name c , name n_0 , verify the inequality.

🔑 **Theorem 3.1 — A quadratic is $O(n^2)$**

$$3n^2 + 10n + 7 = O(n^2).$$

Proof. We must produce c and n_0 with $3n^2 + 10n + 7 \leq cn^2$ for all $n \geq n_0$. For $n \geq 1$ we have $n \leq n^2$ and $1 \leq n^2$, so

$$3n^2 + 10n + 7 \leq 3n^2 + 10n^2 + 7n^2 = 20n^2.$$

Taking $c = 20$ and $n_0 = 1$ satisfies the definition. ■

💡 The witnesses are not unique, and need not be tight

That proof produced $c = 20$. A sharper argument gives a smaller one: for $n \geq 10$, $10n + 7 \leq n^2$, so $c = 4$ and $n_0 = 10$ also work.

Both proofs are equally valid, because the definition asks only that *some* pair exist. Students waste effort hunting for the smallest c ; the definition does not reward it. Pick the crudest bound that closes the inequality — here, replacing every term by the largest one.

🧩 **Lemma 3.2 — Polynomials**

Let $p(n) = \sum_{i=0}^d a_i n^i$ with $a_d > 0$. Then $p(n) = \Theta(n^d)$.

Proof. Upper bound. Let $A = \sum_{i=0}^d |a_i|$. For $n \geq 1$ every $n^i \leq n^d$, so $p(n) \leq An^d$, giving $c_2 = A$.

Lower bound. Write $p(n) = a_d n^d + r(n)$ where $r(n) = \sum_{i<d} a_i n^i$. By the same argument $|r(n)| \leq Bn^{d-1}$ with $B = \sum_{i<d} |a_i|$. Hence

$$p(n) \geq a_d n^d - Bn^{d-1} = n^d \left(a_d - \frac{B}{n} \right) \geq \frac{a_d}{2} n^d \quad \text{whenever } n \geq \frac{2B}{a_d}.$$

So $c_1 = a_d/2$ and $n_0 = \max\{1, 2B/a_d\}$ complete the pair of bounds, and $p(n) = \Theta(n^d)$ by definition. ■

» **Corollary 3.3 — Only the degree survives**

The degree of a polynomial running time is the only thing about it that survives asymptotic analysis. $0.001n^3$ grows faster than $1000n^2$.

Disproof is the other direction, and it is a proof by contradiction.

🔗 Theorem 3.4 — A quadratic is not linear

$$n^2 \neq O(n).$$

Proof. Suppose it were. Then there exist $c > 0$ and $n_0 > 0$ such that $n^2 \leq cn$ for all $n \geq n_0$. Since $n > 0$ we may divide by n :

$$n \leq c \quad \text{for all } n \geq n_0.$$

But c is a fixed constant, so taking $n = \max\{n_0, \lceil c \rceil + 1\}$ gives $n \geq n_0$ and $n > c$ simultaneously — contradicting the line above. No such c exists. ■

🔗 Theorem 3.5 — Θ is exactly O and Ω together

$$f(n) = \Theta(g(n)) \text{ if and only if } f(n) = O(g(n)) \text{ and } f(n) = \Omega(g(n)).$$

Proof. ($\Rightarrow$) Suppose $f = \Theta(g)$ with witnesses c_1, c_2, n_0 . Then $f(n) \leq c_2 g(n)$ for $n \geq n_0$, which is the definition of O with $c = c_2$; and $c_1 g(n) \leq f(n)$ for $n \geq n_0$, which is the definition of Ω with $c = c_1$.

($\Leftarrow$) Suppose $f = O(g)$ with witnesses c_u, n_u and $f = \Omega(g)$ with witnesses c_ℓ, n_ℓ . Put $n_0 = \max\{n_u, n_\ell\}$. For every $n \geq n_0$ both inequalities hold at once, so

$$c_\ell g(n) \leq f(n) \leq c_u g(n),$$

which is the definition of Θ with $c_1 = c_\ell$ and $c_2 = c_u$. ■

💡 Why that theorem earns its place

It is the most-used result in the subject and it is two lines long, so it is tempting to skip. Do not: it is what licenses the standard division of labour in every later analysis.

To establish a Θ bound you prove an upper bound and a lower bound *separately*, by completely different arguments. Chapter 10 proves $\Omega(n \lg n)$ for comparison sorting by a decision-tree argument that has nothing in common with Chapter 8's $O(n \lg n)$ recurrence — and Theorem 3.5 is the reason the two halves combine into $\Theta(n \lg n)$.

3.3 The Strict Bounds: o and ω

O allows f and g to grow at the same rate. The strict forms forbid it, and the change in the definition is the position of one quantifier.

📋 o and ω

$$o(g(n)) = \{ f(n) : \forall c > 0, \exists n_0 > 0, \forall n \geq n_0, 0 \leq f(n) < c g(n) \}$$

$$\omega(g(n)) = \{ f(n) : \forall c > 0, \exists n_0 > 0, \forall n \geq n_0, 0 \leq c g(n) < f(n) \}$$

The whole difference is $\exists c$ against $\forall c$. For O it is enough that *one* constant works. For o , *every* constant must work, however small — so f must become negligible relative to g , not merely bounded by it.

🧮 Worked Example: Testing the Quantifier

Is $2n^2 = o(n^2)$? Take $c = 1$. We would need $2n^2 < n^2$ for all large n , i.e. $2 < 1$. So **no** — and the reasoning shows why: o requires the ratio to vanish, and $2n^2/n^2 = 2$ for every n .

Is $2n = o(n^2)$? Given any $c > 0$, we need $2n < cn^2$, i.e. $n > 2/c$. So $n_0 = \lceil 2/c \rceil + 1$ works. Since we produced an n_0 for *every* c , **yes**.

The pattern. $f = o(g)$ exactly when $f = O(g)$ but $f \neq \Theta(g)$ — the bound holds and is *not* tight.

3.4 Properties, and the Analogy With Inequality

Theorem 3.6 — Transitivity

If $f(n) = O(g(n))$ and $g(n) = O(h(n))$, then $f(n) = O(h(n))$.

Proof. Take witnesses c_1, n_1 for the first and c_2, n_2 for the second, and let $n_0 = \max\{n_1, n_2\}$. For $n \geq n_0$,

$$f(n) \leq c_1 g(n) \leq c_1(c_2 h(n)) = (c_1 c_2) h(n),$$

where the second inequality may be multiplied by $c_1 > 0$ without reversing. So $c = c_1 c_2$ and n_0 are witnesses for $f = O(h)$. The same argument, with the inequalities reversed, gives transitivity for Ω , and Theorem 3.5 then gives it for Θ . ■

Lemma 3.7 — Sum rule

If $f_1 = O(g_1)$ and $f_2 = O(g_2)$, then $f_1 + f_2 = O(\max\{g_1, g_2\})$.

Proof. With witnesses c_1, n_1 and c_2, n_2 , put $n_0 = \max\{n_1, n_2\}$ and $M(n) = \max\{g_1(n), g_2(n)\}$. For $n \geq n_0$,

$$f_1(n) + f_2(n) \leq c_1 g_1(n) + c_2 g_2(n) \leq c_1 M(n) + c_2 M(n) = (c_1 + c_2) M(n). \quad \blacksquare$$

Lemma 3.7 is why “keep the dominant term” is legitimate: a program that does an $O(n^2)$ pass and then an $O(n)$ pass is $O(n^2)$, and we do not have to think about it again.

The analogy with $\leq$, and the one place it fails

Asymptotic	Reads as	Properties it really has
$f = O(g)$	$f \leq g$	transitive; reflexive ($f = O(f)$)
$f = \Omega(g)$	$f \geq g$	transitive; reflexive
$f = \Theta(g)$	$f = g$	transitive, reflexive <i>and</i> symmetric — an equivalence relation, which the other four are not
$f = o(g)$	$f < g$	transitive; <i>not</i> reflexive
$f = \omega(g)$	$f > g$	transitive; not reflexive

Transpose symmetry also holds: $f = O(g)$ exactly when $g = \Omega(f)$, and $f = o(g)$ exactly when $g = \omega(f)$.

But trichotomy fails. For real numbers, exactly one of $a < b$, $a = b$, $a > b$ holds. For functions there is a fourth possibility: *incomparable*. Let

$$f(n) = n^{1+\sin n}.$$

As n runs over the integers $\sin n$ oscillates densely in $[-1, 1]$, so the exponent oscillates in $[0, 2]$. Infinitely often $f(n)$ is near $n^0 = 1$, which kills any $\Omega(n)$ bound; infinitely often it is near n^2 , which kills any $O(n)$ bound. So f is **neither** $O(n)$ **nor** $\Omega(n)$ — the two functions simply cannot be ordered.

Pathological, and worth one paragraph: it is the reason the next section's limit test needs a disclaimer, and the reason you may not argue “ f is not $O(g)$, therefore f is $\Omega(g)$ ”.

3.5 The Limit Test

Constructing witnesses is reliable and slow. For most functions met in practice, a limit settles the question at once.

Theorem 3.8 — Limit test

Suppose $L = \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ exists, with $L = \infty$ allowed. Then

$$L = 0 \implies f = o(g); \quad 0 < L < \infty \implies f = \Theta(g); \quad L = \infty \implies f = \omega(g).$$

Proof. We prove the middle case; the others are similar. Let $0 < L < \infty$. By the definition of a limit, taking $\varepsilon = L/2$ there is an n_0 such that for all $n \geq n_0$,

$$\left| \frac{f(n)}{g(n)} - L \right| < \frac{L}{2} \implies \frac{L}{2} < \frac{f(n)}{g(n)} < \frac{3L}{2}.$$

Multiplying through by $g(n) > 0$ gives $\frac{L}{2}g(n) < f(n) < \frac{3L}{2}g(n)$, so $c_1 = L/2$ and $c_2 = 3L/2$ witness $f = \Theta(g)$. ■

The test is one-directional

Theorem 3.8 assumes the limit *exists*. If it does not, the test says nothing at all — it does not say the functions are incomparable.

For $f(n) = n^{1+\sin n}$ and $g(n) = n$ the ratio $n^{\sin n}$ oscillates forever and the limit does not exist; here the functions do turn out to be incomparable. But $f(n) = n(2 + \sin n)$ also has no limiting ratio against $g(n) = n$, and yet $f = \Theta(n)$ plainly, with $c_1 = 1$ and $c_2 = 3$. When the limit fails, go back to the witnesses.

Worked Example: Logarithms Lose to Every Power

Claim: $\lg n = o(n^\varepsilon)$ for every constant $\varepsilon > 0$ — however small.

Step 1. Both $\lg n \rightarrow \infty$ and $n^\varepsilon \rightarrow \infty$, so the ratio is of the form ∞/∞ and L'Hôpital's rule applies. Treat n as a real variable x and use $\lg x = \ln x / \ln 2$:

$$\lim_{x \rightarrow \infty} \frac{\lg x}{x^\varepsilon} = \lim_{x \rightarrow \infty} \frac{(1/(x \ln 2))}{\varepsilon x^{\varepsilon-1}} = \lim_{x \rightarrow \infty} \frac{1}{\varepsilon \ln 2 \cdot x^\varepsilon} = 0.$$

Step 2. The limit is 0, so by Theorem 3.8, $\lg n = o(n^\varepsilon)$.

Step 3 — what it means. Take $\varepsilon = 0.01$. The claim is that $\lg n$ is eventually negligible beside $n^{0.01}$, a function so flat it looks constant on any plot you can draw. “Eventually” is doing heavy work: at $n = 10^6$, $\lg n \approx 20$ while $n^{0.01} \approx 1.15$, so the inequality has not begun to hold. It holds from about $n = 10^{530}$.

Step 4 — the lesson. Asymptotic statements are about limits, and a limit carries no promise about $n = 10^6$. This is the same lesson as Chapter 1’s crossover at $n = 65$, in a more extreme form.

💡 The hierarchy worth memorising

Each function below is o of the one after it, for any constants $\varepsilon > 0$, $k \geq 1$ and $a > 1$:

$$1 \prec \lg \lg n \prec \lg n \prec (\lg n)^k \prec n^\varepsilon \prec n \prec n \lg n \prec n^2 \prec n^k \prec a^n \prec n!$$

At $n = 10^6$, taking one operation per nanosecond:

$g(n)$	Operations	Time
$\lg n$	20	20 nanoseconds
n	10^6	1 millisecond
$n \lg n$	2.0×10^7	20 milliseconds
n^2	10^{12}	17 minutes
n^3	10^{18}	32 years
2^n	—	no physical comparison exists

The gap from $n \lg n$ to n^2 — 20 milliseconds to 17 minutes — is the one most algorithm design is about.

3.6 What Θ Asserts, Measured

$f = \Theta(g)$ says $f(n)/g(n)$ is eventually trapped between two positive constants. That is a claim about a ratio, so it can be tested directly — and tested against *rival* exponents, which is the sharper experiment.

📊 Worked Example: Ruling Out the Exponents Either Side

`code/ch03_fit.cpp` times insertion sort and divides by three candidate functions. Each column is normalised to 1.000 at the first row, because the constant itself is machine-specific and the question is only whether the column *stays put*. Median of nine runs.

n	T (ms)	$T/n^{1.9}$	$T/n^{2.0}$	$T/n^{2.1}$
4,000	2.45	1.000	1.000	1.000
8,000	10.22	1.116	1.041	0.972
16,000	39.19	1.146	0.998	0.869
32,000	162.82	1.276	1.036	0.842
64,000	678.31	1.424	1.079	0.818
128,000	2595.28	1.460	1.032	0.730

Read the three columns as three hypotheses.

- $T/n^{1.9}$ rises by 46%, without turning over. If $T = \Theta(n^{1.9})$ this column would be trapped between constants; it is climbing, so $n^{1.9}$ is too small.
- $T/n^{2.0}$ stays inside $[0.998, 1.079]$ — a spread of 8% across a 32-fold range of n . That is a ratio trapped between two constants, which is exactly the definition.
- $T/n^{2.1}$ falls by 27%, monotonically. So $n^{2.1}$ is too large: $T = o(n^{2.1})$, an upper bound that is not tight.

The experiment therefore does more than confirm the exponent — it excludes the two exponents either side, which is what distinguishes evidence for $\Theta(n^2)$ from mere consistency with it.

📊 Worked Example: The Same Test for $n \lg n$, Which Is Harder

$n \lg n$ sits only a whisker above n , so this test needs both a wider range of n and more careful timing.

n	T (ms)	T/n	$T/(n \lg n)$	$T/n^{1.2}$
100,000	11.49	1.000	1.000	1.000
200,000	23.89	1.040	0.980	0.905
400,000	52.63	1.145	1.022	0.868
800,000	107.19	1.166	0.988	0.769
1,600,000	237.89	1.294	1.043	0.743
3,200,000	489.47	1.331	1.023	0.666
6,400,000	1036.62	1.410	1.036	0.614

$T/(n \lg n)$ stays within 4.3% of 1 over a 64-fold range of n . The rivals both fail: T/n climbs 41%, $T/n^{1.2}$ falls 39%.

Now the detail that makes this more than a curve fit. If T really is $cn \lg n$, then $T/n = c \lg n$, so the T/n column should itself grow like $\lg n$. Over this range that predicts a rise of

$$\frac{\lg(6,400,000)}{\lg(100,000)} = \frac{22.61}{16.61} = 1.361.$$

Measured: **1.410**, within 4% of the prediction. The drift in the failing column is not noise — it is the $\lg n$ factor, measured.

And what it took to get there. The first draft of this program allocated a fresh vector inside the timed region. At these sizes the allocation cost more than the sort it was supposed to be measuring, and the $T/(n \lg n)$ column came out as 1.000, 1.134, 0.804, 1.342, 0.839 — noise, from which nothing follows. The fix was to allocate once outside the loop, refill with `std::copy`, and *measure the refill separately and subtract it*; the `copy` column in the program's output is that measurement, and it is how you can tell the subtraction was small enough to trust. Appendix A collects the rest of these rules.

3.7 Four Abuses, and What Each Conceals

⚠ 1. Using O when you mean Θ

“Insertion sort is $O(n^2)$ ” is true. So is “insertion sort is $O(n^{100})$ ”, and so is “ $O(2^n)$ ” — an upper bound stays true however loose it gets. If you know the bound is tight, say Θ and claim the credit; if you do not, O is honest and you should expect to be asked.

The reverse error is worse: reading somebody’s $O(\cdot)$ as if it were a $\Theta(\cdot)$, and concluding an algorithm is slower than it is.

⚠ 2. Ω as an excuse

“My algorithm is $\Omega(n)$ ” is a boast about a *lower* bound, which for an algorithm is bad news, not good. Lower bounds are interesting about *problems* — “every comparison sort is $\Omega(n \lg n)$ ”, Chapter 10 — because they say no algorithm can do better. About your own code, $\Omega(\cdot)$ means “at least this slow”.

⚠ 3. Dropping the case

$\Theta(\cdot)$ says nothing about which input you had in mind. Quicksort is $\Theta(n^2)$ in the worst case and $\Theta(n \lg n)$ in expectation; both are true and they differ by a factor of thousands. **Name the case every time** — the notation has no slot for it, so the sentence must.

⚠ 4. Forgetting that the constants went somewhere

Asymptotic notation deliberately discards exactly the information that decides which program to run at your actual n . From `ch03_fit.cpp`: $100n \lg n < n^2$ first holds at $n = 997$ — not at $n = 10$. Below a thousand elements the quadratic algorithm wins, and $\Theta(\cdot)$ cannot tell you that.

The notation is a tool for comparing *ideas*. Choosing an *implementation* needs the constants back, which means measuring — the other half of every chapter in this book.

! Common Pitfalls

- **Writing $O(2n^2)$ or $O(n^2 + n)$.** Legal but illiterate: the point of the notation is that constants and lower terms are already gone. Write $O(n^2)$.
- **Hunting for the smallest c .** The definition asks for existence. Any witness pair is a complete proof.
- **Concluding $f = \Omega(g)$ from “ $f \neq O(g)$ ”.** Trichotomy fails; $n^{1+\sin n}$ is the counterexample.
- **Using the limit test without checking the limit exists.** When it does not, the test is silent, not negative.
- **Comparing $O(\cdot)$ bounds to rank two algorithms.** Both being $O(n^2)$ tells you nothing about which is faster, nor even that either is quadratic.
- **Saying “ $\lg n$ is basically constant”.** It is $o(n^\varepsilon)$ for every ε , and it is also 20 at $n = 10^6$ and 30 at $n = 10^9$. Negligible in the limit is not the same as negligible.
- **Timing inside an allocation.** Not a notation error, but the reason the second worked example nearly reported nonsense.

🔧 Try It Yourself: Fitting a Curve, and Excluding the Rivals

Reproduces both tables in Section 3.6 and the crossover figure. Expect thirty minutes; part A takes about a minute of CPU at $n = 128,000$.

```
// ch03_fit.cpp -- what Theta(g) actually asserts, tested.
// Build: cl /O2 /EHsc /std:c++17 ch03_fit.cpp           (see Appendix A)
```

```

// --- 1. Insertion sort against three candidate exponents -----
// Only one of the three columns can stay constant, and which one
// does is the empirical content of "Theta(n^2)".
//
// Normalising to the FIRST row makes that row's noise everybody's
// noise, so the first row must not be a sub-millisecond timing:
// start at n = 4000 and take the median of nine runs.
double b19 = 0, b20 = 0, b21 = 0;
for (size_t n = 4000; n <= 128000; n *= 2) {
    auto base = random_input(n, 2024);
    std::vector<int> work(n);
    double t = time_ms([&]{
        std::copy(base.begin(), base.end(), work.begin());
        insertion_sort(work);
    }, 9);
    double x = static_cast<double>(n);
    double r19 = t / std::pow(x, 1.9);
    double r20 = t / std::pow(x, 2.0);
    double r21 = t / std::pow(x, 2.1);
    if (b19 == 0) { b19 = r19; b20 = r20; b21 = r21; }
    printf("%8zu %10.2f %10.3f %10.3f %10.3f\n",
        n, t, r19/b19, r20/b20, r21/b21);
}

// --- 2. Merge sort against n*lg(n) and its two rivals -----
// n*lg(n) is only a whisker above n, so this is a far harder test
// than part 1, and it needs better hygiene to pass.
//
// The first draft allocated a fresh vector inside the timed region.
// At these sizes that allocation costs more than it measures -- the
// T/(n lg n) column came out as 1.000, 1.134, 0.804, 1.342, 0.839,
// which says nothing at all.
//
// The fix: allocate ONCE outside the timing and refill with
// std::copy, then measure the refill alone and subtract it.
for (size_t n = 100000; n <= 6400000; n *= 2) {
    auto base = random_input(n, 2024);
    std::vector<int> work(n), buf(n);
    double tc = time_ms([&]{
        std::copy(base.begin(), base.end(), work.begin());
    }, 11);
    double tt = time_ms([&]{
        std::copy(base.begin(), base.end(), work.begin());
        merge_sort(work, buf, 0, n);
    }, 11);
    double t = tt - tc; // the sort, with the refill removed
    /* print tc so the reader can check the subtraction was small,
       then t/n, t/(n lg n) and t/n^1.2 normalised to the first row */
}

// --- 3. Where an asymptotically worse function is still smaller ----
// Theta is a statement about a limit, and the limit can be a long way
// off. Report the crossover rather than pretending there is none:
// 100 n lg n < n^2 requires 100 lg n < n.
for (size_t n = 2; n < 100000; ++n) {
    double x = static_cast<double>(n);
    if (100.0 * x * std::log2(x) < x * x) {
        printf("\n    crossover: n lg n first wins at n = %zu\n", n);
        break;
    }
}

```

What to notice. The `copy` column in part 2 is there to be audited, not admired: if it ever approaches the `total` column, the subtraction is doing too much work and the experiment should be redesigned rather than reported.

Then try to break it. Add a $T/n^{2.05}$ column to part 1. It will drift down, but only slightly — and that is the honest limit of this method. Timing can separate $n^{1.9}$ from $n^{2.1}$ over a 32-fold range; it cannot separate n^2 from $n^{2.02}$, and no amount of care will make it. Say what an experiment can resolve before you claim a result from it.

Checkpoint — Test Yourself

1. Prove that $5n^3 + 200n^2 + 17 = O(n^3)$ by exhibiting witnesses c and n_0 , and verify your inequality at $n = n_0$.
2. Is $n^2 = o(n^2 \lg n)$? Justify your answer from the definition, not from the limit test.
3. A student writes: “my algorithm is $O(n^3)$ and my colleague’s is $O(n^2)$, so mine is slower.” Give the two distinct reasons this conclusion does not follow.

Chapter Summary

- O , Ω and Θ are *sets* of functions, defined by the existence of witnesses c and n_0 . “ $f = O(g)$ ” means $f \in O(g)$ and may not be read backwards.
- To prove a bound, exhibit witnesses — any witnesses. To disprove one, derive a contradiction, as for $n^2 \neq O(n)$.
- $f = \Theta(g)$ iff $f = O(g)$ and $f = \Omega(g)$, which is what lets upper and lower bounds be proved by unrelated arguments and combined.
- o and ω differ from O and Ω only in having $\forall c$ where those have $\exists c$, which forces the ratio to vanish or diverge.
- All five are transitive; only Θ is an equivalence relation; and trichotomy fails — $n^{1+\sin n}$ is comparable to n in no direction at all.
- The limit test decides the question whenever the limit exists, and is silent, not negative, when it does not.
- Measured: $T/n^{2.0}$ for insertion sort held within 8% across a 32-fold range of n while $T/n^{1.9}$ rose 46% and $T/n^{2.1}$ fell 27%. For merge sort $T/(n \lg n)$ held within 4.3% across a 64-fold range, and the drift in the failing T/n column matched the predicted $\lg n$ growth to within 4%.
- The notation discards precisely what decides which program to run: $100n \lg n$ does not beat n^2 until $n = 997$.

Review Questions

1. (MCQ) $f(n) = \Theta(g(n))$ holds if and only if: (a) $f = O(g)$; (b) $f = \Omega(g)$; (c) both of these; (d) $f = o(g)$.
2. (MCQ) Which is *false*? (a) $n = O(n^2)$; (b) $n^2 = O(n)$; (c) $2^{n+1} = O(2^n)$; (d) $\lg n = o(n)$.
3. (Short) State the formal definition of $O(g(n))$ and explain the role of each quantifier (CLO-5).
4. (Short) Explain the difference between $O(g)$ and $o(g)$ in terms of the quantifier on c , and give a function that is in the first but not the second.
5. (Short) Why is a lower bound a desirable result about a problem but an undesirable one about your own algorithm?
6. (Applied) Prove $\Theta(\cdot)$ is transitive, using Theorems 3.5 and 3.6 rather than arguing from the definition directly.

7. (**Applied**) You time an unknown algorithm and find that $T(n)/n^2$ falls steadily while $T(n)/(n \lg n)$ rises steadily across your whole range of n . What can you conclude, what can you not, and what would you measure next?

Correctness: Loop Invariants

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part I — Analysing an Algorithm

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** why testing cannot establish that an algorithm is correct, and **quantify** the input space it would have to cover.
- **State** a loop invariant for a given loop, and **name** the three obligations a proof must discharge.
- **Prove** insertion sort and binary search correct from their invariants, including the permutation clause.
- **Distinguish** partial correctness from total correctness, and **prove** termination by exhibiting a decreasing measure.
- **Diagnose** a loop whose invariant holds but which fails to terminate.

🔗 Before You Start

Chapter 1 for the vocabulary. Section 4.2 leans on mathematical induction; Appendix B has it if you want a refresher first.

📖 Key Terms in This Chapter

correctness • partial correctness • total correctness • loop invariant • initialisation • maintenance • termination • precondition • postcondition • permutation • loop variant • decreasing measure • well-founded

4.1 Why Testing Is Not Enough

A sorting routine takes any array of any length holding any values. That is an infinite set of inputs, and a test suite is a finite list. The gap between them is not a matter of diligence; it is a matter of cardinality.

🧮 Worked Example: Counting What Exhaustive Testing Would Cost

Restrict the problem absurdly: arrays of exactly n elements, each value drawn from 0 to 99. How many distinct sorted inputs must a binary search be tested against?

Step 1. Only the multiset of values matters, so the count is the number of multisets of size n from an alphabet of $m = 100$:

$$\binom{m+n-1}{n}.$$

Step 2. Evaluated by `code/ch04_invariant.cpp`, and each array must then be tried against all 100 possible keys:

n	sorted arrays	$\times$ keys
4	4.42×10^6	4.4×10^8
8	3.26×10^{11}	3.3×10^{13}
16	1.50×10^{19}	1.5×10^{21}
20	2.46×10^{22}	2.5×10^{24}

Step 3. At a billion tests a second, the $n = 20$ row takes 2.5×10^{15} seconds — about **80 million years**. And that is for arrays of twenty elements holding two-digit numbers.

Step 4 — the conclusion. Testing samples the input space; it cannot cover it. A proof covers it, in half a page, for every n and every alphabet at once. The two activities answer different questions, and this chapter is about the one testing cannot answer.

💡 What testing is still for

None of this says do not test. A proof establishes that the *algorithm* is correct; it says nothing about whether your *program* implements that algorithm, which is where most real defects live — an off-by-one in a loop bound, a wrong variable name, an integer overflow the model ignored.

Proof and testing catch disjoint classes of bug. The lab in this chapter contains a defect that *testing found in a third of its trials and a careless proof would have missed*, and Chapter 16's lab contains the reverse. Use both.

4.2 The Loop Invariant

📖 Loop Invariant

A **loop invariant** is an assertion about the program's state that is true immediately before the loop's first iteration and immediately before every subsequent one.

A proof by loop invariant has **three obligations**:

1. **Initialisation.** The invariant is true before the first iteration.
2. **Maintenance.** If it is true before an iteration, it is true before the next one.
3. **Termination.** The loop ends, and when it does the invariant — together with the reason the loop stopped — gives the result you wanted.

💡 It is induction, wearing different clothes

Invariant proof	Induction	On what
initialisation	base case	the loop has run 0 times
maintenance	inductive step	assume after k , show after $k + 1$
termination	—	no counterpart: induction proves a property for all k , and says nothing about the loop stopping

The missing row is the whole point of Section 4.5. Induction on the iteration count gives you “the invariant holds after every iteration that happens”. Whether any particular iteration happens is a different question, and nothing in the first two obligations answers it.

Partial and Total Correctness

An algorithm is **partially correct** if, *whenever it halts*, its output satisfies the postcondition. It is **totally correct** if it is partially correct *and* halts on every input satisfying the precondition.

Initialisation and maintenance establish partial correctness. Termination is what upgrades it, and Section 4.6 measures a program that has the first and not the second.

4.3 Insertion Sort, Proved

INSERTION-SORT(A, n)

```

1 for  $j \leftarrow 2$  to  $n$  do
2    $key \leftarrow A[j]$   $\triangleright$  the element to place
3    $i \leftarrow j - 1$ 
4   while  $i > 0$  and  $A[i] > key$  do
5      $A[i + 1] \leftarrow A[i]$   $\triangleright$  shift right
6      $i \leftarrow i - 1$ 
7   end while
8    $A[i + 1] \leftarrow key$ 
9 end for
```

Figure 4.1: INSERTION-SORT. The outer loop's invariant concerns the prefix $A[1..j-1]$; the inner loop's concerns where key has got to. Lines are numbered so the proof can cite them.

The algorithm is indexed from 1, as the pseudocode throughout this book is, and the C++ in the labs is indexed from 0. Appendix A explains the translation; the mathematics is cleaner from 1 and the code is cleaner from 0, and pretending otherwise helps nobody.

Theorem 4.1 — Insertion sort is correct

Given an array $A[1..n]$ of $n \geq 1$ numbers, INSERTION-SORT terminates with A holding a permutation of its original contents, in non-decreasing order.

Proof. Take as the invariant for the **for** loop:

At the start of each iteration with index j , the subarray $A[1..j-1]$ consists of the elements originally in $A[1..j-1]$, in non-decreasing order.

Initialisation. Before the first iteration $j = 2$, so the subarray is $A[1..1]$: one element, trivially sorted, and trivially the element originally there.

Maintenance. Assume the invariant before the iteration with index j . The inner **while** loop shifts $A[i]$ to $A[i+1]$ for decreasing i while $A[i] > key$, maintaining its own invariant: *the entries $A[i+2..j]$ are those originally in $A[i+1..j-1]$, all of them $> key$, and position $A[i+1]$ is free.* The loop exits when $i = 0$ or $A[i] \leq key$; in either case every entry left of $i+1$ is $\leq key$ and every entry right of it is $> key$, so writing key into the free slot $A[i+1]$ leaves $A[1..j]$ sorted. No element was overwritten without first being copied, and key was saved before the shifting began, so the subarray still holds exactly the original elements. Incrementing j makes this the invariant for the next iteration.

Termination. The loop variable j increases by one each iteration and the loop ends when $j = n + 1$. Substituting into the invariant: $A[1..n]$ consists of the elements originally in $A[1..n]$, in non-decreasing order — which is the postcondition. Termination itself is immediate because j

strictly increases towards the fixed bound $n + 1$, and the inner loop terminates because i strictly decreases towards the guard $i > 0$. ■

⚠ Do not drop the permutation clause

It is tempting to state the invariant as “ $A[1..j-1]$ is sorted” and save eleven words. Then consider this implementation:

overwrite every element of A with zero.

Its output is sorted. It satisfies the shortened invariant at every iteration and terminates. It is not a sorting algorithm.

“Sorted” is only half the postcondition; “a permutation of the input” is the other half, and a proof that omits it proves the wrong theorem. This is the single most common error in student invariant proofs.

4.4 Binary Search, Where the Invariant Is About Exclusion

Insertion sort’s invariant describes what has been *built*. The other common shape describes what has been *ruled out*.

≡ BINARY-SEARCH(A, n, x)

```

1  $lo \leftarrow 1$ ;  $hi \leftarrow n$ 
2 while  $lo \leq hi$  do
3    $mid \leftarrow \lfloor (lo + hi)/2 \rfloor$ 
4   if  $A[mid] = x$  then
5     return  $mid$ 
6   else if  $A[mid] < x$  then
7      $lo \leftarrow mid + 1$  ▷ left half cannot hold  $x$ 
8   else
9      $hi \leftarrow mid - 1$  ▷ right half cannot hold  $x$ 
10  end if
11 end while
12 return NOT-FOUND

```

Figure 4.2: BINARY-SEARCH on a sorted array. The $+1$ on line 7 and the -1 on line 9 are what make the search range strictly smaller every iteration
— Section 4.5 is about what happens when one of them is missing.

🔑 Theorem 4.2 — Binary search is correct

Let $A[1..n]$ be sorted in non-decreasing order. BINARY-SEARCH returns an index mid with $A[mid] = x$ if x occurs in A , and NOT-FOUND otherwise.

Proof. Invariant: *if x occurs anywhere in A , then it occurs in $A[lo..hi]$.*

Initialisation. $lo = 1$ and $hi = n$, so the range is all of A and the implication is trivial.

Maintenance. Suppose the invariant holds and the loop body runs. If $A[mid] = x$ the procedure returns a correct index and there is nothing to maintain. If $A[mid] < x$ then, because A is sorted, every index $k \leq mid$ has $A[k] \leq A[mid] < x$, so no such position holds x ; discarding them by setting $lo \leftarrow mid + 1$ therefore cannot discard an occurrence of x , and the invariant survives. The case $A[mid] > x$ is symmetric.

Termination. Each iteration either returns, or replaces $hi - lo$ by a strictly smaller value: line 7 raises lo past $mid \geq lo$, and line 9 lowers hi below $mid \leq hi$. Since $hi - lo$ is an integer that strictly decreases and the loop runs only while $hi - lo \geq 0$, the loop performs finitely many iterations. If it exits, then $lo > hi$, so the range $A[lo..hi]$ is empty; by the invariant x occurs in A only if it occurs in an empty range, so it does not occur at all, and NOT-FOUND is correct. ■

» Corollary 4.3 — Binary search is logarithmic

Binary search performs at most $\lfloor \lg n \rfloor + 1$ iterations, since $hi - lo + 1$ at least halves each time and the loop stops when it reaches 0.

4.5 Termination: The Obligation People Skip

The termination argument above did real work, and it is worth isolating what kind of work.

≡ Loop Variant

A **loop variant**, or decreasing measure, is a quantity μ computed from the program state such that

- μ takes values in a **well-founded** set — for our purposes, the non-negative integers;
- every iteration strictly decreases μ .

Then the loop terminates, because a strictly decreasing sequence of non-negative integers must be finite.

The variants used so far: $n + 1 - j$ for insertion sort's outer loop, i for its inner loop, and $hi - lo + 1$ for binary search.

≡ EUCLID-GCD(a, b)

Require: $a \geq 0, b \geq 0$, not both zero

```

1 while  $b \neq 0$  do
2    $r \leftarrow a \bmod b$ 
3    $a \leftarrow b$ 
4    $b \leftarrow r$ 
5 end while
6 return  $a$ 
```

Figure 4.3: EUCLID-GCD. Here the invariant is an equality — $\gcd(a, b)$ never changes — and the variant is simply b , whose decrease is what makes the loop finite.

♀ Theorem 4.4 — Euclid's algorithm is totally correct

EUCLID-GCD(a, b) terminates and returns $\gcd(a, b)$.

Proof. **Invariant:** $\gcd(a, b)$ equals the greatest common divisor of the original arguments, and not both of a, b are zero.

Initialisation. Immediate — a and b are the original arguments.

Maintenance. The body replaces (a, b) by $(b, a \bmod b)$. Write $a = qb + r$ with $r = a \bmod b$. Any d dividing both b and r divides $qb + r = a$; any d dividing both a and b divides $a - qb = r$. So the pairs (a, b) and (b, r) have the same set of common divisors, hence the same greatest one, and the invariant is preserved.

Termination. Take $\mu = b$. Since $b \neq 0$ in the body, $r = a \bmod b$ satisfies $0 \leq r < b$, so the new value of b is strictly less than the old one and never negative. A strictly decreasing sequence of non-negative integers is finite, so the loop ends — with $b = 0$. Then $\gcd(a, 0) = a$, and by the invariant a is the gcd of the original arguments, which is what line 6 returns. ■

⚠ Where the measure must live

“Strictly decreasing” is not enough on its own; the measure must decrease within a *well-founded* set. Over the non-negative reals it is not:

$$x \leftarrow x/2 \quad \text{from } x = 1$$

strictly decreases forever and never terminates. Over the non-negative integers no such sequence exists, which is the entire force of the argument.

Use integers, and if your measure is naturally a real number, find an integer that bounds it.

4.6 A Loop Whose Invariant Holds and Which Never Finishes

Here is the case the three obligations exist to catch. Change one character of BINARY-SEARCH: on line 7 write $lo \leftarrow mid$ instead of $lo \leftarrow mid + 1$.

💡 The invariant still holds — and that is the problem

Re-read the maintenance argument of Theorem 4.2. It establishes that positions $k \leq mid$ cannot hold x , and therefore that discarding them is safe. Setting $lo \leftarrow mid$ discards *fewer* positions than that — it keeps mid itself. Keeping a position cannot lose an occurrence of x , so:

Initialisation still holds. Maintenance still holds. The modified program is *partially correct*: if it answers, the answer is right.

Termination fails. When $hi = lo + 1$ we get $mid = \lfloor (2lo + 1)/2 \rfloor = lo$, so $lo \leftarrow mid$ leaves lo and hi unchanged and the next iteration is identical. The measure $hi - lo$ no longer strictly decreases, and the loop spins forever.

📊 Worked Example: Measured: 132 Million Iterations, Zero Wrong Answers

code/ch04_invariant.cpp runs the broken search on a 64-element sorted array, 10,000 trials with keys known to be present and 10,000 with keys known to be absent, and *audits the invariant on every single iteration*. A run that exceeds 10,000 iterations is recorded as non-terminating.

10,000 trials each	non-termination	wrong answer
keys always present	3,228	0
keys always absent	10,000	0
iterations audited	132,311,650	
invariant violations	0	

Read the second column first. In 20,000 trials the broken search returned a wrong answer **not once**. That is not luck; it is precisely what the invariant guarantees, and the audit confirms the invariant never broke across 132 million iterations.

Now read the first column. 13,228 of the 20,000 runs never finished. The program is partially correct and totally useless.

The distribution matters for how fast you find it. Keys drawn from inside the array hang about a third of the time; keys known to be absent hang every time. A test suite built only from present keys would still have caught this — in roughly three trials — which is worth saying plainly, because the tidier story would be that testing misses it. Testing found this bug easily.

What testing could not have told you is *why*, or that the defect is confined entirely to termination with partial correctness intact. And a proof that checked only initialisation and maintenance — which is what most students write — would have certified this program as correct.

! Common Pitfalls

- **Omitting the permutation clause** from a sorting invariant. The all-zeros “sort” satisfies the weakened version.
- **Stating an invariant that mentions no loop variable.** “A is an array of numbers” is true throughout and proves nothing. A useful invariant relates the work done so far to the loop index.
- **Checking the invariant at the wrong moment.** It is asserted *before* each iteration, not in the middle of the body, where it is routinely false — insertion sort’s prefix is genuinely unsorted while the shifting is in progress.
- **Treating termination as a formality.** Section 4.6 is a program that passes the other two obligations and runs forever.
- **A measure that decreases but not in a well-founded set.** $x \leftarrow x/2$ over the reals.
- **Proving the loop correct and forgetting the exit condition.** The invariant alone is not the postcondition; you need the invariant *and* the reason the loop stopped, which is why every termination paragraph above substitutes the final index back in.
- **Assuming a proved algorithm gives a correct program.** The proof is about the algorithm; overflow, off-by-one and typing errors live below it.

🔧 Try It Yourself: A Bug That Is Never Wrong

Reproduces the table in Section 4.6. Expect twenty-five minutes; part C audits 132 million iterations and takes about a minute.

```
// ch04_invariant.cpp -- why the THIRD obligation is not decoration.
// Build: cl /O2 /EHsc /std:c++17 ch04_invariant.cpp (see Appendix A)

// --- 1. The correct version, and the invariant it maintains -----
// Invariant: if x occurs in a, then it occurs in a[lo..hi].
// Initialisation: lo = 0, hi = n-1, so the range is all of a.
// Maintenance:   a is sorted, so discarding the half that cannot
//                contain x preserves the invariant.
// Termination:   hi - lo strictly decreases and is bounded below.
static int bsearch_ok(const std::vector<int>& a, int x) {
    int lo = 0, hi = (int)a.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == x) return mid;
        if (a[mid] < x) lo = mid + 1;    // the correct step
        else           hi = mid - 1;
    }
    return -1;
}

// --- 2. The same search with one character changed -----
// "lo = mid" still discards nothing that could hold x, so the
```

```

// invariant SURVIVES. But when hi == lo + 1 and a[mid] < x, mid
// computes to lo, so lo = mid leaves lo and hi UNCHANGED and the loop
// makes no progress. It is not wrong; it is non-terminating.
static int bsearch_bug(const std::vector<int>& a, int x,
                      bool* inv_ever_broken, long long* iters) {
    int lo = 0, hi = (int)a.size() - 1;
    for (int step = 0; step < CAP; ++step) {
        if (lo > hi) return -1;
        int mid = lo + (hi - lo) / 2;

        // Audit the invariant itself, every single iteration: if x is
        // present in the whole array, is it still inside [lo, hi]?
        bool present = std::binary_search(a.begin(), a.end(), x);
        if (present) {
            bool in_window = false;
            for (int i = lo; i <= hi; ++i)
                if (a[i] == x) { in_window = true; break; }
            if (!in_window) *inv_ever_broken = true;
        }
        ++*iters;

        if (a[mid] == x) return mid;
        if (a[mid] < x) lo = mid;           // <-- the one-character bug
        else             hi = mid - 1;
    }
    return -2;                             // stands in for "forever"
}

// --- 3. Ten thousand tests with keys that are present -----
// --- 4. The same ten thousand tests, keys absent -----
// The only change between parts 3 and 4 is the distribution the keys
// are drawn from, and it changes the hang rate from 32% to 100%.

// --- 5. And the invariant? It never broke once. -----
// inv_ever_broken is set by the audit in part 2 and is never set.
// Printing that zero beside the 13,228 hangs IS the experiment: the
// two numbers together say "partially correct, and useless".
printf("  iterations audited : %lld\n", iters);
printf("  invariant violated : %s\n", broken ? "YES" : "NO");
printf("  wrong answers, A+B : %d\n", wrong_present + wrong_absent);

// --- 6. Why proving beats testing, counted -----
// Exhaustive testing means every sorted array and every key. The
// number of sorted arrays of length n over an alphabet of size m is
// the multiset coefficient C(m+n-1, n).
for (int n : {4, 8, 16, 20}) {
    const int m = 100;
    double c = 1.0;
    for (int i = 1; i <= n; ++i) c *= (double)(m + n - i) / i;
    printf("    n=%2d, values 0..99 : %.3g sorted arrays"
           " x %d keys\n", n, c, m);
}

```

What to notice. The invariant violated line prints NO, and it is the most important line in the output: the audit is not failing to find a violation because it is badly written, it is failing because there is none to find.

Then try to break it. Change the bug from `lo = mid` to `lo = mid + 2`. Now predict, before running it, which of the three obligations fails — and which column of the table changes. This mutation terminates and gives wrong answers, the exact mirror image of the one in the book, and it is the case an invariant proof catches immediately.

? Checkpoint — Test Yourself

1. State a loop invariant for a loop that computes $\sum_{i=1}^n A[i]$ into a variable *total*, and give the measure that proves it terminates.
2. A classmate's invariant for selection sort reads: "after each iteration, $A[1..k]$ is sorted". Give an incorrect program that satisfies it, and repair the invariant.
3. Explain, in terms of the three obligations, how a program can be partially correct but not totally correct, and give the measure whose absence causes it.

✓ Chapter Summary

- Testing samples an infinite input space. For sorted arrays of 20 two-digit numbers, exhaustive testing needs 2.5×10^{24} cases — about 80 million years. A proof covers every n at once.
- A loop invariant proof has three obligations: initialisation, maintenance, termination. The first two are induction; the third has no counterpart in induction and is where the real content often lies.
- Initialisation plus maintenance give *partial* correctness — if it halts, it is right. Termination upgrades that to total correctness.
- Insertion sort's invariant must say the prefix holds *the elements originally there*, in order. Drop the permutation clause and the all-zeros "sort" satisfies it.
- Binary search's invariant is about exclusion: if x is in A at all, it is in $A[lo..hi]$.
- Termination is proved by a loop variant: an integer-valued measure that strictly decreases. Over the reals the argument fails — $x \leftarrow x/2$.
- Measured: a binary search with $lo \leftarrow mid$ gave **zero** wrong answers in 20,000 trials and **zero** invariant violations across 132,311,650 audited iterations — and failed to terminate in 13,228 of those trials. Two obligations out of three is not correctness.

✍ Review Questions

1. (MCQ) The three obligations of a loop invariant proof are: (a) initialisation, iteration, conclusion; (b) initialisation, maintenance, termination; (c) precondition, invariant, postcondition; (d) base case, inductive step, generalisation.
2. (MCQ) Initialisation and maintenance alone establish: (a) total correctness; (b) partial correctness; (c) termination; (d) a running-time bound.
3. (Short) Explain why a loop invariant proof is a proof by induction, and identify the one obligation that has no inductive counterpart.
4. (Short) Why must a sorting algorithm's invariant mention that the sorted prefix contains the original elements? Give the program that makes the point.
5. (Short) State the two conditions a loop variant must satisfy, and give an example of a measure satisfying the first but not the second.
6. (Applied) Prove that the following loop computes a^n for integer $n \geq 0$, discharging all three obligations: $r \leftarrow 1; k \leftarrow n; \text{while } k > 0 \text{ do } r \leftarrow r \cdot a; k \leftarrow k - 1$.
7. (Applied) The measured experiment found a defect that is never wrong and sometimes never finishes. Construct the mirror-image defect — one that always terminates and is sometimes wrong — in binary search, say which obligation it violates, and describe the test that would find it.

PART II

Recursion and Divide-and-Conquer

An algorithm that calls itself is easy to write and hard to cost. Part II is the machinery for costing one — three ways to solve a recurrence, a theorem that solves most of them at sight, and the two sorting algorithms that every later chapter uses as a reference point.



Figure P2.1: Reading path through Part II. Chapter 6 is the tool you will reach for most; Chapter 7 is the baseline that makes the rest worth having.

Recurrences I: Substitution and Recursion Trees

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part II — Recursion and Divide-and-Conquer

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Write** the recurrence that describes a recursive algorithm's running time, and **state** what its base case contributes.
- **Solve** a recurrence by substitution: guess a bound and **prove** it by induction, including the case where the guess must be strengthened.
- **Draw** a recursion tree, **sum** it level by level, and **use** the result as a guess for substitution.
- **Analyse** an unbalanced split, and **explain** why the deepest branch sets the depth while the total work stays $O(n \lg n)$.
- **Justify** ignoring floors, ceilings and boundary conditions.

🔗 Before You Start

Chapter 3 for O , Ω and Θ , and Chapter 4 for induction — a substitution proof is an induction proof, and the three obligations are the same ones. Appendix B has the summation identities.

📖 Key Terms in This Chapter

recurrence • base case • substitution method • inductive hypothesis • strengthening the hypothesis • recursion tree • level • internal node • leaf • balanced split • unbalanced split • boundary condition

5.1 What a Recurrence Is

A recursive algorithm's cost is defined in terms of itself, so its running time is too.

📖 Recurrence

A **recurrence** is an equation that defines a function in terms of its own value at smaller arguments, together with one or more **base cases**.

Merge sort divides its input in half, sorts each half recursively, and merges the results in linear time. So

$$T(n) = \begin{cases} \Theta(1) & n = 1, \\ T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + \Theta(n) & n > 1. \end{cases}$$

Written as algorithm designers actually write it, with the floors dropped and the constants absorbed: $T(n) = 2T(n/2) + n$.

⚠️ Three things that are quietly being ignored

1. **Floors and ceilings.** $n/2$ is not an integer when n is odd. Section 5.5 says why dropping them is safe.
2. **The base case.** $T(1) = \Theta(1)$ is assumed. For every recurrence in this book the base case contributes a constant, which the asymptotics absorb.

3. **The constant on n .** “ $+n$ ” means $+\Theta(n)$. We are solving for an order of growth, so a constant factor on the combining step cannot change the answer — though Chapter 8 measures what it does to the clock.

5.2 The Substitution Method

The substitution method has two steps and no magic in the second one:

1. **Guess** the form of the answer.
2. **Prove** it by induction, finding the constants that make the induction go through.

The guess is where the work is; Section 5.3 is a machine for producing guesses.

Theorem 5.1 — Merge sort’s recurrence

If $T(n) = 2T(n/2) + n$ with $T(1) = 1$, then $T(n) = O(n \lg n)$.

Proof. Guess $T(n) \leq cn \lg n$ for a constant $c > 0$ to be chosen, and all $n \geq n_0$.

Inductive step. Assume the bound holds for $n/2$, that is $T(n/2) \leq c(n/2) \lg(n/2)$. Then

$$\begin{aligned} T(n) &= 2T(n/2) + n \\ &\leq 2\left(c \frac{n}{2} \lg \frac{n}{2}\right) + n = cn \lg \frac{n}{2} + n \\ &= cn \lg n - cn \lg 2 + n = cn \lg n - cn + n. \end{aligned}$$

This is at most $cn \lg n$ provided $-cn + n \leq 0$, that is $c \geq 1$.

Base case. The induction needs some n at which the bound holds outright. At $n = 1$ it does not: $T(1) = 1$ but $c \cdot 1 \cdot \lg 1 = 0$. This is not a flaw in the result — asymptotic bounds need hold only for $n \geq n_0$, so take $n_0 = 2$. Then $T(2) = 2T(1) + 2 = 4$, and $c \cdot 2 \lg 2 = 2c \geq 4$ whenever $c \geq 2$.

Taking $c = 2$ and $n_0 = 2$ satisfies both conditions, so $T(n) \leq 2n \lg n$ for all $n \geq 2$, and $T(n) = O(n \lg n)$. ■

The base case is where students lose marks, not the algebra

Notice that the inductive step needed only $c \geq 1$, and the base case forced $c \geq 2$. Both constraints are real and the proof must satisfy both.

Notice also that $n = 1$ *could not* be made to work and did not have to be. Moving the base case up to $n = 2$ is legitimate precisely because $O(\cdot)$ quantifies over $n \geq n_0$ — the definition in Chapter 3 was built for exactly this.

Worked Example: A Guess That Fails, and the Repair

Claim to test: $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1$ is $O(n)$.

Step 1 — the obvious attempt. Guess $T(n) \leq cn$. Then

$$T(n) \leq c \left\lfloor \frac{n}{2} \right\rfloor + c \left\lceil \frac{n}{2} \right\rceil + 1 = cn + 1.$$

Step 2 — why that is a failure. $cn + 1$ is *not* $\leq cn$, for any c . The algebra came out “nearly” right, and nearly is no good: the induction does not close, so nothing has been proved. No choice of c repairs it, because the leftover $+1$ does not involve c at all.

Step 3 — strengthen the hypothesis. The trick is to prove something *stronger*, which gives the induction more to work with. Guess $T(n) \leq cn - d$ for constants $c, d > 0$:

$$\begin{aligned} T(n) &\leq \left(c \left\lfloor \frac{n}{2} \right\rfloor - d\right) + \left(c \left\lceil \frac{n}{2} \right\rceil - d\right) + 1 \\ &= cn - 2d + 1 \leq cn - d \quad \text{whenever } d \geq 1. \end{aligned}$$

Step 4 — the lesson. Subtracting a lower-order term made the proof work. It looks backwards — proving more should be harder — but the inductive hypothesis is *used* as well as proved, and a stronger hypothesis is a stronger tool. This is the single most useful trick in the method, and the one worth remembering.

5.3 Recursion Trees: a Way to Guess

A recursion tree makes the sum visible. Each node is one call, labelled with the work that call does *excluding* its recursive calls; the total is the sum over all nodes, which is easiest taken level by level.

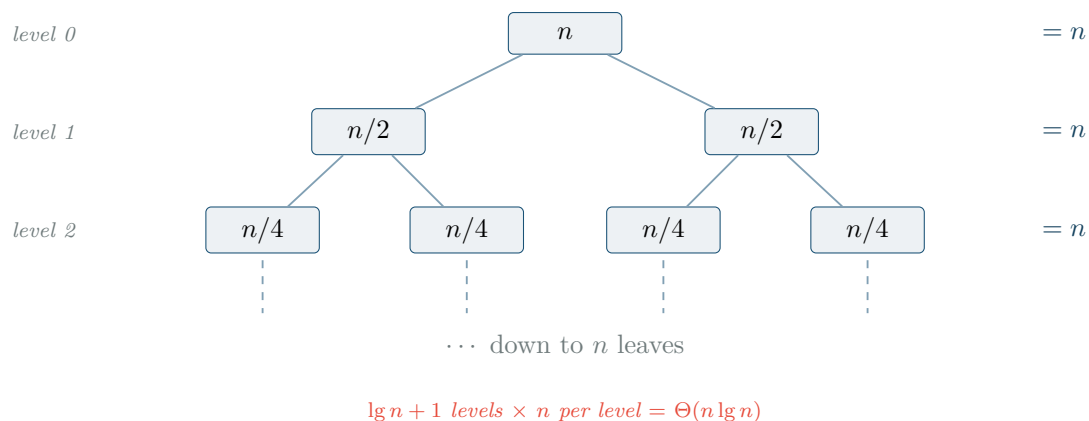


Figure 5.1: The recursion tree for $T(n) = 2T(n/2) + n$. Each level holds twice as many nodes, each doing half as much work, so every level totals n . The depth is $\lg n$ because the argument halves, and the product is the bound.

💡 A tree is a guess, not a proof

The tree gives $\Theta(n \lg n)$ in two lines. It is not a proof: the levels were assumed uniform, the leaves were counted loosely, and floors were ignored throughout.

That is the correct division of labour. **Draw the tree to get the guess; run substitution to prove it.** Theorem 5.1 is the proof for exactly the guess this tree produced.

📊 Worked Example: The Tree, Counted

`code/ch05_rectree.cpp` walks the recursion and records how many calls each level holds and how much work they do. At $n = 1024$:

depth	calls	work	work/ n
0	1	1024	1.000
1	2	1024	1.000
2	4	1024	1.000
...	levels 3 to 9, all	1.000	...
10	1024	1024	1.000
total		11,264	11.000

Every level does exactly n work — the column is 1.000 all the way down, not approximately. There are $11 = \lg 1024 + 1$ levels, and the total is exactly $11n$.

And the quotient against the bound: $11,264 / (1024 \lg 1024) = 1.100$. The tree's reasoning is not an approximation here; it is arithmetic, and the program is checking the arithmetic.

5.4 An Unbalanced Split

Real algorithms do not always divide evenly. Quicksort (Chapter 9) splits wherever the pivot lands. What does a lopsided split cost?

 **Worked Example:** $T(n) = T(n/3) + T(2n/3) + n$

Step 1 — the shallowest leaf. Following the $n/3$ branch, the argument is divided by 3 each time, reaching 1 after $\log_3 n$ steps.

Step 2 — the deepest leaf. Following the $2n/3$ branch, the argument is multiplied by $2/3$ each time, reaching 1 after $\log_{3/2} n$ steps.

Step 3 — what that means for the sum. While *every* branch is still alive — down to depth $\log_3 n$ — each level totals exactly n , because the two children's arguments sum to the parent's. Below that, branches start bottoming out and the levels total less than n . So

$$\underbrace{n \log_3 n}_{\text{full levels}} \leq T(n) \leq \underbrace{n \log_{3/2} n}_{\text{if every level were full}},$$

and since $\log_3 n$ and $\log_{3/2} n$ are both $\Theta(\lg n)$ — they differ from $\lg n$ only by the constants $1/\lg 3$ and $1/\lg(3/2)$ — we get $T(n) = \Theta(n \lg n)$.

Step 4 — the moral. *Any* split into constant fractions gives $\Theta(n \lg n)$. A 1-to-99 split is still $\Theta(n \lg n)$; only the constant changes. What breaks the bound is a split that is not a constant fraction — such as 1 and $n - 1$, which is Chapter 9's worst case.

 **Worked Example:** The Lopsided Tree, Counted

The same program, same $n = 1024$, lopsided split:

depth	calls	work	work/ n
0–6	1 to 64	1024 each	1.000
7	126	1023	0.999
8	210	1002	0.979
9	326	955	0.933
10	342	800	0.781
11	352	634	0.619
12	264	414	0.404
13–17	300	407 total	decaying to 0.002
total		12,403	12.112

Levels 0 to 6 are exactly full, at 1.000 each. And $\log_3 1024 = 6.29$ — so the prediction in Step 3 above, that the levels are full down to depth $\log_3 n$, is confirmed to the level.

Below that the levels decay, exactly as the argument said they would, and the tree runs on to depth 17 rather than 10.

The totals: 12,403 against the balanced tree’s 11,264 — only 10% more work, for a split twice as lopsided. Against $n \lg n$ the quotient is 1.211 versus 1.100. This is what “the constant changes and the order does not” looks like when you measure it.

Worked Example: Depth, Against the Prediction

Predicted depth ratio: $\log_{3/2} n / \lg n = 1 / \lg(3/2) = 1.71$.

n	balanced	$\lg n$	lopsided	$\log_{3/2} n$	ratio
1,024	11	10.0	18	17.1	1.64
4,096	13	12.0	22	20.5	1.69
16,384	15	14.0	25	23.9	1.67
65,536	17	16.0	29	27.4	1.71
262,144	19	18.0	32	30.8	1.68

Measured mean 1.68 against a predicted 1.71. The lopsided recursion is about 70% deeper and still $\Theta(n \lg n)$ — which is the single most useful fact about divide-and-conquer, because it means the split does not have to be perfect.

5.5 Floors, Ceilings and Boundaries

Why dropping them is safe

Every recurrence above was written $T(n/2)$ when the honest version is $T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil)$.

The justification is a sandwich. For the recurrences in this book,

$$T(\lfloor n/2 \rfloor) \leq T(n/2) \leq T(\lceil n/2 \rceil),$$

since T is non-decreasing, and the outer two differ by at most one step of the recursion — a $O(n)$ term already present. Carrying the floors through a substitution proof changes the constants and never the order of growth, which is why textbooks state the clean form and prove the messy one once.

The same goes for the base case. $T(1) = 1$ against $T(1) = 1000$ changes c , not the bound: the base-case work is $O(1)$ per leaf and there are $O(n)$ leaves, contributing $O(n)$, which $n \lg n$ absorbs.

Where this stops being true is a recurrence whose subproblem sizes do not shrink by a constant fraction — $T(n) = T(n-1) + n$, say, whose tree is a path of length n and whose solution is $\Theta(n^2)$. There the “ -1 ” is the whole content and may not be tidied away.

! Common Pitfalls

- **Guessing from the tree and calling it proved.** The tree produces a guess. Substitution proves it. Both steps are required.
- **An inductive step that comes out “nearly” right.** $cn + 1 \leq cn$ is false, and a proof that ends there has proved nothing. Strengthen the hypothesis.
- **Forgetting the base case.** The inductive step above needed $c \geq 1$ and the base case needed $c \geq 2$; a proof that checks only the step is half a proof.
- **Insisting the base case be $n = 1$.** It need not be. $O()$ quantifies over $n \geq n_0$ and you may choose n_0 .
- **Assuming every level of a tree is full.** It is true for a balanced split and false for a lopsided one from depth $\log_{b_{\min}} n$ onward — as the measured table shows, to the level.
- **Dropping a -1 as though it were a $/2$.** $T(n) = T(n-1) + n$ is quadratic, not logarithmic-depth anything.
- **Proving $O()$ and claiming $\Theta()$.** Substitution as used here gives an upper bound. The lower bound is a separate proof, by Theorem 3.5.

🔧 Try It Yourself: Counting a Recursion Tree

Reproduces all three tables above. Expect twenty minutes.

```
// ch05_rectree.cpp -- a recursion tree, counted rather than drawn.
// Build: cl /O2 /EHsc /std:c++17 ch05_rectree.cpp (see Appendix A)

// --- 1. The balanced recursion,  $T(n) = 2T(n/2) + n$  -----
// The body does  $n$  units of work and then halves. Nothing is sorted
// here: the point is the SHAPE, so work is counted, not performed.
static void balanced(long long n, int d) {
    if (n <= 1) { note(d, 1); return; }
    note(d, n);
    balanced(n / 2, d + 1);
    balanced(n - n / 2, d + 1);
}

// --- 2. The lopsided recursion,  $T(n) = T(n/3) + T(2n/3) + n$  -----
// The  $\max(1, \dots)$  is not decoration. Written as  $\text{lopsided}(n/3)$  and
//  $\text{lopsided}(n - n/3)$ , the case  $n = 2$  gives  $2/3 = 0$  and  $2 - 0 = 2$ , so
// the larger branch recurses on 2 for ever: the measure  $n$  never
// decreases and the recursion does not terminate. That is Chapter 4's
// third obligation, failing in a program four lines long.
static void lopsided(long long n, int d) {
    if (n <= 1) { note(d, 1); return; }
    note(d, n);
    long long small = std::max(1LL, n / 3);
    lopsided(small, d + 1);
    lopsided(n - small, d + 1);
}

// --- 3. Each level of a balanced tree does the same work -----
// The "work/ $n$ " column is the one to read: if the recursion-tree
```

```

// argument is sound, it is 1.000 all the way down to the leaves.

// --- 4. A lopsided tree, where the levels stop being full -----
// Now "work/n" holds at 1.000 only while every branch is still alive
// -- down to depth log_3(n) -- and then decays.

// --- 5. Does the depth grow the way the algebra says? -----
// Balanced depth should be lg(n); lopsided depth log_{3/2}(n), about
// 1.71 times as deep.
for (long long n = 1024; n <= 262144; n *= 4) {
    work.assign(1, 0); calls.assign(1, 0);
    balanced(n, 0);
    size_t db = work.size();
    work.assign(1, 0); calls.assign(1, 0);
    lopsided(n, 0);
    size_t dl = work.size();
    double lg = std::log2((double)n);
    printf("%10lld %8zu %8.1f %8zu %8.1f %8.2f\n",
           n, db, lg, dl, lg / std::log2(1.5),
           (double)dl / db);
}

```

What to notice. The first version of the `lopsided` function written for this book had no `max(1, ...)` and did not terminate. It is worth pausing on that: the recurrence $T(n) = T(n/3) + T(2n/3) + n$ is perfectly well defined, and the obvious transcription of it into code is not. Integer division did it.

Then try to break it. Change the split to $n/10$ and $9n/10$ and predict, before running, (a) how many levels are full, (b) the total depth, and (c) whether the total work goes up or down. The answers are $\log_{10} n$, $\log_{10/9} n$, and up — but only by a constant.

Checkpoint — Test Yourself

1. Solve $T(n) = T(n-1) + n$, $T(1) = 1$, by drawing its recursion tree. Why is the answer not $\Theta(n \lg n)$?
2. Prove by substitution that $T(n) = 2T(n/2) + n^2$ is $O(n^2)$, stating the constraint on c that the inductive step produces.
3. Why does a 1-to-99 split still give $\Theta(n \lg n)$, while a 1-to- $(n-1)$ split does not?

Chapter Summary

- A recurrence defines a running time in terms of itself, plus a base case. Floors, ceilings and base constants are dropped because they change constants, not orders of growth.
- Substitution = guess + induction. The inductive step and the base case each constrain the constant, and both constraints must be met.
- When an inductive step comes out “nearly” right — $cn + 1 \leq cn$ — strengthen the hypothesis by subtracting a lower-order term.
- A recursion tree is a device for *guessing*: sum level by level, multiply work-per-level by number of levels.
- Measured: for $T(n) = 2T(n/2) + n$ at $n = 1024$, every level does exactly 1024 units, there are exactly 11 levels, and the total is exactly $11n$.
- Any split into constant fractions is $\Theta(n \lg n)$. Measured, a one-third/two-thirds split was 70% deeper (ratio 1.68 against a predicted 1.71) and did only 10% more work.
- Measured, the lopsided tree’s levels were exactly full down to depth 6, and $\log_3 1024 = 6.29$.

 Review Questions

1. **(MCQ)** The recursion tree for $T(n) = 2T(n/2) + n$ has: (a) n levels each doing $\lg n$ work; (b) $\lg n$ levels each doing n work; (c) $\lg n$ levels each doing $\lg n$ work; (d) n levels each doing n work.
2. **(MCQ)** In a substitution proof, strengthening the hypothesis means: (a) choosing a larger c ; (b) proving a stronger statement, such as $cn - d$ instead of cn ; (c) raising n_0 ; (d) using strong induction.
3. **(Short)** Why is a recursion tree not a proof, and what is it for?
4. **(Short)** Explain why the base case of a substitution proof need not be $n = 1$.
5. **(Short)** State the two depths of the tree for $T(n) = T(n/3) + T(2n/3) + n$ and say which one determines the total work.
6. **(Applied)** Solve $T(n) = 3T(n/4) + n$ by recursion tree, then prove your guess by substitution.
7. **(Applied)** The measured table shows the lopsided tree's levels full to depth 6 at $n = 1024$. Predict the corresponding depth for $n = 10^6$ with the same split, and say what you would expect the work/ n column to be at depth 10.

Recurrences II: The Master Theorem

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part II — Recursion and Divide-and-Conquer

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **State** the master theorem and **identify** a , b and $f(n)$ for a given recurrence.
- **Decide** which of the three cases applies by comparing $f(n)$ with $n^{\log_b a}$, and **apply** it to obtain a Θ bound.
- **Check** the regularity condition that case 3 requires.
- **Recognise** a recurrence that falls into none of the three cases, and **say** what to do instead.
- **Explain** why case 2 produces a logarithmic factor that neither of the others does.

🔗 Before You Start

Chapter 5 — recursion trees especially, since the three cases are three shapes of tree. From Appendix B you need $a^{\log_b n} = n^{\log_b a}$ and the geometric series.

📖 Key Terms in This Chapter

master theorem • a , b , $f(n)$ • watershed function • $n^{\log_b a}$ • polynomially larger • polynomially smaller • regularity condition • the gap • leaf-dominated • root-dominated

6.1 One Theorem for a Whole Family

Most divide-and-conquer algorithms split a problem of size n into a subproblems of size n/b and spend $f(n)$ dividing and combining. That is a family of recurrences, and the master theorem solves the family at once.

🏆 Theorem 6.1 — Master theorem

Let $a \geq 1$ and $b > 1$ be constants, $f(n)$ a non-negative function, and

$$T(n) = aT(n/b) + f(n).$$

Then, writing $w(n) = n^{\log_b a}$ for the **watershed function**:

Case 1. If $f(n) = O(n^{\log_b a - \varepsilon})$ for some constant $\varepsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.

Case 2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \lg n)$.

Case 3. If $f(n) = \Omega(n^{\log_b a + \varepsilon})$ for some constant $\varepsilon > 0$, and $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n (the **regularity condition**), then $T(n) = \Theta(f(n))$.

💡 What the theorem is actually comparing, and why

Think of the recursion tree.

- The **root** does $f(n)$ work.

- The **leaves** number $a^{\log_b n} = n^{\log_b a}$ — that identity is the whole reason $n^{\log_b a}$ appears — and each does $\Theta(1)$, so the leaves do $\Theta(n^{\log_b a})$ in total.

So the theorem compares **the work at the top against the work at the bottom**, and the three cases are the three possible answers:

1. the leaves win, and the total is the leaves: $\Theta(n^{\log_b a})$;
2. it is a tie, every level does the same work, and the total is one level's work times the $\lg n$ levels;
3. the root wins, the sum is dominated by its first term, and the total is $\Theta(f(n))$.

Read that way the theorem is not three rules to memorise but one question — *which end of the tree is heavier?* — with three answers.

Worked Example: Applying It Three Times

(a) Merge sort: $T(n) = 2T(n/2) + n$. $a = 2$, $b = 2$, $f(n) = n$. Watershed: $n^{\log_2 2} = n^1 = n$. Since $f(n) = n = \Theta(n)$, this is **case 2**:

$$T(n) = \Theta(n \lg n).$$

(b) Binary search: $T(n) = T(n/2) + 1$. $a = 1$, $b = 2$, $f(n) = 1$. Watershed: $n^{\log_2 1} = n^0 = 1$. Since $f(n) = 1 = \Theta(1)$, this is **case 2** again:

$$T(n) = \Theta(n^0 \lg n) = \Theta(\lg n).$$

Note that case 2 with $a = 1$ is where the familiar $\lg n$ of binary search comes from — the same case as merge sort, not a different rule.

(c) A bad matrix multiply: $T(n) = 8T(n/2) + n^2$. $a = 8$, $b = 2$, $f(n) = n^2$. Watershed: $n^{\log_2 8} = n^3$. Now $f(n) = n^2 = O(n^{3-\varepsilon})$ with $\varepsilon = 1$, so this is **case 1**:

$$T(n) = \Theta(n^3).$$

The combining work is irrelevant: there are so many leaves that nothing at the top matters. Strassen's algorithm improves this to $T(n) = 7T(n/2) + n^2$, giving $\Theta(n^{\log_2 7}) = \Theta(n^{2.81})$ — the whole gain comes from turning an 8 into a 7.

Worked Example: Case 3, With Its Condition Checked

$$T(n) = 2T(n/2) + n^2.$$

Step 1 — identify. $a = 2$, $b = 2$, $f(n) = n^2$. Watershed $n^{\log_2 2} = n$.

Step 2 — compare. $f(n) = n^2 = \Omega(n^{1+\varepsilon})$ with $\varepsilon = 1$. So case 3 is a candidate.

Step 3 — check regularity, which case 3 requires and the other two do not:

$$af(n/b) = 2\left(\frac{n}{2}\right)^2 = \frac{n^2}{2} = \frac{1}{2}f(n) \leq cf(n) \quad \text{with } c = \frac{1}{2} < 1.$$

It holds, for every n , with room to spare.

Step 4 — conclude. $T(n) = \Theta(f(n)) = \Theta(n^2)$.

Step 5 — read what that says. The recursion is essentially free. The top call alone does n^2 work, the two half-sized calls do $n^2/2$ between them, their children $n^2/4$, and the whole sum is at most $2n^2$. A geometric series with ratio $1/2$ — which is precisely what the regularity condition guarantees exists.

6.2 The Theorem, Verified Exactly

The master theorem's claim is about a sum, so it can be checked by computing the sum. `code/ch06_master.cpp` evaluates each recurrence directly and divides by the predicted bound; if the theorem is right, the quotient settles on a constant.

💡 Why this is counted and not timed

Timing these three recurrences gave ratios wandering by 50% from one doubling to the next — cases 1 and 3 run for seconds, and over seconds a laptop's clock speed is not constant.

Counting the work the recurrence defines has none of that. It is *exact*, deterministic, and identical on every machine, so you can check these tables against your own run line by line. Where the thing being tested is a piece of mathematics rather than a piece of hardware, count it.

📊 Worked Example: Case 1: the Leaves Win

$T(n) = 8T(n/2) + n^2$, predicted $\Theta(n^3)$.

n	work	n^3	work/ n^3
16	7,936	4,096	1.9375
64	520,192	262,144	1.9844
256	33,488,896	16,777,216	1.9961
1,024	2,146,435,072	1,073,741,824	1.9990

The quotient climbs to 2.0000 and stops. $T(n) \rightarrow 2n^3$ exactly, so $T(n) = \Theta(n^3)$ with the constant visible: it is 2.

📊 Worked Example: Case 2: the Tie, and Where the Logarithm Comes From

$T(n) = 2T(n/2) + n$, predicted $\Theta(n \lg n)$.

n	work	$n \lg n$	quotient	$1 + 1/\lg n$
1,024	11,264	10,240	1.1000	1.1000
32,768	524,288	491,520	1.0667	1.0667
262,144	4,980,736	4,718,592	1.0556	1.0556
1,048,576	22,020,096	20,971,520	1.0500	1.0500

The quotient is not constant — and that is correct. It falls steadily from 1.1000 to 1.0500, and the last column says why: it is exactly $1 + 1/\lg n$, to four decimal places, at every size.

This is worth dwelling on, because it is the commonest misunderstanding of Θ . $T(n) = \Theta(g(n))$ does *not* say T/g is constant; it says T/g is eventually trapped between two positive constants. Here it is trapped in $[1, 1.1]$ and converging to 1, which satisfies the definition completely. Chapter 3's measured $T/n^{2.0}$ column wandered by 8% for the same reason, and that was noise; this is not noise at all, it is the exact solution $T(n) = n \lg n + n$.

Worked Example: Case 3: the Root Wins

$T(n) = 2T(n/2) + n^2$, predicted $\Theta(n^2)$.

n	work	work/ n^2
1,024	2,096,128	1.9990
32,768	2,147,450,880	2.0000
262,144	137,438,691,328	2.0000
1,048,576	2,199,022,206,976	2.0000

2.0000 to four places, exactly as the geometric series in Step 5 of the previous worked example predicted: $n^2(1 + \frac{1}{2} + \frac{1}{4} + \dots) \rightarrow 2n^2$.

And the recurrence executed rather than counted — case 2, timed, median of seven runs — gives ratios 2.14, 2.14, 2.20, 2.17, 2.06, 2.12 against the predicted $2 + 2/\lg n \approx 2.1$. The counting and the clock agree.

6.3 The Gap: When the Theorem Does Not Apply

⚠ Cases 1 and 3 need a *polynomial* difference

Case 1 requires $f(n) = O(n^{\log_b a - \varepsilon})$ and case 3 requires $f(n) = \Omega(n^{\log_b a + \varepsilon})$, with $\varepsilon > 0$ *constant*. A function can be smaller than the watershed without being *polynomially* smaller, and then no case applies.

Worked Example: A Recurrence in the Gap

$T(n) = 2T(n/2) + n \lg n$.

Step 1. $a = 2$, $b = 2$, watershed $n^{\log_2 2} = n$.

Step 2 — which case? $f(n) = n \lg n$ is larger than n , so case 1 is out. Is it case 3? That needs $n \lg n = \Omega(n^{1+\varepsilon})$ for some constant $\varepsilon > 0$ — that is, $\lg n = \Omega(n^\varepsilon)$. But Chapter 3 proved the reverse: $\lg n = o(n^\varepsilon)$ for *every* $\varepsilon > 0$. So case 3 fails. And $f(n) \neq \Theta(n)$, so case 2 fails too.

Step 3 — the verdict. f is larger than the watershed, but only by a logarithmic factor, and a logarithm is not a polynomial. **The master theorem does not apply.**

Step 4 — what to do instead. Go back to Chapter 5. The recursion tree has $\lg n$ levels; level i holds 2^i nodes of size $n/2^i$, each doing $(n/2^i) \lg(n/2^i)$ work, so the level totals $n \lg(n/2^i) = n(\lg n - i)$. Summing:

$$T(n) = \sum_{i=0}^{\lg n - 1} n(\lg n - i) = n \sum_{j=1}^{\lg n} j = n \cdot \frac{\lg n (\lg n + 1)}{2} = \Theta(n \lg^2 n).$$

Step 5 — the moral. The master theorem is a shortcut, not a foundation. When it declines to answer, the recursion tree still works, and it is the tree that told us where the extra $\lg n$ came from.

💡 A decision procedure

#	Do this	Then
1	Read off $a, b, f(n)$	a = number of subproblems, b = factor the size shrinks by. Not the same number in general.
2	Compute $w(n) = n^{\log_b a}$	This is the leaf count.
3	Compare f with w	Smaller by a polynomial factor $\Rightarrow$ case 1. Equal up to constants $\Rightarrow$ case 2. Larger by a polynomial factor $\Rightarrow$ case 3.
4	If case 3, check regularity	$af(n/b) \leq cf(n)$, $c < 1$. It holds for every polynomial f ; it can fail for strange ones.
5	If nothing fits	You are in the gap. Use a recursion tree, then substitution.

! Common Pitfalls

- **Comparing $f(n)$ with n instead of with $n^{\log_b a}$.** The watershed is almost never n ; it is n only when $a = b$.
- **Treating a logarithmic difference as polynomial.** $n \lg n$ versus n is *not* a case-3 situation, and Chapter 3's $\lg n = o(n^\epsilon)$ is the reason.
- **Skipping the regularity condition.** It is part of case 3, not a footnote. It holds for every polynomial f , which is why it is easy to forget and occasionally fatal.
- **Using it on a non-divide-and-conquer recurrence.** $T(n) = T(n-1) + n$ has no b . The theorem says nothing.
- **Using it when the subproblems have different sizes.** $T(n) = T(n/3) + T(2n/3) + n$ is not of the form $aT(n/b) + f(n)$; Chapter 5 handled it by tree.
- **Reading case 2's Θ as “the quotient is constant”.** The measured quotient fell from 1.1000 to 1.0500 and the bound is still exactly right.
- **Forgetting a and b must be constants.** A recurrence whose branching factor depends on n is outside the theorem entirely.

🔧 Try It Yourself: The Three Cases, Counted and Timed

Reproduces all four tables. Expect twenty-five minutes.

```
// ch06_master.cpp -- the master theorem's three cases, verified.
// Build: cl /O2 /EHsc /std:c++17 ch06_master.cpp (see Appendix A)

// --- 1. Count the work each recurrence prescribes -----
// No timing, no hardware, no noise: just the sum the recurrence
// defines. Exact, deterministic, and the same on every machine.
static long long c1(long long n) { // 8T(n/2) + n^2
    if (n <= 1) return 1;
    return n * n + 8 * c1(n / 2);
}
static long long c2(long long n) { // 2T(n/2) + n
    if (n <= 1) return 1;
    return n + c2(n / 2) + c2(n - n / 2);
}
static long long c3(long long n) { // 2T(n/2) + n^2
    if (n <= 1) return 1;
    return n * n + c3(n / 2) + c3(n - n / 2);
}
```

```
// --- 2. Work that cannot be optimised away -----
// For the timing part only. Each unit is a cheap arithmetic step
// folded into an accumulator that escapes to a volatile, so the loop
// must actually be performed.
static inline unsigned long long burn(long long units) {
    unsigned long long h = 1469598103934665603ULL;
    for (long long i = 0; i < units; ++i)
        h = (h ^ (unsigned long long)i) * 1099511628211ULL;
    return h;
}

// --- 3. Case 1: the leaves dominate -----
//  $n^{\lg 8} = n^3$  beats  $n^2$ , so  $T = \Theta(n^3)$ , work/ $n^3$  flat.
counted("A. Case 1:  $T(n) = 8T(n/2) + n^2 \rightarrow \Theta(n^3)$ ", c1, "n^3",
    [(double n) { return n * n * n; }, 16, 1024];

// --- 4. Case 2: the tie, where the logarithm comes from -----
//  $n^{\lg 2} = n$  equals  $f(n) = n$ : neither dominates, every level
// does the same work, and the number of levels is  $\lg n$ .
counted("B. Case 2:  $T(n) = 2T(n/2) + n \rightarrow \Theta(n \lg n)$ ", c2,
    "n lg n",
    [(double n) { return n * std::log2(n); }, 1024, 1048576];

// --- 5. Case 3: the root dominates -----
//  $n^{\lg 2} = n$  loses to  $n^2$ , so  $T = \Theta(n^2)$ . The recursion is
// nearly free and the top call is the cost.
counted("C. Case 3:  $T(n) = 2T(n/2) + n^2 \rightarrow \Theta(n^2)$ ", c3, "n^2",
    [(double n) { return n * n; }, 1024, 1048576];

// --- 6. The regularity condition case 3 needs -----
//  $a f(n/b) \leq c f(n)$  for some  $c < 1$ . For  $f(n) = n^2$ ,  $a=2$ ,  $b=2$ :
//  $2(n/2)^2 = n^2/2$ , so  $c = 1/2$ . Print the check, do not assert it.

// --- 7. And the same recurrence, timed -----
// Case 2 only, where timing behaves. Predicted ratio  $2 + 2/\lg n$ .
```

What to notice. In part B, compare the quotient column against $1 + 1/\lg n$ computed by hand. They agree to four decimal places, which means you are not looking at an approximation — you are looking at the exact solution of the recurrence, and the master theorem's $\Theta(n \lg n)$ is a true but deliberately lossy summary of it.

Then try to break it. Add a fourth recurrence, $T(n) = 2T(n/2) + n \lg n$, and divide its work by $n \lg^2 n$. The master theorem cannot touch it, and the quotient should still settle — which is the point of Section 6.3.

Checkpoint — Test Yourself

1. Apply the master theorem to $T(n) = 4T(n/2) + n$. State a , b , $f(n)$, the watershed, the case and the bound.
2. $T(n) = 2T(n/2) + n/\lg n$. Which case applies? Justify your answer.
3. Explain, in terms of the recursion tree, why case 2 is the only one of the three that produces a $\lg n$ factor.

Chapter Summary

- For $T(n) = aT(n/b) + f(n)$, the master theorem compares $f(n)$ with the watershed $n^{\log_b a}$ — the work at the root against the work at the leaves.

- Case 1 (leaves win): $\Theta(n^{\log_b a})$. Case 2 (tie): $\Theta(n^{\log_b a} \lg n)$. Case 3 (root wins, plus regularity): $\Theta(f(n))$.
- $n^{\log_b a}$ appears because the tree has $a^{\log_b n} = n^{\log_b a}$ leaves.
- Cases 1 and 3 need a *polynomial* gap. $n \lg n$ against n is not one, so $T(n) = 2T(n/2) + n \lg n$ falls in the gap — its true answer, $\Theta(n \lg^2 n)$, comes from a recursion tree.
- Measured exactly: case 1's work/ n^3 and case 3's work/ n^2 both converge to 2.0000; case 2's work/ $(n \lg n)$ falls from 1.1000 to 1.0500, matching $1 + 1/\lg n$ to four decimal places.
- That falling quotient is not a failure of the bound. Θ requires the ratio to be trapped between constants, not to be constant.
- The theorem is a shortcut. When it declines, recursion trees and substitution still work.

Review Questions

1. **(MCQ)** For $T(n) = 9T(n/3) + n$, the watershed function is: (a) n ; (b) n^2 ; (c) n^3 ; (d) $n \lg n$.
2. **(MCQ)** The regularity condition is required by: (a) case 1 only; (b) case 2 only; (c) case 3 only; (d) all three.
3. **(Short)** Explain why $n^{\log_b a}$ is the right thing to compare $f(n)$ against, in terms of the recursion tree.
4. **(Short)** Give a recurrence the master theorem cannot solve, and say which condition fails.
5. **(Short)** Binary search and merge sort fall into the same case of the theorem. State the case and explain how one gives $\Theta(\lg n)$ and the other $\Theta(n \lg n)$.
6. **(Applied)** Strassen's algorithm satisfies $T(n) = 7T(n/2) + \Theta(n^2)$. Solve it, compare with the naive $8T(n/2) + \Theta(n^2)$, and say at what n a factor-of-two constant penalty on Strassen would be repaid.
7. **(Applied)** The measured case-2 quotient was $1 + 1/\lg n$ exactly. Derive that from the exact solution $T(n) = n \lg n + n$, and say what the quotient would be for $T(n) = n \lg n + 3n$.

Brute Force, and When It Is the Right Answer

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part II — Recursion and Divide-and-Conquer

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Define** the brute-force strategy and **apply** it to a stated problem (*CLO-6*).
- **Distinguish** removing redundant work within a search from changing the search itself, and **say** which one changes the order of growth.
- **Derive** the running time of the three maximum-subarray algorithms and **verify** the predicted ratios by measurement.
- **State** the conditions under which brute force is the correct engineering choice.
- **Use** a brute-force implementation as a correctness oracle for a cleverer one.

🔗 Before You Start

Chapter 3 for the notation, Chapter 4 for the invariant in Section 7.3. No recursion is needed; this chapter is the baseline the rest of Part II improves on.

📖 Key Terms in This Chapter

brute force • exhaustive search • candidate solution • maximum subarray • redundant work • incremental computation • Kadane’s algorithm • oracle • baseline

7.1 The Strategy With No Ideas In It

📖 Brute Force

Brute force — also **exhaustive search** — solves a problem by generating every candidate solution and testing each one. It needs no insight into the problem’s structure beyond the ability to recognise an answer.

It is the first algorithm you should write and, often, the last one you should ship.

💡 Why a chapter on the obvious

The published HEC outline lists brute force among the design techniques, beside divide-and-conquer, greedy and dynamic programming. That is the right company for it, for three reasons.

1. **It is the baseline.** “Quicksort is fast” is meaningless without something to be fast relative to. Every later chapter is an improvement on an exhaustive search, and you cannot measure an improvement against nothing.
2. **It is the oracle.** A brute-force implementation is short enough to be obviously correct, so it is what you test the clever algorithm *against*. Section 7.4 does exactly this.
3. **It is sometimes the answer.** For small or bounded n , for code that runs once, or for a correctness-critical path, the four-line version you can read in one sitting beats the forty-line version you cannot.

7.2 A Problem With Three Answers

Maximum Subarray

Given an array $A[1..n]$ of numbers, some negative, find the contiguous block $A[i..j]$ with the largest sum.

If all the numbers were positive the answer would be the whole array and there would be no problem. The negatives are what make it one.

MAX-SUBARRAY-BRUTE(A, n)

```

1  $best \leftarrow -\infty$ 
2 for  $i \leftarrow 1$  to  $n$  do
3   for  $j \leftarrow i$  to  $n$  do
4      $s \leftarrow 0$ 
5     for  $k \leftarrow i$  to  $j$  do
6        $s \leftarrow s + A[k]$  ▷ re-added from scratch
7     end for
8      $best \leftarrow \max(best, s)$ 
9   end for
10 end for
11 return  $best$ 
```

Figure 7.1: MAX-SUBARRAY-BRUTE: every block, summed from scratch. Three nested loops, so $\Theta(n^3)$ — and lines 4 to 7 are doing work that lines 4 to 7 of the previous iteration already did.

Theorem 7.1 — The brute force is cubic

MAX-SUBARRAY-BRUTE runs in $\Theta(n^3)$ time.

Proof. For each pair (i, j) with $i \leq j$, the inner loop performs $j - i + 1$ additions. Summing over all pairs,

$$\sum_{i=1}^n \sum_{j=i}^n (j - i + 1) = \sum_{i=1}^n \sum_{m=1}^{n-i+1} m = \sum_{i=1}^n \frac{(n-i+1)(n-i+2)}{2} = \sum_{p=1}^n \frac{p(p+1)}{2}.$$

The last sum is $\frac{1}{2}(\sum p^2 + \sum p) = \frac{1}{2}(\frac{n(n+1)(2n+1)}{6} + \frac{n(n+1)}{2})$, whose leading term is $n^3/6$. By Lemma 3.2 this is $\Theta(n^3)$. Every other operation is $O(1)$ per pair, contributing $O(n^2)$, which the sum rule absorbs. ■

Removing the waste, which is not a new idea

The inner loop recomputes a sum the previous iteration almost had. Extending the block from j to $j + 1$ costs one addition.

⚠ This is the distinction the chapter exists for

Going from n^3 to n^2 removed *redundant work inside* the search. The set of candidates examined did not change: still all $\binom{n}{2}$ pairs.

Going from n^2 to n , next, will change *the search itself* — it will stop examining pairs at all. Both are worth doing and they are different activities. The first is careful programming and gets you a constant or a factor of n ; the second requires an idea about the problem, and it is what the rest of this book is about.

☰ MAX-SUBARRAY-QUADRATIC(A, n)

```

1  $best \leftarrow -\infty$ 
2 for  $i \leftarrow 1$  to  $n$  do
3    $s \leftarrow 0$ 
4   for  $j \leftarrow i$  to  $n$  do
5      $s \leftarrow s + A[j]$  ▷ carried, not rebuilt
6      $best \leftarrow \max(best, s)$ 
7   end for
8 end for
9 return  $best$ 

```

Figure 7.2: MAX-SUBARRAY-QUADRATIC. The search is unchanged — still every pair (i, j) — but each candidate now costs $O(1)$ instead of $O(n)$. A better accountant, not a better idea.

7.3 An Idea About the Problem

💡 Kadane's observation

Ask a smaller question: *what is the best block **ending exactly at position** j ?*

Such a block either extends the best block ending at $j - 1$, or it starts fresh at j . There is no third option — a block ending at j that does not contain $j - 1$ must begin at j . So if H_j is the best sum ending at j ,

$$H_j = \max(A[j], H_{j-1} + A[j]),$$

and the answer is $\max_j H_j$.

That one sentence removes an entire dimension from the search. It is also the first appearance in this book of *optimal substructure* — the best solution built from best solutions to smaller versions — which Chapter 15 makes into a general technique.

☰ MAX-SUBARRAY-KADANE(A, n)

```

1  $here \leftarrow 0; \quad best \leftarrow -\infty$ 
2 for  $j \leftarrow 1$  to  $n$  do
3    $here \leftarrow \max(A[j], here + A[j])$  ▷ extend, or start fresh
4    $best \leftarrow \max(best, here)$ 
5 end for
6 return  $best$ 

```

Figure 7.3: MAX-SUBARRAY-KADANE. One pass, two variables. The loop invariant is what makes it believable, and Theorem 7.2 is what makes it true.

🏆 Theorem 7.2 — Kadane's algorithm is correct

MAX-SUBARRAY-KADANE returns the maximum subarray sum, in $\Theta(n)$ time.

Proof. Invariant: at the start of the iteration for j , $here$ is the maximum sum of a block ending at $j - 1$, and $best$ is the maximum sum of any block within $A[1..j - 1]$.

Initialisation. Before $j = 1$ both ranges are empty; $best = -\infty$ correctly records that no block has been seen, and $here = 0$ is the empty sum.

Maintenance. Assume the invariant before iteration j . Any block ending at j either contains $j - 1$ — in which case its sum is (the sum of a block ending at $j - 1$) + $A[j]$, maximised by *here* + $A[j]$ using the invariant — or it does not, in which case it is $A[j]$ alone. Line 3 takes the larger of exactly these two, so afterwards *here* is the best sum ending at j . Line 4 then extends the running maximum over one more position, re-establishing the invariant for $j + 1$.

Termination. The loop runs n times and stops. Substituting $j = n + 1$ into the invariant, *best* is the maximum sum of any block within $A[1..n]$, which is the postcondition. The body is $O(1)$ and runs n times, so the time is $\Theta(n)$. ■

7.4 The Three, Raced

Worked Example: First, Do They Agree?

A faster algorithm that returns a different answer is not faster. Before any timing, `code/ch07_brute.cpp` runs all three on the same inputs:

n	cubic	quadratic	linear	
1	35	35	35	agree
2	80	80	80	agree
10	265	265	265	agree
100	563	563	563	agree
500	1362	1362	1362	agree

This is the oracle use of brute force, and it is not a formality. The cubic version is three nested loops with no ideas in it and is almost impossible to get wrong; Kadane's is four lines that are easy to get subtly wrong — initialise *best* to 0 instead of $-\infty$ and it silently returns 0 for an all-negative array. The agreement above is evidence, cheaply bought.

Worked Example: The Predicted Ratios: 8, 4, 2

Doubling n should multiply a cubic time by 8, a quadratic by 4 and a linear by 2. Median of 3, 15 and 101 runs respectively; times in milliseconds.

n	cubic	ratio	quadratic	ratio	linear	ratio
500	7.66	—	0.070	—	0.0005	—
1,000	60.25	7.86	0.304	4.32	0.0010	2.00
2,000	471.16	7.82	1.095	3.60	0.0018	1.80
4,000	3355.85	7.12	4.348	3.97	0.0035	1.94
8,000	—	—	17.299	3.98	0.0071	2.03

Means: 7.60, 3.97, 1.94 against predicted 8, 4, 2.

Now read the first and last columns together. At $n = 4,000$ the cubic version takes 3.36 *seconds* and the linear version takes 3.5 *microseconds* — a factor of about a million. At $n = 500$ the factor was about fifteen thousand. The gap is not a constant; it grows as n^2 , which is exactly what “ $\Theta(n^3)$ against $\Theta(n)$ ” means in seconds.

And the row that is missing — cubic at $n = 8,000$ — is missing because it would take about 27 seconds for a result the linear version produces in 7 microseconds.

7.5 When Brute Force Is the Right Answer

The table above makes brute force look indefensible. In absolute terms it often is not.

Worked Example: What It Costs, in Absolute Terms

The same cubic routine, read as a cost rather than as a ratio:

n	cubic	verdict
10	0.1 μ s	imperceptible
30	2.1 μ s	imperceptible
100	78.8 μ s	imperceptible
300	1.6 ms	a visible pause
1,000	52.3 ms	a visible pause
3,000	1470.1 ms	too slow to ship

At $n = 100$ the cubic algorithm takes 79 microseconds. If your arrays are bounded by a hundred elements and the routine runs once per user action, the asymptotically terrible algorithm is the correct engineering choice: it is four lines, it needs no proof, and nobody will ever notice.

The decision turns on the range of n , not on the exponent. Which is why the first question to ask about any algorithm is not “what is its order of growth” but “how big does n get”.

A checklist

Brute force is defensible when *all* of these hold:

- n is small and **bounded by something real** — a board size, a number of columns, a field count — not merely small today;
- the code runs rarely, or off the critical path;
- correctness matters more than speed, and the short version is the one you can convince yourself of;
- you have measured, at the largest n you will actually see.

It is indefensible when n is unbounded or user-supplied, because then you have not chosen a slow algorithm, you have chosen an unbounded one — and Chapter 22 is about problems where that is the only choice anyone knows.

Common Pitfalls

- **Confusing “removed redundant work” with “found a better algorithm”.** The cubic-to-quadratic step did not change which candidates were examined. The quadratic-to-linear step did.
- **Initialising *best* to 0 in Kadane’s algorithm.** It then returns 0 on an all-negative array, which is wrong unless the empty block is allowed — and if it is, say so in the specification.
- **Dismissing brute force without measuring.** 79 microseconds at $n = 100$ is not a performance problem.
- **Keeping brute force when n is user-supplied.** “Small in practice” is a prediction, and users falsify it.

- **Not writing the brute force at all.** You then have nothing to test the clever version against, which is how subtly wrong algorithms reach production.
- **Timing the brute force once and the clever one once.** At microsecond scale one run measures the scheduler.

Try It Yourself: Three Algorithms, One Problem

Reproduces all three tables. Expect twenty-five minutes; the cubic column at $n = 4,000$ takes about ten seconds on its own.

```
// ch07_brute.cpp -- brute force, and the two things that beat it.
// Build: cl /O2 /EHsc /std:c++17 ch07_brute.cpp      (see Appendix A)

// --- 1. Brute force, as literally as it can be written -----
// For every start i and end j, add up a[i..j] from scratch. The inner
// re-addition is pure waste, and it is the  $n^3$ .
static long long max_sub_cubic(const std::vector<int>& a) {
    long long best = std::numeric_limits<long long>::min();
    size_t n = a.size();
    for (size_t i = 0; i < n; ++i)
        for (size_t j = i; j < n; ++j) {
            long long s = 0;
            for (size_t k = i; k <= j; ++k) s += a[k];
            best = std::max(best, s);
        }
    return best;
}

// --- 2. The same search, with the waste removed -----
// Extending the block from j to j+1 costs one addition, not a fresh
// traversal. The SEARCH is unchanged -- still every (i, j) pair -- so
// this is not yet a better idea, only a better accountant.
static long long max_sub_quadratic(const std::vector<int>& a) {
    long long best = std::numeric_limits<long long>::min();
    size_t n = a.size();
    for (size_t i = 0; i < n; ++i) {
        long long s = 0;
        for (size_t j = i; j < n; ++j) {
            s += a[j];
            best = std::max(best, s);
        }
    }
    return best;
}

// --- 3. Kadane: a different idea, not a faster loop -----
// Invariant: after processing a[0..j], "here" is the largest sum of a
// block ENDING at j, and "best" the largest sum anywhere in a[0..j].
// A block ending at j+1 either extends the best block ending at j, or
// starts fresh. That one observation removes a whole dimension.
static long long max_sub_linear(const std::vector<int>& a) {
    long long best = std::numeric_limits<long long>::min(), here = 0;
    for (int x : a) {
        here = std::max((long long)x, here + x);
        best = std::max(best, here);
    }
    return best;
}

// --- 4. All three agree, which is what makes the race fair -----
// A faster algorithm that gets a different answer is not faster.
```

```
// --- 5. The ratios each order of growth predicts -----
// Doubling n should multiply the times by 8, 4 and 2.

// --- 6. Where brute force is the right answer -----
// Report the ABSOLUTE cost, not a ratio against a linear time too
// small to measure, and say where it crosses thresholds a person
// would actually notice.
```

What to notice. Part A costs almost nothing to run and is the most valuable part of the program. Delete it and the other two parts are a race between three functions you have not checked.

Then try to break it. Change `best` in `max_sub_linear` from `numeric_limits::min()` to 0, and run part A again. Every row still agrees — because the random inputs all contain a positive block. Now generate an all-negative array and watch part A catch it. An oracle only tests the inputs you give it, which is Chapter 2’s point about distributions arriving again in a new costume.

🔍 Checkpoint — Test Yourself

1. The cubic algorithm performs $\sum_i \sum_{j \geq i} (j - i + 1)$ additions. Evaluate this sum for $n = 100$ and check it against $n^3/6$.
2. Explain why going from n^3 to n^2 did not require any insight into the maximum-subarray problem, while going from n^2 to n did.
3. Give a concrete situation in which you would ship the $\Theta(n^3)$ version, and state the one fact that would change your mind.

📝 Chapter Summary

- Brute force generates every candidate and tests it. It needs no insight into the problem, which is both its weakness and its value.
- It earns its place as the *baseline* to improve on and the *oracle* to test against — and sometimes as the answer.
- Maximum subarray has three solutions: $\Theta(n^3)$ exhaustive, $\Theta(n^2)$ with the redundant re-summation removed, and $\Theta(n)$ by Kadane’s observation.
- Removing redundant work *within* a search is different from changing *the search*. Only the second needs an idea about the problem, and it is the one this book is mostly about.
- Measured ratios on doubling: 7.60, 3.97, 1.94 against predicted 8, 4, 2. At $n = 4,000$ the cubic version took 3.4 seconds and the linear one 3.5 microseconds.
- Measured absolutely, the cubic version costs 79 μs at $n = 100$ and 1.5 s at $n = 3,000$. For a bounded small n it is the right choice.
- The first question about an algorithm is how big n gets, not what its exponent is.

🔧 Review Questions

1. (MCQ) The brute-force maximum-subarray algorithm runs in: (a) $\Theta(n)$; (b) $\Theta(n \lg n)$; (c) $\Theta(n^2)$; (d) $\Theta(n^3)$.
2. (MCQ) Kadane’s algorithm improves on the quadratic version by: (a) using a faster inner loop; (b) examining fewer candidate blocks; (c) using recursion; (d) sorting the input first.
3. (Short) Define the brute-force strategy and give two reasons a course on algorithm design includes it (CLO-6).
4. (Short) State Kadane’s loop invariant and explain why only two cases need be considered in its maintenance step.

5. **(Short)** Explain the oracle use of a brute-force implementation, and its one limitation.
6. **(Applied)** Give a brute-force algorithm for finding the closest pair among n points in the plane, state its running time with justification, and say what you would need to beat it.
7. **(Applied)** Your team has a $\Theta(n^3)$ routine running on $n \leq 80$, taking about $40 \mu s$. A colleague proposes replacing it with a $\Theta(n)$ version, 60 lines long. Argue both sides, and state the single piece of information that should decide it.

Divide and Conquer: Merge Sort and the Pattern

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part II — Recursion and Divide-and-Conquer

🕒 Learning Outcomes

By the end of this chapter you will be able to:

- **State** the three steps of the divide-and-conquer pattern and **apply** it to a stated problem (CLO-6).
- **Prove** merge sort correct from the merge step's invariant, and **derive** its $\Theta(n \lg n)$ bound in all three cases.
- **Explain** why merge sort's bound is case-independent and **contrast** it with insertion sort's.
- **Measure** the effect of an insertion-sort cutoff and **choose** one for a given machine.
- **Account** for a measured gap between two runs of the same algorithm in terms of work done and hardware behaviour.

🔗 Before You Start

Chapter 5 and Chapter 6 — the bound here is case 2 of the master theorem. Chapter 4 for the correctness argument.

📖 Key Terms in This Chapter

divide and conquer • divide • conquer • combine • merge • stability • in-place • auxiliary space • cutoff • hybrid sort • top-down • bottom-up • branch misprediction

8.1 The Pattern

📑 Divide and Conquer

A **divide-and-conquer** algorithm has three steps:

1. **Divide** the problem into subproblems that are smaller instances of the *same* problem.
2. **Conquer** the subproblems by solving them recursively; when they are small enough, solve them directly.
3. **Combine** the subproblem solutions into a solution for the original.

The running time is then $T(n) = aT(n/b) + D(n) + C(n)$, with D and C the cost of dividing and combining — which is exactly the form Chapter 6 solves.

💡 Where the work goes, and why it matters

The three steps are not equally expensive, and which one carries the weight distinguishes the two great sorting algorithms.

Merge sort divides trivially (compute the midpoint, $O(1)$) and combines expensively (merge two runs, $\Theta(n)$). *Easy split, hard join.*

Quicksort (Chapter 9) divides expensively (partition around a pivot, $\Theta(n)$) and combines trivially (the pieces are already in place, $O(1)$). *Hard split, easy join.*

Both give $T(n) = 2T(n/2) + \Theta(n)$ when the split is even. The difference is that merge sort *guarantees* the even split and quicksort only hopes for it — which is the whole of Chapter 9.

8.2 Merge Sort

≡ MERGE-SORT(A, lo, hi)

```

1 if  $hi - lo < 2$  then
2   return                                ▷ 0 or 1 element: already sorted
3 end if
4  $mid \leftarrow \lfloor (lo + hi)/2 \rfloor$ 
5 MERGE-SORT( $A, lo, mid$ )                    ▷ conquer the left
6 MERGE-SORT( $A, mid, hi$ )                    ▷ conquer the right
7 MERGE( $A, lo, mid, hi$ )                    ▷ combine

```

Figure 8.1: MERGE-SORT. The divide is one line of arithmetic; all the work is in MERGE.

≡ MERGE(A, lo, mid, hi)

```

1  $i \leftarrow lo; j \leftarrow mid; k \leftarrow lo$ 
2 while  $i < mid$  and  $j < hi$  do
3   if  $A[i] \leq A[j]$  then
4      $B[k] \leftarrow A[i]; i \leftarrow i + 1$     ▷  $\leq$  keeps it stable
5   else
6      $B[k] \leftarrow A[j]; j \leftarrow j + 1$ 
7   end if
8    $k \leftarrow k + 1$ 
9 end while
10 while  $i < mid$  do
11    $B[k] \leftarrow A[i]; i \leftarrow i + 1; k \leftarrow k + 1$ 
12 end while
13 while  $j < hi$  do
14    $B[k] \leftarrow A[j]; j \leftarrow j + 1; k \leftarrow k + 1$ 
15 end while
16 copy  $B[lo..hi]$  back into  $A$ 

```

Figure 8.2: MERGE: two sorted runs become one in a single pass. Line 3's $\leq$ rather than $<$ is what makes merge sort stable, and it is the only difference.

🔗 Theorem 8.1 — Merge is correct

If $A[lo..mid]$ and $A[mid..hi]$ are each sorted, then after MERGE the range $A[lo..hi]$ is sorted and holds the same multiset of values.

Proof. Invariant for the first loop: $B[lo..k]$ holds the $k - lo$ smallest elements of the two runs, in sorted order; and $A[i]$ and $A[j]$ are the smallest remaining elements of their respective runs.

Initialisation. $k = lo$, so $B[lo..lo]$ is empty and holds the 0 smallest elements vacuously. i and j point at the fronts of two sorted runs, so they are the smallest remaining in each.

Maintenance. The smallest remaining element overall is $\min(A[i], A[j])$, since each run is sorted and $A[i], A[j]$ are their minima. The body copies exactly that element into $B[k]$ and advances the corresponding index, so B gains the next-smallest element and the two pointers again indicate their runs' minima.

Termination. The loop ends when one run is exhausted. The measure $(mid - i) + (hi - j)$ strictly decreases each iteration and is bounded below, so the loop is finite. The two tail loops append the remainder of whichever run is left — already sorted, and all of it $\geq$ everything in B by the invariant. Every element is copied exactly once and none is overwritten before being copied, so the multiset is preserved. ■

🔗 Theorem 8.2 — Merge sort's running time

MERGE-SORT runs in $\Theta(n \lg n)$ in the best, worst and average cases, and uses $\Theta(n)$ auxiliary space.

Proof. MERGE performs between $\min(n_1, n_2)$ and $n_1 + n_2 - 1$ comparisons and exactly $n_1 + n_2$ moves, so it is $\Theta(n)$ regardless of the data. The divide is $O(1)$ and the two subproblems have sizes $\lfloor n/2 \rfloor$ and $\lceil n/2 \rceil$. Hence

$$T(n) = 2T(n/2) + \Theta(n)$$

for *every* input, with no case distinction. By Chapter 6 with $a = b = 2$ and $f(n) = n$, the watershed is $n^{\log_2 2} = n = \Theta(f(n))$, so case 2 applies and $T(n) = \Theta(n \lg n)$. Since the recurrence holds for every input, the bound is simultaneously best-, worst- and average-case. The buffer B holds at most n elements, giving $\Theta(n)$ auxiliary space. ■

⚠️ The bound is case-independent, the running time is not

Theorem 8.2 says the *order of growth* does not depend on the input. It does not say the running time is the same for every input, and Section 8.4 measures a factor of 3.9 between two inputs of the same size.

Keeping those two statements apart is the main intellectual work of this chapter.

8.3 Measured: the Bound Holds, the Clock Does Not

📊 Worked Example: Three Input Shapes, $n = 2,000,000$

code/ch08_merge.cpp sorts the same two million elements arranged three ways, counting comparisons exactly and timing the sort.

shape	time (ms)	comparisons	/ $n \lg n$	vs sorted
sorted	73.1	20,769,984	0.496	—
reversed	73.5	21,132,864	0.505	$\times 1.006$
random	285.7	39,348,350	0.940	$\times 3.910$

($n \lg n = 41,863,137$.)

First, the good news. Compare this with Chapter 2, where insertion sort on these same three shapes varied by a factor of **6,600**. Merge sort's comparison count varies by a factor of 1.9 and its time by 3.9. The guarantee is real.

Second, the comparison counts are exactly what theory says. On sorted input every merge exhausts its *left* run first — every element of the left run is smaller than every element of the right — so the comparison loop performs exactly n_1 comparisons instead of about

$n_1 + n_2$. Half as many. Measured: $0.496 n \lg n$ against random input's $0.940 n \lg n$, a ratio of 1.89.

Third, the time ratio is 3.91 and the comparison ratio is 1.89. Those do not match. Roughly half the slowdown is not accounted for by work done, and Section 8.5 is about finding out what it is.

Worked Example: The Cutoff, Tuned

Merge sort's recursion bottoms out at $n = 1$, which means a great many calls doing almost nothing. Chapter 1 measured insertion sort beating merge sort below $n \approx 65$; so stop recursing at some cutoff and insertion-sort the piece instead.

cutoff	time (ms)	vs none
0	275.5	1.000
4	257.1	0.933
8	243.1	0.882
16	232.5	0.844
32	226.4	0.822
64	231.0	0.838
128	257.0	0.933
256	247.7	0.899

The best cutoff on this machine is 32, and it is worth 18%. The curve is shallow between 16 and 64 — anything in that range gets most of the benefit — and turns back upward beyond it, as insertion sort's quadratic term begins to bite.

Note what this is and is not. It is a constant-factor improvement: the algorithm is $\Theta(n \lg n)$ before and after, and Chapter 3's notation cannot see the difference. It is also the reason every production sort in the world does it.

And note that 32 is a fact about this laptop, not about merge sort. A different cache, a different element size or a different compiler moves it. That is why the lab measures it rather than quoting it.

Worked Example: Four Implementations of One Bound

implementation	time (ms)	relative
top-down, no cutoff	279.2	1.00
top-down, cutoff 32	225.1	0.81
bottom-up	263.3	0.94
std::sort	169.1	0.61

All four are $\Theta(n \lg n)$ and they span a factor of 1.65. Bottom-up merge sort — same recurrence, no call stack — is 6% faster than top-down, which is less than most people expect; the recursion was never the expensive part.

std::sort wins by 39% because it is not merge sort at all. It is introsort: quicksort (Chapter 9) with an insertion-sort cutoff and a heapsort fallback, sorting in place and so touching half as much memory. The comparison is here to make a point that $\Theta(\cdot)$ cannot:

within one order of growth there is still a factor of nearly two to be had, and it is won by caring about memory.

8.4 Where the Missing Factor of Two Went

The gap in Section 8.4 was 3.91 in time and 1.89 in comparisons. Something costs time without being a comparison.

💡 A hypothesis, and how to test it

Hypothesis. The culprit is the branch `if (A[i] <= A[j])` in the merge loop. On sorted input it goes the same way for an entire run and the processor’s branch predictor gets it right every time. On random input it is a coin flip, mispredicted about half the time, at a cost of roughly fifteen cycles each.

This is exactly the third lie of Chapter 1’s RAM model — the one that says instructions execute one at a time — coming back to charge for itself.

The test. Rewrite the merge loop so that the choice is arithmetic rather than a branch:

```
bool left = A[i] <= A[j];
B[k++] = left ? A[i] : A[j]; i += left; j += !left;
```

This compiles to a conditional move. *Same comparisons, same output, no branch to mispredict.* If the hypothesis is right, the gap should close. If it is wrong, nothing will change and we will have learnt that too.

📊 Worked Example: The Test, Run

merge variant	time (ms)	vs branchy random
branchy, random input	272.9	1.000
branchless, random input	152.0	0.557
branchy, sorted input	72.1	0.264

Removing the branch cut the time on random input by 44%. The hypothesis is confirmed.

And now the gap decomposes cleanly:

$$\underbrace{3.78}_{\text{random/sorted, branchy}} = \underbrace{1.89}_{\text{more comparisons}} \times \underbrace{2.00}_{\text{branch misprediction}}$$

Measured: random/sorted was 3.78× with the branch and 2.11× without it. So 3.78/2.11 = 1.79 is attributable to the branch, and the residual 2.11 matches the comparison-count ratio of 1.89 to within 12%.

Two lessons. First, a factor of two was hiding in a line of code that $\Theta(\cdot)$ cannot see and that no amount of algorithm analysis would have revealed — only measurement found it, and only a designed experiment explained it. Second, the branchless version is not simply better: it does the *same* comparisons, so on sorted input it is slower than the branchy one, which gets its branches free. Which version wins depends on the data, and that is a sentence this book has now written in four different chapters.

! Common Pitfalls

- **Reading “case-independent bound” as “case-independent time”.** Measured, the same sort on the same n varied by 3.9.
- **Allocating the buffer inside the recursion.** One buffer for the whole sort, allocated once. Chapter 3’s lab nearly reported nonsense for exactly this reason.
- **Using $<$ instead of $\leq$ in the merge.** It still sorts; it is no longer stable, and stability is often why merge sort was chosen.
- **Claiming merge sort is in-place.** It needs $\Theta(n)$ auxiliary space. In-place merging exists and is slower and much harder.
- **Quoting a cutoff from a book.** The best value here was 32, on this machine, for 4-byte elements. Measure your own.
- **Tuning the cutoff before fixing the algorithm.** 18% from a cutoff is worth having; it is not worth having instead of a better order of growth.
- **Assuming recursion is the overhead.** Bottom-up was only 6% faster. The memory traffic is the cost, not the call stack.

🔧 Try It Yourself: Merge Sort, Tuned and Explained

Reproduces all four tables. Expect thirty minutes; it sorts two million elements many times.

```
// ch08_merge.cpp -- the divide-and-conquer pattern, and what tuning buys.
// Build: cl /O2 /EHsc /std:c++17 ch08_merge.cpp      (see Appendix A)

// --- 1. The merge, which is the whole idea -----
// Two sorted runs become one in a single pass: look at both fronts,
// take the smaller. Linear in the total length, and the only place
// the algorithm compares anything.
static void merge(std::vector<int>& a, std::vector<int>& buf,
                 size_t lo, size_t mid, size_t hi, bool count) {
    size_t i = lo, j = mid, k = lo;
    while (i < mid && j < hi) {
        if (count) ++comparisons;
        buf[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
    }
    while (i < mid) buf[k++] = a[i++];
    while (j < hi)  buf[k++] = a[j++];
    std::copy(buf.begin() + lo, buf.begin() + hi, a.begin() + lo);
}

// --- 2. Top-down: the recurrence written out as code -----
// T(n) = 2T(n/2) + Theta(n), which Chapter 6 settles as Theta(n lg n).
// The cutoff is the one tuning knob: below it, insertion sort's
// smaller constant wins, exactly as Chapter 1 measured.
static void msort(std::vector<int>& a, std::vector<int>& buf,
                 size_t lo, size_t hi, size_t cutoff, bool count) {
    if (hi - lo < 2) return;
    if (hi - lo <= cutoff) { insertion(a, lo, hi); return; }
    size_t mid = lo + (hi - lo) / 2;
    msort(a, buf, lo, mid, cutoff, count);
    msort(a, buf, mid, hi, cutoff, count);
    merge(a, buf, lo, mid, hi, count);
}

// --- 3. The same merge with the branch removed -----
// Same comparisons and same output; the choice becomes arithmetic
// that compiles to a conditional move, so there is no branch to
// mispredict. Used only by part 7, to test what the gap is made of.
static void merge_bl(std::vector<int>& a, std::vector<int>& buf,
```

```

        size_t lo, size_t mid, size_t hi) {
    size_t i = lo, j = mid, k = lo;
    while (i < mid && j < hi) {
        bool take_left = a[i] <= a[j];
        buf[k++] = take_left ? a[i] : a[j];
        i += take_left ? 1 : 0;
        j += take_left ? 0 : 1;
    }
    while (i < mid) buf[k++] = a[i++];
    while (j < hi)  buf[k++] = a[j++];
    std::copy(buf.begin() + lo, buf.begin() + hi, a.begin() + lo);
}

// --- 4. The bound does not care what the input looks like -----
// Chapter 2 found insertion sort varying by 6,600x across these same
// three shapes. Merge sort should vary by almost nothing.

// --- 5. What is the cutoff worth, and where is it? -----
// The best value is a property of this machine, not of the algorithm,
// which is exactly why it has to be measured rather than looked up.

// --- 6. Top-down against bottom-up, and against the library -----
// Same recurrence, same Theta. Everything that differs is constant
// factors -- recursion overhead, memory order, and in std::sort's
// case a different algorithm entirely (Chapter 9).

// --- 7. Why is random input 4x slower than sorted input? -----
// Part 4 found 3.9x in time but only 1.9x in comparisons, so half the
// slowdown is NOT work done. Hypothesis: the branch in the merge
// loop, perfectly predicted on sorted input and a coin flip on
// random. Replace it with arithmetic and see whether the gap closes.

```

What to notice. Part 7 is the shape of a real investigation: a measurement that does not fit, a hypothesis that names a mechanism, and an experiment whose two possible outcomes would tell you different things. The answer — 0.557 — was not knowable in advance.

Then try to break it. Run the branchless merge on *sorted* input and predict the result first. It should be *slower* than the branchy one, because a perfectly predicted branch costs almost nothing while a conditional move always costs something. If your prediction was “faster”, the thing to revise is the belief that branchless code is simply better.

? Checkpoint — Test Yourself

1. Why is merge sort’s recurrence $T(n) = 2T(n/2) + \Theta(n)$ valid for every input, while insertion sort needs three separate analyses?
2. MERGE performs between $\min(n_1, n_2)$ and $n_1 + n_2 - 1$ comparisons. Describe an input achieving each bound, and relate your answer to the measured 0.496 and 0.940.
3. The measured best cutoff was 32 and Chapter 1’s measured crossover was 65. Give a reason the two numbers differ.

✓ Chapter Summary

- Divide and conquer: divide into smaller instances of the same problem, conquer recursively, combine. The cost is $aT(n/b) + D(n) + C(n)$.
- Merge sort has an easy split and a hard join; quicksort the reverse. Merge sort *guarantees* the even split, which is why its bound holds in every case.
- $T(n) = 2T(n/2) + \Theta(n)$ for every input, so by case 2 of the master theorem merge sort is $\Theta(n \lg n)$ best, worst and average, with $\Theta(n)$ auxiliary space.

- Measured at $n = 2,000,000$: comparisons were $0.496 n \lg n$ on sorted input and $0.940 n \lg n$ on random, and times were 73 ms and 286 ms. The bound is case-independent; the clock is not.
- An insertion-sort cutoff of 32 was worth 18% on this machine. The best value is a property of the machine and must be measured.
- The $3.78\times$ sorted-to-random time gap decomposes as $1.89\times$ (more comparisons) $\times 2.0\times$ (branch misprediction). Replacing the branch with a conditional move cut random-input time by 44%, which confirmed the mechanism.
- `std::sort` beat all four merge sorts by 39% — the same order of growth, won back by sorting in place.

Review Questions

1. **(MCQ)** Merge sort's auxiliary space requirement is: (a) $\Theta(1)$; (b) $\Theta(\lg n)$; (c) $\Theta(n)$; (d) $\Theta(n \lg n)$.
2. **(MCQ)** Merge sort's worst-case running time is: (a) $\Theta(n)$; (b) $\Theta(n \lg n)$; (c) $\Theta(n^2)$; (d) dependent on the input order.
3. **(Short)** State the three steps of divide and conquer and apply them to merge sort, naming the cost of each (*CLO-6*).
4. **(Short)** Explain why merge sort is stable, and which single character in MERGE is responsible.
5. **(Short)** A colleague reports that their merge sort is twice as slow on one dataset as on another of the same size, and concludes the implementation is buggy. Give two innocent explanations.
6. **(Applied)** Derive the number of comparisons MERGE performs on already-sorted input, and use it to predict the $0.496 n \lg n$ measured above.
7. **(Applied)** Design an experiment to find the best insertion-sort cutoff for sorting 64-byte records rather than 4-byte integers. Predict whether the best cutoff will be larger or smaller than 32, and say why.

Quicksort, Randomisation and the Average Case

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part II — Recursion and Divide-and-Conquer

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Trace** Lomuto partitioning and **state** its invariant.
- **Derive** quicksort's $\Theta(n^2)$ worst case and $\Theta(n \lg n)$ expected case, and **identify** the input that causes each (*CLO-1*, *CLO-2*).
- **Explain** why randomising the pivot converts an assumption about the data into an assumption about a random number generator.
- **Account** for quicksort's dominance in practice despite a worse worst case than merge sort.
- **Predict** the recursion depth of a sort and **relate** it to the stack a program is given.

🔗 Before You Start

Chapter 2 for the distinction between an average over inputs and an expectation over an algorithm's own coins — this chapter is that distinction made concrete. Chapter 8 for the pattern and the baseline.

📖 Key Terms in This Chapter

quicksort • partition • Lomuto • pivot • in-place • unbalanced partition • expected running time • randomised algorithm • adversary • median-of-three • introsort • tail recursion • recursion depth

9.1 Hard Split, Easy Join

Quicksort is merge sort's mirror image. It does all its work in the divide step and nothing in the combine step.

☰ QUICKSORT(A, lo, hi)

```

1 if  $hi - lo < 2$  then
2   return
3 end if
4  $p \leftarrow \text{PARTITION}(A, lo, hi)$  ▷ all the work is here
5 QUICKSORT( $A, lo, p$ )
6 QUICKSORT( $A, p + 1, hi$ ) ▷ no combine step at all
```

Figure 9.1: QUICKSORT. There is no line after the two recursive calls: once the pieces are sorted, the array is sorted, because PARTITION already put them in the right places.

≡ PARTITION(A, lo, hi)

```

1  $x \leftarrow A[hi - 1]$   $\triangleright$  the pivot: the last element
2  $i \leftarrow lo$ 
3 for  $j \leftarrow lo$  to  $hi - 2$  do
4   if  $A[j] \leq x$  then
5     exchange  $A[i] \leftrightarrow A[j]$ 
6      $i \leftarrow i + 1$ 
7   end if
8 end for
9 exchange  $A[i] \leftrightarrow A[hi - 1]$ 
10 return  $i$ 

```

Figure 9.2: PARTITION (Lomuto). Everything $\leq$ the pivot is swept to the left; the pivot is then dropped into the gap. Linear, in place, and the only place quicksort compares anything.

🔗 Theorem 9.1 — Partition is correct

After PARTITION returns i , every element of $A[lo \dots i]$ is $\leq A[i]$, every element of $A[i + 1 \dots hi]$ is $> A[i]$, and $A[i]$ is the original pivot.

Proof. Invariant for the **for** loop: every element of $A[lo \dots i]$ is $\leq x$; every element of $A[i \dots j]$ is $> x$; and $A[hi - 1] = x$.

Initialisation. $i = j = lo$, so both ranges are empty and both clauses hold vacuously; the pivot has not been moved.

Maintenance. If $A[j] > x$, the loop body does nothing and j advances, extending the second range by an element that is indeed $> x$. If $A[j] \leq x$, the body exchanges $A[i]$ with $A[j]$: position i — which held an element $> x$, or nothing if the range was empty — receives an element $\leq x$, extending the first range, and the displaced element lands at j , which is now inside the second range and is $> x$. Incrementing i and j restores the invariant.

Termination. j increases by one each iteration towards the fixed bound $hi - 2$, so the loop ends. At that point $A[lo \dots i]$ holds elements $\leq x$ and $A[i \dots hi - 1]$ elements $> x$. Line 8 exchanges the pivot into position i ; everything left of i is still $\leq x$ and everything right of it is $> x$, which is the claim. ■

9.2 Two Running Times, Both True

🔗 Theorem 9.2 — Quicksort's worst case

QUICKSORT performs $\Theta(n^2)$ comparisons in the worst case, and that worst case occurs on already-sorted input.

Proof. If at every level the pivot is the largest (or smallest) remaining element, the partition splits n elements into $n - 1$ and 0, giving

$$T(n) = T(n - 1) + T(0) + \Theta(n) = T(n - 1) + \Theta(n).$$

Expanding, $T(n) = \sum_{k=1}^n \Theta(k) = \Theta(n^2)$ — the recursion tree is a path of length n , not a balanced tree of depth $\lg n$.

That this is attained by sorted input is immediate: the pivot is $A[hi - 1]$, the last element, which in a sorted array is the largest. Every element satisfies $A[j] \leq x$, so i advances every iteration and the final exchange returns $i = hi - 1$. The split is $(n - 1, 0)$ at every level. ■

⚠ The worst case is the commonest input shape in practice

This is the part that matters. Quicksort's worst case is not an exotic adversarial construction — it is *data that is already in order*, which is what you get from a database query with an `ORDER BY`, a log file, a sorted import, or any list a user has already sorted once.

An algorithm whose worst case is a rare accident is tolerable. An algorithm whose worst case is Tuesday is not.

🔗 Theorem 9.3 — Randomised quicksort's expected time

If the pivot is chosen uniformly at random from the subarray, QUICKSORT performs $O(n \lg n)$ comparisons in expectation, on *every* input.

Proof. Let $z_1 < z_2 < \dots < z_n$ be the elements in sorted order and let X_{ij} indicate that z_i and z_j are ever compared. Two elements are compared at most once, since a comparison happens only against a pivot and the pivot is then excluded from both subproblems. So the total is $\sum_{i < j} X_{ij}$ and, by linearity of expectation, $\mathbb{E}[\text{comparisons}] = \sum_{i < j} \Pr[z_i \text{ and } z_j \text{ compared}]$.

Consider the set $Z_{ij} = \{z_i, \dots, z_j\}$, of size $j - i + 1$. While all of Z_{ij} sits in one subproblem, no element of it has been a pivot. The first pivot chosen from Z_{ij} decides matters: if it is z_i or z_j , those two are compared; if it is any of the $j - i - 1$ elements strictly between, then z_i and z_j are separated into different subproblems and are never compared. Each element of Z_{ij} is equally likely to be the first pivot chosen from it, so

$$\Pr[z_i \text{ and } z_j \text{ compared}] = \frac{2}{j - i + 1}.$$

Therefore

$$\mathbb{E}[\text{comparisons}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j - i + 1} < \sum_{i=1}^{n-1} \sum_{k=1}^n \frac{2}{k} = 2(n-1)H_n = O(n \lg n),$$

using $H_n = \Theta(\lg n)$ for the harmonic number. The expectation is over the pivot choices only, with the input fixed, so the bound holds for every input. ■

💡 What randomising actually bought

Compare the two statements carefully.

Without randomisation: “ $\Theta(n \lg n)$ on average, assuming the input is a uniform random permutation.” That assumption is about the world, and the world violates it constantly — sorted data is common, not rare.

With randomisation: “ $O(n \lg n)$ expected, on every input.” No assumption about the data at all. An adversary who has read your source code, knows your algorithm and chooses the worst possible input still cannot make it slow, because he does not know your coin flips.

This is Chapter 2's distinction, and it is the single most valuable idea in the chapter: **randomisation converts an assumption you cannot enforce into one you can.**

9.3 Measured: the Gap, and Closing It

Worked Example: Comparisons on Already-Sorted Input

code/ch09_quick.cpp counts comparisons exactly — no timing, no noise.

n	last-pivot	$/(n^2/2)$	random pivot	$/(n \lg n)$	speed-up
1,000	499,500	0.9990	12,001	1.2042	41.6
2,000	1,999,000	0.9995	25,000	1.1399	80.0
4,000	7,998,000	0.9998	52,511	1.0971	152.3
8,000	31,996,000	0.9999	121,896	1.1752	262.5
16,000	127,992,000	0.9999	271,253	1.2139	471.9

The last-pivot column is $n(n-1)/2$ exactly — 0.9990 $\rightarrow$ 0.9999, converging to 1. Theorem 9.2 predicted the split $(n-1, 0)$ at every level, giving $\sum_{k=1}^{n-1} k = n(n-1)/2$ comparisons, and that is precisely what was counted. This is not an asymptotic estimate; it is the exact number.

The random-pivot column is about $1.15 n \lg n$, as Theorem 9.3's $2n \ln n = 1.39 n \lg n$ bound permits.

And the speed-up grows without limit: 41.6 at $n = 1,000$ and 471.9 at $n = 16,000$. It is multiplying by about 1.8 per doubling, which is what $n^2/(n \lg n) = n/\lg n$ does. Extrapolated to a million elements the factor is about 30,000.

Worked Example: The Worst Case Does Not Merely Run Slowly

Recursion depth on the same sorted input:

n	last-pivot	random pivot	$\lg n$
1,000	999	21	10.0
2,000	1,999	26	11.0
4,000	3,999	27	12.0
8,000	7,999	30	13.0
16,000	15,999	30	14.0

The naive version's depth is exactly $n-1$. Each of those is a live stack frame. The program measured its own frame size by comparing the address of a local at depth 0 with one at the deepest depth:

depth reached	15,999
stack span	1,023,912 bytes
bytes per frame	64.0
default stack	1,048,576 bytes (1 MB)
frames that fit	16,384

At $n = 16,000$ this program used 99.9% of its stack. It was 385 frames — six elements of input — from dying.

And at $n = 32,000$ it did die, with 0xC00000FD, STATUS_STACK_OVERFLOW, which is why the timing table in the lab stops at 16,000. The prediction “ $n > 16,384$ crashes” and the observation agree.

This is the practical form of the worst case. A quadratic sort is a performance bug you can file. A stack overflow is a crash, with no stack trace worth reading, on the day a customer's data happened to arrive in order.

📊 Worked Example: And the Trap: On Random Input the Naive Version Wins

n	last-pivot (ms)	random pivot (ms)	ratio
100,000	7.39	8.86	0.83
200,000	16.33	17.77	0.92
400,000	33.82	37.04	0.91
800,000	73.40	77.72	0.94

On random input the unsafe version is 6–17% faster, because it skips a random number generation and a swap per partition.

That is the trap in its purest form. A developer benchmarking on shuffled test data will find randomisation costs 10% and remove it, and the benchmark will support the decision every single time — until the day real, ordered data arrives and the program crashes.

Ten per cent is the insurance premium. The thing insured against is a factor of 472 and a stack overflow.

9.4 What Is Actually Shipped

💡 Introsort, and the three defences

`std::sort` beat every merge sort in Chapter 8 by 39%. It is **introsort**, and it stacks three ideas:

1. **Quicksort** as the main engine, for its small constant and its in-place operation — it touches half the memory merge sort does, which is where most of the 39% comes from.
2. **An insertion-sort cutoff** at around 16–32 elements, for exactly the reason Chapter 1 measured and Chapter 8 tuned.
3. **A heapsort fallback.** The recursion depth is tracked, and if it exceeds about $2 \lg n$ the implementation abandons quicksort and finishes with heapsort, which is $O(n \lg n)$ in the worst case.

The third is the interesting one: rather than make the bad case unlikely, it makes it *survivable*. The guarantee becomes $O(n \lg n)$ worst case, with quicksort's constants almost all of the time.

Median-of-three — taking the median of the first, middle and last elements as pivot — is a cheaper partial defence, and it fixes the sorted-input case specifically. It does not give a worst-case guarantee: an adversary who knows you use it can still construct a quadratic input. Use it with randomisation, not instead of it.

💡 Merge sort or quicksort?

	Merge sort	Quicksort (randomised)
worst case	$\Theta(n \lg n)$ guaranteed	$\Theta(n^2)$, probability vanishingly small

expected	$\Theta(n \lg n)$	$\Theta(n \lg n)$, smaller constant
space	$\Theta(n)$ auxiliary	$\Theta(\lg n)$ stack, in place
stable	yes	no
depth	$\Theta(\lg n)$ always	$\Theta(\lg n)$ expected
choose it when	stability matters; worst case must be bounded; data is on disk or a linked list	memory is tight; average speed matters; data is an array in memory

! Common Pitfalls

- **Benchmarking only on shuffled data.** It is the one distribution on which the dangerous version looks good — measured, 10% better.
- **Believing median-of-three is a guarantee.** It defeats sorted input, not an adversary.
- **Forgetting recursion depth.** Depth $n - 1$ is a crash, not a slow program, and it arrives at 16,384 elements on a 1 MB stack.
- **Not recursing on the smaller side first.** Recursing into the smaller part and looping on the larger bounds the stack at $O(\lg n)$ even in the worst case — a two-line change that removes the crash, though not the quadratic time.
- **Using quicksort when stability is required.** It is not stable, and no amount of pivot selection makes it so.
- **Seeding the generator with a constant.** An adversary who can read your binary can then reproduce your pivots, and the guarantee of Theorem 9.3 is gone.
- **Confusing “expected $\Theta(n \lg n)$ ” with “average-case $\Theta(n \lg n)$ ”.** The first holds on every input; the second assumes a distribution.

🔧 Try It Yourself: The Price of Insurance

Reproduces all five tables. Expect thirty minutes.

```
// ch09_quick.cpp -- quicksort, and why randomising is not superstition.
// Build: cl /O2 /EHsc /std:c++17 ch09_quick.cpp      (see Appendix A)

// --- 1. Lomuto partition, the textbook one -----
// Everything <= the pivot is swept to the left. Returns where the
// pivot ended up. Linear, and where every comparison happens.
static size_t partition_last(std::vector<int>& a, size_t lo, size_t hi) {
    int pivot = a[hi - 1];
    size_t i = lo;
    for (size_t j = lo; j + 1 < hi; ++j) {
        ++comps;
        if (a[j] <= pivot) std::swap(a[i++], a[j]);
    }
    std::swap(a[i], a[hi - 1]);
    return i;
}

// --- 2. The same partition, after moving a random element to the end
// One extra swap. That is the entire cost of the change, and it buys
// an expected-time bound that holds for EVERY input -- no assumption
// about the data at all (Chapter 2's distinction between an average
// over inputs and an expectation over the algorithm's own coins).
static size_t partition_random(std::vector<int>& a, size_t lo, size_t hi) {
    std::uniform_int_distribution<size_t> d(lo, hi - 1);
    std::swap(a[d(rng)], a[hi - 1]);
    return partition_last(a, lo, hi);
}
```

```

}

// --- 3. Comparisons counted: the two bounds, seen directly -----
// On sorted input the last-element pivot splits n into 0 and n-1
// every single time, so the count is 1+2+...+(n-1) = n(n-1)/2
// exactly. Compare that with the random pivot on the same input.

// --- 4. Recursion depth, which is what actually crashes -----
// The worst case is not only slow. Depth n means n stack frames.

// --- 5. And the same thing in seconds -----
// Capped at 16,000 deliberately: see part 6 for why 32,000 is not an
// option on a default stack.

// --- 6. The worst case does not just run slowly; it crashes -----
// Measure how big one frame actually is by looking at where a local
// variable sits at depth 0 and at the deepest depth, then work out
// how many fit in the default 1 MB stack. This is a prediction that
// can be checked, and it was: at n = 32,000 this program died with
// 0xC00000FD, STATUS_STACK_OVERFLOW.
double span = (double)(stack_top - stack_deep);
double per  = span / maxdepth;
printf("    bytes per frame      : %.1f\n", per);
printf("    default stack        : %.0f bytes (1 MB)\n", LIMIT);
printf("    frames that fit         : %.0f\n", LIMIT / per);

// --- 7. On RANDOM input the naive pivot is fine -----
// Which is the trap: the defect is invisible until the day someone
// feeds your sort data that is already in order, and that day comes.

```

What to notice. Part 6 measures the stack rather than assuming it. The frame came out at exactly 64.0 bytes, which is not a number you could have looked up — it depends on this compiler, these optimisation flags and the number of live locals.

Then try to break it. Make the recursion sort the *smaller* side recursively and loop on the larger one. Predict first what happens to (a) the comparison count on sorted input and (b) the recursion depth. The answer is that (a) does not change at all and (b) drops to $O(\lg n)$ — so the crash disappears and the quadratic time does not. Fixing a symptom is not fixing a bug, and knowing which you have done is the point.

🔍 Checkpoint — Test Yourself

1. On sorted input the measured comparison count was exactly $n(n-1)/2$. Derive that from the split Theorem 9.2 describes.
2. Explain why “expected $O(n \lg n)$ on every input” is a stronger claim than “average-case $O(n \lg n)$ ”, and why an adversary makes the difference (*CLO-2*).
3. The measured frame was 64 bytes and the stack 1 MB. At what n does the naive sort on sorted input crash, and what would doubling the stack buy you?

📋 Chapter Summary

- Quicksort is merge sort inverted: a hard split (partition, $\Theta(n)$, in place) and no join at all.
- Worst case $\Theta(n^2)$, attained on *already-sorted* input — which is a common input shape, not an exotic one. Expected $O(n \lg n)$ with a random pivot, on every input.
- Randomisation converts an assumption about the data, which the world violates, into an assumption about a random number generator, which it does not.

- Measured: on sorted input the naive pivot made exactly $n(n-1)/2$ comparisons ($0.9999 \times n^2/2$) against the random pivot's $1.15 n \lg n$ — a speed-up of 41.6 at $n=1,000$ rising to 471.9 at $n=16,000$.
- Measured: the naive version's recursion depth is exactly $n-1$. At $n=16,000$ it used 99.9% of a 1 MB stack, at 64.0 bytes per frame; at $n=32,000$ it crashed with `STATUS_STACK_OVERFLOW`.
- Measured: on *random* input the unsafe version is 6–17% faster. That 10% is the insurance premium, and the thing insured against is a factor of 472 and a crash.
- `std::sort` is introsort: quicksort, plus an insertion-sort cutoff, plus a heapsort fallback once the depth exceeds $2 \lg n$ — making the bad case survivable rather than merely unlikely.

Review Questions

1. **(MCQ)** Quicksort with a last-element pivot achieves its worst case on: (a) random input; (b) already-sorted input; (c) input with many duplicates only; (d) no realistic input.
2. **(MCQ)** Randomised quicksort's expected $O(n \lg n)$ bound assumes: (a) the input is a random permutation; (b) the input has no duplicates; (c) nothing about the input; (d) the input is partly sorted.
3. **(Short)** State Lomuto partition's loop invariant and explain what the final exchange accomplishes.
4. **(Short)** Explain why quicksort is preferred in practice despite a worse worst case than merge sort (*CLO-1*).
5. **(Short)** Why is median-of-three a defence against sorted input but not against an adversary?
6. **(Applied)** In the proof of Theorem 9.3, explain why z_i and z_j are compared exactly when the first pivot chosen from Z_{ij} is one of them, and evaluate the resulting probability for $i=3$, $j=10$.
7. **(Applied)** Your team benchmarks a sort on shuffled data and finds randomisation costs 10%, so proposes removing it. Write the reply, citing the measured numbers in this chapter, and name the one experiment that would settle the question.

PART III

Sorting, Selection and the Structures Beneath

Part II costed particular algorithms. Part III asks the harder question — how fast could any algorithm be? — proves an answer for sorting, and then shows exactly what buying an extra assumption gets you.

the first result about a problem, not an algorithm

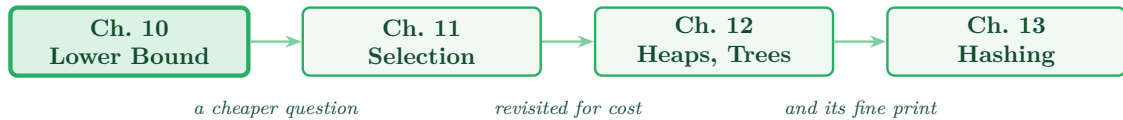


Figure P3.1: Reading path through Part III. Chapter 10 is the hinge of the whole book; Chapters 12 and 13 revisit Data Structures for their analysis, not their implementation.

Lower Bounds and Sorting in Linear Time

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part III — Sorting, Selection and the Structures Beneath

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** what a decision tree models and **state** why a comparison sort's execution corresponds to a root-to-leaf path.
- **Prove** the $\Omega(n \lg n)$ lower bound for comparison sorting, and **state** precisely which algorithms it binds.
- **Distinguish** a lower bound on a *problem* from an upper bound on an *algorithm*.
- **Describe** counting sort and radix sort, and **name** the assumption each buys in exchange for linearity.
- **Assess** how close a real comparison sort comes to the information-theoretic floor.

🔗 Before You Start

Chapter 3 for Ω , and Chapter 8 for a sort that attains the bound. Appendix B has Stirling's approximation, which Section 10.2 uses.

📖 Key Terms in This Chapter

decision tree • comparison sort • lower bound • information-theoretic bound • leaf • height • Stirling's approximation • counting sort • radix sort • bucket sort • stability • key universe

10.1 A Different Kind of Question

Every result so far has been about an algorithm: *this* sort takes $\Theta(n \lg n)$, *that* one $\Theta(n^2)$. This chapter asks something else.

💡 Algorithms and problems

An upper bound is a statement about an algorithm. “Merge sort is $O(n \lg n)$ ” is proved by exhibiting merge sort and analysing it. It says sorting can be done at least this fast.

A lower bound is a statement about a problem. “Comparison sorting is $\Omega(n \lg n)$ ” must hold for *every* comparison sort — including the ones nobody has invented yet. It cannot be proved by examining algorithms one at a time, because there are infinitely many. That difference is why this chapter is the hinge of the book. Up to here the subject has been ingenuity; from here it also has limits, and Chapter 22 is about the largest open question concerning them.

10.2 The Decision Tree

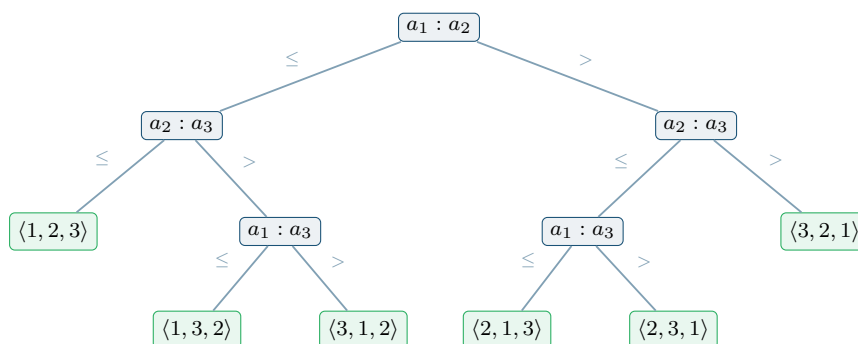
Comparison Sort

A **comparison sort** determines the order of its input using only comparisons between pairs of elements. It may ask “is $a_i \leq a_j$?” and move elements about, and it may do nothing else with their values — no arithmetic on them, no indexing by them, no inspection of their bits. Insertion sort, merge sort, quicksort, heapsort and every sort in Part II are comparison sorts.

Decision Tree

Fix a comparison sort and an input size n . Its **decision tree** is a binary tree in which each internal node is a comparison $a_i : a_j$, the two edges are the two outcomes, and each leaf is a permutation — the answer the algorithm outputs when it reaches that leaf.

Running the algorithm on an input traces one root-to-leaf path. The number of comparisons made is the length of that path, and the worst-case number of comparisons is the **height** of the tree.



$3! = 6$ leaves, so height $\geq \lceil \lg 6 \rceil = 3$

Figure 10.1: The decision tree of a comparison sort on three elements. Every one of the $3! = 6$ orderings must appear as a leaf, or some input would be sorted wrongly — and a binary tree with 6 leaves must have height at least $\lceil \lg 6 \rceil = 3$.

Theorem 10.1 — The comparison sorting lower bound

Any comparison sort requires $\Omega(n \lg n)$ comparisons in the worst case.

Proof. Consider the decision tree of any comparison sort on n elements, and let h be its height and ℓ its number of leaves.

Every permutation must appear as a leaf. There are $n!$ distinct orderings of n distinct elements. If some ordering labelled no leaf, then on the input requiring that ordering the algorithm would reach a leaf labelled with a different permutation and output it — producing an unsorted array. So $\ell \geq n!$.

A binary tree of height h has at most 2^h leaves, by induction on h : a tree of height 0 is a single leaf, and a tree of height h has two subtrees of height at most $h - 1$, hence at most $2 \cdot 2^{h-1} = 2^h$ leaves between them.

Combining, $2^h \geq \ell \geq n!$, so

$$h \geq \lg(n!).$$

And $\lg(n!) = \Omega(n \lg n)$, by Stirling's approximation $n! > (n/e)^n$:

$$\lg(n!) > \lg\left(\frac{n}{e}\right)^n = n \lg n - n \lg e = n \lg n - 1.4427n = \Omega(n \lg n).$$

Since h is the worst-case number of comparisons, the bound follows. ■

» Corollary 10.2

Merge sort and heapsort are asymptotically optimal comparison sorts, and randomised quicksort is optimal in expectation.

⚠ What the theorem does *not* say

It does not say sorting is $\Omega(n \lg n)$. It says *comparison* sorting is.

The proof's entire force comes from the branching factor being 2: each comparison yields one bit, and distinguishing $n!$ possibilities needs $\lg(n!)$ bits. An algorithm that extracts more than one bit per operation — by using a key's value as an array index, say — is not modelled by this tree at all, and Section 10.4 presents two such algorithms. They are not counter-examples; they are outside the hypothesis.

10.3 How Close Does a Real Sort Get?

The bound says at least $\lg(n!)$ comparisons. Merge sort achieves $O(n \lg n)$. Those agree asymptotically — but by what constant does a real sort miss the floor?

📊 Worked Example: The Floor Itself

`code/ch10_lowerbound.cpp` computes $\lg(n!)$ by summing logarithms (forming $n!$ would overflow at $n = 21$).

n	$\lg(n!)$	$n \lg n$	$n \lg n - 1.443n$	$\lg(n!)/n \lg n$
1,000	8,529	9,966	8,523	0.8559
10,000	118,458	132,877	118,450	0.8915
100,000	1,516,704	1,660,964	1,516,695	0.9131
1,000,000	18,488,885	19,931,569	18,488,874	0.9276

Stirling's form is essentially exact. $n \lg n - 1.443n$ differs from $\lg(n!)$ by 11 out of 18 million at $n = 10^6$ — a relative error of 6×10^{-7} .

The crude form is not. $\lg(n!)/(n \lg n)$ is only 0.86 at $n = 1,000$, creeping to 0.93 at a million. The $-1.443n$ term matters at every size you will ever sort, which is why the sharper form is the one to quote.

📊 Worked Example: Merge Sort Against the Floor

n	merge sort comparisons	$\lg(n!)$	excess
1,000	8,702	8,529	2.0%
10,000	120,505	118,458	1.7%
100,000	1,536,346	1,516,704	1.3%
1,000,000	18,672,689	18,488,885	1.0%

Merge sort is within 1% of the information-theoretic minimum at $n = 10^6$, and the gap is shrinking.

That is a stronger statement than “asymptotically optimal”. Asymptotic optimality permits a constant factor of 10; this is a constant factor of 1.01. No comparison sort — none that exists, none that will ever be invented — can beat merge sort’s comparison count by more than one per cent at this size.

Which puts the remaining engineering in its place. Chapter 8 found `std::sort` beating merge sort by 39% in *time*. Since the comparison count cannot improve by more than 1%, essentially all of that 39% is memory traffic and branch behaviour — not comparisons. The lower bound tells you where the remaining performance is *not*.

10.4 Sorting Without Comparing

To beat $\Omega(n \lg n)$ you must leave the comparison model. Both ways of doing so buy linearity with an assumption about what the keys *are*.

```

≡ COUNTING-SORT( $A, n, K$ )
1 let  $C[0 \dots K-1]$  be a new array of zeros
2 for  $i \leftarrow 1$  to  $n$  do
3    $C[A[i]] \leftarrow C[A[i]] + 1$                                 ▷ how many of each value
4 end for
5 for  $v \leftarrow 1$  to  $K-1$  do
6    $C[v] \leftarrow C[v] + C[v-1]$                                 ▷ running totals: where each value ends
7 end for
8 for  $i \leftarrow n$  downto 1 do
9    $C[A[i]] \leftarrow C[A[i]] - 1$ 
10   $B[C[A[i]]] \leftarrow A[i]$                                 ▷ downwards keeps it stable
11 end for
12 return  $B$ 

```

Figure 10.2: COUNTING-SORT. No element is ever compared with another: line 3 uses a key as an *index*, which the decision-tree model does not permit and which is why the bound does not apply. Running time $\Theta(n + K)$.

Theorem 10.3 — Counting sort

If every key is an integer in $[0, K)$, COUNTING-SORT sorts n keys in $\Theta(n + K)$ time and $\Theta(n + K)$ space, and is stable.

Proof. The three loops run n , $K-1$ and n times respectively with $O(1)$ bodies, giving $\Theta(n + K)$; the arrays B and C give the space.

For correctness, observe after the second loop that $C[v]$ holds the number of keys $\leq v$. Hence the keys equal to v belong in positions $C[v-1], \dots, C[v]-1$ of the output. The third loop places each key $A[i]$ at $C[A[i]]-1$ and then decrements $C[A[i]]$, so successive keys of the same value fill that block from its top downwards. Scanning i from n down to 1 therefore places the *last* occurrence highest, preserving input order among equal keys — which is stability. ■

 **Read the hypothesis, which is where the cost went**

$\Theta(n + K)$ is linear only when $K = O(n)$.

Sorting a million 32-bit integers with counting sort means $K = 2^{32}$ and an array of four billion counters — 16 GB, to sort 4 MB of data. The algorithm is not fast there; it is unusable.

Counting sort is the right choice for ages, exam marks, pixel values, bytes, days of the year — small known universes. Radix sort is what lifts the restriction.

💡 Radix sort: counting sort, one digit at a time

Treat a 32-bit key as four 8-bit digits. Counting-sort the keys by the least significant digit, then by the next, and so on. Each pass has $K = 256$, so each is $\Theta(n + 256) = \Theta(n)$, and there are 4 passes, giving $\Theta(n)$ overall — more precisely $\Theta(d(n + R))$ for d digits of radix R .

Stability is not a nicety here; it is the mechanism. The pass on digit i must not disturb the relative order established by the passes on digits $0 \dots i - 1$, or the earlier work is destroyed. This is why Theorem 10.3 bothered to prove stability, and why the third loop of COUNTING-SORT counts downwards.

And the assumption has not vanished, only changed shape: from “keys come from a small universe” to “keys have a fixed width”. Sorting variable-length strings by radix is a more delicate business.

📊 Worked Example: Beating the Bound, Measured

One million to 6.4 million keys drawn uniformly from $[0, 1000)$, so counting sort’s $K = 1000$ is genuinely small. Times in milliseconds, median of 5.

n	<code>std::sort</code>	counting	radix	best speed-up
100,000	4.0	0.6	1.1	6.74
400,000	17.1	1.8	4.7	9.64
1,600,000	68.6	7.3	23.2	9.45
6,400,000	259.7	29.0	81.0	8.94

Counting sort is 7–10× faster than the best comparison sort in the standard library, and all three produce identical output.

No theorem was violated. `std::sort` is bound by Theorem 10.1; counting sort is not, because it never compares two keys. The speed-up is the price the comparison model was charging, made visible.

And the ratio is roughly constant at about 9, not growing — because both algorithms are within a logarithmic factor here and $\lg n$ moves slowly. The gain is a large constant, not a change of order; at $n = 6.4$ million, $\lg n = 22.6$, and counting sort recovers about 40% of that.

⚠️ Common Pitfalls

- **Stating the bound as “sorting is $\Omega(n \lg n)$ ”.** It is *comparison* sorting. Dropping the qualifier makes Section 10.4 look like a contradiction.
- **Thinking counting sort breaks the theorem.** It is outside the hypothesis, not a counter-example to the conclusion.
- **Forgetting the $+K$.** Counting sort on 32-bit keys needs a four-billion-entry array.
- **An unstable counting sort inside a radix sort.** The radix passes then destroy each other’s work and the output is wrong, not merely slow.
- **Quoting $\lg(n!) \approx n \lg n$ at small n .** At $n = 1,000$ the ratio is 0.86. Use $n \lg n - 1.443n$.

- **Hunting for a faster comparison sort.** Merge sort is within 1% of the floor at $n = 10^6$; the remaining performance is in memory behaviour, not comparisons.
- **Assuming linear-time sorts are always faster.** They were $9\times$ faster here on a 1000-value universe. Widen the universe and counting sort loses, then becomes impossible.

Try It Yourself: The Floor, and Going Under It

Reproduces all three tables. Expect twenty-five minutes.

```
// ch10_lowerbound.cpp -- the comparison lower bound, and sorting past it.
// Build: cl /O2 /EHsc /std:c++17 ch10_lowerbound.cpp (see Appendix A)

// --- 1. lg(n!) exactly, by summing logs -----
// n! overflows at n = 21, so the factorial is never formed: the log of
// a product is the sum of the logs, and that stays in range for ever.
static double lg_factorial(long long n) {
    double s = 0;
    for (long long k = 2; k <= n; ++k) s += std::log2((double)k);
    return s;
}

// --- 2. A comparison sort, counting every comparison -----
// Merge sort, because Chapter 8 proved it Theta(n lg n) in every case.
// The question here is how close its CONSTANT is to the floor.

// --- 3. Counting sort: no comparisons at all -----
// It never asks "is x < y". It asks "how many keys equal v", which is
// a different question and not covered by the theorem. The price is
// the assumption that keys are integers in a known, small range.
static void counting_sort(const std::vector<int>& in,
                        std::vector<int>& out, int K) {
    std::vector<int> cnt(K, 0);
    for (int x : in) ++cnt[x];
    for (int v = 1; v < K; ++v) cnt[v] += cnt[v - 1];
    for (size_t i = in.size(); i-- > 0;) out[--cnt[in[i]]] = in[i];
}

// --- 4. Radix sort: counting sort, one digit at a time -----
// Lifts counting sort's "small range" assumption to "fixed width":
// 32-bit keys in four passes of an 8-bit counting sort. Still no
// comparisons, so still outside the theorem.
static void radix_sort(std::vector<int>& a, std::vector<int>& tmp) {
    const int BITS = 8, R = 1 << BITS;
    size_t n = a.size();
    for (int shift = 0; shift < 32; shift += BITS) {
        std::vector<size_t> cnt(R + 1, 0);
        for (size_t i = 0; i < n; ++i)
            ++cnt[((unsigned)a[i] >> shift & (R - 1)) + 1];
        for (int v = 0; v < R; ++v) cnt[v + 1] += cnt[v];
        for (size_t i = 0; i < n; ++i)
            tmp[cnt[((unsigned)a[i] >> shift & (R - 1)) + 1]] = a[i];
        a.swap(tmp);
    }
}

// --- 5. The bound itself, and Stirling's sharper form -----
// lg(n!) >= n lg n - n lg e = n lg n - 1.4427n.

// --- 6. How close does a real sort get to the floor? -----
// Merge sort is Theta(n lg n); so is the bound. The question is the
// constant, and the answer is "within a few per cent".
```

```
// --- 7. Beating the bound, by not comparing -----
// Counting and radix sort are not counter-examples to the theorem.
// They are outside it: they never compare two keys. What they pay
// instead is an assumption about what the keys ARE.
```

```
// --- 8. And they really did sort -----
```

What to notice. Part 6's excess column *falls* — 2.0%, 1.7%, 1.3%, 1.0%. Merge sort is not merely within a constant factor of optimal; it is converging to optimal. There is nothing left to win in comparisons.

Then try to break it. Change K from 1000 to 10,000,000 and re-run part 7. Counting sort's time should barely move for small n and then be dominated entirely by the $+K$ term — at $n = 100,000$ it will be *slower* than `std::sort`, because it is allocating and sweeping an array a hundred times longer than its input. The $+K$ is not a footnote.

🔍 Checkpoint — Test Yourself

1. A comparison sort's decision tree on $n = 4$ elements must have at least how many leaves, and hence what minimum height? Compare with $\lg(4!)$.
2. Explain why counting sort does not contradict Theorem 10.1, naming the step of the proof that fails for it.
3. Merge sort was measured 1.0% above $\lg(n!)$ at $n = 10^6$, yet `std::sort` is 39% faster in time. Reconcile these.

📋 Chapter Summary

- A lower bound is a statement about a *problem* and must hold for every algorithm, including undiscovered ones. This is the first such result in the book.
- A comparison sort's decision tree must have $n!$ leaves; a binary tree of height h has at most 2^h ; hence $h \geq \lg(n!) = \Omega(n \lg n)$.
- Merge sort, heapsort and randomised quicksort are therefore asymptotically optimal comparison sorts.
- Measured: $\lg(n!)$ matches $n \lg n - 1.443n$ to a relative error of 6×10^{-7} , while the crude $n \lg n$ is 14% high at $n = 1,000$.
- Measured: merge sort uses 1.0% more comparisons than the floor at $n = 10^6$, and the excess is falling. No comparison sort can do appreciably better.
- Counting sort ($\Theta(n + K)$) and radix sort ($\Theta(d(n + R))$) are linear because they never compare keys — they index by them. Each buys that with an assumption: a small universe, or a fixed width.
- Measured: counting sort beat `std::sort` by 7–10× on a 1000-value universe, with identical output and no theorem harmed.
- Radix sort requires its per-digit pass to be stable, or earlier passes are destroyed.

🔧 Review Questions

1. (MCQ) The decision tree of a comparison sort on n elements must have at least: (a) n leaves; (b) n^2 leaves; (c) $n!$ leaves; (d) 2^n leaves.
2. (MCQ) Counting sort runs in: (a) $\Theta(n)$ always; (b) $\Theta(n + K)$; (c) $\Theta(n \lg n)$; (d) $\Theta(K \lg n)$.
3. (Short) Explain the difference between a lower bound on a problem and an upper bound on an algorithm, with an example of each.

4. **(Short)** Why must the counting sort inside a radix sort be stable? Describe what goes wrong if it is not.
5. **(Short)** State the assumption counting sort buys in exchange for linearity, and give a key type for which it is a bad bargain.
6. **(Applied)** Prove that any comparison-based algorithm for finding the *minimum* of n elements requires at least $n - 1$ comparisons. (The decision-tree argument is not the easiest route; think about what an adversary can arrange.)
7. **(Applied)** You must sort 50 million 64-bit timestamps spanning one year. Decide between `std::sort`, counting sort and radix sort, giving the arithmetic for each, and state what you would measure before committing.

Selection and Order Statistics

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part III — *Sorting, Selection and the Structures Beneath*

🕒 Learning Outcomes

By the end of this chapter you will be able to:

- **Define** the i -th order statistic and **distinguish** selection from sorting.
- **Derive** quickselect's $\Theta(n)$ expected and $\Theta(n^2)$ worst-case running times from the recurrences, and **explain** why recursing on one side changes the master-theorem case.
- **Describe** the median-of-medians algorithm and **state** why it gives a worst-case linear bound.
- **Judge** when the deterministic algorithm is worth its constant.
- **Predict** how selection's cost varies with k .

🔗 Before You Start

Chapter 9 — quickselect is quicksort with one recursive call deleted, and the randomisation argument is the same one. Chapter 6 for the recurrence.

📖 Key Terms in This Chapter

order statistic • selection • median • quickselect • prune and search • median of medians • introselect • worst-case linear • tail call

11.1 Asking for Less

📑 Order Statistic, Selection

The i -th **order statistic** of a set of n elements is the i -th smallest. The 1st is the minimum, the n -th the maximum, and the $\lfloor (n+1)/2 \rfloor$ -th the **median**.

The **selection problem** is: given an array and an index i , return the i -th order statistic.

💡 Sorting answers more than was asked

Selection is obviously solvable in $\Theta(n \lg n)$: sort, then index. The question is whether that is wasteful, and it plainly is — sorting also determines the relative order of every *other* pair, which nobody requested.

Chapter 10 proved that *sorting* cannot be done in less than $\lg(n!)$ comparisons. That theorem says nothing about selection, because selection has only n possible answers rather than $n!$, so its decision tree needs only $\lg n$ leaves' worth of height from the counting argument — a useless bound, but an honest sign that the problem is easier.

And it is: selection is $\Theta(n)$, in the worst case, deterministically. Doing *less* than sorting turns out to cost asymptotically less.

11.2 Quickselect

```

1  QUICKSELECT( $A, lo, hi, k$ )
2  while true do
3    if  $hi - lo = 1$  then
4      return  $A[lo]$ 
5    end if
6     $p \leftarrow \text{RANDOM-PARTITION}(A, lo, hi)$ 
7    if  $p = k$  then
8      return  $A[p]$   $\triangleright$  the pivot is the answer
9    else if  $k < p$  then
10      $hi \leftarrow p$   $\triangleright$  discard the right side entirely
11   else
12      $lo \leftarrow p + 1$   $\triangleright$  discard the left side entirely
13   end if
14 end while

```

Figure 11.1: QUICKSELECT. Identical to QUICKSORT except that one of the two recursive calls is *deleted* rather than made. Written as a loop because the remaining call is in tail position — which also removes Chapter 9’s stack-overflow failure.

🔗 Theorem 11.1 — Quickselect’s expected time

With a uniformly random pivot, QUICKSELECT runs in $\Theta(n)$ expected time on every input, and $\Theta(n^2)$ in the worst case.

Proof. Worst case. If every partition is maximally unbalanced, the range shrinks by one each iteration and the partitions cost $n, n-1, n-2, \dots$, summing to $\Theta(n^2)$.

Expected case. Call a pivot *good* if it falls in the middle half of the range, that is between the 25th and 75th percentiles. A uniformly random pivot is good with probability $1/2$, and a good pivot leaves a subproblem of size at most $\frac{3}{4}$ of the range. Hence the expected number of partitions needed to reduce the range by a factor of $\frac{3}{4}$ is at most 2, and if $T(n)$ denotes the expected cost,

$$T(n) \leq T\left(\frac{3}{4}n\right) + \Theta(n),$$

where the $\Theta(n)$ absorbs the expected two partitions. By Chapter 6 with $a = 1$, $b = 4/3$, $f(n) = n$: the watershed is $n^{\log_{4/3} 1} = n^0 = 1$, and $f(n) = n = \Omega(n^{0+\varepsilon})$ with $\varepsilon = 1$, so case 3 applies provided regularity holds — $1 \cdot f(3n/4) = \frac{3}{4}n \leq cn$ with $c = \frac{3}{4} < 1$, which it does. Therefore $T(n) = \Theta(f(n)) = \Theta(n)$. ■

💡 One deleted line, one changed case

This is worth stating plainly, because it is the whole idea.

Quicksort solves both subproblems: $T(n) = 2T(n/2) + n$. Watershed $n^{\log_2 2} = n$, equal to $f(n)$, so **case 2** — and case 2 is the one that produces a logarithm. Result $\Theta(n \lg n)$.

Quickselect solves one: $T(n) = T(n/2) + n$. Watershed $n^{\log_2 1} = 1$, far below $f(n) = n$, so **case 3** — the root dominates and the sum is geometric. Result $\Theta(n)$.

The $\lg n$ factor is not saved by cleverness. It is the number of levels in the tree, and discarding one branch collapses the tree to a path whose work halves each step: $n + n/2 + n/4 + \dots < 2n$.

11.3 Measured

 Worked Example: Comparisons to Find the Median

code/ch11_select.cpp, mean of 25 independent runs — an *expected* value is being measured, and a single run of a randomised algorithm is not an answer.

n	comparisons	$/n$	spread over runs	$/(n \lg n)$
10,000	33,640	3.36	1.80–5.84	0.2532
100,000	318,800	3.19	1.78–5.09	0.1919
1,000,000	3,344,844	3.34	2.01–5.67	0.1678
10,000,000	32,623,584	3.26	1.91–5.60	0.1403

The cost per element is flat at about 3.3 across three orders of magnitude of n . That flatness *is* linearity: $\Theta(n)$ means comparisons/ n is bounded, and a column that neither rises nor falls over a thousandfold range of n is the evidence. The classical expected value for median selection is $3.39n$.

Meanwhile $/(n \lg n)$ falls steadily — 0.2532 to 0.1403 — which is what a linear quantity does when divided by an $n \lg n$ one.

Read the spread column too. Individual runs ranged from $1.8n$ to $5.8n$ comparisons. The expectation is 3.3; the variance is real, and a single unlucky run costs three times a lucky one. Expected-time guarantees are about the average of many runs, not about the next one.

 Worked Example: Selecting Against Sorting, in Seconds

n	quickselect	nth_element	full sort	speed-up
100,000	0.99	0.95	6.79	6.84
800,000	7.76	6.12	61.26	7.89
3,200,000	29.94	29.76	284.57	9.50
6,400,000	62.83	64.42	584.80	9.31

Selection beats sorting by 7–9.5 $\times$, and the factor grows with n , because it is essentially $\lg n$ divided by a constant.

And the hand-written quickselect matches std::nth_element — within a few per cent either way. The library version is introselect, which adds a median-of-medians fallback for the worst case; on random data that fallback never fires, so the two are doing the same work.

 Worked Example: The Worst Case, and What It Costs

A last-element pivot on already-sorted input, selecting the median:

n	naive comparisons	$/(n^2/2)$	random pivot	$/n$
2,000	1,499,500	0.7498	9,402	4.70
8,000	23,998,000	0.7499	18,914	2.36
16,000	95,996,000	0.7500	67,744	4.23
32,000	383,992,000	0.7500	137,461	4.30

The naive count is $0.7500 \times n^2/2 = 3n^2/8$ **exactly**, and that number can be derived. On sorted input the last-element pivot is always the largest, so the range shrinks from the top by one each time. To reach the median the range must fall from n to $n/2$, costing

$$n + (n-1) + \cdots + \frac{n}{2} = \frac{(n + n/2)}{2} \cdot \frac{n}{2} = \frac{3n^2}{8}.$$

Measured 0.7500, predicted 0.75. The analysis is not approximately right; it is right.

One difference from Chapter 9 worth noting. There the same input caused a *crash*, because the recursion was n deep. Here QUICKSELECT is written as a loop, so the worst case is merely slow. Writing a tail call as a loop did not fix the algorithm — it is still quadratic — but it converted a crash into a performance problem, which is a real improvement in its own right.

Worked Example: Does k Matter?

Mean of 25 runs, $n = 1,000,000$:

k/n	comparisons	$/n$
0.00	2,124,066	2.12
0.10	2,615,691	2.62
0.25	3,622,715	3.62
0.50	2,992,344	2.99
0.99	1,911,099	1.91
1.00	1,945,088	1.95

The ends are cheaper than the middle — about $2n$ for the minimum or maximum against about $3n$ for the median — but not dramatically so, and the variation is less than a factor of two.

The reason is a floor nothing can remove: the *first* partition examines all n elements whatever k is. Even finding the minimum by quickselect costs at least n comparisons in that first pass, and the remaining passes add about another n . So the answer to “is it cheaper to find the 10th smallest than the median?” is yes, by about a third — not by the factor people expect.

(For the minimum specifically, a single sweep costs exactly $n - 1$ comparisons and is the obvious choice. Quickselect is for the cases a sweep cannot do.)

11.4 Linear in the Worst Case

Quickselect’s expected bound is excellent and its worst case is quadratic. A deterministic algorithm achieves $\Theta(n)$ in the *worst* case.

Median of medians

To partition well you need a good pivot. To find a good pivot you could find the median — but that is the problem you are trying to solve. The way out is to find a pivot that is merely *guaranteed decent*, recursively.

1. Divide the n elements into $\lceil n/5 \rceil$ groups of 5.
2. Find each group’s median by sorting 5 elements — $O(1)$ each, $O(n)$ in total.

3. **Recursively select** the median of those $\lceil n/5 \rceil$ medians. Call it x .
4. Partition around x and recurse into the side containing the answer.

Why x is a decent pivot. At least half the groups have their median $\leq x$, and in each such group at least 3 of the 5 elements are $\leq$ that median. So at least $3 \cdot \frac{1}{2} \cdot \frac{n}{5} = \frac{3n}{10}$ elements are $\leq x$, and symmetrically $\frac{3n}{10}$ are $\geq x$. Therefore the recursive call in step 4 is on at most $\frac{7n}{10}$ elements — always, with no probability involved.

The recurrence is then

$$T(n) \leq \underbrace{T(n/5)}_{\text{step 3}} + \underbrace{T(7n/10)}_{\text{step 4}} + \Theta(n),$$

which the master theorem cannot touch (two different subproblem sizes). But $\frac{1}{5} + \frac{7}{10} = \frac{9}{10} < 1$, and substitution with the guess $T(n) \leq cn$ gives $T(n) \leq \frac{cn}{5} + \frac{7cn}{10} + an = \frac{9cn}{10} + an \leq cn$ whenever $c \geq 10a$. So $T(n) = O(n)$.

The groups of 5 are not arbitrary. With groups of 3 the sum becomes $\frac{1}{3} + \frac{2}{3} = 1$, the substitution fails, and the bound is $\Theta(n \lg n)$. The constant 5 is the smallest odd group size that makes the fractions sum to less than 1.

⚠ And yet almost nobody uses it

Median-of-medians is $\Theta(n)$ worst case, which is strictly better than quickselect's guarantee. Its constant is roughly 5–10× larger: it sorts every group of five, makes a second recursive call, and touches memory in a scattered pattern.

So in practice:

- **randomised quickselect** for ordinary work, at about $3.3n$ comparisons measured;
- **introselect** — quickselect that counts its iterations and switches to median-of-medians if progress is too slow — for library code. This is what `std::nth_element` does, and it is the same design as `std::sort`'s heapsort fallback in Chapter 9.

The pattern is worth naming, because it recurs: **use the algorithm with the better constant, and keep the one with the better guarantee as a fallback.** You get the common case fast and the worst case bounded.

! Common Pitfalls

- **Sorting to find one element.** Measured, 7–9.5× more expensive, growing with n .
- **Recurring into both sides.** That is quicksort. The deleted call is the entire algorithm.
- **Judging an expected bound from one run.** Measured runs ranged from $1.8n$ to $5.8n$ for a mean of $3.3n$.
- **Using a fixed pivot.** Sorted input then gives $3n^2/8$ comparisons, measured exactly.
- **Reaching for median-of-medians by default.** Its constant is 5–10× worse; it is a fallback, not a first choice.
- **Using groups of 3.** The fractions sum to 1 and the bound degrades to $\Theta(n \lg n)$.
- **Expecting selection to be much cheaper near the ends.** Measured $2.1n$ at $k = 0$ against $3.0n$ at the median — the first partition costs n regardless.

🔧 Try It Yourself: Selection Against Sorting

Reproduces all four tables. Expect twenty-five minutes.

```
// ch11_select.cpp -- finding the k-th smallest without sorting.
// Build: cl /O2 /EHsc /std:c++17 ch11_select.cpp      (see Appendix A)
```

```

// --- 1. Partition, exactly as in Chapter 9 -----

// --- 2. Quickselect: recurse into ONE side, not both -----
// This single change is the whole algorithm. Quicksort solves both
// subproblems and gets  $T(n) = 2T(n/2) + n = \Theta(n \lg n)$ .
// Quickselect discards one and gets  $T(n) = T(n/2) + n$ , which the
// master theorem settles as  $\Theta(n)$  -- case 3, the root dominating.
//
// Written as a loop rather than a recursion, because the recursive
// call is in tail position and there is no reason to spend a stack
// frame on it. That also removes Chapter 9's stack-overflow failure.
static int quickselect(std::vector<int>& a, size_t k, bool randomised) {
    size_t lo = 0, hi = a.size();
    for (;;) {
        if (hi - lo == 1) return a[lo];
        size_t p = randomised ? partition_random(a, lo, hi)
                               : partition_last(a, lo, hi);
        if (p == k) return a[p];
        if (k < p) hi = p;           // the k-th is on the left
        else      lo = p + 1;       // ... or on the right
    }
}

// --- 3. Comparisons: selection against sorting -----
// An EXPECTED value is being measured, so one run of a randomised
// algorithm is not an answer: single runs of this gave 2.52, 3.92,
// 3.73 and 2.73 comparisons per element, which says only that the
// answer is somewhere near 3. Average over 25 independent runs.

// --- 4. The same thing in seconds -----
// std::nth_element is the library's version: introselect, which is
// quickselect with a median-of-medians fallback, exactly as
// std::sort is quicksort with a heapsort fallback.

// --- 5. The worst case, for Chapter 9's reason -----
// A last-element pivot on sorted input shrinks the range by one each
// time:  $T(n) = T(n-1) + n = \Theta(n^2)$ . The loop form means it is
// slow rather than a crash, which is an improvement but not a fix.

// --- 6. Does k matter? -----
// The expected cost is smaller towards the ends -- but not
// dramatically, because the first partition costs  $n$  whatever  $k$  is.
// That first  $n$  is a floor nothing removes.

```

What to notice. The spread column in part 3. The mean is a stable 3.3 across four orders of magnitude, and individual runs vary by a factor of three. Both facts are properties of the algorithm, and quoting only the first would be misleading.

Then try to break it. Implement median-of-medians and race it against randomised quickselect on random input. Predict first which wins and by how much. Then build the input that makes quickselect quadratic and race them again. The crossover between “better constant” and “better guarantee” is the entire argument for introselect, and you will have measured both ends of it.

🔍 Checkpoint — Test Yourself

1. Quicksort's recurrence is $T(n) = 2T(n/2) + n$ and quickselect's is $T(n) = T(n/2) + n$. Identify the master-theorem case for each and explain where the $\lg n$ went.
2. Derive the $3n^2/8$ comparison count for a last-element pivot selecting the median of sorted input.

3. Why does median-of-medians use groups of 5 rather than 3? Show the arithmetic that fails.

Chapter Summary

- The i -th order statistic is the i -th smallest. Selection asks for one of them; sorting computes all of them and is asymptotically more expensive.
- Quickselect is quicksort with one recursive call deleted. That moves the recurrence from master-theorem case 2 ($\Theta(n \lg n)$) to case 3 ($\Theta(n)$): the tree collapses to a path and the work is geometric.
- Expected $\Theta(n)$ on every input with a random pivot; $\Theta(n^2)$ worst case.
- Measured: 3.3 comparisons per element for the median, flat across $n = 10^4$ to 10^7 — and individual runs ranged from $1.8n$ to $5.8n$.
- Measured: 7–9.5× faster than a full sort, a factor that grows with n ; and indistinguishable from `std::nth_element`.
- Measured: the naive pivot on sorted input used exactly $3n^2/8$ comparisons, matching the derivation to four decimal places.
- Median-of-medians is worst-case $\Theta(n)$ via $T(n) \leq T(n/5) + T(7n/10) + \Theta(n)$, where $\frac{1}{5} + \frac{7}{10} < 1$ is what makes the substitution close. Its constant is 5–10× worse, so it is used as a fallback, not a default.
- Cost varies with k by less than a factor of two, because the first partition costs n whatever k is.

Review Questions

1. (MCQ) Randomised quickselect's expected running time is: (a) $\Theta(\lg n)$; (b) $\Theta(n)$; (c) $\Theta(n \lg n)$; (d) $\Theta(n^2)$.
2. (MCQ) Median-of-medians partitions around an element guaranteed to exceed at least: (a) $n/5$; (b) $3n/10$; (c) $n/2$; (d) $7n/10$ of the elements.
3. (Short) Explain why deleting one recursive call changes the master-theorem case, and hence removes the logarithm.
4. (Short) State the trade-off between randomised quickselect and median-of-medians, and say what introselect does about it.
5. (Short) The measured spread was $1.8n$ to $5.8n$ for a mean of $3.3n$. What does that tell you about relying on an expected-time bound for a single latency-critical request?
6. (Applied) Prove that at least $n - 1$ comparisons are needed to find the minimum of n elements, and explain why you would use a single sweep rather than quickselect for $k = 1$.
7. (Applied) You need the 99th percentile latency from 50 million recorded request times, once per minute. Choose an algorithm, justify it with the measurements in this chapter, and say what would change your choice.

Heaps and Search Trees, Revisited for Analysis

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part III — Sorting, Selection and the Structures Beneath

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** why a binary heap’s height is exactly $\lfloor \lg n \rfloor$ and why that cannot fail.
- **Prove** that BUILD-HEAP runs in $\Theta(n)$, not $\Theta(n \lg n)$, and **identify** the step where the naive analysis is loose.
- **Contrast** a heap’s structural guarantee with a binary search tree’s dependence on insertion order (*CLO-2*).
- **State** the expected height of a randomly built BST and **assess** measured data against it.
- **Decide** between a heap, a balanced tree and a sorted array for a stated workload.

🔗 Before You Start

Data Structures built all of these; this chapter assumes you can recall what a heap and a BST are. Chapter 5 for the summation in Section 12.2, and Chapter 2 for the best/worst-case vocabulary.

🔑 Key Terms in This Chapter

binary heap • complete binary tree • heap property • sift down • build-heap • height • binary search tree • degenerate tree • internal path length • randomly built BST • balanced tree • rotation

⚠️ What this chapter is, and is not

It is **not** a reimplementaion of heaps and search trees. Appendix F records the boundary: *Data Structures* owns the structures and their operations; this course owns their analysis.

So the question here is never “how do you insert into a heap”. It is *why is the height what it is, what guarantees it, and what does a broken guarantee cost?* — and the answers divide these two structures sharply.

12.1 A Heap’s Height Is a Guarantee

📖 Binary Heap

A **binary heap** is a **complete binary tree** — every level full except possibly the last, which is filled from the left — satisfying the **heap property**: every node’s key is $\geq$ (max-heap) or $\leq$ (min-heap) its children’s.

It is stored in an array, with the children of index i at $2i + 1$ and $2i + 2$.

🏆 Theorem 12.1 — Heap height

A binary heap on n elements has height exactly $\lfloor \lg n \rfloor$.

Proof. A complete binary tree of height h has at least 2^h nodes — levels 0 to $h - 1$ are full, contributing $2^h - 1$, plus at least one node on level h — and at most $2^{h+1} - 1$, when every level is full. So

$$2^h \leq n \leq 2^{h+1} - 1 \implies h \leq \lg n < h + 1 \implies h = \lfloor \lg n \rfloor. \quad \blacksquare$$

💡 Why this cannot go wrong

Theorem 12.1 has no hypothesis about the data. Compare that with everything else in this book.

The height is $\lfloor \lg n \rfloor$ because the tree is *complete*, and it is complete because of the *array layout*: node i 's children are at $2i + 1$ and $2i + 2$, and a new element goes at index n . There is no insertion order that produces an unbalanced heap, because there is no representation in which an unbalanced heap could be stored.

The structure is enforced by the addressing scheme, not maintained by an algorithm. That is why heaps need no rebalancing, no rotations, no colouring and no amortised argument — and it is the sharpest contrast in this chapter with Section 12.3.

📊 Worked Example: Measured, Because It Costs Nothing To Check

`code/ch12_structures.cpp` walks from the last node to the root:

n	height	$\lfloor \lg n \rfloor$	
1,000	9	9	equal
100,000	16	16	equal
10,000,000	23	23	equal
100,000,000	26	26	equal

Exact at every size, as a theorem with no hypotheses should be. A hundred million elements are 26 levels deep.

12.2 Build-Heap Is Linear, and the Obvious Analysis Says Otherwise

☰ BUILD-HEAP(A, n)

```

1 for  $i \leftarrow \lfloor n/2 \rfloor - 1$  downto 0 do
2   SIFT-DOWN( $A, i, n$ )
3 end for
```

▷ nodes below $\lfloor n/2 \rfloor$ are leaves

Figure 12.1: BUILD-HEAP. The loop runs $n/2$ times and SIFT-DOWN costs $O(\lg n)$, so the obvious bound is $O(n \lg n)$. The obvious bound is loose, and Theorem 12.2 says by how much.

🏆 Theorem 12.2 — Build-Heap

BUILD-HEAP runs in $\Theta(n)$ time.

Proof. SIFT-DOWN on a node at height j — counting height upward from the leaves — costs $O(j)$, since it travels at most j levels down.

The key observation is that *most nodes are near the bottom*. In a heap of n elements there are at most $\lceil n/2^{j+1} \rceil$ nodes of height j . So the total cost is

$$\sum_{j=0}^{\lfloor \lg n \rfloor} \left\lceil \frac{n}{2^{j+1}} \right\rceil O(j) = O\left(n \sum_{j=0}^{\lfloor \lg n \rfloor} \frac{j}{2^j}\right).$$

The series $\sum_{j \geq 0} j/2^j$ converges — its value is exactly 2, from $\sum_{j \geq 0} jx^j = x/(1-x)^2$ at $x = 1/2$. Hence the total is $O(2n) = O(n)$, and it is $\Omega(n)$ because every element is examined, giving $\Theta(n)$. ■

💡 Where the naive analysis was loose

“ $n/2$ calls, each $O(\lg n)$, therefore $O(n \lg n)$ ” is valid — it is a correct upper bound. It is simply not tight, and the reason is worth internalising.

It charges *every* node the cost of the *worst* node. But half the nodes are leaves and cost nothing, a quarter are one level up and cost 1, an eighth cost 2. The expensive nodes are the rare ones, and the sum $\sum j/2^j$ converges precisely because the cost grows linearly while the population falls geometrically.

This pattern — a bound that is loose because it multiplies a worst case by a count — recurs throughout the book, and Chapter 17 is entirely devoted to it. Heapsort, by contrast, really is $\Theta(n \lg n)$: it calls SIFT-DOWN n times from the *root*, where the height really is $\lg n$ every time.

12.3 A Search Tree’s Height Is a Hope

⚠️ The same keys, the same code, two different structures

A BST has no array layout to enforce its shape. Its height is decided entirely by the *order the keys arrive in* — which is Chapter 2’s lesson, applied to a data structure rather than to a sort.

Insert $1, 2, 3, \dots, n$ in that order and every key is larger than everything before it, so every key goes down the right spine. The result has height n : it is a linked list that costs more to traverse than a linked list, because the pointers are scattered.

📊 Worked Example: Measured: Random Keys Against Sorted Keys

n	$\lg n$	random h	$h/\lg n$	sorted h	sorted h/n
1,000	10.0	21	2.11	1,000	1.000
8,000	13.0	27	2.08	8,000	1.000
64,000	16.0	37	2.32	64,000	1.000
256,000	18.0	44	2.45	256,000	1.000

The sorted column is n exactly — $h/n = 1.000$ at every size. Not “about n ”; exactly n .

The random column is about $2.3 \lg n$, which is a tree. At $n = 256,000$: height 44 against height 256,000. A factor of 5,800, from nothing but the order the same keys were handed over in.

💡 The constants for a randomly built BST

Two classical results, both asymptotic:

- **Expected height** $\sim 4.311 \ln n - 1.953 \ln \ln n + O(1)$ (Reed, 2003). The leading term is $2.988 \lg n$.
- **Expected node depth** $\sim 2 \ln n - 3 + O(1)$. The leading term is $1.386 \lg n$, and this is the average cost of a successful search.

Note that “randomly built” means the keys were *inserted* in random order. It does not mean the keys are random — sorted keys inserted in sorted order is the disaster above, and the keys there were as random as you like.

📊 Worked Example: The Constants, Measured Honestly

Mean of 25 independently shuffled trees. Both measured columns are divided by $\lg n$.

n	height	corrected	asympt.	mean depth	corrected	asympt.
2,000	2.324	2.627	2.988	1.231	1.113	1.386
8,000	2.311	2.657	2.988	1.242	1.155	1.386
32,000	2.459	2.683	2.988	1.276	1.186	1.386
128,000	2.461	2.704	2.988	1.278	1.209	1.386

Quoting only the asymptotic constants would have made the measurement look wrong. Measured height/ $\lg n$ is 2.46 against an asymptote of 2.99 — an 18% shortfall, which looks like a failed prediction until you include the $-1.953 \ln \ln n$ term. At $n = 128,000$, $\ln \ln n = 2.46$, and that term alone subtracts 0.28 from the ratio.

The mean-depth column is the cleaner story. Measured $1.231 \rightarrow 1.278$, rising steadily towards the asymptote 1.386, and bracketed above the corrected form $(2 \ln n - 3)/\lg n$. It is converging, slowly, from below — and “slowly” here means like $1/\lg n$, so reaching within 1% of the asymptote would need n around 10^{30} .

The general lesson is about reading asymptotic constants. An asymptotic constant is a limit, and the lower-order terms it discards can dominate at every size you will ever run. When a measurement misses a theoretical constant by 20%, the first question is not “is the theory wrong” but “what did the $O(1)$ swallow”.

📊 Worked Example: What the Height Costs, in Seconds

100,000 successful searches against each tree:

n	random BST (ms)	sorted BST (ms)	ratio
2,000	6.65	532.05	80.0
8,000	10.02	1,886.29	188.3
16,000	9.40	3,790.47	403.3
32,000	11.72	7,618.81	650.1

The random tree’s time barely moves — 6.6 ms to 11.7 ms while n grows 16-fold — which is what $\lg n$ looks like on a clock.

The degenerate tree’s time doubles with n , and the ratio reaches $650\times$ at only 32,000 elements.

And it is worse than the $n/\lg n$ ratio predicts. At $n = 32,000$, $n/(2.4 \lg n) \approx 890$ steps against 44, a ratio of about 20 in *steps* — yet the measured time ratio is 650. The extra factor is memory: each step in the degenerate tree is a dependent pointer dereference to an unpredictable address, so the processor can neither prefetch it nor overlap it with the next. Chapter 1’s second lie — that all memory costs the same — charging interest again.

12.4 Choosing

💡 Three structures, one table

Operation	Sorted array	Binary heap	Unbalanced BST	Balanced BST
find min/max	$\Theta(1)$	$\Theta(1)$	$\Theta(h)$	$\Theta(\lg n)$
search	$\Theta(\lg n)$	$\Theta(n)$	$\Theta(h)$	$\Theta(\lg n)$
insert	$\Theta(n)$	$\Theta(\lg n)$	$\Theta(h)$	$\Theta(\lg n)$
delete min	$\Theta(n)$	$\Theta(\lg n)$	$\Theta(h)$	$\Theta(\lg n)$
in-order walk	$\Theta(n)$	$\Theta(n \lg n)$	$\Theta(n)$	$\Theta(n)$
worst-case h	—	$\lfloor \lg n \rfloor$ <i>guaranteed</i>	n	$O(\lg n)$ <i>guaranteed</i>

A heap is not a search structure. It gives $\Theta(1)$ access to the extremum and $\Theta(n)$ search for anything else — the heap property orders each node against its children only, never its siblings. Use it for priority queues (Chapter 19, Chapter 20) and nothing else.

An unbalanced BST is a gamble on insertion order, and the measured penalty for losing is $650\times$. Balanced variants — AVL, red-black, treaps — restore the guarantee by rotating on insertion, at a constant-factor cost. *Data Structures* covers the rotations; what this course adds is that the guarantee is worth paying for, and by how much.

⚠️ Common Pitfalls

- **Assuming BUILD-HEAP is $O(n \lg n)$.** It is $\Theta(n)$; the loose analysis charges every node the root’s cost.
- **Searching a heap.** It is $\Theta(n)$. Heaps order parents against children, not siblings.
- **Expecting a BST to be balanced.** Its height is decided by the insertion order, and sorted input gives height exactly n .
- **Testing a BST only with shuffled keys.** The disaster case is *ordered* input, which is what real data often is.
- **Comparing a measured constant with a bare asymptote.** The height ratio was 2.46 against an asymptote of 2.99 and the theory was right; the $\ln \ln n$ term accounted for the difference.
- **Predicting slowdown from step counts alone.** The measured $650\times$ came from about $20\times$ in steps and the rest from unpredictable memory access.
- **Reaching for a balanced tree when a sorted array would do.** If the data is static, binary search on an array is $\Theta(\lg n)$ with a far better constant and no pointers at all.

Try It Yourself: Height, Measured

Reproduces all four tables. Expect twenty-five minutes; part D is slow by design — it is timing a deliberately degenerate structure.

```
// ch12_structures.cpp -- heaps and search trees, analysed not rebuilt.
// Build: cl /O2 /EHsc /std:c++17 ch12_structures.cpp (see Appendix A)

// --- 1. A BST as three parallel arrays -----
// No pointers and no allocation: left[i], right[i] and key[i]. The
// shape is identical to a pointer tree, and the measurement is about
// shape, not about allocators.
void insert(int k) {
    if (root < 0) { root = make(k); return; }
    int cur = root;
    for (;;) {
        if (k < key[cur]) {
            if (left[cur] < 0) { left[cur] = make(k); return; }
            cur = left[cur];
        } else {
            if (right[cur] < 0) { right[cur] = make(k); return; }
            cur = right[cur];
        }
    }
}

// Iterative height, so a degenerate tree does not overflow the stack
// the way Chapter 9's quicksort did.

// --- 2. A heap's height is not a hope, it is a guarantee -----
// A binary heap is a COMPLETE tree stored in an array: node i has
// children 2i+1 and 2i+2. Completeness is not maintained by clever
// rebalancing -- it is maintained by the array layout, and cannot
// fail. Hence height = floor(lg n), exactly, always.

// --- 3. A BST's height depends entirely on the insertion order ----
// Same keys, same code, two orders. This is Chapter 2's lesson --
// the nature of the input, not its size -- applied to a structure
// rather than to a sort.

// --- 4. The constant, averaged -----
// Quoting only the leading constants (2.988 and 1.386) would make the
// measurement look wrong. Both have lower-order corrections that are
// still large at these sizes:
// height ~ 4.311 ln n - 1.953 ln ln n + O(1) (Reed, 2003)
// mean depth ~ 2 ln n - 3 + O(1)
// So the honest comparison is against the corrected forms, with the
// bare asymptote shown beside them to make the gap visible.

// --- 5. And what the height costs, in seconds -----
// Searching a degenerate BST is a linear scan with pointer chasing,
// which is worse than a linear scan of an array: same number of
// steps, none of them predictable to the prefetcher.
```

What to notice. Part 4 prints three columns for each quantity: measured, corrected prediction, and bare asymptote. That is deliberate. A table with only the first and last would have read as a 20% failure of a well-established theorem.

Then try to break it. Insert the keys in *bit-reversal* order — 0, 8, 4, 12, 2, 10, ... for $n = 16$ — and measure the height. Predict first. This order is neither sorted nor random, and it produces a *perfectly balanced* tree, height exactly $\lg n$: better than random. The lesson is that “not sorted” and “random” are not the same assumption, and only one of them is what the theorem requires.

? Checkpoint — Test Yourself

1. Explain why a heap cannot become unbalanced, in terms of its representation rather than its operations.
2. The naive bound for BUILD-HEAP is $O(n \lg n)$ and the true answer $\Theta(n)$. Identify precisely which step of the naive argument is loose, and evaluate $\sum_{j \geq 0} j/2^j$.
3. A colleague's BST benchmark shows excellent performance. What single question would you ask about the test data (*CLO-2*)?

✓ Chapter Summary

- A binary heap's height is exactly $\lfloor \lg n \rfloor$, guaranteed by the array layout rather than maintained by an algorithm. Measured exact at every size up to 10^8 .
- BUILD-HEAP is $\Theta(n)$, not $\Theta(n \lg n)$: most nodes are near the leaves, and $\sum_{j \geq 0} j/2^j = 2$ converges. The naive bound is correct but loose — the pattern Chapter 17 generalises.
- A BST has no such guarantee. Its height is decided by the insertion order: measured $2.3 \lg n$ for shuffled keys and exactly n for sorted ones.
- Measured: 100,000 searches took 11.7 ms in a random tree and 7.6 seconds in a degenerate one at $n = 32,000$ — a factor of 650, of which only about 20 is step count and the rest unpredictable memory access.
- Measured height/ $\lg n$ was 2.46 against an asymptote of 2.99; the $-1.953 \ln \ln n$ correction accounts for the gap. Asymptotic constants are limits, and the discarded terms can dominate at every realistic size.
- A heap is a priority queue, not a search structure: $\Theta(1)$ to the extremum and $\Theta(n)$ to anything else.

✍ Review Questions

1. (MCQ) BUILD-HEAP on n elements runs in: (a) $\Theta(\lg n)$; (b) $\Theta(n)$; (c) $\Theta(n \lg n)$; (d) $\Theta(n^2)$.
2. (MCQ) Inserting $1, 2, 3, \dots, n$ into an unbalanced BST gives height: (a) $\lfloor \lg n \rfloor$; (b) $\Theta(\lg n)$; (c) $\Theta(\sqrt{n})$; (d) n .
3. (Short) Explain why a heap needs no rebalancing while a BST does, in terms of what enforces the shape.
4. (Short) Why is searching a heap $\Theta(n)$ despite its height being $\lg n$?
5. (Short) State what “randomly built BST” means, and give a key set that is random but produces a degenerate tree.
6. (Applied) Prove that a heap on n elements has at most $\lceil n/2^{j+1} \rceil$ nodes of height j , and use it to complete the $\Theta(n)$ bound for BUILD-HEAP.
7. (Applied) You must support insert, delete-min and lookup-by-key on up to 10^7 records arriving in timestamp order. Choose a structure, justify it against the table in Section 12.4, and say which measurement from this chapter most influenced you.

Hashing, and What Constant Time Actually Promises

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part III — Sorting, Selection and the Structures Beneath

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **State** the simple uniform hashing assumption and **explain** what the $\Theta(1)$ claim depends on (*CLO-2*).
- **Derive** the expected probe length $1 + \alpha/2$ for a successful search under chaining, and **verify** it by measurement.
- **Identify** the key patterns that defeat a division-method hash, and **explain** why they are common rather than exotic.
- **Describe** how an adversary degrades a hash table to a linked list, and **name** the defence.
- **Distinguish** expected, worst-case and amortised cost for hash table operations.

🔗 Before You Start

Chapter 2 for the distinction between an average over inputs and an expectation over an algorithm's coins — this chapter is that distinction again, and the resolution is the same one. Chapter 12 for the comparison with trees.

📖 Key Terms in This Chapter

hash function • hash table • collision • chaining • open addressing • load factor • simple uniform hashing • division method • multiplicative hashing • universal hashing • adversarial input • hash flooding

13.1 The Claim, and Its Fine Print

Every programmer knows a hash table is $O(1)$. That sentence has three things wrong with it, and this chapter is about all three.

📑 Chaining, Load Factor

A **hash table** with **chaining** stores n keys in m slots; each slot holds a list of the keys hashing to it. The **load factor** is $\alpha = n/m$, the average chain length.

Simple uniform hashing is the assumption that each key is equally likely to hash to any of the m slots, independently of the others.

🏆 Theorem 13.1 — Expected search cost under chaining

Under simple uniform hashing, an unsuccessful search examines $\Theta(1 + \alpha)$ elements in expectation, and a successful search $\Theta(1 + \alpha)$ — more precisely $1 + \alpha/2 - \alpha/2n$.

Proof. Unsuccessful. The key hashes to a slot whose expected chain length is $\alpha = n/m$, and the whole chain is scanned. With the $\Theta(1)$ to compute the hash and index, the cost is $\Theta(1 + \alpha)$.

Successful. Suppose the keys were inserted in order $x_1, \dots, x_n$ and we search for x_i , chosen uniformly. Appending at the head, the elements examined before reaching x_i are exactly those

inserted *after* it that landed in the same slot. For $j > i$, the probability that x_j shares x_i 's slot is $1/m$. So the expected number examined is

$$1 + \frac{1}{n} \sum_{i=1}^n \sum_{j=i+1}^n \frac{1}{m} = 1 + \frac{1}{nm} \sum_{i=1}^n (n-i) = 1 + \frac{n(n-1)}{2nm} = 1 + \frac{\alpha}{2} - \frac{\alpha}{2n}. \quad \blacksquare$$

⚠ Three things wrong with “a hash table is $O(1)$ ”

1. **It is $\Theta(1 + \alpha)$, not $\Theta(1)$.** Constant only if α is held constant — which requires resizing, which is Chapter 17's subject.
2. **It is an *expectation*, not a bound.** The worst case is $\Theta(n)$: all keys in one slot. Section 13.4 produces exactly that on purpose.
3. **The expectation is over an assumption about your data.** Simple uniform hashing is not a property of the hash function alone; it is a property of the hash function *together with the keys*. Section 13.3 shows a perfectly reasonable hash failing on perfectly reasonable keys.

13.2 Measured: the Assumption, and the Prediction

📊 Worked Example: Does the Distribution Look Like the Analysis Assumes?

With n keys scattered uniformly into m slots, a slot's load is $\text{Binomial}(n, 1/m)$, which for large m is $\text{Poisson}(\alpha)$. At $\alpha = 1$ the predicted number of slots with chain length L is $m e^{-1}/L!$. `code/ch13_hashing.cpp` with $n = m = 1,000,000$ and a multiplicative hash:

chain length	slots observed	Poisson(1) predicts	ratio
0	368,092	367,879	1.001
1	367,372	367,879	0.999
2	184,047	183,940	1.001
3	61,795	61,313	1.008
4	15,035	15,328	0.981
5	3,054	3,066	0.996
6	510	511	0.998

The whole distribution matches, not just its mean. Seven ratios, all within 2% of 1. This is the strongest form of evidence for a modelling assumption: not that the average came out right, but that the shape did.

The longest chain was 9, against $\lg n / \lg \lg n = 4.6$ — the classical order of the maximum load. So even with a million keys and a good hash, *some* lookup costs nine probes while the average costs 1.5. An expectation is not a guarantee, and tail latency lives in that gap.

📊 Worked Example: The One Knob: Load Factor

Theorem 13.1 predicts $1 + \alpha/2$ probes for a successful search, with no fitted constants. $m = 2^{20}$, keys sampled uniformly at random from the table:

α	mean probes measured	$1 + \alpha/2$ predicted	lookup (ms)
0.25	1.125	1.125	8.13
0.50	1.252	1.250	10.45
1.00	1.500	1.500	9.18
2.00	1.996	2.000	10.72
4.00	2.998	3.000	12.97
8.00	5.001	5.000	16.24

Six predictions, six matches, to three decimal places. This is as close as an analysis and a measurement get.

And note what the time column does *not* do. Probes rise by $4.4\times$ from $\alpha = 0.25$ to $\alpha = 8$; time rises by only $2.0\times$. Longer chains are contiguous-ish in memory and cheap to walk, while the initial slot access is a cache miss paid once however short the chain is. The step count and the clock disagree again, in the opposite direction from Chapter 12.

A caution about how this was measured. The first version of this experiment probed the *first* 200,000 keys inserted. Those sit at the front of their chains, so it reported 1.096 probes at every load factor — a perfectly flat column that looked like a result. The fix was to sample keys uniformly at random. A benchmark that samples its inputs in insertion order is measuring insertion order.

13.3 When the Assumption Quietly Fails

Simple uniform hashing is an assumption about the *relationship* between the hash and the keys. Real keys are not random: they are IDs, timestamps, pointers, offsets into arrays of structs — and they very often share a stride.

Worked Example: A Reasonable Hash Meeting Reasonable Keys

Keys 0, 64, 128, 192, ... — which is what the addresses of an array of 64-byte records look like — into a table of $m = 2^{20}$ slots, which is what “round the capacity up to a power of two” gives you. One million keys.

hash	mean probes	longest chain	slots used
$k \bmod m$	31.0	62	16,384
mix, then $\bmod m$	1.5	8	644,230

The division hash used 16,384 of its 1,048,576 slots — 1.6% — and left the other 98.4% empty. Mean probes 31 instead of 1.5: a $20\times$ slowdown, and the longest chain is 62.

The arithmetic is not subtle. $\gcd(64, 2^{20}) = 64$, so $k \bmod 2^{20}$ for k a multiple of 64 can only ever produce multiples of 64. Taking the modulus by a power of two keeps only the low bits, and the low six bits of these keys are always zero. The hash discarded the only part of the key that varied.

Neither choice was foolish. “Table size a power of two” makes the modulus a bitmask, which is fast and universal practice. “Hash is the key modulo the table size” is the textbook division method. Each is defensible; the combination, meeting keys with a stride of 64, is a $20\times$ slowdown nobody wrote down.

The defence is in the second row: destroy the key's structure before taking the modulus. The mix used here is MurmurHash3's finaliser — three shift-xor steps and two multiplications — and it moves high bits down into low ones, so no bit of the key is ignored.

💡 Rules that follow

- **Never use $k \bmod m$ with m a power of two.** It is a bitmask: it reads the low bits and throws the rest away.
- **If you use the division method, make m prime** and not close to a power of 2 — then a stride s collides only if s is a multiple of m .
- **Better: mix, then mask.** A multiplicative finaliser then $\&(m-1)$ gives you the fast mask *and* uses every bit.
- **Hash the whole key.** A hash that reads only a prefix of a string collides on every key sharing that prefix — and URLs, file paths and identifiers share prefixes constantly.

13.4 An Adversary

Section 13.3's collisions were an accident. They can also be arranged.

🧮 Worked Example: Hash Flooding

If the hash function is fixed and public — as it is in every language's standard library — then the keys landing in any given slot can simply be computed. For $h(k) = k \bmod m$, the multiples of m all hash to slot 0. No cryptography, no timing attack: arithmetic.

n	random keys (ms)	chosen keys (ms)	ratio
2,000	0.01	0.62	64.0
8,000	0.06	10.19	162.3
16,000	0.13	41.95	313.5
32,000	0.45	147.27	324.2

The ratio grows with n — 64 to 324 — because the chosen-key case is $\Theta(n^2)$ for n lookups while the random case is $\Theta(n)$. The table has become a linked list, and every operation on it is a linear scan.

This is a real attack, not a thought experiment. It is called **hash flooding**: send a web server a few thousand form fields chosen to collide, and its parameter dictionary — every request — turns quadratic. Published against PHP, Python, Ruby, Java and Node in 2011–12, and the reason those languages now randomise their hash seeds at process start.

💡 Universal hashing, and why it is Chapter 9 again

The defence is to stop having a fixed hash function. Choose h at random, at run time, from a carefully designed family $\mathcal{H}$.

$\mathcal{H}$ is **universal** if for any two distinct keys $x \neq y$,

$$\Pr_{h \in \mathcal{H}} [h(x) = h(y)] \leq \frac{1}{m}.$$

That is, a randomly chosen member collides on any particular pair no more often than a uniformly random function would.

With a universal family, Theorem 13.1's expected bounds hold *for every input*, with the expectation taken over the *choice of hash function* rather than over an assumption about the

keys. An adversary who knows your source code still cannot choose colliding keys, because he does not know which h you drew.

This is precisely Chapter 9’s move. There, randomising the pivot converted “ $\Theta(n \lg n)$ if your data is unsorted” into “ $\Theta(n \lg n)$ expected, on every input”. Here, randomising the hash converts “ $\Theta(1)$ if your keys suit my hash” into “ $\Theta(1)$ expected, on every key set”. Same technique, same payoff: **an assumption about the world, which can be violated, becomes an assumption about a random number generator, which cannot.**

💡 Hash table or balanced tree?

	Hash table	Balanced BST
search, expected	$\Theta(1)$	$\Theta(\lg n)$
search, worst case	$\Theta(n)$ (or $\Theta(\lg n)$ with tree bins)	$\Theta(\lg n)$ <i>guaranteed</i>
ordered traversal	impossible without sorting	$\Theta(n)$, free
predecessor, range query	not supported	$\Theta(\lg n)$
needs	a good hash for <i>your</i> keys	only a comparison
degrades on	adversarial or structured keys	nothing

Hash tables win on raw lookup and lose everything about order. Where a worst case must be bounded — latency budgets, untrusted input — the tree’s guarantee may be worth the logarithm. Java’s `HashMap` hedges: chains longer than 8 become red–black trees, so the worst case is $O(\lg n)$ rather than $O(n)$.

❗ Common Pitfalls

- **Saying “ $O(1)$ ” without saying “expected”.** The worst case is $\Theta(n)$ and it is reachable on purpose.
- **$k \bmod m$ with m a power of two.** Measured: 1.6% of slots used and a $20\times$ slowdown on keys with a stride of 64.
- **Testing only with random keys.** Random keys make every hash look good. Real keys have structure; test with yours.
- **Treating the assumption as a property of the hash alone.** Simple uniform hashing is a claim about the hash *and* the key distribution together.
- **A fixed, public hash on untrusted input.** That is the hash flooding vulnerability, measured here at $324\times$.
- **Ignoring the maximum chain.** Measured 9 with a million keys and a good hash, against a mean of 1.5. Tail latency is not the average.
- **Benchmarking lookups in insertion order.** The first draft of this chapter’s experiment did, and reported a flat 1.096 probes at every load factor.
- **Forgetting that keeping α bounded requires resizing,** whose cost is amortised — Chapter 17.

Try It Yourself: What Voids the Guarantee

Reproduces all four tables. Expect thirty minutes; part C builds tables holding up to eight million keys.

```
// ch13_hashing.cpp -- what  $O(1)$  actually promises, and what voids it.
// Build: cl /O2 /EHsc /std:c++17 ch13_hashing.cpp (see Appendix A)

// --- 1. A chaining hash table, as plainly as possible -----
// Buckets of vectors. No resizing, no tricks: the table size is fixed
// so the load factor is exactly what the experiment says it is.

// --- 2. Two hash functions, both entirely reasonable -----
// "Modulo the table size" is the textbook division method and what
// most people write first. It is fine when the table size is prime
// and the keys unstructured, and catastrophic when the keys share a
// factor with the table size.
static unsigned h_mod(unsigned k, unsigned m) { return k % m; }

// A multiplicative mix (MurmurHash3's finaliser). It destroys
// structure in the key before taking the modulus, which is the whole
// job of a hash function.
static unsigned h_mix(unsigned k, unsigned m) {
    k ^= k >> 16; k *= 0x85ebca6bu;
    k ^= k >> 13; k *= 0xc2b2ae35u;
    k ^= k >> 16;
    return k % m;
}

// --- 3. The distribution the analysis assumes -----
// With  $n$  keys in  $m$  slots and perfect scattering, a slot's load is
// Binomial( $n$ ,  $1/m$ ), which for large  $m$  is Poisson( $\alpha$ ). The analysis
// of "expected  $O(1)$ " rests on that shape, so check the shape.

// --- 4. The same keys, a hash that meets the data's structure -----
// Real keys are not random: IDs, timestamps, pointers, struct
// offsets -- and they very often share a stride. Here every key is a
// multiple of 64, which is what an array of 64-byte records gives
// you, and the table size is a power of two, which is what every
// "round up to the next power of 2" gives you.

// --- 5. Load factor is the one knob you actually turn -----
// Sample the keys at RANDOM, not the first 200,000. The
// first-inserted keys sit at the front of their chains, so probing
// them reports 1.096 probes whatever the load factor is -- which is
// what the first draft of this program did, and the flat column was
// the clue.

// --- 6. An adversary, who does not need to be clever -----
// If the hash is fixed and public, the keys that land in one slot can
// be enumerated. Here they are simply the multiples of  $m$ . No
// cryptography, no timing attack: arithmetic.
for (size_t i = 0; i < n; ++i)
    ak[i] = (unsigned)((i + 1) * m); // all hash to slot 0
```

What to notice. Part 3 compares a whole distribution against Poisson, not just a mean. Any broken hash can still produce the right average chain length — the average is n/m whatever the hash does, since every key lands somewhere. Only the *shape* distinguishes a good hash from a bad one, and part 4's table makes that concrete: both rows have the same mean chain length by definition, and one of them is $20\times$ slower.

Then try to break it. Change the table size in part 4 from 2^{20} to the prime 1,048,573 and re-run with the plain $k \% m$ hash. Predict first. The stride-64 keys should now scatter well,

because 64 and 1,048,573 are coprime — which tells you that the division method is not the problem, the *power of two* is.

🔍 Checkpoint — Test Yourself

1. State the simple uniform hashing assumption, and explain why it is a claim about the hash function *and* the keys rather than about the hash alone (*CLO-2*).
2. Derive the expected probe count $1 + \alpha/2$ for a successful search, and check it against the measured 2.998 at $\alpha = 4$.
3. The measured mean chain length was identical for both hash functions in Section 13.3 by definition. What quantity did distinguish them, and why is it the right one to look at?

📋 Chapter Summary

- A chaining hash table's search cost is $\Theta(1 + \alpha)$ *expected*, under the simple uniform hashing assumption. Each of those three qualifications can fail.
- Measured: with a good hash, the chain-length distribution matched Poisson(1) to within 2% across seven lengths — the shape, not just the mean. Longest chain 9 against a mean of 1.5.
- Measured: probes per successful search were 1.125, 1.252, 1.500, 1.996, 2.998, 5.001 against a predicted $1 + \alpha/2$ of 1.125, 1.250, 1.500, 2.000, 3.000, 5.000.
- Measured: $k \bmod 2^{20}$ on keys with a stride of 64 used 1.6% of the slots and cost 31 probes instead of 1.5 — because a power-of-two modulus keeps only the low bits, and those keys' low bits are constant.
- Measured: deliberately chosen colliding keys made lookups $324\times$ slower at $n = 32,000$, and growing. This is hash flooding, a published attack against every major language.
- Universal hashing — choosing h at random at run time — restores the expected bounds for *every* input, exactly as randomising the pivot did in Chapter 9.
- Use a tree where the worst case must be bounded or where order matters; a hash table buys lookup speed by giving up order entirely.

🔧 Review Questions

1. (MCQ) The expected number of probes for a successful search in a chaining table with load factor α is about: (a) 1; (b) α ; (c) $1 + \alpha/2$; (d) $\lg \alpha$.
2. (MCQ) Hashing with $h(k) = k \bmod m$ where $m = 2^{20}$ is a poor choice because: (a) the modulus is slow; (b) it uses only the low 20 bits of the key; (c) it cannot hash negative keys; (d) it is not deterministic.
3. (Short) Give the three qualifications hidden in the claim “hash table lookup is $O(1)$ ”.
4. (Short) Explain hash flooding and the defence, and relate it to randomised quicksort.
5. (Short) Why does comparing the chain-length distribution with Poisson test a hash function better than comparing mean chain lengths?
6. (Applied) Your keys are file paths that nearly all begin `/var/log/app/`. Explain what will happen with a hash that reads the first 16 characters, and give two fixes.
7. (Applied) A service parses untrusted JSON into a hash map and shows $300\times$ latency spikes under some requests. Diagnose it, state the measurement that would confirm your diagnosis, and give the fix.

PART IV

The Design Paradigms

Divide-and-conquer splits a problem into independent pieces. Part IV is what to do when the pieces are not independent — when a local choice can be trusted, when it cannot and the subproblems overlap, and how to cost a sequence of operations rather than one.

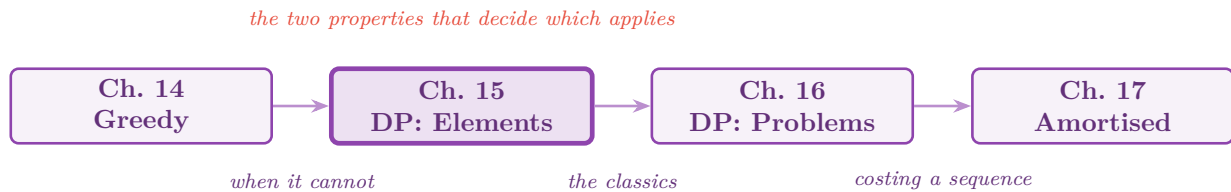


Figure P4.1: Reading path through Part IV. Chapter 15 is the one to understand rather than memorise: the two properties in it decide whether greedy or dynamic programming applies at all.

Greedy Algorithms

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part IV — The Design Paradigms

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Define** the greedy strategy and **apply** it to a stated problem (*CLO-6*).
- **State** the greedy-choice property and optimal substructure, and **explain** why both are needed.
- **Prove** a greedy algorithm optimal by an exchange argument.
- **Construct** a counter-example showing a plausible greedy rule is not optimal.
- **Describe** Huffman coding and **relate** its output to the entropy bound.

🔗 Before You Start

Chapter 7 for the baseline and the oracle technique. Nothing else in Part II is required, though Chapter 12's heap is the data structure Huffman's algorithm runs on.

📖 Key Terms in This Chapter

greedy algorithm • greedy-choice property • optimal substructure • exchange argument • activity selection • counter-example • Huffman code • prefix code • entropy • matroid

14.1 Commit and Never Reconsider

📖 Greedy Algorithm

A **greedy algorithm** builds a solution one step at a time, at each step taking the option that looks best *right now*, by some local rule, and never revisiting that decision.

It is the simplest strategy in the book and the one most often wrong. What makes it interesting is that when it works, it is usually both the fastest algorithm available and the easiest to implement — and whether it works is decidable by a proof, not by testing.

💡 The two properties

A greedy algorithm is optimal when the problem has both of these.

1. **The greedy-choice property.** A globally optimal solution can be reached by making a locally optimal choice. More precisely: there exists an optimal solution that *contains* the greedy first choice. Note the “there exists” — greedy need not find *the* optimal solution, only *an* optimal one.
2. **Optimal substructure.** An optimal solution to the problem contains within it optimal solutions to the subproblems that remain after the choice.

Both are required and they are different. Optimal substructure alone gives you dynamic programming (Chapter 15); it is the greedy-choice property that lets you commit without exploring alternatives, and it is the one that usually fails. Chapter 15 returns to this comparison in full.

14.2 Activity Selection

Activity Selection

Given n activities, activity i occupying the half-open interval $[s_i, f_i)$, choose a largest set of mutually compatible activities — no two overlapping.

Three local rules suggest themselves, and they sound equally sensible:

- take the activity that **finishes earliest**;
- take the **shortest** activity;
- take the activity that **starts earliest**.

ACTIVITY-SELECT(s, f, n)

```

1 sort the activities so that  $f_1 \leq f_2 \leq \dots \leq f_n$ 
2  $A \leftarrow \{1\}; \quad k \leftarrow 1$   $\triangleright$  the first to finish is always taken
3 for  $m \leftarrow 2$  to  $n$  do
4   if  $s_m \geq f_k$  then
5      $A \leftarrow A \cup \{m\}$   $\triangleright$  compatible: commit, never reconsider
6      $k \leftarrow m$ 
7   end if
8 end for
9 return  $A$ 
```

Figure 14.1: ACTIVITY-SELECT, greedy by earliest finishing time. After the sort it is a single pass, so the whole algorithm is $\Theta(n \lg n)$.

Theorem 14.1 — Earliest finish is optimal

Let S be a set of activities and let a_1 be one with the earliest finishing time. Then some maximum-size set of mutually compatible activities contains a_1 .

Proof. This is an **exchange argument**: take any optimal solution and show it can be modified to contain the greedy choice without getting worse.

Let A be a maximum-size compatible set, and let a_j be the activity in A with the earliest finishing time. If $a_j = a_1$ there is nothing to prove. Otherwise consider

$$A' = (A \setminus \{a_j\}) \cup \{a_1\}.$$

A' is compatible. The activities of A other than a_j all start at or after f_j , because a_j finishes earliest in A and A is compatible. Since a_1 has the earliest finishing time in all of S , we have $f_1 \leq f_j$, so every other activity in A also starts at or after f_1 and does not overlap a_1 .

A' has the same size as A , since one element was exchanged for one. So A' is also a maximum-size compatible set, and it contains a_1 .

Optimal substructure then completes the induction: after committing to a_1 , the remaining problem is activity selection on $\{a_i : s_i \geq f_1\}$, an instance of the same problem, and the same argument applies to it. ■

Worked Example: Measured: the Three Rules, Against an Oracle

Rules 2 and 3 have no such proof. Rather than assert they are wrong, `code/ch14_greedy.cpp` checks all three against exhaustive search (Chapter 7’s oracle) on 20,000 random instances of 10 activities:

greedy rule	instances solved optimally
earliest finish time	100.0%
shortest activity	94.5%
earliest start time	62.1%

Read the middle row carefully. “Take the shortest” is right 94.5% of the time. Any amount of testing on random inputs would leave you confident in it, and it is wrong. A 94.5% algorithm is not a 94.5%-correct algorithm; it is an incorrect algorithm that is usually lucky.

Read the last row too. “Take whatever starts first” — the most natural rule of all, and the one a scheduler written in a hurry implements — is optimal barely three-fifths of the time.

And the top row is 100.0%, not 100%-ish, because it is a theorem. That is the difference between a proof and a benchmark, and it is the reason Theorem 14.1 is in this chapter.

14.3 When Greedy Fails: Coin Change

Worked Example: The Same Algorithm, Two Coin Systems

Make an amount from coins of given denominations, using as few coins as possible. Greedy: take the largest coin that fits, repeatedly.

coin system	greedy wrong on	first failure
{1, 5, 10, 25}	0 of 200 amounts	—
{1, 3, 4}	49 of 200 amounts	amount 6

The counter-example is six. Greedy takes 4, then 1, then 1 — three coins. The optimum is 3 + 3 — two coins. Taking the largest coin was the mistake, and no amount of care in the implementation can fix a wrong strategy.

The lesson is not that greedy is unreliable. It is that greediness is a property of the *problem instance*, not of the approach. The identical algorithm is provably optimal for {1, 5, 10, 25} — which is why it works for every currency you have handled — and provably wrong for {1, 3, 4}. You cannot tell which case you are in by looking at the code.

And when greedy fails, dynamic programming does not. Chapter 15’s method solves coin change optimally for any denomination set, in $\Theta(nW)$ — slower, and always right.

How to approach a greedy algorithm

1. **Write the greedy rule down precisely.** “Greedy” names a family; the three activity-selection rules are three different algorithms with three different success rates.
2. **Try to prove the greedy-choice property** by exchange argument. If the proof goes through, you are finished.

3. **If it does not, look for the counter-example** where the proof stuck. Counter-examples for greedy algorithms are usually tiny — $\{1, 3, 4\}$ and the amount 6.
4. **Never conclude from testing.** 94.5% looked like success.

14.4 Huffman Coding

Prefix Code

A **prefix code** assigns each symbol a binary string such that no codeword is a prefix of another. That property is what makes a code unambiguously decodable without separators: reading left to right, the first codeword you complete is the right one.

A prefix code corresponds exactly to a binary tree whose leaves are the symbols; the codeword is the path, with 0 for left and 1 for right. The cost of a tree T is

$$B(T) = \sum_c f(c) d_T(c),$$

the total encoded length, where $f(c)$ is c 's frequency and $d_T(c)$ its depth.

HUFFMAN(C)

```

1  $Q \leftarrow$  a min-priority queue of all symbols, keyed by frequency
2 for  $i \leftarrow 1$  to  $|C| - 1$  do
3   allocate a new node  $z$ 
4    $z.left \leftarrow \text{EXTRACT-MIN}(Q)$ 
5    $z.right \leftarrow \text{EXTRACT-MIN}(Q)$   $\triangleright$  the two rarest
6    $z.freq \leftarrow z.left.freq + z.right.freq$ 
7    $\text{INSERT}(Q, z)$ 
8 end for
9 return  $\text{EXTRACT-MIN}(Q)$   $\triangleright$  the root
```

Figure 14.2: HUFFMAN. The greedy choice is on line 5: the two *least* frequent symbols are merged first, which puts them deepest. With a binary heap the running time is $\Theta(n \lg n)$.

Lemma 14.2 — The greedy choice is safe

Let x and y be two symbols of lowest frequency in C . Then there is an optimal prefix code for C in which x and y have the same (maximal) depth and differ only in the last bit.

Proof. Let T be an optimal tree, and let a and b be two sibling leaves of maximum depth in T (they exist: the deepest leaf has a sibling, or the tree could be improved by promoting it). Assume without loss of generality $f(a) \leq f(b)$ and $f(x) \leq f(y)$.

Exchange x with a to form T' . The change in cost is

$$B(T) - B(T') = (f(a) - f(x))(d_T(a) - d_T(x)) \geq 0,$$

because $f(x) \leq f(a)$ (x is of lowest frequency) and $d_T(x) \leq d_T(a)$ (a is of maximal depth), making both factors non-negative. So $B(T') \leq B(T)$. Exchanging y with b in T' gives T'' with $B(T'') \leq B(T') \leq B(T)$ by the same argument.

Since T was optimal, $B(T'') = B(T)$, so T'' is also optimal — and in it x and y are sibling leaves at maximum depth. ■

Theorem 14.3 — Huffman’s algorithm is optimal

HUFFMAN produces a prefix code of minimum total length.

Proof. By induction on $|C|$. The base case $|C| = 2$ is immediate: both codewords have length 1 and no code can do better.

For the inductive step, let x, y be the two least frequent symbols and let C' be C with x and y replaced by a single symbol z of frequency $f(x) + f(y)$. By the inductive hypothesis, HUFFMAN produces an optimal tree T' for C' . Replacing z ’s leaf by an internal node with children x and y gives a tree T for C with

$$B(T) = B(T') + f(x) + f(y),$$

since x and y sit one level below where z was. The $+f(x) + f(y)$ is a constant independent of the tree, so minimising $B(T')$ minimises $B(T)$ over all trees in which x and y are siblings — and Lemma 14.2 says some optimal tree is of that form. Hence T is optimal. ■

Worked Example: Huffman Against the Entropy Bound

Shannon’s source coding theorem says no prefix code can average fewer than $H = -\sum_c p(c) \lg p(c)$ bits per symbol, and Huffman is always within one bit of that. Both halves are checkable.

text	symbols	entropy H	Huffman	saving vs 8-bit
English prose	22	3.9670	4.0130	49.8%
DNA (4 symbols)	4	1.9999	2.0000	75.0%
95% one symbol	6	0.4065	1.1202	86.0%
uniform bytes	256	7.9953	8.0000	0.0%

Every row satisfies $H < \text{Huffman} < H + 1$, as the theory requires. Three of the four are within 0.05 bits of the floor.

The third row is the interesting one. Entropy 0.4065, Huffman 1.1202 — a gap of 0.71 bits, nearly the whole slack the theorem allows. The reason is structural: Huffman assigns each symbol a *whole number* of bits, and the minimum is 1. When one symbol has probability 0.95 its information content is $-\lg 0.95 = 0.074$ bits, and Huffman must spend 1. It cannot do better, and the loss is a factor of 2.8.

That is the known limit of Huffman coding, and the reason arithmetic coding exists — it can spend fractional bits per symbol and approaches H arbitrarily closely. A greedy algorithm that is provably optimal *within its class* can still be far from the information-theoretic bound, because the class itself is the constraint. Compare Chapter 10, where merge sort came within 1% of its floor; here the restriction to whole bits costs 175%.

And the last row. With 256 equally likely bytes, $H = 8$ and Huffman produces exactly 8 bits per symbol — it saves nothing, correctly. Compression is possible only where there is structure to exploit, and a greedy algorithm cannot create any.

Common Pitfalls

- **Saying “use a greedy algorithm” without naming the rule.** Three rules for activity selection scored 100%, 94.5% and 62.1%.
- **Concluding optimality from testing.** “Take the shortest” was right in 94.5% of 20,000 random trials and is wrong.

- **Assuming greedy works because it works for coins.** It does — for $\{1, 5, 10, 25\}$. For $\{1, 3, 4\}$ it fails at the amount 6.
- **Confusing the two properties.** Optimal substructure alone gives dynamic programming. It is the greedy-choice property that permits committing without exploring.
- **Proving the wrong direction in an exchange argument.** You must show the *optimal* solution can be modified to contain the greedy choice — not that the greedy solution can be modified.
- **Expecting Huffman to reach the entropy.** It cannot spend fractional bits; measured, it was 175% above H on a highly skewed source.
- **Using Huffman on a uniform source.** Measured saving: 0.0%.

Try It Yourself: Three Rules, Two Coin Systems, One Code

Reproduces all three tables. Expect twenty-five minutes; part A runs 20,000 brute-force searches.

```
// ch14_greedy.cpp -- when a local choice can be trusted, and when not.
// Build: cl /O2 /EHsc /std:c++17 ch14_greedy.cpp (see Appendix A)

// --- 1. Activity selection, greedy by earliest finish -----
// Sort by finishing time and take anything compatible. The claim is
// that the activity finishing first is in SOME optimal solution --
// the greedy-choice property, and what the chapter proves.
static int greedy_finish(std::vector<Act> a) {
    std::sort(a.begin(), a.end(),
        [](const Act& x, const Act& y) {
            return x.finish < y.finish; });
    int count = 0, last = -1;
    for (const Act& x : a)
        if (x.start >= last) { ++count; last = x.finish; }
    return count;
}

// --- 2. Two plausible rules that are simply wrong -----
// Both "take the shortest" and "take the earliest starting" sound
// just as reasonable as rule 1. Neither is optimal, and the program
// finds the counter-examples by brute force rather than asserting it.

// --- 3. The oracle: every subset, checked -----
// Chapter 7's lesson. n <= 18 so 2^n is affordable, and it is the
// only way to be sure the greedy answers are right, not plausible.

// --- 4. Coin change: greedy, and the optimum it sometimes misses ---
// Greedy takes the largest coin that fits. For {1, 5, 10, 25} that is
// optimal; for {1, 3, 4} it is not, and the smallest counter-example
// is six. Dynamic programming (Chapter 15) gets it right for any
// system.
static int coins_greedy(std::vector<int> c, int amount) {
    std::sort(c.rbegin(), c.rend());
    int used = 0;
    for (int v : c) while (amount >= v) { amount -= v; ++used; }
    return amount == 0 ? used : -1;
}

// --- 5. Huffman coding -----
// Repeatedly merge the two least frequent symbols. The greedy choice
// is that the two rarest symbols can be put deepest, as siblings.
// Total encoded length = sum over leaves of freq * depth, which is
// also the sum of every internal node's frequency -- a neater way to
```

```
// compute it than walking the tree.

// --- 6. Is the greedy rule actually optimal? -----
// --- 7. Coin change: the same algorithm, two coin systems -----
// --- 8. Huffman against the entropy bound -----
// Shannon: no prefix code can average fewer than  $H$  bits per symbol,
// and Huffman is always within 1 bit of it. Both are checkable.
```

What to notice. Part 6’s middle column. A rule that is right 94.5% of the time is not 94.5% of a correct algorithm — it is wrong, and the 5.5% is where it will meet your data.

Then try to break it. Find the smallest instance on which “take the shortest” fails, by searching exhaustively over instances rather than sampling. Three activities suffice: $[0, 10)$, $[9, 12)$ and $[11, 20)$ — the shortest is the middle one, taking it forbids both others, and the optimum takes the outer two. Having the counter-example in hand is worth more than the 94.5%.

🔍 Checkpoint — Test Yourself

1. State the greedy-choice property and optimal substructure, and give a problem that has the second but not the first.
2. Greedy coin change fails for $\{1, 3, 4\}$ at the amount 6. Find the next amount at which it fails, and say why the counter-examples are small.
3. Huffman achieved 1.1202 bits per symbol against an entropy of 0.4065. Explain the gap without blaming the algorithm.

📋 Chapter Summary

- A greedy algorithm commits to the locally best choice and never reconsiders. It is optimal when the problem has both the greedy-choice property and optimal substructure.
- Optimality is proved by an *exchange argument*: show that some optimal solution can be modified to contain the greedy choice without getting worse.
- Measured: of three plausible activity-selection rules, earliest-finish was optimal on 100.0% of 20,000 random instances, shortest-first on 94.5% and earliest-start on 62.1%. Testing cannot distinguish a theorem from luck.
- Measured: greedy coin change is optimal for $\{1, 5, 10, 25\}$ and wrong on 49 of 200 amounts for $\{1, 3, 4\}$, first failing at 6. Greediness is a property of the instance, not of the code.
- Huffman’s algorithm is greedy — merge the two rarest symbols — and provably produces a minimum-length prefix code.
- Measured: Huffman lay in $[H, H + 1)$ on all four corpora. On a 95%-skewed source it used 1.1202 bits against an entropy of 0.4065, because it cannot spend fractional bits; on a uniform source it saved nothing, correctly.

🔧 Review Questions

1. (MCQ) The greedy-choice property states that: (a) the greedy solution is unique; (b) some optimal solution contains the greedy first choice; (c) every optimal solution is greedy; (d) the problem has no optimal substructure.
2. (MCQ) Huffman coding produces a code whose average length is: (a) always equal to the entropy; (b) at least the entropy and less than entropy +1; (c) always 8 bits; (d) unrelated to the entropy.
3. (Short) Define the greedy strategy and apply it to activity selection, stating the rule precisely (CLO-6).

4. **(Short)** Explain an exchange argument, and say which direction the modification goes.
5. **(Short)** Why is a greedy rule that succeeds on 94.5% of random instances not acceptable?
6. **(Applied)** Prove that greedy coin change is optimal for $\{1, 5, 10, 25\}$, or give the argument's outline and identify where it would fail for $\{1, 3, 4\}$.
7. **(Applied)** You must schedule jobs with deadlines and profits, one machine, each job taking unit time, to maximise profit. Propose a greedy rule, attempt an exchange argument, and either prove it or give a counter-example.

Dynamic Programming I: The Elements

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part IV — The Design Paradigms

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **State** the two properties a problem must have for dynamic programming to apply, and **test** a given problem for them.
- **Distinguish** overlapping subproblems from the independent subproblems of divide-and-conquer.
- **Convert** a naive recursion into a memoised and then a bottom-up solution, and **compare** the three (*CLO-6*).
- **Derive** the running time of a dynamic program from the number of subproblems and the work per subproblem.
- **Decide** between greedy and dynamic programming for a stated problem.

🔗 Before You Start

Chapter 5 for recursion trees, Chapter 14 for optimal substructure and the comparison that closes this chapter.

📖 Key Terms in This Chapter

dynamic programming • optimal substructure • overlapping subproblems • memoisation • top-down • bottom-up • subproblem graph • rod cutting • reconstruction • table

15.1 The Two Conditions

📋 When Dynamic Programming Applies

1. **Optimal substructure.** An optimal solution to the problem contains optimal solutions to subproblems.
2. **Overlapping subproblems.** A recursive solution revisits the *same* subproblems repeatedly — the total number of *distinct* subproblems is small, while the number of recursive calls is large.

Dynamic programming solves each distinct subproblem once and stores the answer.

💡 Condition 2 is what makes it a technique

Merge sort has optimal substructure: sorting an array optimally requires sorting its halves. Memoising merge sort would buy exactly nothing, because the two halves are *different* subproblems and no subproblem ever recurs.

That is the dividing line:

- **divide and conquer** — subproblems are independent and distinct, so recursion is the whole answer;

- **dynamic programming** — subproblems overlap, so recursion alone does exponential redundant work.

And unlike the greedy-choice property, overlap is *measurable*: instrument the recursion, count how many times each subproblem is solved, and look. That is what Section 15.3 does, and the numbers are startling.

15.2 Rod Cutting

Rod Cutting

A rod of length n can be cut into integer pieces. A piece of length i sells for p_i . Cut the rod to maximise total revenue (cutting is free).

Optimal substructure is easy to see: after the first cut of length i , the remaining rod of length $n - i$ must itself be cut optimally, or the whole solution could be improved. So, writing r_n for the best revenue,

$$r_n = \max_{1 \leq i \leq n} (p_i + r_{n-i}), \quad r_0 = 0.$$

CUT-ROD-NAIVE(p, n)

```

1 if  $n = 0$  then
2   return 0
3 end if
4  $q \leftarrow -\infty$ 
5 for  $i \leftarrow 1$  to  $n$  do
6    $q \leftarrow \max(q, p_i + \text{CUT-ROD-NAIVE}(p, n - i))$     ▷ re-solves  $n - i$  every time
7 end for
8 return  $q$ 
```

Figure 15.1: CUT-ROD-NAIVE: a direct transcription of the recurrence. It is correct, and it takes $\Theta(2^n)$ time, because line 5 re-solves subproblems that other branches have already solved.

Theorem 15.1 — The naive recursion is exponential

CUT-ROD-NAIVE makes exactly 2^n calls on an input of length n .

Proof. Let $T(n)$ be the number of calls, counting the top-level one. $T(0) = 1$, and for $n \geq 1$ the body makes one call on each of $n - 1, n - 2, \dots, 0$:

$$T(n) = 1 + \sum_{j=0}^{n-1} T(j).$$

We show $T(n) = 2^n$ by induction. The base case holds, $T(0) = 1 = 2^0$. Assuming it for all $j < n$,

$$T(n) = 1 + \sum_{j=0}^{n-1} 2^j = 1 + (2^n - 1) = 2^n. \quad \blacksquare$$

Worked Example: Measured: How Often Is Each Subproblem Solved?

code/ch15_dp1.cpp instruments the recursion and counts visits to each subproblem. At $n = 22$:

subproblem k	times solved
22	1
21	1
20	2
19	4
18	8
17	16
... doubling all the way down ...	
3	262,144
2	524,288
1	1,048,576
0	2,097,152
total calls	4,194,304

The subproblem “a rod of length 0” was solved **2,097,152** times. There is only one of it, and its answer is 0.

Total calls $4,194,304 = 2^{22}$ **exactly**, confirming Theorem 15.1 to the unit.

And there are only 23 distinct subproblems — the rod lengths 0 to 22. Four million calls to solve twenty-three problems. That ratio is the saving dynamic programming is about to collect, and it is why “overlapping subproblems” is a measurable condition rather than an aesthetic judgement.

15.3 Two Ways to Collect the Saving

💡 Memoisation and bottom-up

Top-down with memoisation. Keep the recursion exactly as written, and add a table. Before computing, check whether the answer is known; after computing, record it. One line of difference.

Bottom-up. Sort the subproblems by size and solve them in that order, so every subproblem’s dependencies are already in the table when it is reached. No recursion at all.

They have the same asymptotic running time, and in practice:

- memoisation is a smaller edit to working code and only solves the subproblems actually needed — which matters when the table is sparse;
- bottom-up has a smaller constant (no call overhead, no cache checks), touches memory in order, and **cannot overflow the stack** — which for large n is the deciding argument, as Chapter 9 showed.

🏆 Theorem 15.2 — The dynamic program

CUT-ROD-DP computes r_n in $\Theta(n^2)$ time and $\Theta(n)$ space.

☰ CUT-ROD-DP(p, n)

```

1 let  $r[0..n]$  be a new array
2  $r[0] \leftarrow 0$ 
3 for  $j \leftarrow 1$  to  $n$  do
4    $q \leftarrow -\infty$ 
5   for  $i \leftarrow 1$  to  $j$  do
6      $q \leftarrow \max(q, p_i + r[j-i])$   $\triangleright r[j-i]$  is already known
7   end for
8    $r[j] \leftarrow q$ 
9 end for
10 return  $r[n]$ 

```

Figure 15.2: CUT-ROD-DP, bottom-up. Two nested loops, so $\Theta(n^2)$ — and the only change from the recurrence is that subproblems are visited in increasing order of size, so each is solved once.

Proof. The outer loop runs n times and the inner loop at most $j \leq n$ times, with an $O(1)$ body, so the time is $O(n^2)$; it is $\Omega(n^2)$ because the inner loop runs exactly j times, giving $\sum_{j=1}^n j = n(n+1)/2$ iterations. The array r gives $\Theta(n)$ space.

Correctness is an induction on j : assuming $r[0], \dots, r[j-1]$ hold the optimal revenues for those lengths, line 6 evaluates $\max_i(p_i + r_{j-i})$, which by the recurrence is r_j . ■

💡 The running time formula worth memorising

time = (number of distinct subproblems) $\times$ (work per subproblem).

Rod cutting: n subproblems $\times O(n)$ choices each = $\Theta(n^2)$. This formula handles nearly every dynamic program in Chapter 16, and it is usually quicker than writing down a recurrence.

15.4 Measured

📊 Worked Example: The Explosion and the Collapse

Calls made by each version:

n	naive calls	memoised calls	ratio
10	1,024	56	18.3
14	16,384	106	154.6
18	262,144	172	1,524.1
22	4,194,304	254	16,513.0
26	67,108,864	352	190,650.2

The naive column is 2^n ; the memoised column grows like n^2 . Their ratio therefore grows like $2^n/n^2$ — and does, from 18 to 190,000 over sixteen units of n .

📊 Worked Example: In Seconds

<i>n</i>	naive (ms)	memoised (ms)	bottom-up (ms)	naive/bottom-up
20	4.113	0.0009	0.0004	10,281
24	72.090	0.0012	0.0005	144,180
28	1,130.981	0.0015	0.0006	1,884,968
32	16,640.171	0.0017	0.0006	27,733,618

At $n = 32$: 16.6 seconds against 0.6 microseconds. A factor of 27.7 million, for a change that adds one array and one `if`.

Bottom-up beats memoised by about $2.5\times$ at every size — the constant-factor advantage of no recursion and no cache checks, exactly as Section 15.3 claimed.

And the naive column’s ratios are 17.5, 15.7, 14.7 per four units of n , against the predicted $2^4 = 16$. The exponential is not a figure of speech.

📊 Worked Example: Where the Naive Version Cannot Go

Bottom-up alone, at sizes the recursion could not reach in the lifetime of the universe:

<i>n</i>	bottom-up (ms)
10,000	0.263
40,000	1.054
160,000	4.802

The times are *linear* in n , not quadratic, because the price table here is capped at 30 entries — so the inner loop runs at most 30 times rather than j times, and $\Theta(n^2)$ becomes $\Theta(30n)$. A reminder to check what the parameters actually are before quoting a bound: the algorithm is $\Theta(n \cdot \min(n, |p|))$, and which factor dominates depends on the instance.

At $n = 160,000$ the naive version would need about 2^{160000} calls.

15.5 Greedy or Dynamic Programming?

💡 The comparison, finally

	Greedy	Dynamic programming
needs	greedy-choice property <i>and</i> optimal substructure	optimal substructure <i>and</i> overlapping subproblems
decides	commits to one choice, never reconsiders	considers all choices, keeps the best
cost	usually $\Theta(n)$ or $\Theta(n \lg n)$	usually $\Theta(n^2)$ or worse
when it fails	silently returns a suboptimal answer	does not
proof obligation	an exchange argument	the recurrence is correct

Both require optimal substructure; that is the shared half. They differ in the second condition, and the second conditions are not opposites — a problem can have both, or neither.

The practical rule: try greedy first, because it is faster and simpler. Attempt the exchange argument. If it goes through, you are done; if it does not, the place it stuck usually hands you the counter-example, and dynamic programming is the fallback that always works. Coin change in Chapter 14 is exactly this story: greedy for $\{1, 5, 10, 25\}$, dynamic programming for $\{1, 3, 4\}$.

⚠ Reconstructing the solution, not just its value

Every algorithm in this chapter returns a *number* — the best revenue — not the cuts that achieve it. That is almost never what is wanted.

The fix is to store, alongside each $r[j]$, the choice i that achieved the maximum. Walking back from $j = n$ then produces the cuts. The cost is $\Theta(n)$ extra space and no extra time.

This matters more than it sounds, and Chapter 16 returns to it: the space-saving tricks that reduce a DP table from $\Theta(mn)$ to $\Theta(\min(m, n))$ work only if you do *not* need to reconstruct the solution, because reconstruction needs the whole table.

! Common Pitfalls

- **Memoising a problem with no overlap.** Merge sort gains nothing; the table is pure overhead.
- **Assuming optimal substructure.** It must be argued. Longest *simple* path in a graph does not have it, and the usual DP recurrence for it is simply wrong.
- **Forgetting the base case in the table.** $r[0] = 0$ is load bearing.
- **Recurring memoised at large n .** The first call descends n frames deep before any memo is written — Chapter 9's stack overflow, in a new costume. Bottom-up has no stack.
- **Returning only the value.** Store the choices too, or the answer is unusable.
- **Quoting $\Theta(n^2)$ without checking the parameters.** Measured, rod cutting with a 30-entry price table is linear.
- **Reaching for DP before trying greedy.** Greedy is faster when it works, and the attempt to prove it tells you whether it does.

🔧 Try It Yourself: Counting the Overlap

Reproduces all five tables. Expect twenty-five minutes; the naive version at $n = 32$ takes about seventeen seconds on its own.

```
// ch15_dp1.cpp -- overlapping subproblems, counted.
// Build: cl /O2 /EHsc /std:c++17 ch15_dp1.cpp (see Appendix A)

// --- 1. The naive recursion, instrumented -----
// Rod cutting: a rod of length n, a price p[i] for a piece of length
// i. Cut it to maximise revenue. The obvious recursion tries every
// first cut and recurses on the remainder.
static long long rod_naive(const std::vector<long long>& p, int n) {
    ++calls;
    if (n < (int)visits.size()) ++visits[n];
    if (n == 0) return 0;
    long long best = 0;
    for (int i = 1; i <= n && i < (int)p.size(); ++i)
        best = std::max(best, p[i] + rod_naive(p, n - i));
}
```

```

    return best;
}

// --- 2. The same recursion, with an answer cache -----
// One line of difference. Each subproblem is now solved once and
// looked up thereafter, which is why the running time collapses from
// exponential to quadratic.
static long long rod_memo(const std::vector<long long>& p, int n,
                        std::vector<long long>& memo) {
    ++calls;
    if (memo[n] >= 0) return memo[n];
    long long best = 0;
    if (n > 0)
        for (int i = 1; i <= n && i < (int)p.size(); ++i)
            best = std::max(best, p[i] + rod_memo(p, n - i, memo));
    return memo[n] = best;
}

// --- 3. Bottom-up: the same table, filled in dependency order -----
// No recursion and no cache lookups, just two loops. Identical
// answers, identical asymptotics, smaller constant -- and it cannot
// overflow the stack, which for large n matters (Chapter 9).
static long long rod_bottom_up(const std::vector<long long>& p, int n) {
    std::vector<long long> best(n + 1, 0);
    for (int j = 1; j <= n; ++j) {
        long long b = 0;
        for (int i = 1; i <= j && i < (int)p.size(); ++i)
            b = std::max(b, p[i] + best[j - i]);
        best[j] = b;
    }
    return best[n];
}

// --- 4. How often is each subproblem solved? -----
// This is the measurement that DEFINES "overlapping". If every
// subproblem were solved once, DP would buy nothing.

// --- 5. The explosion, and the collapse -----
// --- 6. And in seconds -----
// --- 7. All three agree, which is the point -----
// --- 8. Bottom-up scales where the others cannot -----

```

What to notice. Part 4's last line: subproblem 0 solved 2,097,152 times. Every one of those calls returns 0 after doing nothing. The whole of dynamic programming is the observation that this is avoidable.

Then try to break it. Remove the memo check from `rod_memo` but keep the memo *write*. Predict what happens to the call count before running it. The answer is that nothing improves at all — writing to a cache you never read is not memoisation, and the program will still be exponential. The lookup is the technique; the table is just where it keeps things.

🔍 Checkpoint — Test Yourself

1. State the two conditions for dynamic programming and give a problem satisfying the first but not the second.
2. CUT-ROD-NAIVE made $2^{22} = 4,194,304$ calls to solve 23 distinct subproblems. Derive the 2^n from the recurrence $T(n) = 1 + \sum_{j < n} T(j)$.
3. Give the running time of a dynamic program with mn subproblems each requiring a maximum over k choices, and apply the formula to rod cutting.

Chapter Summary

- Dynamic programming needs optimal substructure *and* overlapping subproblems. The second distinguishes it from divide-and-conquer, where subproblems are distinct and memoising buys nothing.
- Overlap is measurable. Measured for rod cutting at $n = 22$: subproblem 0 was solved 2,097,152 times, total calls were exactly 2^{22} , and there were 23 distinct subproblems.
- Memoisation keeps the recursion and adds a table; bottom-up sorts the subproblems by size and uses loops. Same asymptotics; bottom-up has the smaller constant and no stack.
- Time = (number of distinct subproblems) $\times$ (work per subproblem). Rod cutting: $n \times O(n) = \Theta(n^2)$.
- Measured: at $n = 32$ the naive version took 16.6 s and bottom-up 0.0006 ms — a factor of 27.7 million. Bottom-up beat memoised by about $2.5\times$ throughout.
- Greedy and dynamic programming both need optimal substructure. Try greedy first and attempt the exchange argument; dynamic programming is the fallback that always works.
- Store the choices, not only the values, or the answer cannot be reconstructed.

Review Questions

1. **(MCQ)** Dynamic programming requires: (a) optimal substructure only; (b) overlapping subproblems only; (c) both; (d) the greedy-choice property.
2. **(MCQ)** Memoising merge sort would: (a) make it $\Theta(n)$; (b) make it $\Theta(\lg n)$; (c) buy nothing, because its subproblems do not overlap; (d) make it exponential.
3. **(Short)** Explain the difference between overlapping and independent subproblems, with an algorithm of each kind.
4. **(Short)** Compare memoised top-down with bottom-up on three axes, and say when each is preferable (*CLO-6*).
5. **(Short)** Why does the space-saving trick of keeping only two rows of a DP table prevent reconstructing the solution?
6. **(Applied)** Write the recurrence for coin change with denominations $c_1, \dots, c_k$ and amount W , state the number of subproblems and the work per subproblem, and give the running time.
7. **(Applied)** A colleague memoises a recursive function and reports no speed-up at all. Give three distinct explanations, and the measurement that would distinguish them.

Dynamic Programming II: Classic Problems

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part IV — The Design Paradigms

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Derive** the recurrence for longest common subsequence and edit distance, and **implement** both (*CLO-6*).
- **Reduce** a dynamic program's space from $\Theta(mn)$ to $\Theta(\min(m, n))$, and **state** what that costs.
- **Formulate** 0/1 knapsack as a dynamic program and **analyse** its $\Theta(nW)$ running time.
- **Explain** why $\Theta(nW)$ is *pseudo-polynomial* rather than polynomial, in terms of input size.
- **Choose** a DP formulation by identifying the subproblem parameters.

🔗 Before You Start

Chapter 15 for the method. Chapter 2 is essential for Section 16.4 — the pseudo-polynomial argument is exactly that chapter's point about a number's size being its bit length.

🔑 Key Terms in This Chapter

longest common subsequence • edit distance • Levenshtein • alignment • 0/1 knapsack • matrix-chain multiplication • pseudo-polynomial • rolling array • reconstruction • subproblem parameters

16.1 Longest Common Subsequence

📖 Subsequence, LCS

A **subsequence** of a sequence is obtained by deleting zero or more elements without reordering the rest. ALRIT is a subsequence of ALGORITHM.

The **longest common subsequence** of X and Y is a longest sequence that is a subsequence of both. Note that it need not be contiguous — that would be the longest common *substring*, a different and easier problem.

💡 Finding the recurrence

Let $c[i][j]$ be the LCS length of the prefixes $X_{1..i}$ and $Y_{1..j}$. Consider the last characters.

If $x_i = y_j$, there is an LCS ending with that character — including it can never hurt — so $c[i][j] = c[i-1][j-1] + 1$.

If $x_i \neq y_j$, the LCS cannot end with both, so at least one of x_i, y_j is unused, and $c[i][j] = \max(c[i-1][j], c[i][j-1])$.

$$c[i][j] = \begin{cases} 0 & i = 0 \text{ or } j = 0, \\ c[i-1][j-1] + 1 & x_i = y_j, \\ \max(c[i-1][j], c[i][j-1]) & \text{otherwise.} \end{cases}$$

Both conditions of Chapter 15 hold. Optimal substructure is the argument above. Overlap is severe: $c[i-1][j]$ and $c[i][j-1]$ both depend on $c[i-1][j-1]$, so a naive recursion solves it twice, and the duplication compounds exponentially.

≡ LCS-LENGTH(X, Y, m, n)

```

1 for i ← 0 to m do
2   c[i][0] ← 0
3 end for
4 for j ← 0 to n do
5   c[0][j] ← 0
6 end for
7 for i ← 1 to m do
8   for j ← 1 to n do
9     if xi = yj then
10      c[i][j] ← c[i-1][j-1] + 1
11    else
12      c[i][j] ← max(c[i-1][j], c[i][j-1])
13    end if
14  end for
15 end for
16 return c[m][n]
```

Figure 16.1: LCS-LENGTH. mn subproblems, $O(1)$ work each, so $\Theta(mn)$ time — and $\Theta(mn)$ space, which Section 16.3 reduces.

🏆 Theorem 16.1 — LCS

LCS-LENGTH computes the LCS length of X and Y in $\Theta(mn)$ time and $\Theta(mn)$ space.

Proof. The nested loops perform mn iterations with $O(1)$ bodies, and the initialisation is $\Theta(m+n)$, giving $\Theta(mn)$ time; the table gives the space.

Correctness follows by induction on $i + j$. For $i = 0$ or $j = 0$ one prefix is empty and the LCS length is 0. For $i, j \geq 1$, assume the entries with smaller $i + j$ are correct. If $x_i = y_j$: any common subsequence of $X_{1..i}, Y_{1..j}$ not ending in that character can be extended by it, so some LCS does end in it, and removing it leaves an LCS of $X_{1..i-1}, Y_{1..j-1}$ — giving $c[i-1][j-1] + 1$. If $x_i \neq y_j$: an LCS cannot use both last characters, so it is an LCS of $X_{1..i-1}, Y_{1..j}$ or of $X_{1..i}, Y_{1..j-1}$, and the maximum of the two is correct. ■

📊 Worked Example: A Small Instance, Checkable by Eye

code/ch16_dp2.cpp on ALGORITHM and ALTRUISTIC:

LCS length = 5, the subsequence ALRIT; edit distance = 6.

Trace ALRIT through both words to confirm it is a subsequence of each, and satisfy yourself that no common subsequence of length 6 exists. Small instances you can check by hand are worth more than large ones you cannot.

16.2 Edit Distance

📖 Edit Distance

The **edit distance** (Levenshtein distance) between two strings is the minimum number of single-character insertions, deletions and substitutions that turn one into the other.

💡 The same table, three choices instead of two

Let $d[i][j]$ be the distance between $X_{1..i}$ and $Y_{1..j}$. To account for the last position there are exactly three possibilities:

$$d[i][j] = \min \begin{cases} d[i-1][j] + 1 & \text{delete } x_i, \\ d[i][j-1] + 1 & \text{insert } y_j, \\ d[i-1][j-1] + [x_i \neq y_j] & \text{substitute, or match free.} \end{cases}$$

with $d[i][0] = i$ and $d[0][j] = j$ — deleting or inserting everything.

LCS and edit distance are the same algorithm with a different cost table, which is worth noticing: once you can write one, you can write spell-checking, DNA alignment, **diff**, and fuzzy search. All of them are $\Theta(mn)$ with different weights on the three moves.

📊 Worked Example: Measured: Both Are Quadratic

Random strings over a 4-letter alphabet; doubling both lengths should quadruple the time.

n	LCS full (ms)	ratio	LCS 2-row (ms)	ratio	edit dist (ms)
500	1.4	—	0.7	—	1.3
1,000	5.5	4.08	3.2	4.65	5.2
2,000	24.4	4.43	14.6	4.59	22.1
4,000	107.0	4.39	61.2	4.19	82.4
8,000	403.6	3.77	240.2	3.93	320.3

Mean ratios 4.17 and 4.34 against a predicted 4. The two-row version is consistently about $1.7\times$ faster than the full table — not because it does less arithmetic, but because it touches $\Theta(n)$ memory instead of $\Theta(n^2)$ and stays in cache.

16.3 Saving Space, and What It Costs

💡 The rolling array

Row i of the LCS table depends only on row $i-1$ and on earlier entries of row i . So only two rows need exist at a time, and the space falls from $\Theta(mn)$ to $\Theta(n)$ — or $\Theta(\min(m, n))$, by iterating over the longer string in the outer loop.

At $m = n = 100,000$ that is the difference between 40 GB and 400 KB. It is not an optimisation; it is the difference between possible and impossible.

And here is the cost. Reconstructing the actual subsequence requires walking *backwards* through the table, which needs the whole table. With two rows you get the length and nothing else.

So the choice is:

- need the length only $\Rightarrow$ two rows, $\Theta(\min(m, n))$ space;

- need the sequence $\Rightarrow$ full table, $\Theta(mn)$ space;
- need the sequence and cannot afford the table $\Rightarrow$ Hirschberg's algorithm, which combines the rolling array with divide-and-conquer to reconstruct in $\Theta(mn)$ time and $\Theta(\min(m, n))$ space. It is the right answer and it is twice the code.

16.4 0/1 Knapsack, and a Word That Matters

0/1 Knapsack

Given n items, item i having integer weight w_i and value v_i , and a knapsack of integer capacity W , choose a subset of maximum total value whose total weight is at most W . Each item is taken whole or not at all — hence 0/1.

 KNAPSACK(w, v, n, W)

```

1 for cap  $\leftarrow$  0 to  $W$  do
2    $b[\text{cap}] \leftarrow 0$ 
3 end for
4 for  $i \leftarrow 1$  to  $n$  do
5   for cap  $\leftarrow W$  downto  $w_i$  do
6      $b[\text{cap}] \leftarrow \max(b[\text{cap}], b[\text{cap} - w_i] + v_i)$ 
7   end for
8 end for
9 return  $b[W]$ 
```

Figure 16.2: KNAPSACK, one row at a time. The inner loop counts *down*, which is what enforces 0/1: counting up would let item i be used more than once, solving the unbounded knapsack problem instead.

Theorem 16.2 — Knapsack

KNAPSACK computes the optimal value in $\Theta(nW)$ time and $\Theta(W)$ space.

Proof. There are nW subproblems — one per (item prefix, capacity) pair — each resolved by one comparison, giving $\Theta(nW)$ by the formula of Chapter 15. The single array gives $\Theta(W)$ space.

For the downward inner loop: when $b[\text{cap}]$ is updated it reads $b[\text{cap} - w_i]$, which at that moment still holds the value from iteration $i - 1$, because indices below cap have not yet been visited in this pass. So each item is considered at most once per capacity, as 0/1 requires. ■

Worked Example: Checked Against an Oracle

Chapter 7's technique again: 300 random instances with $n = 18$, where all 2^{18} subsets can be enumerated.

300 instances, 0 disagreements.

Worth the twenty lines. The downward loop is exactly the kind of detail that is easy to get backwards and produces plausible-but-wrong answers.

⚠️ $\Theta(nW)$ is not a polynomial running time

This is the most important sentence in the chapter, and it takes Chapter 2 to understand.

$\Theta(nW)$ *looks* polynomial: no exponents, no factorials. But a running time is polynomial when it is polynomial in the **size of the input**, and Chapter 2 established that a number's size is its *bit length*, not its value.

W is one number. Writing it takes $b \approx \lg W$ bits. So

$$\Theta(nW) = \Theta(n \cdot 2^b)$$

— **exponential in the length of the input**. Add one bit to W and the running time doubles.

An algorithm polynomial in the numeric value of its input but exponential in its length is called **pseudo-polynomial**. Chapter 22 explains why this distinction decides whether a problem counts as solved: 0/1 knapsack is NP-complete, and this algorithm is not a counter-example to that.

📊 Worked Example: The Claim, Measured

n fixed at 200. Only the capacity changes, and the capacity is a single number.

W	bits	time (ms)	ratio
65,536	17	16.4	—
131,072	18	34.9	2.13
262,144	19	64.1	1.84
524,288	20	134.0	2.09
1,048,576	21	275.9	2.06
2,097,152	22	630.5	2.29
4,194,304	23	1,138.3	1.81
8,388,608	24	2,278.9	2.00
16,777,216	25	4,540.1	1.99

Eight ratios, mean 2.02. Each row adds exactly one bit of input and doubles the running time.

Compare this table with Chapter 2's trial-division table, where adding two bits to a prime doubled the work, mean ratio 2.02. They are the same phenomenon, reached from opposite directions: there, an algorithm that looked exponential and was; here, an algorithm that looks polynomial and is not.

Extrapolate. A 64-bit capacity — an entirely ordinary thing to find in a struct — is 2^{39} times the last row: about 7×10^{11} seconds, or **22,000 years**. The algorithm is excellent for small capacities and useless for large ones, and no amount of optimisation changes that, because the table has nW entries and they must be written.

💡 How to find the subproblem parameters

Every DP in this chapter is defined by *what the subproblems are indexed by*, and finding that is the whole design task.

Problem	Subproblem indexed by	Count	Time
rod cutting	remaining length	n	$\Theta(n^2)$
LCS	two prefix lengths	mn	$\Theta(mn)$
edit distance	two prefix lengths	mn	$\Theta(mn)$
0/1 knapsack	item prefix, capacity	nW	$\Theta(nW)$
matrix chain	a range (i, j)	n^2	$\Theta(n^3)$

Matrix-chain multiplication is the one with $\Theta(n^3)$: there are $\Theta(n^2)$ ranges and each requires a maximum over $O(n)$ split points. The formula of Chapter 15 gives the time directly once the parameters are named.

! Common Pitfalls

- **Calling $\Theta(nW)$ polynomial.** It is pseudo-polynomial, and measured, one more bit of W doubles the time.
- **Counting the knapsack inner loop upwards.** That solves *unbounded* knapsack — each item reusable. Both are legitimate problems; they are not the same problem.
- **Confusing subsequence with substring.** LCS allows gaps; longest common substring does not and has a different recurrence.
- **Using the two-row trick and then trying to reconstruct.** The table is gone. Use the full table or Hirschberg's algorithm.
- **Getting the edit-distance base cases wrong.** $d[i][0] = i$, not 0 — deleting i characters costs i .
- **Skipping the brute-force oracle.** 300 instances at $n = 18$ cost seconds and caught nothing here, which is the point: it is cheap evidence that the loop direction is right.
- **Assuming a DP's space equals its time.** LCS is $\Theta(mn)$ time and can be $\Theta(\min(m, n))$ space.

🔧 Try It Yourself: Three Classics and One Surprise

Reproduces all four tables. Expect thirty minutes; part D takes about nine seconds in its last row.

```
// ch16_dp2.cpp -- the classic dynamic programs, and one nasty surprise.
// Build: cl /O2 /EHsc /std:c++17 ch16_dp2.cpp          (see Appendix A)

// --- 1. LCS: the full table, so the subsequence can be recovered ---
for (size_t i = 1; i <= m; ++i)
    for (size_t j = 1; j <= n; ++j)
        c[i][j] = (a[i-1] == b[j-1]) ? c[i-1][j-1] + 1
                                     : std::max(c[i-1][j], c[i][j-1]);

// --- 2. LCS in two rows -----
// Row i depends only on row i-1, so only two rows need to exist. The
// LENGTH survives; the subsequence does not, because reconstructing
// it needs the whole table. That is the trade, and it is typical.

// --- 3. Edit distance: the same shape, three choices per cell -----
// Insert, delete or substitute. The recurrence differs from LCS only
// in what the three predecessors cost.
cur[j] = std::min({prev[j] + 1,          // delete
                  cur[j - 1] + 1,        // insert
```

```

        prev[j-1] + (a[i-1] != b[j-1]))});

// --- 4. 0/1 knapsack: Theta(nW), and W is a number not a count -----
// The inner loop counts DOWN, which is what enforces 0/1: counting up
// would let item i be used more than once.
static long long knapsack(const std::vector<int>& w,
    const std::vector<long long>& v, int W) {
    std::vector<long long> best(W + 1, 0);
    for (size_t i = 0; i < w.size(); ++i)
        for (int cap = W; cap >= w[i]; --cap)
            best[cap] = std::max(best[cap], best[cap - w[i]] + v[i]);
    return best[W];
}

// --- 5. The same problem by brute force, as an oracle -----
// 2^n subsets. Only usable for n <= 22, which is exactly why DP
// exists -- and exactly why it is a trustworthy oracle.

// --- 6. LCS and edit distance are Theta(mn) -----
// --- 7. A worked instance, small enough to check by eye -----
// --- 8. Knapsack: DP against brute force -----

// --- 9. The surprise: doubling W doubles the time -----
// n is FIXED at 200 here. Only the capacity changes, and the capacity
// is one number. Writing 2W instead of W adds ONE BIT to the input
// and doubles the running time. Theta(nW) is polynomial in the VALUE
// of W and exponential in its LENGTH: pseudo-polynomial.

```

What to notice. Part 9's bits column is the whole lesson. The input is not getting appreciably bigger — it gains one bit per row, while the 200 items stay fixed — and the running time doubles each time.

Then try to break it. Change the inner loop of `knapsack` to count *up* and re-run part 8 against the brute-force oracle. It will disagree, and the answers will be too *large* — the algorithm now takes some items repeatedly. Then work out which problem the upward loop solves, and note that it is a perfectly good algorithm for a different question.

? Checkpoint — Test Yourself

1. Write the LCS recurrence and explain why the case $x_i = y_j$ does not need a maximum.
2. Explain why $\Theta(nW)$ is pseudo-polynomial, using the measured doubling table, and relate it to Chapter 2's trial division.
3. Give one problem for which the two-row space trick is appropriate and one for which it is not, and say what distinguishes them.

✓ Chapter Summary

- LCS and edit distance share one table shape: mn subproblems indexed by two prefix lengths, $O(1)$ work each, so $\Theta(mn)$. They differ only in the cost of the three moves.
- Measured: both quadratic, mean ratios 4.17 and 4.34 on doubling against a predicted 4; the two-row version was $1.7\times$ faster through cache behaviour alone.
- The rolling array cuts space from $\Theta(mn)$ to $\Theta(\min(m, n))$ and makes reconstruction impossible. Hirschberg's algorithm gets both, at twice the code.
- 0/1 knapsack is $\Theta(nW)$ with nW subproblems. The inner loop must count down, or it solves the unbounded problem instead.
- $\Theta(nW)$ is **pseudo-polynomial**: W is a value, its input size is $\lg W$ bits, so the time is $\Theta(n2^b)$.

- Measured with n fixed at 200: eight doublings of W gave time ratios of mean 2.02, each row adding one bit of input. A 64-bit capacity would take about 22,000 years.
- Find the subproblem parameters first; the running time follows from (count) $\times$ (work each).

Review Questions

1. **(MCQ)** The LCS of two strings of lengths m and n is computed in: (a) $\Theta(m+n)$; (b) $\Theta(mn)$; (c) $\Theta(mn \lg n)$; (d) $\Theta(2^n)$.
2. **(MCQ)** $\Theta(nW)$ for 0/1 knapsack is: (a) polynomial; (b) pseudo-polynomial; (c) logarithmic; (d) constant in W .
3. **(Short)** Derive the edit-distance recurrence, naming the three operations and their costs (*CLO-6*).
4. **(Short)** Why must the knapsack inner loop count downwards? State what the upward version computes.
5. **(Short)** Explain the trade-off between the full LCS table and the two-row version.
6. **(Applied)** Give the dynamic program for matrix-chain multiplication: state the subproblem parameters, the recurrence, the number of subproblems and the running time.
7. **(Applied)** A colleague proposes shipping the $\Theta(nW)$ knapsack solver for a logistics system where capacities are given in grams and can reach 10^8 . Using the measured table, estimate the running time, and propose two alternatives.

Amortised Analysis

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part IV — The Design Paradigms

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** why multiplying a worst-case operation cost by the number of operations can give a badly loose bound.
- **Apply** the aggregate, accounting and potential methods to a sequence of operations (CLO-3).
- **Prove** that n appends to a doubling array cost $\Theta(n)$ in total.
- **Distinguish** amortised cost from average-case and worst-case cost.
- **Judge** when an amortised guarantee is adequate and when it is not.

🔗 Before You Start

Chapter 2 for the three cases — amortised cost is a fourth thing and must be kept apart from all of them. Chapter 12 for the BUILD-HEAP analysis, which is the same looseness in miniature.

📖 Key Terms in This Chapter

amortised analysis • aggregate method • accounting method • credit • potential method • potential function • dynamic array • growth factor • table doubling • worst case versus amortised

⚠️ Why this chapter is here, when HEC does not name it

The published outline does not list amortised analysis. It is taught anyway, for three reasons, and the Course Specification in the front matter records them:

1. Chapter 13's claim that a hash table keeps α bounded depends on resizing, whose cost is amortised. Without this chapter that claim is an assertion.
2. Chapter 19's union-find is the standard example of a structure whose cost is *only* meaningful amortised.
3. CLO-3 asks for the time complexity of simple algorithms, and for a sequence of operations on a growing structure that is an amortised question. Answering it per-operation gives the wrong answer.

17.1 A Loose Bound

💡 The problem, stated

Appending to a dynamic array is almost always $O(1)$: write at the end, increment the length. Occasionally the buffer is full, and then the append costs $\Theta(n)$ — allocate a bigger buffer and copy everything.

So the worst case of a single append is $\Theta(n)$, and

$$n \text{ appends} \leq n \times \Theta(n) = \Theta(n^2).$$

That bound is **correct and useless**. It is correct because no append exceeds $O(n)$; it is useless because the expensive appends cannot all happen — after one, the buffer is twice as big and cannot fill again for another n appends.

Amortised analysis bounds the *total* cost of a sequence directly, without pretending every operation is the worst one.

 **Amortised Cost**

The **amortised cost** of an operation is the total cost of a worst-case sequence of n operations, divided by n .

It is a **worst-case** quantity, not an average over a distribution. No probability is involved and no assumption about the input is made — which is what distinguishes it sharply from the average case of Chapter 2 and from the expected time of Chapter 9.

 **Four costs, kept apart**

Cost	Averaged over	Assumes
worst case	nothing — it is a maximum	nothing
average case	a distribution on inputs	that distribution
expected	the algorithm’s own coin flips	nothing about the input
amortised	a <i>sequence</i> of operations	nothing at all

An amortised bound is as strong as a worst-case one: it holds for every sequence, with no probability anywhere. What it does *not* promise is that any individual operation is fast, and Section 17.4 measures exactly how slow one of them gets.

17.2 Three Methods, One Answer

 APPEND(A, x)

```
1 if A.len = A.cap then
2   cap' ← max(1, 2 · A.cap)
3   allocate B of size cap'
4   copy A[0 .. A.len] into B
5   A.buf ← B; A.cap ← cap'
6 end if
7 A[A.len] ← x; A.len ← A.len + 1
```

▷ the expensive step

Figure 17.1: APPEND with table doubling. Line 4 runs rarely, and when it does it costs $\Theta(n)$. The three methods below are three ways of saying that the rarity exactly pays for the cost.

The aggregate method: add it all up

🔗 Theorem 17.1 — Doubling array

Starting from an empty array, n calls to APPEND cost $\Theta(n)$ in total, so the amortised cost per append is $\Theta(1)$.

Proof. Copying happens only when the length reaches the capacity, which with doubling occurs at lengths 1, 2, 4, 8, ... — the powers of two up to n . The copy at length 2^k moves 2^k elements. So the total copying over n appends is

$$\sum_{k=0}^{\lfloor \lg n \rfloor} 2^k = 2^{\lfloor \lg n \rfloor + 1} - 1 < 2n.$$

Adding the n constant-time writes gives total cost under $3n = \Theta(n)$, and dividing by n gives $\Theta(1)$ amortised. ■

💡 The accounting method: pay in advance

Charge each append **3 units** of amortised cost, and spend them like this:

- 1 unit pays for writing the new element — the actual work;
- 1 unit is saved as **credit** on the new element;
- 1 unit is saved as credit on an element in the older half of the array.

When the array of capacity m fills, it last doubled at size $m/2$, so the $m/2$ most recent elements have each saved 2 units: m units of credit in all, exactly enough to pay for copying m elements.

The credit never goes negative, so the total charged ($3n$) is an upper bound on the total spent, giving $O(1)$ amortised. This is the same accounting a business does when it sets aside money each month against an annual bill.

💡 The potential method: a bank balance

Define a **potential function** Φ on the data structure's state — roughly, how much trouble it is currently storing up. For the dynamic array,

$$\Phi = 2 \text{ len} - \text{cap}.$$

The amortised cost of an operation is

$$\hat{c}_i = c_i + \Phi_i - \Phi_{i-1},$$

its real cost plus the change in potential.

Cheap append (no resize): $c_i = 1$, len rises by 1 so Φ rises by 2, giving $\hat{c}_i = 1 + 2 = 3$.

Expensive append (resize from m to $2m$, with $\text{len} = m$ before): $c_i = m + 1$. Before, $\Phi = 2m - m = m$; after, $\Phi = 2(m + 1) - 2m = 2$. So

$$\hat{c}_i = (m + 1) + 2 - m = 3.$$

Both cases give exactly 3. The potential was built up by the cheap operations and spent by the expensive one. Since $\Phi \geq 0$ always (the array is at least half full after any doubling) and $\Phi_0 = 0$, the total real cost is at most the total amortised cost, $3n$.

Why bother with a third method? Because the potential method is the one that generalises. The aggregate method needs you to see the whole sequence; the accounting

method needs a story about who pays. The potential method needs only a function, and it is how the union–find bound in Chapter 19 and the Fibonacci heap bound in Chapter 20 are proved.

17.3 Measured

 **Worked Example: The Total, Counted Exactly**

code/ch17_amortised.cpp counts every element copy.

n	doubling copies	$/n$	grow-by-one copies	$/(n^2/2)$
1,000	1,023	1.023	499,500	0.9990
8,000	8,191	1.024	31,996,000	0.9999
32,000	32,767	1.024	511,984,000	1.0000
64,000	65,535	1.024	2,047,968,000	1.0000

Doubling: copies/ n is **1.024, flat**. Theorem 17.1 predicted under 2; the actual constant is just over 1, because $\sum_{k < \lg n} 2^k = n - 1$ when n is a power of two.

Grow-by-one: exactly $n^2/2$. The quotient is $0.9990 \rightarrow 1.0000$, confirming $\sum_{k=1}^{n-1} k = n(n-1)/2$ to four decimal places.

So the $\Theta(n^2)$ bound was not paranoia — it is exactly right for the wrong growth policy. What amortised analysis shows is that it is *not* right for doubling, and the difference between the two policies is one line of code.

 **Worked Example: Amortised $O(1)$ Does Not Mean Every Operation Is Fast**

One million appends, doubling:

quantity	value
appends	1,000,000
reallocations	21
appends that copied anything	20 (0.0020%)
total copies	1,048,575 (1.049 per append)
most expensive single append	524,288 copies
worst case $\times n$ (the useless bound)	524,288,000,000

Read the last two rows together. The amortised cost is 1.049 copies per append. The worst single append copied 524,288 elements — half a million times the amortised figure.

Both numbers are true and they describe different things. If you are computing throughput, 1.049 is the number you want. If you are meeting a latency budget — a frame deadline, an interrupt handler, a trading system — then 1.049 is irrelevant and 524,288 is the number that will end your career.

This is the chapter’s main caution. Amortised $O(1)$ is a guarantee about totals. When individual operations must be bounded, the answer is a different structure — an incrementally resized array that copies a few elements on every append, trading a worse constant for a real worst-case bound.

And note how rare the expensive ones are: 20 of a million appends, 0.002%. A latency histogram with 1,000 samples would very likely contain none of them, and the system would look fine until it was not.

Worked Example: The Growth Factor Is a Real Decision

$n = 1,000,000$ appends under different growth factors:

factor	total copies	per append	wasted capacity
1.10	10,170,704	10.171	1.7%
1.25	4,489,047	4.489	12.2%
1.50	2,099,753	2.100	5.0%
2.00	1,048,575	1.049	4.9%
3.00	797,161	0.797	59.4%
4.00	349,525	0.350	4.9%

Every factor above 1 gives $\Theta(1)$ amortised — the sum is geometric with ratio $1/f$ and converges for any $f > 1$. The asymptotics cannot distinguish these rows at all.

What distinguishes them is the constant and the memory. A factor of 1.1 does ten times as much copying as doubling; a factor of 3 wastes 59% of its memory at this particular n . (The waste figures are lumpy because they depend on where n falls relative to the growth sequence — 2.0 and 4.0 both land near a power of two here.)

Several standard libraries use 1.5 rather than 2, and the reason is not in this table: with a factor of 2, the new block is always larger than the sum of all previously freed blocks, so the allocator can never reuse them. With 1.5 it eventually can. That is an argument about the allocator, not about the algorithm, and it is a good example of a decision that asymptotic analysis cannot reach.

Worked Example: And in Seconds

n	doubling (ms)	ratio	n	grow-by-one (ms)
1,000,000	3.5	—	20,000	84.1
4,000,000	14.7	2.02	40,000	350.5
16,000,000	58.2	1.93	80,000	1,432.8
32,000,000	119.3	2.05	160,000	5,759.3

Doubling's ratios average 2.03 — linear. Grow-by-one's average 4.09 — quadratic. At $n = 160,000$ the grow-by-one version takes 5.8 seconds; the doubling version handles 32 million elements in 119 milliseconds.

Note the two columns use different ranges of n . Doubling at $n = 160,000$ takes under a millisecond, which is too small to measure — the first version of this experiment reported ratios of 0.92, 1.19, 1.81 and 4.04 for a linear algorithm, all of it noise.

17.4 Where Else This Pattern Appears

💡 A list worth keeping

Structure	Operation	Worst case	Amortised
dynamic array	append	$\Theta(n)$	$\Theta(1)$
hash table with resizing	insert	$\Theta(n)$	$\Theta(1)$
binary counter	increment	$\Theta(\lg n)$	$\Theta(1)$
union-find	find	$\Theta(\lg n)$	$O(\alpha(n))$
splay tree	search	$\Theta(n)$	$\Theta(\lg n)$
BUILD-HEAP	whole operation	—	$\Theta(n)$ not $\Theta(n \lg n)$

The last row is Chapter 12's result, and it is the same phenomenon: multiplying a count by a worst case gave $O(n \lg n)$, and the truth was $\Theta(n)$ because the expensive cases are rare.

The union-find row is Chapter 19's, where $\alpha(n)$ is the inverse Ackermann function — below 5 for any n that can be written down. It is the most dramatic amortised bound in common use, and it is proved by the potential method.

⚠ Common Pitfalls

- **Multiplying worst case by count.** Measured, that bound was 524,288,000,000 against a true cost of 1,048,575 — a factor of half a million.
- **Calling amortised cost an average case.** There is no distribution and no probability; it is a worst-case bound on a total.
- **Promising low latency from an amortised bound.** The worst single append copied 524,288 elements.
- **Growing by a constant amount.** Measured exactly $n^2/2$ copies. The policy, not the structure, is what makes it quadratic.
- **A potential function that can go negative.** The argument requires $\Phi \geq \Phi_0$, or the total is not bounded.
- **Assuming the growth factor does not matter because the asymptotics agree.** Measured, 1.1 did ten times the copying of 2.0.
- **Benchmarking with too few samples to see the rare case.** 0.002% of appends were expensive; most histograms will miss them entirely.

🔧 Try It Yourself: The Rare Expensive Operation

Reproduces all five tables. Expect twenty-five minutes.

```
// ch17_amortised.cpp -- a sequence of operations, costed as a sequence.
// Build: cl /O2 /EHsc /std:c++17 ch17_amortised.cpp (see Appendix A)

// --- 1. A dynamic array with an adjustable growth policy -----
// Deliberately hand-rolled so every copy can be counted. "factor" is
// the multiplier on resize; a factor of 0 means "grow by one", which
// is the policy the quadratic analysis describes.
void push(int x) {
    if (len == cap) {
        size_t ncap = (factor <= 1.0) ? cap + 1
            : std::max<size_t>(1, (size_t)(cap * factor));
        if (ncap == cap) ncap = cap + 1;
    }
}
```

```

        std::vector<int> nb(ncap);
        for (size_t i = 0; i < len; ++i) { nb[i] = buf[i]; ++copies; }
        buf.swap(nb);
        cap = ncap;
        ++reallocs;
    }
    buf[len++] = x;
}

// --- 2. Total copies: the aggregate method, measured -----
// Doubling: the copies are 1 + 2 + 4 + ... + n/2 < n, so the total
// work for n appends is under 2n and the amortised cost per append is
// under 2. Grow-by-one: 1 + 2 + ... + (n-1) = n(n-1)/2.

// --- 3. Amortised O(1) does not mean every operation is cheap -----
// Print the cost of the single most expensive append, and how many
// appends are expensive at all. This is the distinction between
// amortised and worst-case that the chapter exists for.
    long long worst = *std::max_element(cost.begin(), cost.end());
    size_t expensive = 0;
    for (long long c : cost) if (c > 0) ++expensive;

// --- 4. The growth factor is a real engineering choice -----
// Larger factor: fewer copies, more wasted space. Smaller factor: the
// reverse. 1.5 is what several standard libraries use; 2.0 is what
// most textbooks describe.

// --- 5. And in seconds, against the real thing -----
// The two policies need different ranges. Doubling at n = 160,000
// takes under a millisecond, too small to measure -- the first
// version of this table reported ratios of 0.92, 1.19, 1.81 and 4.04
// for a linear algorithm, all of it noise.

```

What to notice. Part 3 prints `worst case x n` beside the true total. Those two numbers — 524 billion and 1.05 million — are the bound amortised analysis replaces and the answer it gives. The whole chapter is the gap between them.

Then try to break it. Add a growth factor of exactly 1.0 and predict what happens before running. The guard `if (ncap == cap) ncap = cap + 1` catches it and the policy degenerates to grow-by-one — but remove that guard and the capacity never increases, the `push` never terminates its `resize`, and you have written Chapter 4’s non-terminating loop again. A measure that fails to decrease is still the commonest way to hang a program.

🔍 Checkpoint — Test Yourself

1. Using the aggregate method, show that n appends to a doubling array perform fewer than $2n$ copies, and compare with the measured 1.024 per append.
2. Compute the amortised cost of an append under the potential function $\Phi = 2 \text{ len} - \text{cap}$, for both the cheap and the expensive case, and say why they must come out equal.
3. Give a situation in which an amortised $O(1)$ guarantee is inadequate, and say what the measured data in Section 17.3 tells you about it (*CLO-3*).

📋 Chapter Summary

- Multiplying a worst-case operation cost by the number of operations can be enormously loose, because the expensive operations cannot all happen.
- Amortised cost is the total cost of a worst-case *sequence*, divided by its length. It assumes nothing about inputs and involves no probability — unlike average-case or expected cost.

- Three methods: aggregate (sum it), accounting (charge extra and save credit), potential (define Φ and let $\hat{c} = c + \Delta\Phi$). The potential method is the one that generalises.
- n appends to a doubling array cost $\Theta(n)$: the copies are $1 + 2 + 4 + \dots < 2n$. Measured exactly 1.024 copies per append.
- Measured: grow-by-one costs exactly $n^2/2$ copies. The policy makes the difference, not the structure.
- Measured: of one million appends, 20 copied anything, the amortised cost was 1.049, and the worst single append copied 524,288 elements. Amortised $O(1)$ says nothing about any individual operation.
- Any growth factor > 1 gives $\Theta(1)$ amortised; measured, a factor of 1.1 did ten times the copying of 2.0, and 3.0 wasted 59% of its memory.

Review Questions

1. **(MCQ)** The amortised cost of an append to a doubling array is: (a) $\Theta(1)$; (b) $\Theta(\lg n)$; (c) $\Theta(n)$; (d) $\Theta(n^2)$.
2. **(MCQ)** Amortised analysis differs from average-case analysis in that it: (a) assumes a uniform input distribution; (b) assumes nothing about the input; (c) applies only to randomised algorithms; (d) gives a lower bound.
3. **(Short)** State the three methods of amortised analysis and give the one-line idea of each (*CLO-3*).
4. **(Short)** Explain why an amortised $O(1)$ guarantee may be unacceptable for a real-time system, citing the measured worst single append.
5. **(Short)** Why does any growth factor greater than 1 give $\Theta(1)$ amortised appends, while growing by a constant does not?
6. **(Applied)** Analyse the amortised cost of incrementing a binary counter n times, using both the aggregate and the potential method (take Φ = the number of 1 bits).
7. **(Applied)** A team reports their append-heavy service has a 99.99th-percentile latency $500\times$ its median. Explain the likely cause from this chapter, say what measurement would confirm it, and give two fixes with their trade-offs.

PART V

Graph Algorithms

Everything so far has been applied to arrays. Part V applies it to graphs, where one decision — how the graph is stored — changes every bound in the part, and where the design paradigms of Part IV reappear wearing different clothes.

sparsity changes every bound that follows

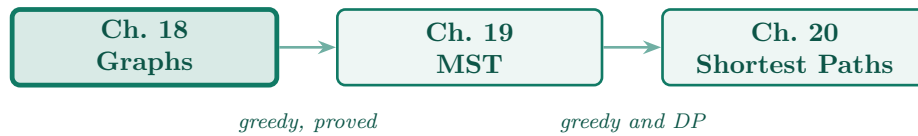


Figure P5.1: Reading path through Part V. Chapter 18 comes first because the representation decides the bounds: the same algorithm is a different cost on a sparse graph and a dense one.

Graphs: Representation, Traversal and Sparsity

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part V — Graph Algorithms

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Compare** the adjacency matrix and adjacency list on space, edge-test cost and neighbour-sweep cost.
- **Derive** the running time of BFS and DFS from each representation, and **explain** why the bound is quoted in two parameters.
- **Define** sparse and dense, and **predict** which representation wins for a given density (*CLO-2*).
- **Apply** BFS to find shortest paths in an unweighted graph and DFS to produce a topological order (*CLO-7*).
- **Account** for a measured result that contradicts the textbook prediction.

🔗 Before You Start

Chapter 2 for the two-parameter bound, Chapter 12 for the queue and stack. No Part IV material is required.

📖 Key Terms in This Chapter

graph • vertex • edge • directed • undirected • adjacency matrix • adjacency list • sparse • dense • density • degree • breadth-first search • depth-first search • topological order • bit-packing

18.1 Why Every Bound Here Has Two Parameters

📖 Density, Sparse and Dense

A graph $G = (V, E)$ has $|V|$ vertices and $|E|$ edges. For an undirected simple graph, $0 \leq |E| \leq \binom{|V|}{2}$, and the **density** is $|E|/\binom{|V|}{2}$.

G is **sparse** when $|E| = O(|V|)$ — each vertex has a bounded average degree — and **dense** when $|E| = \Theta(|V|^2)$.

💡 The same bound, two different answers

Chapter 2 made this point with a road network and a social graph; it is worth restating now that the algorithms are at hand.

BFS is $O(|V| + |E|)$ from an adjacency list. On a road network with 2 million junctions and average degree 3, that is 5×10^6 steps. On a social graph with the same vertex count and average degree 500, it is 5×10^8 — a hundred times more, for the same algorithm at the same $|V|$.

This is why $O(|V|^2)$ and $O(|V| + |E|)$ must be distinguished even though they are equal on dense graphs. The first says nothing about sparsity; the second says you have exploited it.

18.2 Two Representations

Adjacency Matrix and Adjacency List

The **adjacency matrix** is a $|V| \times |V|$ array A with $A[u][v] = 1$ exactly when (u, v) is an edge.

The **adjacency list** stores, for each vertex, a list of its neighbours.

	Matrix	List
space	$\Theta(V ^2)$	$\Theta(V + E)$
“is (u, v) an edge?”	$\Theta(1)$	$O(\deg(u))$
list u ’s neighbours	$\Theta(V)$	$\Theta(\deg(u))$
add an edge	$\Theta(1)$	$\Theta(1)$
BFS or DFS	$\Theta(V ^2)$	$\Theta(V + E)$

The third row is the one that decides traversals. A matrix must read all $|V|$ entries of a row to find a vertex’s neighbours, whether it has 2 or 2,000.

Worked Example: Space, Which Usually Decides First

Average degree 8 — a sparse graph:

$ V $	matrix (MB)	list (MB)	ratio
1,000	1.0	0.04	25
4,000	15.3	0.15	100
16,000	244.1	0.61	400
64,000	3,906.2	2.44	1,600

At 64,000 vertices the matrix needs 3.9 GB and the list needs 2.4 MB. The ratio grows as $|V|$, because the matrix is quadratic and the list linear.

For most real graphs this table ends the discussion before any timing is done. A social network with ten million users cannot be stored as a matrix on any machine — 10^{14} bytes — and can be stored as a list in a few gigabytes.

18.3 Breadth-First Search

Theorem 18.1 — BFS

BFS runs in $\Theta(|V| + |E|)$ from an adjacency list, and $d[v]$ is the number of edges on a shortest path from s to v .

Proof. Time. Each vertex is enqueued at most once, since *seen* is set when it is, so the **while** loop iterates at most $|V|$ times. For each dequeued u the inner loop runs $\deg(u)$ times, and $\sum_u \deg(u) = 2|E|$. With the $\Theta(|V|)$ initialisation this gives $\Theta(|V| + |E|)$.

Correctness. Invariant: *the queue holds vertices of distance k followed by vertices of distance $k + 1$, for some k , and d is correct for every vertex already dequeued.* Initially the queue holds s alone with $d[s] = 0$. When u at distance k is dequeued, each unseen neighbour v gets $d[v] = k + 1$; this is correct because a shortest path to v cannot be shorter than $k + 1$ (its predecessor would then

```

≡ BFS( $G, s$ )
1 for each vertex  $u$  do
2    $seen[u] \leftarrow \text{false}; d[u] \leftarrow \infty$ 
3 end for
4  $seen[s] \leftarrow \text{true}; d[s] \leftarrow 0$ 
5  $Q \leftarrow$  queue containing  $s$ 
6 while  $Q$  is not empty do
7    $u \leftarrow \text{DEQUEUE}(Q)$ 
8   for each  $v$  adjacent to  $u$  do
9     if not  $seen[v]$  then
10       $seen[v] \leftarrow \text{true}$ 
11       $d[v] \leftarrow d[u] + 1$ 
12       $\text{ENQUEUE}(Q, v)$ 
13    end if
14  end for
15 end while

```

Figure 18.1: BFS. Line 7 — “for each v adjacent to u ” — is the only line whose cost depends on the representation, and it is the whole of the asymptotic difference.

have distance $< k$ and v would already have been seen, as all such vertices were dequeued earlier), and it is achievable via u . The vertices appended therefore all have distance $k + 1$, preserving the invariant. ■

💡 DFS, and what it is for

Depth-first search replaces the queue with a stack (or recursion) and is also $\Theta(|V| + |E|)$. It is not an alternative way to visit vertices; it computes different things.

- **BFS** gives shortest paths in an *unweighted* graph, and the layer structure. Chapter 20 generalises it to weights.
- **DFS** gives a **topological order** — reverse order of finishing times, for a directed acyclic graph — and detects cycles, finds strongly connected components, and identifies articulation points. All of these depend on the *timing* of the recursion, which BFS does not have.

A practical warning carried from Chapter 9: recursive DFS on a graph with a long path uses $\Theta(|V|)$ stack frames. On a million-vertex graph that is a stack overflow, and the fix is an explicit stack.

18.4 Measured: Where the Textbook Prediction Fails

📅 Worked Example: BFS Across Densities

$|V| = 8,000$ fixed; only the edge count varies. `code/ch18_graphs.cpp`:

avg degree	$ E $	matrix (ms)	list (ms)	speed-up
2	8,000	61.46	0.17	353.4
8	32,000	76.37	0.27	278.8
64	256,000	79.60	0.88	90.5
256	1,024,000	94.06	3.02	31.2
1024	4,096,000	152.53	10.15	15.0

The matrix column is nearly flat — 61 to 153 ms while the edge count grows 512-fold. That is $\Theta(|V|^2)$ made visible: the work does not depend on $|E|$ at all, because every row is read in full regardless.

The list column grows with $|E|$, from 0.17 to 10.15 ms, which is $\Theta(|V| + |E|)$.

On a sparse graph the list is 353× faster. For a road network or a social graph that is the entire argument.

Worked Example: A Result That Contradicted the Textbook, and Why

The standard claim is that the matrix wins on dense graphs, since $\Theta(|V|^2)$ and $\Theta(|V| + |E|)$ agree there. The first version of this experiment found otherwise — the list won at *every* density, up to 95% of complete.

The cause was the representation of the representation. A matrix stored one byte per potential edge reads $|V|^2$ bytes and tests each with an unpredictable branch. Stored one *bit* per potential edge it reads $|V|^2/8$ bytes and can test 64 candidates in a single word operation. That bit-packing is the real reason matrices are recommended for dense graphs, and leaving it out made the comparison unfair.

With the bit-packed matrix, $|V| = 4,000$:

avg deg	density	byte matrix	bit matrix	list	winner
512	0.128	39.87	4.31	2.86	list
1,024	0.256	58.93	5.13	5.02	<i>tied</i>
2,048	0.512	92.65	8.68	11.31	bit matrix
3,000	0.750	107.16	10.51	17.43	bit matrix
3,990	0.998	97.06	13.59	22.85	bit matrix

The crossover is at a density of about 0.26, and the textbook claim is vindicated — but only for the bit-packed matrix, which is 7× faster than the byte one at high density.

There is a second lesson buried in the first attempt. The bit-packed version was *slower* than the byte version until its bit-scan was fixed: finding the index of the lowest set bit with a shift loop costs up to 64 iterations per edge, and the processor has a single instruction that does it. One line of code moved the bit matrix from 115 ms to 8.7 ms at density 0.512.

Both lessons are the same lesson. “Adjacency matrix” and “adjacency list” name data structures; they do not name implementations, and the gap between a good and a poor implementation of the same structure was larger here than the gap between the two structures.

Worked Example: The One Thing the Matrix Is Better At

“Is (u, v) an edge?” — ten million random queries:

avg degree	matrix (ms)	list (ms)	list/matrix
4	74.9	146.0	1.9
32	74.3	349.4	4.7
256	78.9	2,218.7	28.1

The matrix is flat and the list grows with degree, exactly as $\Theta(1)$ against $O(\deg(u))$ predicts. At average degree 256 the matrix is $28\times$ faster.

So the choice is not “lists are better”. It is:

- **traversal-heavy, sparse** $\Rightarrow$ adjacency list;
- **edge-query-heavy, or dense and small** $\Rightarrow$ matrix, bit-packed;
- **both** $\Rightarrow$ keep a list for traversal and a hash set of edges for queries, which is what most real graph libraries do.

Common Pitfalls

- **Quoting $O(|V|^2)$ when you mean $O(|V| + |E|)$.** They differ by a factor of $|V|$ on a sparse graph — measured, 353.
- **Using a matrix for a large sparse graph.** 3.9 GB against 2.4 MB at $|V| = 64,000$.
- **Using a byte-per-entry matrix on a dense graph.** Measured $7\times$ slower than bit-packing, which is the only reason to choose a matrix at all.
- **Finding the lowest set bit with a loop.** It cost more than the entire rest of the traversal.
- **Recursive DFS on a large graph.** $\Theta(|V|)$ stack frames; use an explicit stack.
- **Using BFS for weighted shortest paths.** It gives fewest *edges*, not least weight. Chapter 20 is the weighted case.
- **Forgetting $\sum_u \deg(u) = 2|E|$.** It is what turns a sum over vertices into a bound in $|E|$, and it is the key step in Theorem 18.1.

Try It Yourself: Representation, Measured

Reproduces all four tables. Expect thirty minutes.

```
// ch18_graphs.cpp -- the representation decides the bound.
// Build: cl /O2 /EHsc /std:c++17 ch18_graphs.cpp      (see Appendix A)

// --- 1. The two representations -----
// The matrix is a vector<char> so one entry is one byte:  $V^2$  bytes,
// which at  $V = 64,000$  is already 3.9 GB.

// --- 2. The matrix, bit-packed -----
// One BIT per potential edge rather than one byte, so  $V^2/8$  bytes, and
// a whole 64-bit word of candidate neighbours tested at once. This is
// the real reason adjacency matrices are recommended for dense graphs,
// and without it the comparison is unfair to them.
struct BitMatrix {
    size_t V, W;                // W = words per row
    std::vector<unsigned long long> a;
    void add(size_t u, size_t w) {
        a[u * W + (w >> 6)] |= 1ULL << (w & 63);
    }
};
```

```

    a[w * W + (u >> 6)] |= 1ULL << (u & 63);
}
};

// The bit-packed scan: skip 64 non-neighbours at a time, and within a
// word walk only the bits that are set.
while (bits) {
    // The index of the lowest set bit, as ONE instruction.
    // Finding it with a shift loop instead costs up to 64
    // iterations per edge and makes the bit-packed matrix
    // slower than the byte one -- which is what the first
    // version of this program measured.
    unsigned long idx;
    _BitScanForward64(&idx, bits);
    size_t w = wi * 64 + idx;
    if (w < g.V && !seen[w]) {
        seen[w] = 1; ++reached; q.push_back((unsigned)w);
    }
    bits &= bits - 1;    // clear it
}

// --- 3. BFS from each representation -----
// The algorithms are identical. The only difference is how "for each
// neighbour of u" is implemented, and that is the whole gap.
for (size_t w = 0; w < g.V; ++w)    // <-- Theta(V) per vertex
for (size_t i = g.start[u]; i < g.start[u+1]; ++i) // Theta(deg)

// --- 4. Space, which usually decides before speed does -----
// --- 5. BFS across a range of densities -----
// V is FIXED. Only the number of edges changes, so the matrix version
// should take the same time throughout while the list grows with E.

// --- 6. Where is the crossover? -----
// --- 7. The trade, in two numbers -----
// The matrix answers "is there an edge u-w?" in one memory access.
// The list must scan u's neighbours.

```

What to notice. Part 5's matrix column is flat while its list column grows 60-fold. Two columns of the same table, measuring the same algorithm on the same graphs, and one of them does not depend on $|E|$ at all.

Then try to break it. Replace the compressed adjacency list with a vector of vectors — one separately allocated neighbour list per vertex — and re-measure part 5 at average degree 2. Predict the direction first. It will be slower, perhaps 2–3×, with identical asymptotics, because $|V|$ separate allocations scatter the neighbour lists across memory and the prefetcher cannot follow. Same bound, same algorithm, different memory layout.

🔍 Checkpoint — Test Yourself

1. Explain why $\sum_{u \in V} \deg(u) = 2|E|$, and where Theorem 18.1 uses it.
2. A graph has $|V| = 10^6$ and average degree 6. Give the space for both representations, and say which BFS bound applies to each.
3. The measured crossover was at density 0.26 for a bit-packed matrix and did not exist at all for a byte-packed one. What does that tell you about quoting crossover densities from a textbook (CLO-2)?

Chapter Summary

- Graph bounds carry two parameters because $|E|$ ranges from 0 to $\Theta(|V|^2)$, and $O(|V| + |E|)$ says you have exploited sparsity where $O(|V|^2)$ does not.
- Matrix: $\Theta(|V|^2)$ space, $\Theta(1)$ edge test, $\Theta(|V|)$ neighbour sweep. List: $\Theta(|V| + |E|)$ space, $O(\deg)$ edge test, $\Theta(\deg)$ sweep.
- BFS and DFS are both $\Theta(|V| + |E|)$ from a list and $\Theta(|V|^2)$ from a matrix. BFS gives unweighted shortest paths; DFS gives topological order and cycle detection.
- Measured: at $|V| = 64,000$, average degree 8, the matrix needs 3.9 GB and the list 2.4 MB.
- Measured: with $|V|$ fixed at 8,000, the matrix BFS time was flat (61–153 ms) while the list grew with $|E|$ (0.17–10.15 ms). On the sparse graph the list was 353× faster.
- Measured: the matrix wins on dense graphs *only when bit-packed*, with the crossover at density ≈ 0.26 . A byte-per-entry matrix never won, and a bit-packed matrix with a naive bit-scan was slower still.
- Measured: for single edge queries the matrix was flat and the list grew with degree — 28× slower at average degree 256.

Review Questions

1. **(MCQ)** BFS from an adjacency list runs in: (a) $\Theta(|V|)$; (b) $\Theta(|E|)$; (c) $\Theta(|V| + |E|)$; (d) $\Theta(|V|^2)$.
2. **(MCQ)** An adjacency matrix for a graph with $|V| = 10^5$ requires about: (a) 100 KB; (b) 10 MB; (c) 10 GB; (d) 1 TB.
3. **(Short)** Define sparse and dense, and say which representation suits each and why (*CLO-2*).
4. **(Short)** Give one task BFS does that DFS does not, and one that DFS does that BFS does not (*CLO-7*).
5. **(Short)** The measured matrix BFS time barely changed as $|E|$ grew 512-fold. Explain.
6. **(Applied)** Prove that DFS on a directed acyclic graph, emitting vertices in reverse order of finishing time, produces a topological order.
7. **(Applied)** You are choosing a representation for a graph of 50 million web pages with average out-degree 40, for a workload that is 90% traversal and 10% edge queries. Decide, giving the arithmetic, and say what you would measure to check.

Minimum Spanning Trees

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part V — Graph Algorithms

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **State** and **prove** the cut property, and **explain** why it makes both MST algorithms greedy (*CLO-7*).
- **Trace** Kruskal’s and Prim’s algorithms and **derive** their running times.
- **Describe** union by rank and path compression, and **state** the amortised bound they give.
- **Measure** the effect of each union–find optimisation and **relate** it to Chapter 17.
- **Choose** between Kruskal and Prim for a given density.

🔗 Before You Start

Chapter 14 for the exchange argument — the cut property is one. Chapter 17 for the union–find bound, which is meaningless without it. Chapter 18 for representations.

📖 Key Terms in This Chapter

spanning tree • minimum spanning tree • cut • crossing edge • light edge • cut property • safe edge • Kruskal • Prim • disjoint-set union • union by rank • path compression • inverse Ackermann

⚠️ Why this chapter exists although the outline omits it

The HEC 2023 outline’s graph clause names “graph algorithms, shortest paths, sparse graphs” and does not mention spanning trees. **CLO-7 requires them by name**, and the Course Specification records the discrepancy.

It would earn its place regardless: the cut property is the cleanest exchange argument in the book, and union–find is where Chapter 17 is paid back.

19.1 The Cut Property

📖 Spanning Tree, MST

A **spanning tree** of a connected undirected graph is a subset of edges that connects every vertex and contains no cycle. It has exactly $|V| - 1$ edges.

A **minimum spanning tree** is one of least total weight.

📖 Cut, Crossing, Light

A **cut** $(S, V \setminus S)$ is a partition of the vertices. An edge **crosses** the cut if its endpoints lie on opposite sides. A crossing edge of least weight is a **light** edge for that cut.

🔗 Theorem 19.1 — The cut property

Let A be a subset of some minimum spanning tree, and let $(S, V \setminus S)$ be any cut that no edge of A crosses. Then any light edge for that cut is safe: $A \cup \{e\}$ is still contained in some minimum spanning tree.

Proof. An exchange argument, in the pattern of Chapter 14.

Let T be an MST containing A . If $e \in T$ there is nothing to prove, so assume $e = (u, v) \notin T$.

Adding e to T creates exactly one cycle, since T is a tree and e joins two vertices already connected by a unique path. That cycle contains the path in T from u to v , and since u and v lie on opposite sides of the cut, that path must cross the cut at least once — say at edge e' .

Now e' is a crossing edge and e is a *light* crossing edge, so $w(e) \leq w(e')$. Form

$$T' = (T \setminus \{e'\}) \cup \{e\}.$$

Removing e' from the cycle leaves a tree spanning the same vertices, so T' is a spanning tree, and

$$w(T') = w(T) - w(e') + w(e) \leq w(T).$$

Since T was minimum, $w(T') = w(T)$ and T' is also an MST.

Finally $A \subseteq T'$: no edge of A crosses the cut, so $e' \notin A$, and removing e' did not disturb A . Hence $A \cup \{e\} \subseteq T'$, an MST. ■

💡 One theorem, two algorithms

Theorem 19.1 says a light crossing edge may always be taken. It does not say *which* cut to use, and the two classical algorithms are two answers.

Prim takes S = the vertices of the tree built so far. The light edge is the cheapest edge leaving the tree, so the structure needed is a **priority queue**.

Kruskal considers edges in increasing weight order; when an edge joins two different components, the cut separating those components is crossed by no chosen edge, and this edge is lightest among those remaining. The structure needed is one that answers “**are these two vertices already connected?**” — disjoint-set union.

Both are greedy, both are optimal, and both are optimal *for the same reason*. The difference is entirely which data structure the chosen cut requires.

19.2 Kruskal, and the Structure It Needs

📖 Disjoint-Set Union

Maintains a partition of the vertices under three operations: $\text{MAKE-SET}(v)$, $\text{FIND}(v)$ returning the representative of v 's set, and $\text{UNION}(u, v)$ merging two sets.

Each set is a tree whose root is the representative. FIND walks to the root, so its cost is the depth.

💡 Two optimisations, both two lines

Union by rank. When merging, attach the shorter tree under the taller. This alone bounds the depth by $O(\lg |V|)$, since a tree of rank r has at least 2^r nodes.

```

≡ KRUSKAL( $G, w$ )
1  $A \leftarrow \emptyset$ 
2 for each vertex  $v$  do
3   MAKE-SET( $v$ )
4 end for
5 sort the edges by non-decreasing weight
6 for each edge  $(u, v)$  in that order do
7   if FIND( $u$ )  $\neq$  FIND( $v$ ) then
8      $A \leftarrow A \cup \{(u, v)\}$   $\triangleright$  safe, by the cut property
9     UNION( $u, v$ )
10  end if
11 end for
12 return  $A$ 

```

Figure 19.1: KRUSKAL. The sort is $\Theta(|E| \lg |E|)$ and dominates; the $O(|E|)$ union-find operations cost $O(|E| \alpha(|V|))$, which is why the amortised bound matters.

Path compression. After FIND locates the root, make every node on the path point directly at it. The walk is done anyway; flattening costs one extra pass and makes all future finds on those nodes $O(1)$.

Together they give an amortised cost of $O(\alpha(|V|))$ per operation, where α is the inverse Ackermann function. $\alpha(n) \leq 4$ for every n that can be written down — $\alpha(n) = 4$ requires n larger than the number of atoms in the universe.

This bound is amortised and nothing else. A single FIND can cost $\Theta(\lg |V|)$; it is only over a sequence that the average falls to α . It is proved by the potential method of Chapter 17, and it is the most dramatic amortised result in common use.

🔗 Theorem 19.2 — Kruskal's running time

With union by rank and path compression, KRUSKAL runs in $O(|E| \lg |E|)$ time, which is $O(|E| \lg |V|)$.

Proof. Sorting the edges is $\Theta(|E| \lg |E|)$. There are $|V|$ MAKE-SET operations and at most $2|E|$ FIND plus $|V| - 1$ UNION operations, costing $O((|E| + |V|) \alpha(|V|))$ in total. Since $\alpha(|V|) = O(\lg |E|)$ trivially, the sort dominates, giving $O(|E| \lg |E|)$. Finally $|E| \leq |V|^2$ gives $\lg |E| \leq 2 \lg |V|$, so the bound is $O(|E| \lg |V|)$. ■

⚠️ Note what the theorem says about the clever part

The union-find work is *not* the bottleneck — the sort is. So why bother with path compression at all?

Because without it the union-find work *becomes* the bottleneck, by a long way. Section 19.4 measures a naive implementation running $1,254\times$ slower than the optimised one on the same graph. An optimisation that makes a non-dominant term stay non-dominant is still doing essential work.

19.3 Prim

```

PRIM( $G, w, r$ )
1  for each vertex  $u$  do
2       $key[u] \leftarrow \infty$ ;  $\pi[u] \leftarrow \text{NIL}$ 
3  end for
4   $key[r] \leftarrow 0$ 
5   $Q \leftarrow$  min-priority queue of all vertices, keyed by  $key$ 
6  while  $Q$  is not empty do
7       $u \leftarrow \text{EXTRACT-MIN}(Q)$   $\triangleright$  the light edge leaving the tree
8      for each  $v$  adjacent to  $u$  do
9          if  $v \in Q$  and  $w(u, v) < key[v]$  then
10              $\pi[v] \leftarrow u$ ;  $\text{DECREASE-KEY}(Q, v, w(u, v))$ 
11          end if
12      end for
13 end while

```

Figure 19.2: PRIM. With a binary heap this is $O((|V| + |E|) \lg |V|)$; the cut is always “tree so far” against “everything else”.

💡 Which heap, and what it is worth

Priority queue	Extract-Min	Decrease-Key	Prim total
unsorted array	$O(V)$	$O(1)$	$O(V ^2)$
binary heap	$O(\lg V)$	$O(\lg V)$	$O(E \lg V)$
Fibonacci heap	$O(\lg V)$	$O(1)$ amortised	$O(E + V \lg V)$

The Fibonacci heap’s $O(1)$ DECREASE-KEY is amortised — another debt to Chapter 17 — and it gives the best known asymptotic bound. Its constants are poor enough that binary heaps win in practice for all but enormous dense graphs, which is a recurring theme: Chapter 11’s median-of-medians, Chapter 9’s heapsort fallback, and now this.

19.4 Measured

📊 Worked Example: First, Do They Agree?

Two different greedy rules; the cut property says they must produce the same total weight.

2,000 random connected graphs. Disagreements: 0.

Cheap evidence that Theorem 19.1 is doing what it claims — and, more practically, that neither implementation has a bug.

📊 Worked Example: Union-Find: the Optimisations, Counted

code/ch19_mst.cpp counts pointer hops inside FIND, divided by the number of operations:

$ V $	naive	by rank	rank + compression	naive/both
10,000	1,406.77	2.89	1.62	868.6
20,000	2,880.47	3.05	1.66	1,733.3
40,000	6,044.05	3.28	1.73	3,495.4
80,000	12,542.07	3.46	1.79	7,025.0
160,000	25,469.42	3.44	1.82	14,008.8

The naive column doubles when $|V|$ doubles — it is $\Theta(|V|)$ per find, because an unbalanced union can build a path. The ratio to the optimised version grows without limit: 869 to 14,009.

Union by rank alone drops it to about 3, an improvement of four orders of magnitude from two lines of code.

And rank plus compression is essentially flat. Carried further, with the naive variant omitted because it would not finish:

$ V $	by rank	rank + compression	$\alpha(V) \leq$
320,000	3.43	1.78	4
1,280,000	3.61	1.85	4
5,120,000	3.68	1.88	4

From $|V| = 10,000$ to $|V| = 5,120,000$ — a 512-fold range — the cost per find went from 1.62 to 1.88. That is the inverse Ackermann bound, made visible. It is not constant, and it is not distinguishable from constant by any measurement you can make.

Worked Example: And in Seconds

$ V $	naive (ms)	by rank (ms)	rank + compr. (ms)	speed-up
25,000	1,064.4	8.9	8.4	127.2
50,000	4,910.3	19.5	17.3	283.5
100,000	44,306.2	44.1	35.3	1,254.4

44 seconds against 35 milliseconds. The naive column is quadratic — each doubling of $|V|$ multiplies it by about 9, not 2.

Note also that rank alone gets most of the benefit (44.1 ms against 35.3 ms). Path compression adds a further 20%. The first optimisation is transformative and the second is an improvement — worth knowing when you are deciding what to implement first.

Worked Example: Kruskal Against Prim

$|V| = 100,000$, density varying:

avg degree	$ E $	Kruskal (ms)	Prim (ms)	winner
2	100,000	10.0	4.9	Prim
4	200,000	19.1	36.2	Kruskal
8	400,000	39.2	64.4	Kruskal
16	800,000	76.2	109.6	Kruskal
32	1,600,000	148.3	155.2	Kruskal
64	3,200,000	303.1	226.4	Prim

Neither wins everywhere, and the pattern is not monotonic. Prim wins at the extreme sparse end and again when dense; Kruskal wins in the middle.

Both are $O(|E| \lg |V|)$, so asymptotic analysis predicts no winner at all — and is right, in the sense that the measured ratio never exceeds about 2 in either direction. Everything in this table is constants.

The practical rules that come out of it:

- **Kruskal** if the edges are *already sorted*, or can be sorted cheaply — then its dominant term vanishes;
- **Prim** on dense graphs, where it never materialises an edge list at all and touches the adjacency structure in place;
- **either**, in the broad middle, where the difference is under $2\times$ and other considerations should decide.

! Common Pitfalls

- **Union–find without union by rank.** Measured $14,000\times$ more pointer hops and 44 seconds against 35 milliseconds.
- **Quoting $O(\alpha(n))$ as a worst case.** It is amortised; a single find can still be $\Theta(\lg |V|)$.
- **Assuming the MST is unique.** It is unique only if all edge weights are distinct. With ties there may be many, and two correct algorithms may return different trees of the same weight.
- **Running an MST algorithm on a disconnected graph.** There is no spanning tree; you get a spanning *forest*, and the code should say which it computed.
- **Using Kruskal on a dense graph without thinking about the sort.** $\Theta(|E| \lg |E|)$ with $|E| = \Theta(|V|^2)$ is $\Theta(|V|^2 \lg |V|)$.
- **Reaching for a Fibonacci heap.** Better asymptotically, worse in practice at every size you will meet — the same story as median-of-medians.
- **Forgetting that the cut property needs A to cross no edge of the cut.** That hypothesis is what makes the exchange safe.

🔧 Try It Yourself: Where Amortised Analysis Pays Off

Reproduces all four tables. Expect forty minutes — the naive union–find at $|V| = 100,000$ takes 44 seconds per run, three times over.

```
// ch19_mst.cpp -- Kruskal, Prim, and where Chapter 17 pays off.
// Build: cl /O2 /EHsc /std:c++17 ch19_mst.cpp           (see Appendix A)

// --- 1. Union-find, three ways -----
// The only differences are whether union balances the trees and
// whether find flattens the path behind it. Both are two lines, and
// together they take the amortised cost from  $O(\lg n)$  to  $O(\alpha(n))$ .
```

```

int find(int x) {
    if (!compress) {
        while (parent[x] != x) { x = parent[x]; ++steps; }
        return x;
    }
    int root = x;
    while (parent[root] != root) { root = parent[root]; ++steps; }
    while (parent[x] != root) {          // second pass: flatten
        int next = parent[x];
        parent[x] = root;
        x = next;
    }
    return root;
}

bool unite(int a, int b) {
    a = find(a); b = find(b);
    if (a == b) return false;
    if (!by_rank) { parent[a] = b; return true; }
    if (rank_[a] < rank_[b]) std::swap(a, b);
    parent[b] = a;
    if (rank_[a] == rank_[b]) ++rank_[a];
    return true;
}

// --- 2. Kruskal: sort, then add anything that is safe -----
// The sort is  $\Theta(E \lg E)$  and dominates; the union-find work is
//  $E * \alpha(V)$ , which is why the structure's amortised cost matters.

// --- 3. Prim: grow one tree from a root -----
// The "lazy" form pushes every edge and discards stale entries when
// popped, which is simpler than decrease-key and no worse
// asymptotically.

// --- 4. Do the two algorithms agree? -----
// They are different greedy rules. If the cut property is right, they
// must produce the same total weight.

// --- 5. The three union-find variants -----
// "steps" counts pointer hops inside find(). This is the measurement
// Chapter 17 predicts: naive grows, rank alone is much better, and
// rank + compression is essentially flat.
//
// The naive variant's find() is LINEAR in the component size, so the
// whole run is quadratic and  $V = 320,000$  does not finish in any
// reasonable time. It stops at 160,000; the other two carry on, and
// that asymmetry is itself the result.

// --- 6. And the same thing in seconds -----
// --- 7. Kruskal against Prim, by density -----

```

What to notice. The rank+compress column in part 5 goes from 1.62 to 1.88 while $|V|$ multiplies by 512. If you plotted it you would draw a horizontal line, and the theory says it is growing — just slower than any function you can name.

Then try to break it. Implement path compression *without* union by rank and measure it. Predict first. It is far better than naive and slightly worse than the pair — compression alone gives $O(\lg n)$ amortised, and it is the combination that gives α . Two optimisations that each help and only together give the famous bound is an unusual and instructive situation.

? Checkpoint — Test Yourself

1. State the cut property and identify the hypothesis that makes the exchange in its proof safe.
2. Kruskal and Prim use the same theorem on different cuts. Describe each algorithm's cut, and say which data structure it forces (*CLO-7*).
3. The measured cost per find was 1.62 at $|V| = 10^4$ and 1.88 at $|V| = 5 \times 10^6$. Why is it wrong to call this $O(1)$, and why does it not matter in practice?

✓ Chapter Summary

- The cut property: a light edge crossing any cut uncrossed by the current partial tree is safe. It is proved by an exchange argument, and it is why both MST algorithms are greedy and optimal.
- Kruskal takes the cut between components (needing union–find); Prim takes the cut between the tree and the rest (needing a priority queue). One theorem, two data structures.
- Union by rank gives $O(\lg |V|)$ per find; adding path compression gives $O(\alpha(|V|))$ amortised, with $\alpha \leq 4$ for any writable n .
- Measured: pointer hops per find were 25,469 (naive), 3.44 (rank) and 1.82 (rank + compression) at $|V| = 160,000$ — a ratio of 14,009.
- Measured: from $|V| = 10^4$ to 5.12×10^6 , the optimised cost per find rose only from 1.62 to 1.88.
- Measured: Kruskal took 44.3 seconds with naive union–find and 35 milliseconds with the optimisations, at $|V| = 100,000$. Rank alone got most of the way; compression added a further 20%.
- Measured: Kruskal and Prim never differed by more than about $2\times$ across densities, and neither won everywhere. Both are $O(|E| \lg |V|)$; the table is entirely constants.

✍ Review Questions

1. **(MCQ)** With union by rank and path compression, the amortised cost of a disjoint-set operation is: (a) $O(1)$ worst case; (b) $O(\alpha(n))$; (c) $O(\lg n)$; (d) $O(n)$.
2. **(MCQ)** Kruskal's running time is dominated by: (a) the union–find operations; (b) sorting the edges; (c) the priority queue; (d) the initialisation.
3. **(Short)** State and prove the cut property (*CLO-7*).
4. **(Short)** Explain why Kruskal needs union–find and Prim needs a priority queue, in terms of the cut each one uses.
5. **(Short)** An MST is unique under what condition? Give a graph with two distinct minimum spanning trees.
6. **(Applied)** Prove that a disjoint-set tree of rank r built with union by rank contains at least 2^r nodes, and deduce the $O(\lg n)$ depth bound.
7. **(Applied)** You must build an MST on a graph of 10 million vertices and 60 million edges, already sorted by weight. Choose an algorithm, give the arithmetic, and say which measurement in this chapter most supports your choice.

Shortest Paths: Single-Source and All-Pairs

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part V — Graph Algorithms

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Solve** single-source shortest-path problems with Dijkstra’s and Bellman–Ford’s algorithms (CLO-7).
- **Prove** Dijkstra’s algorithm correct and **identify** the step that requires non-negative weights.
- **Construct** a graph on which Dijkstra’s algorithm returns a wrong answer, and **explain** why.
- **Apply** Floyd–Warshall to the all-pairs problem and **justify** the loop order.
- **Choose** among the three algorithms given density, weights and the number of sources required.

🔗 Before You Start

Chapter 18 for representation and BFS — Dijkstra is BFS with a priority queue. Chapter 14 for the exchange argument. Chapter 15 for Floyd–Warshall, which is a dynamic program.

📖 Key Terms in This Chapter

shortest path • relaxation • Dijkstra • Bellman–Ford • Floyd–Warshall • negative edge • negative cycle • single-source • all-pairs • optimal substructure • early termination

20.1 Relaxation, the Common Operation

📄 Relaxation

Each algorithm here maintains an estimate $d[v] \geq \delta(s, v)$, the true shortest distance. **Relaxing** an edge (u, v) asks whether going via u improves it:

if $d[u] + w(u, v) < d[v]$ then $d[v] \leftarrow d[u] + w(u, v)$.

All three algorithms do only this. They differ entirely in *the order* in which edges are relaxed, and that order is what buys or costs the running time.

💡 Optimal substructure, and where it comes from

Any subpath of a shortest path is itself a shortest path: if p is a shortest path from s to v passing through u , the portion from s to u must be shortest, or substituting a shorter one would improve p .

That is a cut-and-paste argument, and it is the optimal substructure of Chapter 15. It is why shortest paths yield to both greedy (Dijkstra) and dynamic programming (Floyd–Warshall) — and why *longest simple paths* do not, since a subpath of a longest simple path need not be a longest simple path.

20.2 Dijkstra

 DIJKSTRA(G, w, s)

```

1 for each vertex  $v$  do
2    $d[v] \leftarrow \infty$ 
3 end for
4  $d[s] \leftarrow 0$ ;  $S \leftarrow \emptyset$ ;  $Q \leftarrow$  all vertices
5 while  $Q$  is not empty do
6    $u \leftarrow \text{EXTRACT-MIN}(Q)$   $\triangleright$  smallest estimate
7    $S \leftarrow S \cup \{u\}$   $\triangleright$   $d[u]$  is now final
8   for each  $v$  adjacent to  $u$  do
9      $\text{relax}(u, v)$ 
10  end for
11 end while

```

Figure 20.1: DIJKSTRA. Line 7 is the greedy commitment: once a vertex is extracted its distance is declared final and never reconsidered. Theorem 20.1 is what justifies that, and Section 20.4 is what happens when its hypothesis fails.

 Theorem 20.1 — Dijkstra is correct

If every edge weight is non-negative, then when DIJKSTRA adds u to S , $d[u] = \delta(s, u)$.

Proof. Suppose not, and let u be the first vertex added to S with $d[u] \neq \delta(s, u)$. Then $u \neq s$, and $\delta(s, u) < \infty$ (otherwise $d[u] = \infty = \delta(s, u)$).

Consider a shortest path p from s to u . Since $s \in S$ and $u \notin S$ at the moment before u is added, p must leave S at some point: let y be the first vertex on p not in S , and $x \in S$ its predecessor.

Because x was added to S earlier and u is the *first* mistake, $d[x] = \delta(s, x)$. The edge (x, y) was relaxed when x was added, so

$$d[y] = \delta(s, x) + w(x, y) = \delta(s, y),$$

the second equality because the prefix of a shortest path is a shortest path.

Now y appears on a shortest path to u and **all remaining edge weights are non-negative**, so

$$\delta(s, y) \leq \delta(s, u).$$

But u was chosen by EXTRACT-MIN over y , so $d[u] \leq d[y]$. Chain them:

$$d[u] \leq d[y] = \delta(s, y) \leq \delta(s, u) \leq d[u],$$

the last because d never underestimates. So all are equal and $d[u] = \delta(s, u)$ — contradicting the choice of u . ■

 Exactly one step used non-negativity

The inequality $\delta(s, y) \leq \delta(s, u)$. It says that extending a path can never make it cheaper.

With a negative edge that is false: the rest of the path from y to u might have negative total weight, so $\delta(s, u)$ could be *less* than $\delta(s, y)$, and a vertex finalised early may later turn out to have been reachable more cheaply. The greedy commitment on line 7 is then unjustified.

Section 20.4 builds the graph where this happens.

Worked Example: Which Priority Queue, Measured

$|V| = 4,000$, density varying. `code/ch20_sp.cpp` compares a binary heap with an unsorted array (extract-min by linear scan):

avg degree	$ E $	heap (ms)	array (ms)	winner
2	7,998	0.36	46.77	heap
32	127,970	1.97	53.39	heap
512	2,047,499	6.40	62.63	heap
2000	7,998,035	15.32	63.47	heap

The array column is flat — 47 to 63 ms while $|E|$ grows 1000-fold — which is $O(|V|^2)$ behaving exactly as advertised, just as the adjacency matrix did in Chapter 18.

But the textbook crossover does not appear. The standard claim is that the $O(|V|^2)$ array version wins on dense graphs, since $|E| \approx |V|^2$ makes the heap's $\lg |V|$ factor pure overhead. Here the heap won at every density, including $|E| = 8 \times 10^6$ — half of complete.

The honest reading is that at $|V| = 4,000$ the array's $|V|^2 = 1.6 \times 10^7$ branch-laden scans cost more than $|E| \lg |V| = 10^8$ well-predicted heap operations on this processor. Chapter 18 found the same thing and fixed it by bit-packing; there is no equivalent trick here, because the array version must examine real distances rather than single bits. The crossover is real asymptotically and did not arrive at any size tested.

20.3 Bellman–Ford

BELLMAN-FORD(G, w, s)

```

1 for each vertex  $v$  do
2    $d[v] \leftarrow \infty$ 
3 end for
4  $d[s] \leftarrow 0$ 
5 for  $i \leftarrow 1$  to  $|V| - 1$  do
6   for each edge  $(u, v)$  do
7     relax  $(u, v)$ 
8   end for
9 end for
10 for each edge  $(u, v)$  do
11   if  $d[u] + w(u, v) < d[v]$  then
12     return NEGATIVE-CYCLE
13   end if
14 end for
15 return  $d$ 
```

Figure 20.2: BELLMAN-FORD. No priority queue and no assumption about signs: $O(|V||E|)$. The final pass is not decoration — it is the only way to detect a negative cycle, which Dijkstra cannot do at all.

Theorem 20.2 — Bellman–Ford

If G has no negative-weight cycle reachable from s , then after $|V| - 1$ passes $d[v] = \delta(s, v)$ for every v .

Proof. A shortest path has at most $|V| - 1$ edges: if it had more it would repeat a vertex, and the cycle so formed has non-negative weight (by hypothesis) and can be removed without lengthening the path.

Now argue by induction that after i passes, $d[v] = \delta(s, v)$ for every v whose shortest path uses at most i edges. After 0 passes this holds for $v = s$. For the step: let v have a shortest path $s \rightsquigarrow u \rightarrow v$ using $i + 1$ edges; its prefix to u uses i edges and is shortest, so by hypothesis $d[u] = \delta(s, u)$ after i passes. Pass $i + 1$ relaxes (u, v) and sets $d[v] \leq \delta(s, u) + w(u, v) = \delta(s, v)$; since d never underestimates, equality holds.

After $|V| - 1$ passes every shortest path is covered. If a further relaxation still improves some $d[v]$, then some path uses $|V|$ or more edges and is still improving, which requires a negative cycle. ■

Worked Example: Bellman–Ford Is Faster Than Its Bound Suggests

On random graphs with non-negative weights, average degree 8:

$ V $	Dijkstra (ms)	Bellman–Ford (ms)	ratio	agree?
2,000	0.40	0.3	0.8	yes
8,000	1.89	1.7	0.9	yes
32,000	11.21	11.0	1.0	yes

Bellman–Ford matched Dijkstra, despite a bound of $O(|V||E|)$ against $O(|E| \lg |V|)$ — a predicted factor of $|V|/\lg |V| \approx 2,000$.

The reason is the early exit: the implementation stops when a pass changes nothing, and on a random graph the distances settle after a handful of passes rather than $|V| - 1$. The $O(|V||E|)$ bound is a worst case that random graphs do not come close to.

So does the bound ever materialise? It needs a graph that forces every pass — a long path whose edges are relaxed in the wrong order, so information advances one vertex per pass:

$ V $	Dijkstra (ms)	Bellman–Ford (ms)	ratio	passes used
1,000	0.03	1.4	44.9	999
2,000	0.03	5.5	190.8	1,999
4,000	0.06	21.8	386.9	3,999
8,000	0.11	87.2	789.5	7,999

There it is: 789× slower, and the ratio doubling with $|V|$. The graph is not adversarial in its weights — every edge costs 1. Only the *order* the adjacency lists are visited in is unfavourable.

The lesson is about benchmarking, not about Bellman–Ford. Measuring only on random inputs would have shown the two algorithms as equals. It is the third time this book has made that point — Chapter 9’s sorted input, Chapter 12’s sorted insertions, Chapter 13’s structured keys — and it will not be the last.

20.4 One Negative Edge

 Worked Example: Dijkstra, Measurably Wrong

Four vertices. $0 \rightarrow 1$ costs 3, $0 \rightarrow 2$ costs 4, $2 \rightarrow 1$ costs -2 , and $1 \rightarrow 3$ costs 1. The true distances are $d(1) = 2$ (via vertex 2) and therefore $d(3) = 3$.

vertex	Dijkstra (array)	Dijkstra (heap)	Bellman–Ford	correct
0	0	0	0	0
1	2	2	2	2
2	4	4	4	4
3	4	3	3	3 wrong

Trace it. The array version pops vertex 1 at distance 3 and *finalises* it, relaxing $1 \rightarrow 3$ to give $d(3) = 4$. It then pops vertex 2 and lowers $d(1)$ to 2 — but vertex 1 is already done, so its outgoing edge is never re-examined and $d(3)$ stays at 4.

Notice that $d(1)$ itself ends up correct. The error is only visible at vertex 3, because the wrong value had already *propagated*. The three-vertex example people usually reach for does not fail, for exactly this reason: the wrong value is overwritten before anything depends on it. A counter-example has to let the mistake escape.

And the heap version got it right — which is an accident worth understanding. The lazy heap implementation never marks anything final; it re-pushes whenever an estimate improves. On this graph that lets it recover. It is behaving like Bellman–Ford, not like Dijkstra, and on adversarial graphs with negative edges it can take exponential time. **“Dijkstra is wrong on negative edges” is a statement about the algorithm; whether your *program* visibly fails depends on which implementation you wrote.**

Negative cycles. On this graph Bellman–Ford reported none, correctly. Adding a cycle $0 \rightarrow 1 \rightarrow 2 \rightarrow 0$ of total weight -1 , it reported one — also correctly. Dijkstra cannot detect a negative cycle at all, and with one present “shortest path” is undefined, since going round again is always cheaper.

20.5 All Pairs

 FLOYD-WARSHALL(W)

```
1  $D \leftarrow W$ 
2 for  $k \leftarrow 1$  to  $n$  do
3   for  $i \leftarrow 1$  to  $n$  do
4     for  $j \leftarrow 1$  to  $n$  do
5        $D[i][j] \leftarrow \min(D[i][j], D[i][k] + D[k][j])$ 
6     end for
7   end for
8 end for
9 return  $D$ 
```

$\triangleright k$ must be outermost

Figure 20.3: FLOYD-WARSHALL. Five lines, $\Theta(n^3)$ regardless of density, and the smallest constant of any algorithm in this chapter — three nested loops over contiguous memory.

💡 Why k must be outermost

This is a dynamic program, and k indexes the subproblem.

Let $d^{(k)}[i][j]$ be the shortest distance from i to j using only vertices $1 \dots k$ as *intermediates*. Then

$$d^{(k)}[i][j] = \min(d^{(k-1)}[i][j], d^{(k-1)}[i][k] + d^{(k-1)}[k][j]),$$

— either the best path avoids k , or it goes through k exactly once.

The k loop therefore advances the subproblem, and i and j merely fill in the table for the current one. Putting k inside would compute something that is not a shortest path for any well-defined subproblem.

(The single array works because $d^{(k)}[i][k] = d^{(k-1)}[i][k]$ — a path to k gains nothing by using k as an intermediate — so the row and column being read are unchanged by the current pass.)

📊 Worked Example: All Pairs, Measured

$|V| = 600$. Floyd–Warshall against running Dijkstra from every vertex:

avg degree	$ E $	Floyd–Warshall (ms)	$ V \times$ Dijkstra (ms)	winner
2	1,198	35.8	19.3	Dijkstra
8	4,793	125.7	59.3	Dijkstra
32	19,170	156.6	112.1	Dijkstra
128	76,667	170.7	204.6	Floyd
300	179,711	170.3	299.8	Floyd

The crossover is near average degree 64, or a density of about 0.1.

Floyd–Warshall’s column is nearly flat — 36 to 171 ms — because $\Theta(|V|^3)$ does not depend on $|E|$. (It is not perfectly flat because the early rows have many ∞ entries, which the implementation skips.)

Repeated Dijkstra grows with $|E|$, as $O(|V|(|V| + |E|) \lg |V|)$ requires.

And the practical point. For a dense graph, five lines of Floyd–Warshall beat $|V|$ invocations of a far more sophisticated algorithm. Its constant is tiny: three nested loops over a contiguous array, perfectly predictable, perfectly prefetchable. This is the opposite of Chapter 12’s pointer-chasing penalty, and it is the same cause.

💡 Choosing

Algorithm	Time	Weights	Use when
BFS	$\Theta(V + E)$	unweighted	all weights equal
Dijkstra (heap)	$O((V+E) \lg V)$	non-negative	one source, the common case
Bellman–Ford	$O(VE)$	any	negative edges, or you must detect a negative cycle
Floyd–Warshall	$\Theta(V^3)$	any (no neg. cycle)	all pairs on a dense graph

If all weights are equal, use BFS — a priority queue over identical keys is pure overhead, and Chapter 18's measured 0.17 ms against Dijkstra's 0.36 ms on the same sparse graph is the price of not noticing.

! Common Pitfalls

- **Using Dijkstra with negative edges.** Measured wrong, and the failure can hide: $d(1)$ was right and $d(3)$ was not.
- **Testing for that bug on too small a graph.** The obvious three-vertex example self-corrects. The error must be allowed to propagate.
- **Assuming your Dijkstra will visibly fail.** A lazy heap version may silently behave like Bellman–Ford — correct here, exponential elsewhere.
- **Benchmarking Bellman–Ford only on random graphs.** Measured equal to Dijkstra there and $789\times$ slower on a path.
- **Putting k anywhere but outermost in Floyd–Warshall.** It indexes the subproblem; the result is not a shortest path.
- **Running Dijkstra $|V|$ times on a dense graph.** Measured $1.8\times$ slower than five lines of Floyd–Warshall.
- **Using Dijkstra on an unweighted graph.** BFS is simpler and faster.
- **Asking for shortest paths with a negative cycle present.** The question has no answer; detect it and say so.

🔧 Try It Yourself: Three Algorithms, and One That Is Wrong

Reproduces all five tables. Expect thirty minutes.

```
// ch20_sp.cpp -- three shortest-path algorithms, and when each wins.
// Build: cl /O2 /EHsc /std:c++17 ch20_sp.cpp          (see Appendix A)

// --- 1. Dijkstra with a binary heap -----
// The "lazy" form: push a new entry on every improvement and skip
// stale ones when popped.  $O((V+E) \lg V)$ , simpler than decrease-key.
//
// The invariant is the whole correctness argument: when a vertex is
// popped with the smallest tentative distance, that distance is final.
// It holds only because every edge weight is  $\geq 0$ .

// --- 2. Dijkstra with a linear scan -----
// Same algorithm; the priority queue is an unsorted array, so
// extract-min is  $O(V)$  and the total is  $O(V^2 + E)$ .

// --- 3. Bellman-Ford: relax every edge,  $V-1$  times -----
// No priority queue and no assumption about signs. The bound is
//  $O(VE)$  -- the price of handling negative edges and of being able to
// DETECT a negative cycle, which Dijkstra cannot.
//         if (!changed) break;           // early exit: often much faster

// --- 4. Floyd-Warshall: all pairs, three nested loops -----
// The  $k$  loop must be OUTERMOST. That is not a style choice:  $k$  indexes
// the subproblem ("paths using only intermediate vertices  $< k$ "), and
// putting it inside would compute something that is not a shortest
// path.

// --- 5. Heap against array, across densities -----
// --- 6. Dijkstra against Bellman-Ford -----
// --- 6b. The graph that forces every pass -----
// A single path, with the adjacency lists visited in an order that
```

```
// propagates exactly one vertex per pass. Nothing adversarial about
// the weights; only the ORDER matters.

// --- 7. One negative edge, and Dijkstra is wrong -----
// 0 -> 1 costs 3, 0 -> 2 costs 4, 2 -> 1 costs -2, 1 -> 3 costs 1.
// True distances: d(1) = 2 via vertex 2, so d(3) = 3.
//
// Array Dijkstra pops vertex 1 at distance 3 and finalises it,
// relaxing 1 -> 3 to give d(3) = 4. It then pops vertex 2 and lowers
// d(1) to 2 -- but vertex 1 is done, so its edge to 3 is never
// re-examined and d(3) stays at 4. The error has PROPAGATED, which is
// why this graph fails where the obvious three-vertex example does
// not: there the wrong value was overwritten before anything
// depended on it.

// --- 8. All pairs: Floyd-Warshall against V x Dijkstra -----
```

What to notice. Part 7 prints both Dijkstras. They disagree, and the one that agrees with Bellman–Ford is the one that is not really Dijkstra. An experiment that had shown only the heap version would have concluded, wrongly, that negative edges are harmless.

Then try to break it. Add a negative cycle to the graph in part 7 and run the lazy heap Dijkstra on it. Predict first. It will not terminate in any reasonable time — each trip round the cycle lowers an estimate, so it re-pushes for ever. Bellman–Ford detects the cycle in one extra pass and says so. That is the practical difference between an algorithm that assumes something and one that checks.

🔍 Checkpoint — Test Yourself

1. Identify the single step in the proof of Theorem 20.1 that uses non-negativity, and state what goes wrong without it.
2. Why does Bellman–Ford need exactly $|V| - 1$ passes, and what does the $|V|$ -th pass detect (CLO-7)?
3. Measured, Bellman–Ford matched Dijkstra on random graphs and was $789\times$ slower on a path. What does this say about the value of a worst-case bound?

📋 Chapter Summary

- All three algorithms do nothing but relax edges; they differ only in the order, and the order is what costs or saves.
- Dijkstra is greedy: the extracted minimum is declared final. The proof uses non-negativity at exactly one step — that extending a path cannot make it cheaper.
- Measured: a binary heap beat an unsorted array at every density tested, including half of complete. The array's flat 47–63 ms is $O(|V|^2)$ visible; the textbook crossover did not arrive.
- Bellman–Ford is $O(|V||E|)$, handles negative edges, and is the only one of the three that can detect a negative cycle.
- Measured: with early termination it matched Dijkstra on random graphs, and was $789\times$ slower on a path that forces all $|V| - 1$ passes. Benchmarking only on random inputs would have missed this entirely.
- Measured: array Dijkstra returned $d(3) = 4$ where the answer is 3, on a four-vertex graph with one negative edge — and the error was visible only because it had propagated.
- Floyd–Warshall is $\Theta(|V|^3)$ with a tiny constant; k must be the outermost loop because it indexes the subproblem. Measured, it beat $|V|$ runs of Dijkstra above a density of about 0.1.

 Review Questions

1. **(MCQ)** Dijkstra's algorithm requires: (a) a connected graph; (b) non-negative edge weights; (c) an acyclic graph; (d) integer weights.
2. **(MCQ)** Floyd–Warshall runs in: (a) $\Theta(V^2)$; (b) $\Theta(VE)$; (c) $\Theta(V^3)$; (d) $\Theta(E \lg V)$.
3. **(Short)** State the relaxation operation and explain why all three algorithms are built from it (*CLO-7*).
4. **(Short)** Give a four-vertex graph on which Dijkstra fails, and explain why a three-vertex one will not do.
5. **(Short)** Why must k be the outermost loop in Floyd–Warshall?
6. **(Applied)** Prove that if a graph has no negative cycle, some shortest path between any two vertices uses at most $|V| - 1$ edges, and explain how Bellman–Ford's final pass uses this.
7. **(Applied)** A routing system has 2,000 nodes, 8,000 links, needs all-pairs distances, and some link costs are negative subsidies. Choose an algorithm, give the arithmetic from this chapter's measurements, and state what you would check about the input first.

PART VI

Strings, Limits and Practice

The book closes where the analysis runs out. Chapter 21 is one more well-solved problem; Chapter 22 is the class of problems nobody can solve efficiently and nobody can prove cannot be — the largest open question in the subject, stated precisely enough to recognise.

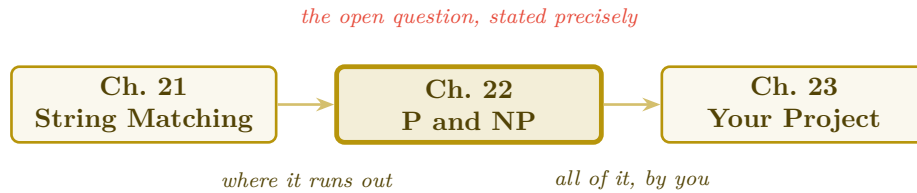


Figure P6.1: Reading path through Part VI. Chapter 22 is the only chapter whose central claim is that nobody knows; Chapter 23 is where you use the rest.

String Matching

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part VI — Strings, Limits and Practice

🕒 Learning Outcomes

By the end of this chapter you will be able to:

- **Trace** the naive, Rabin–Karp and Knuth–Morris–Pratt algorithms on a given text and pattern (*CLO-8*).
- **Derive** the prefix function and **explain** why it gives the correct shift after a mismatch.
- **Prove** that KMP runs in $\Theta(n + m)$ by an amortised argument.
- **Identify** the inputs on which the naive and Rabin–Karp algorithms degrade, and **explain** why KMP has none.
- **Choose** an algorithm for a stated matching workload.

🔗 Before You Start

Chapter 13 for Rabin–Karp — it is a rolling hash and inherits every caveat of Chapter 13. Chapter 17 for the KMP running-time proof, which is an amortised argument.

📖 Key Terms in This Chapter

pattern • text • occurrence • naive matching • rolling hash • Rabin–Karp • spurious hit • prefix function • border • Knuth–Morris–Pratt • shift

21.1 The Problem, and the Obvious Algorithm

📖 String Matching

Given a text T of length n and a pattern P of length $m \leq n$, find every shift s such that $T[s + 1..s + m] = P$.

📖 NAIVE-MATCH(T, P)

```

1 for  $s \leftarrow 0$  to  $n - m$  do
2    $i \leftarrow 1$ 
3   while  $i \leq m$  and  $T[s + i] = P[i]$  do
4      $i \leftarrow i + 1$ 
5   end while
6   if  $i = m + 1$  then
7     report an occurrence at  $s$            ▷ then restart at  $s + 1$  anyway
8   end if
9 end for

```

Figure 21.1: NAIVE-MATCH. After a mismatch at shift s it begins again at $s + 1$, re-reading characters it has already examined. That re-reading is the whole of the $O(nm)$.

💡 What the naive algorithm forgets

Suppose $P = \text{abcabcd}$ and the first mismatch at shift s occurs at position 7 — so $T[s+1..s+6] = \text{abcabc}$.

The naive algorithm now tries shift $s+1$, comparing $P[1] = \text{a}$ with $T[s+2] = \text{b}$. But we *already know* $T[s+2] = \text{b}$, because we just read it. Shifts $s+1$ and $s+2$ are both hopeless and could have been skipped without looking at the text at all.

The information to skip them is in the pattern, not the text. Both better algorithms exploit this, in different ways: Rabin–Karp summarises the window with a hash, and KMP precomputes exactly how far it may safely jump.

21.2 Rabin–Karp

💡 Compare numbers, not strings

Treat each window of m characters as a number in base d modulo q . Then comparing a window with the pattern is *one* integer comparison rather than up to m character comparisons.

The hash of the next window is computed from the current one in $O(1)$:

$$h_{s+1} = (d(h_s - T[s+1]d^{m-1}) + T[s+m+1]) \bmod q,$$

— drop the leading character, shift, add the new trailing one. That is why it is a **rolling** hash.

Unequal hashes prove a mismatch. Equal hashes prove nothing, so every hash match must be verified character by character. A verification that fails is a **spurious hit**, and the running time is

$$O(n + m) + O(m \times (\text{number of spurious hits})).$$

⚠️ Rabin–Karp inherits every caveat of Chapter 13

Its expected $O(n + m)$ assumes spurious hits are rare, which assumes the hash scatters windows well. That is Chapter 13's simple uniform hashing assumption, with the same two ways to fail: a weak modulus by accident, or a text chosen by an adversary who knows q .

The defence is also the same: a large modulus, and if the input is untrusted, a modulus chosen at random at run time.

21.3 Knuth–Morris–Pratt

📖 Prefix Function

For a pattern P of length m , the **prefix function** $\pi[i]$ is the length of the longest proper prefix of $P[1..i]$ that is also a suffix of it.

For $P = \text{ababaca}$:

i	1	2	3	4	5	6	7
$P[i]$	a	b	a	b	a	c	a
$\pi[i]$	0	0	1	2	3	0	1

$\pi[5] = 3$ because **aba** is both a prefix and a suffix of **ababa**.

💡 Why that is exactly the right shift

Suppose the pattern has matched k characters and then fails. The matched text is $P[1..k]$. Any shift that could still succeed must align a prefix of P with a *suffix* of what has already matched — because that suffix is the only part of the text still under the pattern.

So the next viable shift aligns the longest proper prefix of $P[1..k]$ that is also a suffix of it. That is $\pi[k]$, by definition. Shifting to anything shorter would skip a possible occurrence; shifting to anything longer is impossible.

The text pointer never moves backwards. That single property gives the $\Theta(n + m)$ bound, with no hypothesis about the input at all.

≡ PREFIX-FUNCTION(P)

```

1  $\pi[1] \leftarrow 0$ ;  $k \leftarrow 0$ 
2 for  $i \leftarrow 2$  to  $m$  do
3   while  $k > 0$  and  $P[i] \neq P[k + 1]$  do
4      $k \leftarrow \pi[k]$                                 ▷ fall back to a shorter border
5   end while
6   if  $P[i] = P[k + 1]$  then
7      $k \leftarrow k + 1$ 
8   end if
9    $\pi[i] \leftarrow k$ 
10 end for
11 return  $\pi$ 

```

Figure 21.2: PREFIX-FUNCTION. It is KMP run against the pattern itself, and it is $\Theta(m)$ by the same amortised argument.

≡ KMP-MATCH(T, P)

```

1  $\pi \leftarrow$  PREFIX-FUNCTION( $P$ );  $k \leftarrow 0$ 
2 for  $i \leftarrow 1$  to  $n$  do
3   while  $k > 0$  and  $T[i] \neq P[k + 1]$  do
4      $k \leftarrow \pi[k]$                                 ▷  $i$  never decreases
5   end while
6   if  $T[i] = P[k + 1]$  then
7      $k \leftarrow k + 1$ 
8   end if
9   if  $k = m$  then
10    report an occurrence at  $i - m$ 
11     $k \leftarrow \pi[k]$ 
12  end if
13 end for

```

Figure 21.3: KMP-MATCH. The text index i advances on every iteration and never retreats, which is what makes the whole thing linear.

🏆 Theorem 21.1 — KMP

KMP-MATCH runs in $\Theta(n + m)$ time in the worst case.

Proof. An amortised argument, in the pattern of Chapter 17: take the potential $\Phi = k$, the number of characters currently matched.

Φ begins at 0, never goes negative, and each iteration of the **for** loop increases it by at most 1 (line 7). So over n iterations the total increase is at most n .

Each iteration of the inner **while** loop *decreases* Φ by at least 1, since $\pi[k] < k$ always. A quantity that rises by at most n in total and never goes below 0 can fall at most n times. So the inner loop executes at most n times *in total across the whole run* — not n times per iteration.

Hence the matching phase is $O(n)$, and the same argument applied to PREFIX-FUNCTION gives $O(m)$. The $\Omega(n + m)$ is immediate since every character is read. ■

💡 The same looseness as BUILD-HEAP

The naive bound for KMP is “ n iterations, each with a **while** loop that could run m times, so $O(nm)$ ”. That is correct and badly loose, for exactly the reason Chapter 12’s BUILD-HEAP analysis was: it charges every iteration the worst case, and the expensive iterations are paid for by the cheap ones that preceded them.

Three appearances now — BUILD-HEAP, the dynamic array, and KMP — of one idea. If a bound looks like (count) $\times$ (worst case) and the expensive operations consume something the cheap ones built up, the bound is probably loose.

21.4 Measured

📊 Worked Example: First, Do They Agree?

2,000 random texts and patterns over a 4-letter alphabet. Disagreements: 0.

All three return identical lists of occurrences. The naive version is the oracle, as in Chapter 7.

📊 Worked Example: Ordinary Text, Where Nothing Happens

Random text over a 4-letter alphabet, $m = 8$. Comparisons divided by n :

n	naive	Rabin–Karp	KMP	naive (ms)
1,000,000	1.333	0.000	1.250	6.7
2,000,000	1.334	0.000	1.250	12.8
8,000,000	1.333	0.000	1.250	53.7

All three are linear, and the naive algorithm is barely worse than KMP — 1.333 comparisons per character against 1.250. On random text a mismatch arrives after about $1/(1 - 1/|\Sigma|)$ characters, so the naive algorithm almost never re-reads anything.

Rabin–Karp’s column is 0.000 because the hash rejects every window in one integer comparison and the character-comparison counter never moves.

This is the table that makes people think the choice does not matter. It is also the table that any benchmark on English text would produce.

📊 Worked Example: The Text That Destroys the Naive Algorithm

$T = \text{aaaa} \dots \text{a}$ of length 200,000, and $P = \text{aa} \dots \text{ab}$ of length m . Every shift matches $m - 1$ characters and then fails.

m	naive comparisons	$/(nm)$	KMP comparisons	naive (ms)	KMP (ms)
10	1,999,910	1.0000	399,991	3.55	0.85
100	19,990,100	0.9995	399,901	28.98	1.10
1,000	199,001,000	0.9950	399,001	279.22	0.80

The naive count is exactly nm — the ratio is 1.0000, 0.9995, 0.9950. The worst case is not an upper bound being approached; it is being attained.

KMP's count is flat at about $2n$, independent of m . The pattern got a hundred times longer and KMP did the same work.

At $m = 1,000$, KMP is $349\times$ faster. And the input is not contrived: runs of repeated characters occur in DNA, in log files, in padded records and in compressed streams.

Worked Example: And the Input That Destroys Rabin–Karp

Rabin–Karp's expected bound assumes spurious hits are rare. Shrink the modulus and watch:

modulus q	spurious hits	comparisons	time (ms)
13	153,522	204,708	53.1
1,009	2,007	2,917	47.6
1,000,003	0	208	47.8
1,000,000,007	0	208	47.8

The verification count explodes a thousandfold and the clock barely moves — 53.1 ms against 47.8. That is worth pausing on, because it is an honest negative result: on *random* text a spurious candidate is rejected after about 1.3 characters, so a hundred thousand of them cost almost nothing.

The $O(nm)$ bound needs both many candidates *and* expensive verification. Here is an input with both — $T = \mathbf{a}^n$, $P = \mathbf{a}^m$, so every window is a genuine match and every verification runs the full m :

m	RK comparisons	$/(nm)$	RK (ms)	KMP (ms)
10	1,999,910	1.0000	8.48	2.02
100	19,990,100	0.9995	40.21	2.14
1,000	199,001,000	0.9950	368.07	2.28

Exactly nm again, and KMP is $161\times$ faster at $m = 1,000$.

The first attempt at this experiment used $P = \mathbf{a}^{m-1}\mathbf{b}$ — the naive algorithm's worst case — and measured *zero* comparisons, because that pattern's hash differs from every window's and Rabin–Karp rejected all of them instantly. The two algorithms have different worst cases, and assuming they share one was the mistake.

💡 Choosing

Algorithm	Typical	Worst case	Use when
naive	$O(n)$	$O(nm)$	m is tiny, the alphabet is large, and the text is not repetitive
Rabin–Karp	$O(n + m)$	$O(nm)$	searching for <i>many</i> patterns at once — one pass, one hash set
KMP	$\Theta(n + m)$	$\Theta(n + m)$	a guarantee is needed, or the text is repetitive, or it is streaming

Rabin–Karp’s real advantage is not speed — KMP beats it and has a guarantee. It is that a *set* of patterns can be matched in one pass by putting all their hashes in a hash set, which neither of the others can do. That generalises to the Rabin–Karp fingerprint used in rsync and in plagiarism detection.

KMP’s real advantage is that the text pointer never moves backwards, so it works on a stream that cannot be rewound — a socket, a pipe, a file too large to hold.

! Common Pitfalls

- **Benchmarking only on English text.** Measured, the naive algorithm was within 7% of KMP there and $349\times$ worse on repetitive input.
- **Assuming two algorithms share a worst case.** The pattern that is worst for naive matching made Rabin–Karp do *zero* comparisons.
- **Forgetting to verify a Rabin–Karp hash match.** Equal hashes do not mean equal strings. Skipping the check makes it fast and wrong.
- **A small or fixed modulus on untrusted input.** Chapter 13’s flooding attack, in a new costume.
- **Getting π off by one.** It is the longest *proper* prefix that is also a suffix; $\pi[i] < i$ always, and $\pi[i] = i$ would make the matching loop hang.
- **Believing KMP is $O(nm)$ because of the nested **while**.** The amortised argument is the whole point.
- **Reaching for KMP when `std::search` will do.** For one short pattern in a non-repetitive text, the library is fine and it is already written.

🔧 Try It Yourself: Three Algorithms and Two Traps

Reproduces all five tables. Expect thirty minutes.

```
// ch21_strings.cpp -- naive, Rabin-Karp and KMP, on text and on traps.
// Build: cl /O2 /EHsc /std:c++17 ch21_strings.cpp (see Appendix A)

// --- 1. Naive matching: try every alignment -----
// After a mismatch at offset s it restarts at s+1, re-reading
// characters it has already looked at. That re-reading is the  $O(nm)$ .

// --- 2. Rabin-Karp: compare numbers, not strings -----
// A rolling hash of each window. Equal hashes mean a POSSIBLE match
// that must be verified; unequal hashes mean a certain mismatch,
// decided in one comparison. Expected  $O(n+m)$ ; the worst case is every
```

```

// window colliding, which is Chapter 13's problem in a new costume.
    if (hp == ht) { // candidate -- verify it
        size_t i = 0;
        while (i < m) { ++cmps; if (t[s+i] != p[i]) break; ++i; }
        if (i == m) hits.push_back(s);
        else if (spurious) ++spurious; // hash agreed, text did not
    }
    ht = (ht + mod - (high * (unsigned char)t[s]) % mod) % mod;
    ht = (ht * base + (unsigned char)t[s + m]) % mod;

// --- 3. KMP: never re-read a character of the text -----
// pi[i] is the length of the longest proper prefix of p[0..i] that is
// also a suffix of it. On a mismatch, that is exactly how far the
// pattern can be shifted without missing an occurrence -- so the text
// pointer only ever moves forward, giving O(n+m) with no hypothesis
// about the input at all.
static std::vector<size_t> prefix_function(const std::string& p) {
    std::vector<size_t> pi(p.size(), 0);
    for (size_t i = 1; i < p.size(); ++i) {
        size_t k = pi[i - 1];
        while (k > 0 && p[i] != p[k]) k = pi[k - 1];
        if (p[i] == p[k]) ++k;
        pi[i] = k;
    }
    return pi;
}

// --- 4. All three agree, on every input used below -----
// --- 5. Ordinary text, where nothing much happens -----
// This is the case that makes people think the choice does not matter.

// --- 6. The text that destroys the naive algorithm -----
// text = aaaa...a, pattern = aaa...ab. Every alignment matches m-1
// characters and then fails, so the naive version does n*m
// comparisons -- the worst case, reached exactly.

// --- 7. The input that destroys Rabin-Karp -----
// Shrink the modulus and every window becomes a candidate that must
// be verified. This is Chapter 13's adversarial hashing again.

// --- 8. Rabin-Karp's actual worst case -----
// The pattern a^(m-1)b does NOT do it: its hash differs from every
// window of a^n, so Rabin-Karp rejects all of them in one comparison
// each and never verifies anything. (The first version of this
// program measured exactly zero comparisons, which was the clue.)
//
// The bound needs every window to be a CANDIDATE. Take p = a^m: now
// every window hashes equal, every verification runs the full m
// characters, and the total is n*m.

```

What to notice. Parts 7 and 8 together. Part 7 shows a thousandfold rise in verifications that costs 11% of the running time; part 8 shows the input where the same bound actually bites. A measurement that moves a counter dramatically and the clock barely at all is telling you the counter is not what costs.

Then try to break it. Use KMP to search for *all* occurrences of 10,000 different patterns in one text, and compare with a Rabin–Karp that puts all 10,000 hashes in a `unordered_set` and makes a single pass. Predict first. KMP must run 10,000 times; Rabin–Karp runs once. This is the workload where the algorithm with the worse guarantee is the right answer, and it is why Rabin–Karp is still in use.

? Checkpoint — Test Yourself

1. Compute the prefix function for `aabaaab` and state the shift KMP uses after a mismatch at position 6 (*CLO-8*).
2. Why does KMP's inner **while** loop not make it $O(nm)$? Give the potential function.
3. The pattern $\mathbf{a}^{m-1}\mathbf{b}$ is the naive algorithm's worst case and makes Rabin–Karp do no work at all. Explain both facts from the algorithms' structure.

✓ Chapter Summary

- Naive matching re-reads text after a mismatch, giving $O(nm)$ worst case and near-linear behaviour on random text.
- Rabin–Karp compares rolling hashes: $O(n + m)$ expected, with every hash match verified. It inherits all of Chapter 13's assumptions.
- KMP's prefix function gives the exact safe shift, so the text pointer never moves backwards and the bound is $\Theta(n + m)$ with no hypothesis about the input.
- KMP's bound is proved by an amortised argument with $\Phi = k$ — the same looseness that BUILD-HEAP and the dynamic array had.
- Measured on random text: 1.333 comparisons per character for naive and 1.250 for KMP. The choice looks irrelevant.
- Measured on $T = \mathbf{a}^n$: the naive count was exactly nm and KMP's was flat at $2n$. At $m = 1,000$ KMP was $349\times$ faster.
- Measured: a weak Rabin–Karp modulus produced 153,522 spurious hits and cost only 11% more time, because on random text each verification fails immediately. The $O(nm)$ bound needs an input with both many candidates and expensive verification — $P = \mathbf{a}^m$ gives exactly nm , and KMP is $161\times$ faster.
- Rabin–Karp's real advantage is matching many patterns in one pass; KMP's is working on a stream that cannot be rewound.

🔧 Review Questions

1. (MCQ) KMP's worst-case running time is: (a) $\Theta(nm)$; (b) $\Theta(n \lg m)$; (c) $\Theta(n + m)$; (d) $\Theta(n^2)$.
2. (MCQ) In Rabin–Karp, equal hash values mean: (a) a confirmed occurrence; (b) a candidate that must be verified; (c) a certain mismatch; (d) the modulus is too small.
3. (Short) Define the prefix function and explain why it gives the correct shift after a mismatch (*CLO-8*).
4. (Short) Give the amortised argument for KMP's linear bound.
5. (Short) Name the input that defeats the naive algorithm and the input that defeats Rabin–Karp, and say why they are different.
6. (Applied) Trace KMP matching `ababaca` in `abababacaba`, listing the value of k after each text character.
7. (Applied) You must scan a 50 GB log stream, which cannot be rewound, for any of 200 known error signatures. Choose an algorithm (or combination), justify it against this chapter's measurements, and say what you would check about the signatures first.

Complexity Classes: P, NP and NP-Completeness

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part VI — Strings, Limits and Practice

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Define** P, NP, NP-hard and NP-complete, and **contrast** them (*CLO-4*).
- **Distinguish** verifying a solution from finding one, and **explain** why that distinction defines NP.
- **Explain** what a polynomial-time reduction is and what it proves.
- **Recognise** when a problem is likely NP-complete, and **state** what to do about it.
- **Explain** why a pseudo-polynomial algorithm does not settle P versus NP.

🔗 Before You Start

Chapter 2 for input size as bit length, and Chapter 16 for the pseudo-polynomial knapsack — Section 22.5 depends on both. Chapter 10 for what a statement about a *problem* looks like.

📖 Key Terms in This Chapter

decision problem • P • NP • certificate • verifier • polynomial-time reduction • NP-hard • NP-complete • SAT • Cook–Levin • pseudo-polynomial • approximation

⚠️ Where this chapter stops, and why

Appendix F records the boundary. **Theory of Automata** owns computability — Turing machines, decidability, the halting problem. This course owns *complexity*: among the problems that *can* be solved, which can be solved *efficiently*.

The distinction matters. The halting problem has no algorithm at all. NP-complete problems all have algorithms; the algorithms are just too slow, and nobody knows whether that is necessary.

22.1 Decision Problems, and Two Classes

📖 Decision Problem

A **decision problem** has a yes/no answer. “Is there a subset of these numbers summing to t ?” is one; “what is the largest such subset?” is not.

The theory is built on decision problems because they are the simplest form with the same essential difficulty: an efficient algorithm for the decision version almost always yields one for the optimisation version, by binary search on the answer.

📖 P and NP

P is the class of decision problems solvable by an algorithm running in time polynomial in the input size.

NP is the class of decision problems for which a *yes* answer has a **certificate** that can be *verified* in polynomial time.

Formally, $L \in \text{NP}$ if there is a polynomial-time verifier V and a polynomial p such that

$$x \in L \iff \exists c, |c| \leq p(|x|), V(x, c) = \text{YES}.$$

⚠ NP does not stand for “not polynomial”

It stands for **nondeterministic polynomial**. Every problem in P is also in NP — if you can solve it quickly you can verify a certificate quickly by ignoring the certificate and solving it. So $P \subseteq \text{NP}$, and the open question is whether the inclusion is strict.

💡 Checking against finding

This is the whole of NP in one sentence: **it is the class of problems where recognising a good answer is easy, even if producing one is not.**

- A jigsaw: checking that a completed puzzle is correct takes a glance; completing it does not.
- A sudoku: verifying a filled grid is a few hundred comparisons; solving one is a search.
- A factorisation: checking $p \times q = n$ is one multiplication; finding p and q is what RSA’s security rests on.

$P = \text{NP}$ would say that whenever a solution can be recognised quickly it can also be *found* quickly — that there is never any real difficulty in searching, only in knowing what you are looking for. Almost nobody believes it, and nobody can prove it either way.

22.2 Measured: the Gap Between Checking and Finding

📋 Subset Sum

Given a set of integers S and a target t , is there a subset of S summing to exactly t ?
The certificate is the subset itself. Verifying it is one pass — $O(n)$ additions.

🧮 Worked Example: The Two Activities, Timed

`code/ch22_complexity.cpp`, on instances with a planted solution so the answer is always *yes* and the search cannot exit early by luck:

n	verify (ns)	find by search (ms)	ratio
16	12.1	0.78	6.4×10^4
20	13.4	16.82	1.3×10^6
24	20.7	877.58	4.2×10^7
28	18.9	5221.69	2.8×10^8

The verify column is flat — 12 to 21 nanoseconds — because verification is linear in n and n is under 30.

The find column doubles with every element added, from 0.78 ms to 5.2 seconds.

And the ratio grows without limit: 64 thousand at $n = 16$ and 280 million at $n = 28$. That ratio *is* the P versus NP question, measured on one problem. Nobody knows whether it can be made to stop growing.

(A note on method: verification takes tens of nanoseconds, below the clock's resolution. Timing it one call at a time reported 0.0000 and a ratio of infinity, which is what the first version of this program printed. The table above times a million verifications and divides.)

Worked Example: Where the Wall Is

Extrapolating the brute-force column:

n	estimated time
30	65.7 seconds
40	18.7 hours
50	2.18 years
60	2,240 years
80	2.4×10^9 years
100	2.5×10^{15} years

At $n = 50$ — fifty numbers — exhaustive search takes two years. At $n = 80$ it takes longer than the age of the universe.

And a faster computer barely helps. A machine a thousand times faster moves the two-year mark from $n = 50$ to $n = 60$. This is Chapter 1's exponential alertbox, now attached to a problem thousands of people would like to solve.

Worked Example: Exponential Is Not One Speed

Meet-in-the-middle splits the set in half, enumerates all subset sums of each, and looks for a complementary pair — $O(2^{n/2})$ time and space.

n	brute force (ms)	meet-in-the-middle (ms)	speed-up
20	55.0	0.20	279
24	557.4	0.64	864
28	8,114.2	3.04	2,668
32	(too slow)	12.80	—

Halving the exponent doubles the reachable n , and at $n = 28$ it is $2,668\times$ faster.

But it is still exponential. $2^{n/2}$ at $n = 100$ is 2^{50} — the two-year row of the previous table. Meet-in-the-middle moves the wall; it does not remove it, and no known technique does.

The practical lesson is worth separating from the theoretical one. “NP-complete” does not mean “give up”. It means no *polynomial* algorithm is known, and a better exponential algorithm can still be the difference between $n = 28$ and $n = 56$ — which may be the difference between useless and useful.

22.3 Reductions, and NP-Completeness

Polynomial-Time Reduction

Problem A **reduces** to problem B — written $A \leq_P B$ — if there is a polynomial-time function f transforming any instance x of A into an instance $f(x)$ of B such that

$$x \text{ is a yes-instance of } A \iff f(x) \text{ is a yes-instance of } B.$$

Which way round, and why it is easy to get backwards

$A \leq_P B$ means B is **at least as hard as** A .

If you had a fast algorithm for B you could solve A fast: transform, then solve. So a fast B gives a fast A — meaning B cannot be easier than A .

The direction students reverse. To prove your new problem X is hard, you reduce a *known hard* problem to X — that is, $\text{SAT} \leq_P X$. Reducing X to SAT proves only that X is no harder than SAT, which is true of every problem in NP and says nothing.

Worked Example: A Reduction, Actually Run

Independent set and vertex cover are the same problem wearing different clothes: S is an independent set of G exactly when $V \setminus S$ is a vertex cover.

Why. S is independent iff no edge has both endpoints in S , iff every edge has at least one endpoint outside S , iff $V \setminus S$ covers every edge.

So “is there an independent set of size $\geq k$?” and “is there a vertex cover of size $\leq |V| - k$?” have the same answer, and the transformation $k \mapsto |V| - k$ takes constant time. A reduction is a translation, and a translation can be tested:

3,000 random graphs. Answers differ in 0.

That is not a proof — the argument above is the proof — but it is the kind of check that catches a reduction stated with an off-by-one, which is the commonest way to get one wrong.

NP-hard, NP-complete

A problem B is **NP-hard** if $A \leq_P B$ for *every* $A \in \text{NP}$. It is **NP-complete** if it is NP-hard *and* in NP.

NP-hard problems need not be in NP — the halting problem is NP-hard and is not in NP, since it is not decidable at all.

Theorem 22.1 — Why one NP-complete problem settles them all

If any NP-complete problem is in P, then $P = \text{NP}$.

Proof. Let B be NP-complete and suppose $B \in P$, solvable in time $O(n^k)$. Let A be any problem in NP. Since B is NP-hard there is a polynomial-time reduction f from A to B , computable in $O(n^j)$ for some j ; note $|f(x)| = O(n^j)$ too, since a machine running for $O(n^j)$ steps cannot write more than that.

To decide $x \in A$: compute $f(x)$, then run B ’s algorithm on it. The total is $O(n^j) + O((n^j)^k) = O(n^{jk})$ — polynomial. So $A \in P$, and since A was arbitrary, $\text{NP} \subseteq P$. With $P \subseteq \text{NP}$ always, they are equal. ■

💡 Cook–Levin, and the chain since

Theorem 22.1 needs at least one NP-complete problem to exist. **Cook and Levin** (1971) supplied it: **SAT** — given a Boolean formula, is there an assignment making it true? — is NP-complete. The proof encodes the computation of an arbitrary polynomial-time verifier as a formula.

After that, every further proof is a reduction from a problem already known to be complete. Karp’s 1972 paper gave 21, and the list now runs to thousands:

SAT $\rightarrow$ 3-SAT $\rightarrow$ clique $\rightarrow$ vertex cover $\rightarrow$ Hamiltonian cycle $\rightarrow$ TSP $\rightarrow$ subset sum $\rightarrow$ knapsack $\dots$

This is why the question is one question. A polynomial algorithm for any single one of these thousands of problems would give one for all of them. Fifty years of distinguished failure on every one is the strongest evidence available that $P \neq NP$ — and it is not a proof.

22.4 Why Knapsack Does Not Settle It

⚠ Chapter 16’s algorithm is not a counter-example

Chapter 16 gave a $\Theta(nW)$ dynamic program for 0/1 knapsack, which is NP-complete. That looks like a polynomial algorithm for an NP-complete problem — and therefore like a proof that $P = NP$.

It is not, and Chapter 2 explains why. A running time is polynomial when it is polynomial in the *size of the input*, and a number’s size is its bit length. W is one number of $b \approx \lg W$ bits, so

$$\Theta(nW) = \Theta(n \cdot 2^b),$$

exponential in the input size. The algorithm is **pseudo-polynomial**.

And Chapter 16 measured it. With n fixed at 200, eight successive doublings of W — each adding exactly one bit of input — multiplied the running time by a mean factor of 2.02. That table is the proof that the algorithm is exponential, performed rather than argued.

💡 Three things to do when a problem is NP-complete

Recognising NP-completeness is not the end of the engineering; it is the start of a different kind.

1. **Approximate.** Accept a provably near-optimal answer. Vertex cover has a 2-approximation that is six lines long; metric TSP has a 1.5-approximation. These come with *guarantees*, which is what distinguishes them from the next item.
2. **Use a heuristic.** Simulated annealing, genetic algorithms, branch and bound, SAT solvers. No guarantee, often excellent in practice — modern SAT solvers routinely dispatch instances with millions of variables, which is a fact about real instances rather than about SAT.
3. **Restrict the problem.** Many NP-complete problems are polynomial on a restricted class: vertex cover is easy on trees, graph colouring is easy for two colours, knapsack is fine when W is small. The pseudo-polynomial algorithm is this third strategy, honestly labelled.

What you must not do is keep looking for the exact polynomial algorithm. That is the practical value of this chapter: it tells you when to stop.

! Common Pitfalls

- **Reading NP as “not polynomial”.** It is *nondeterministic* polynomial, and $P \subseteq NP$.
- **Reducing in the wrong direction.** To show X is hard, reduce a known hard problem to X .
- **Calling a problem NP-complete without checking it is in NP.** Without that it is merely NP-hard, and the halting problem shows the difference matters.
- **Treating a pseudo-polynomial algorithm as polynomial.** Measured in Chapter 16: one extra bit of W doubles the time.
- **Concluding that NP-complete means hopeless.** Measured here, meet-in-the-middle was $2,668\times$ faster than brute force and still exponential — and that factor is often the whole engineering problem.
- **Confusing undecidable with intractable.** The halting problem has no algorithm; SAT has many, all slow in the worst case.
- **Proving your problem is in NP and thinking you have shown it is hard.** Every problem in P is in NP.

🔧 Try It Yourself: Checking Against Finding

Reproduces all four tables. Expect thirty minutes; the brute-force column at $n = 28$ takes about five seconds per run.

```
// ch22_complexity.cpp -- checking against finding, measured.
// Build: cl /O2 /EHsc /std:c++17 ch22_complexity.cpp (see Appendix A)

// --- 1. Verifying a certificate: linear, and that is the point ----
// Given a claimed subset (as a bitmask), check it sums to the target.
// No search, no cleverness: one pass. This is what "in NP" means.
static bool verify(const std::vector<long long>& a,
                  unsigned long long cert, long long target) {
    long long s = 0;
    for (size_t i = 0; i < a.size(); ++i)
        if (cert >> i & 1ULL) s += a[i];
    return s == target;
}

// --- 2. Finding a certificate by exhaustive search: 2^n -----
// Every subset. Nothing is wasted and nothing is clever; this is the
// only method known that works for every instance.

// --- 3. Meet in the middle: 2^(n/2) -----
// Split the set in half, enumerate all subset sums of each half, and
// look for a pair that adds to the target. Still exponential -- but
// the exponent is halved, which doubles the largest n you can handle.
// A reminder that "exponential" is not a single speed.

// --- 4. Checking against finding -----
// Verification takes tens of NANOseconds, below the clock's
// resolution -- timing it one call at a time reports 0.0000 and a
// ratio of infinity, which is what the first version printed. Time a
// million verifications and divide.
const int REPS = 1000000;
double tv_ns = time_ms([&] {
    int ok = 0;
    for (int r = 0; r < REPS; ++r) ok += verify(a, plant, target);
    return ok;
}, 3) * 1e6 / REPS;           // ms for REPS -> ns per call

// --- 5. Where the wall is -----
```

```
// The point is not that the numbers are large; it is how FAST they
// become large.

// --- 6. Exponential is not one speed -----

// --- 7. A reduction, actually run -----
// S is an independent set of G exactly when V \ S is a vertex cover.
// So "is there an independent set of size k?" and "is there a vertex
// cover of size |V| - k?" are the same question. A reduction is a
// translation, and translations can be tested.
```

What to notice. Part 4’s ratio column is the P versus NP question in numerical form. It grows from 6×10^4 to 3×10^8 over twelve values of n , and the open problem is whether any algorithm can stop it growing.

Then try to break it. Make the instances *unsatisfiable* — pick a target no subset can reach — and re-run part 4. Predict first. The brute force gets *slower*, because it can no longer stop when it finds a solution and must examine all 2^n subsets. And verification of a *no* answer has no certificate at all: there is nothing short to check. That asymmetry between yes and no is why NP has a mirror class, co-NP, and why “verify in polynomial time” is always stated for one answer only.

🔍 Checkpoint — Test Yourself

1. Define P and NP, and explain why $P \subseteq NP$ (CLO-4).
2. To prove a new problem X is NP-hard, which direction must the reduction go? Explain what the other direction would prove instead.
3. Chapter 16’s knapsack algorithm runs in $\Theta(nW)$ and knapsack is NP-complete. Reconcile these, citing the measured doubling table.

📋 Chapter Summary

- P is what can be *solved* in polynomial time; NP is what can be *verified* in polynomial time given a certificate. NP means nondeterministic polynomial, and $P \subseteq NP$.
- Measured on subset sum: verification was flat at 12–21 ns while exhaustive search went from 0.78 ms to 5.2 s, a ratio growing from 6×10^4 to 2.8×10^8 . That growing ratio is the open question.
- Measured: exhaustive search reaches $n = 50$ in two years and $n = 80$ in longer than the age of the universe. A thousandfold faster machine buys ten more elements.
- Measured: meet-in-the-middle was 2,668× faster at $n = 28$ and is still exponential. NP-complete does not mean hopeless.
- $A \leq_P B$ means B is at least as hard as A . To show X is hard, reduce a known hard problem *to* X .
- NP-complete = NP-hard + in NP. By Theorem 22.1 a polynomial algorithm for any one NP-complete problem gives one for all of NP. Cook–Levin supplied the first; thousands follow by reduction.
- $\Theta(nW)$ knapsack is pseudo-polynomial, not polynomial, because W ’s input size is $\lg W$ bits. It settles nothing.
- When a problem is NP-complete: approximate with a guarantee, use a heuristic without one, or restrict the problem — and stop looking for the exact polynomial algorithm.

 Review Questions

1. **(MCQ)** NP is the class of problems that can be: (a) solved in polynomial time; (b) verified in polynomial time; (c) not solved in polynomial time; (d) solved in exponential time only.
2. **(MCQ)** If an NP-complete problem were shown to be in P, then: (a) nothing would follow; (b) $P = NP$; (c) $P \neq NP$; (d) the halting problem would be decidable.
3. **(Short)** Contrast P, NP, NP-hard and NP-complete, giving an example of a problem that is NP-hard but not in NP (*CLO-4*).
4. **(Short)** Explain what a polynomial-time reduction proves, and which direction it must go to establish hardness.
5. **(Short)** Why does a pseudo-polynomial algorithm for an NP-complete problem not prove $P = NP$?
6. **(Applied)** Show that S is an independent set of G if and only if $V \setminus S$ is a vertex cover, and use it to give a reduction in both directions.
7. **(Applied)** Your team must schedule 300 exams with no clashes and suspects the problem is NP-complete. Describe how you would confirm that suspicion, and set out the three strategies you would then consider, with what each would guarantee.

The Semester Project

Part 1

Part 2

Part 3

Part 4

Part 5

Part 6

Part VI — *Strings, Limits and Practice*

🎯 Learning Outcomes

By the end of this chapter you will be able to:

- **Choose** a project whose claim can be settled by measurement.
- **Design** an experiment that tests an order of growth rather than a machine.
- **Build** a measurement harness with a correctness oracle, median timing and multiple input shapes.
- **Report** results so that a reader can check them, including the ones that did not come out as predicted.
- **Defend** your analysis orally against questions about method.

🔗 Before You Start

All of it. The project is where the three strands of this course — analysis, proof and measurement — are used together on a problem nobody has worked out for you.

📖 Key Terms in This Chapter

hypothesis • measurement harness • oracle • input shape • crossover • reproducibility • confound • negative result • defence

23.1 What the Project Is

📄 The Deliverable

Choose two or more algorithms for one problem. **Implement** them, **analyse** them, **measure** them, and **defend** the result.

Four artefacts are submitted:

- **Code** that compiles with the flags of Appendix A and runs unattended.
- **A report** of 8–12 pages: the analysis, the experiment, the tables, and the discussion.
- **The raw output** of every run quoted in the report.
- **A 15-minute defence**, in which the questions will be about method.

💡 The one requirement that matters

The project must make a claim that measurement can settle.

“I implemented three sorting algorithms” is not a claim. “Introsort’s insertion-sort cutoff should be larger for 64-byte records than for 4-byte integers, and I measured where” is one: it predicts a direction, it can come out false, and a reader can check it.

Every chapter of this book has done the same thing — predicted a ratio, measured it, and reported the result including the times it did not fit. That is the pattern to copy.

23.2 Choosing

💡 Projects that work

Project	The claim	Builds on
Sorting cutoffs by record size	the best cutoff grows with the record size	Chapter 1, Chapter 8
Hash function against real keys	structured keys defeat a division hash; mixing fixes it	Chapter 13
BST shapes	the height of a BST built from <i>your</i> data, against $2.99 \lg n$	Chapter 12
Graph representation crossover	where the crossover is for <i>your</i> graphs	Chapter 18
Dijkstra's priority queue	heap against bucket queue against array, by density	Chapter 20
Knapsack's wall	where $\Theta(nW)$ stops being usable	Chapter 16
String search for many patterns	Rabin–Karp's one-pass multi-pattern trick against repeated KMP	Chapter 21
An NP-complete problem	how far exact search reaches, and what an approximation costs	Chapter 22

A project of your own is encouraged, provided it states a claim and a way of being wrong.

⚠️ Projects that do not work

- **“Compare all the sorting algorithms.”** No claim, no hypothesis, and the answer is in every textbook.
- **Reimplementing a chapter.** The labs are done; the project is for a question the labs did not answer.
- **A claim that cannot come out false.** “Quicksort is fast” is not refutable.
- **Anything needing hardware you do not have.** Measure what you can measure.
- **A claim settled by a single number.** One measurement is an anecdote; a project needs a *trend*.

23.3 The Method, Collected

💡 Every rule this book arrived at the hard way

1. **Build with optimisation on.** A debug build is slower by *different* factors for different code, so the comparison is meaningless (Chapter 1).
2. **Take the median of several runs.** A mean is dragged by one scheduler interruption; one run measures the weather (Chapter 1).

3. **Generate the input outside the timed region.** Chapter 3's first draft allocated inside it and reported pure noise.
4. **Measure the setup separately and subtract it,** and print the setup cost so a reader can see the subtraction was small (Chapter 3).
5. **Check correctness against a slow obvious reference, first.** A faster algorithm that gets a different answer is not faster (Chapter 7).
6. **Report ratios, not absolute times.** The ratio cancels the machine-specific constant; it is the part that is about the algorithm (Chapter 3).
7. **Use more than one input shape.** Chapter 2 measured a $6,600\times$ spread across three; Chapter 9 found a crash on one; Chapter 20 found a $789\times$ slowdown on a shape nobody would generate at random.
8. **State what your clock can resolve,** and claim nothing within an order of magnitude of it (Chapter 3).
9. **Sample randomly, not in insertion order.** Chapter 13's first draft probed the first-inserted keys and reported a flat column that looked like a result.
10. **Count as well as time, where you can.** Chapter 6 abandoned timing for counting and got four decimal places instead of 50% noise.

Worked Example: The Harness, Run

`code/ch23_harness.cpp` implements all ten rules. Run unmodified, with a bottom-up merge sort as the placeholder candidate:

clock resolution	0.000100 ms
smallest trustworthy measurement	0.0100 ms
correctness, 2,000 instances across 5 shapes	all agree

n	random	sorted	reversed	99% sorted	few distinct
125,000	12.40	2.89	2.77	3.89	6.43
500,000	59.74	16.14	14.09	17.28	27.61
2,000,000	245.74	65.78	58.11	77.76	119.45

And the ratios on doubling, which are what the claim rests on:

n	random	sorted	reversed	99% sorted	few distinct	predicted
250,000	2.04	2.05	2.23	2.22	2.15	2.11
1,000,000	2.19	2.29	2.39	2.33	2.12	2.10
2,000,000	2.12	2.37	2.44	2.33	2.17	2.10

Read what this does and does not establish. All five ratio columns sit near the predicted $2 + 2/\lg n$, so the implementation is $\Theta(n \lg n)$ on all five shapes — which is the guarantee Chapter 8 proved.

But the absolute times differ by $4.2\times$ between random and reversed input at $n = 2,000,000$. Same order of growth, same algorithm, four-fold spread — and Chapter 8 identified the mechanism as branch misprediction. A report that gave only the first table would have missed it; a report that gave only the second would have claimed the shapes were equivalent.

Both tables, always. The ratios test the analysis; the absolute times show what a user will experience.

23.4 The Report

💡 Structure

Section	Pages	Contains
Problem and claim	1	what you are comparing and what you predict, stated so it could be false
Analysis	2–3	the asymptotic argument for each algorithm, with the recurrence or the counting argument worked
Method	1–2	machine, compiler, flags, input generation, repetitions, what is timed and what is subtracted
Results	2–3	tables with ratio columns; one figure if it helps
Discussion	2	where prediction and measurement agreed, where they did not, and what you think happened
Threats	0.5	what could have confounded the result

⚠️ Report what did not work

The strongest sections of this book are the ones where the first attempt failed:

- Chapter 3: a benchmark that allocated inside the timed region and produced noise.
- Chapter 18: a bit-packed matrix that was *slower* than the byte version until its bit-scan was fixed.
- Chapter 13: a load-factor experiment that sampled keys in insertion order and reported a flat 1.096 at every load factor.
- Chapter 21: a worst case for Rabin–Karp that produced zero comparisons, because it was the *naïve* algorithm’s worst case.
- Chapter 20: a counter-example for Dijkstra on which Dijkstra got the right answer.

Every one of those is in the handout, with the failure described. They are more instructive than the successes, and a report that admits one is more credible than one that does not — because the alternative hypothesis, that you had no failures, is not believable.

A negative result is a result. “I predicted the crossover at $n \approx 1000$ and did not find one up to $n = 10^6$ ” is a perfectly good project outcome, provided you say what you would conclude from it.

23.5 The Defence

💡 What you will be asked

Fifteen minutes. The questions are about method, not about whether you can recite a bound.

1. *What would have falsified your claim?* If nothing would, it was not a claim.
2. *Why those input shapes?* Name the case each one is probing.
3. *How do you know your implementation is correct?* Point at the oracle.
4. *Your ratio column reads 3.8 and you predicted 4. Is that agreement?* Know your noise level and your clock resolution.
5. *What is the slowest part, and is it the part you analysed?* Many projects measure their input generator.
6. *What did you expect that did not happen?*

7. What would you measure next, with another week?

💡 Marking

Component	Weight	Looking for
Correctness of implementation	20%	an oracle, and it passes
Quality of the analysis	25%	recurrences solved, bounds justified, not quoted
Experimental design	25%	repetitions, shapes, setup excluded, ratios reported
Honesty of reporting	15%	anomalies shown and discussed, not smoothed
Defence	15%	answers about method

Note that honesty carries more weight than the result. A careful negative result marks higher than a confident claim unsupported by its own tables.

! Common Pitfalls

- **Timing a debug build.** The single commonest fatal error.
- **Measuring your input generator.** Shuffling 10^7 elements costs more than sorting them in some designs.
- **Reporting absolute times only.** They are a fact about your laptop.
- **One input shape.** Usually “random”, which is the shape on which the interesting failures do not occur.
- **Claiming a crossover from two points.** Two points determine a line and nothing else.
- **Smoothing an anomaly out of the table.** It is the most interesting thing you found.
- **Quoting an asymptotic constant from a textbook as though it applied at your n .** Chapter 12 measured 2.46 against an asymptote of 2.99, and the theory was right.
- **Leaving the defence until the report is finished.** The questions above are better used as a checklist while you work.

🔧 Try It Yourself: The Harness

A complete measurement harness. Copy it, replace the two algorithms in part 3, and it produces the tables your report needs.

```
// ch23_harness.cpp -- the measurement harness for your semester project.
// Build: cl /O2 /EHsc /std:c++17 ch23_harness.cpp (see Appendix A)

// --- 1. Timing: median, and honest about its own limits -----
// A mean is dragged by one scheduler interruption; a minimum flatters
// the cache; one run measures the weather. The median of an odd
// number of runs is the cheapest estimator that is not actively
// misleading.
//
// And: what is the smallest interval this clock can resolve? Any
// measurement within an order of magnitude of it is not evidence.
static double clock_resolution_ms() {
    using C = std::chrono::steady_clock;
    double best = 1e9;
    for (int i = 0; i < 1000; ++i) {
        auto a = C::now(), b = C::now();
```

```

    while (b == a) b = C::now();
    best = std::min(best,
        std::chrono::duration<double, std::milli>(b - a).count());
}
return best;
}

// --- 2. The input shapes that catch things -----
// Chapter 2 measured a 6,600x spread across these on one algorithm;
// Chapter 9 found a crash on one of them; Chapter 20 found a 789x
// slowdown on a shape nobody would have generated at random. Testing
// only on "random" is how all of those get missed.
enum Shape { RANDOM, SORTED, REVERSED, NEARLY_SORTED, FEW_DISTINCT };

// --- 3. The thing under test, and the thing it is checked against --
// REPLACE THESE TWO. The reference must be the version you can read
// in one sitting and be sure of; the candidate is the one you are
// claiming something about. Chapter 7 is why both exist.
static void reference(std::vector<int>& a) {          // obviously correct
    std::sort(a.begin(), a.end());
}
static void candidate(std::vector<int>& a) {          // under test
    /* your algorithm here */
}

// --- 4. Correctness first, timing second -----
// A faster algorithm that gets a different answer is not faster. This
// runs before anything is timed, and a failure here stops the report.

// --- 5. Gate: do not report timings for a wrong algorithm -----

// --- 6. The table: every shape, every size, with ratios -----
// Setup (the copy) is measured separately and subtracted, so what is
// reported is the algorithm and not the memcopy.
double tc = time_ms([&] {
    std::copy(base.begin(), base.end(), work.begin()); }, 7);
double tt = time_ms([&] {
    std::copy(base.begin(), base.end(), work.begin());
    candidate(work);
}, 7);
double t = tt - tc;

// --- 7. The ratios, which is what the claim rests on -----
// Report the ratio column, not the absolute times: the absolute times
// are a fact about this laptop and the ratios are a fact about the
// algorithm.

```

What to notice. The harness refuses to print timings if the correctness check fails. That is deliberate, and it is the single most useful line in the file.

Then make it yours. Replace `candidate` with an algorithm you expect to be $\Theta(n^2)$ and check that the predicted-ratio column, which still prints $2 + 2/\lg n$, now visibly disagrees with every measured column. A harness that cannot show you are wrong is not measuring anything.

🔍 Checkpoint — Test Yourself

1. Take one of the project ideas in Section 23.2 and state it as a claim that could come out false.
2. Your ratio column reads 3.6, 4.4, 3.9, 4.1 and you predicted 4. What would you need to know before calling that agreement?

3. Name three input shapes you would test a hash table with, and say what each one is probing.

✓ Chapter Summary

- The project implements, analyses, measures and defends. It must make a claim that measurement can settle and that could come out false.
- Ten rules of method, each one arrived at by a failure recorded in this book: optimise, take medians, generate input outside the timing, subtract setup, check against an oracle, report ratios, use several input shapes, state the clock resolution, sample randomly, and count rather than time where you can.
- `code/ch23_harness.cpp` implements all ten and refuses to report timings if the correctness check fails.
- Measured with the harness: five input shapes all gave ratios near the predicted $2 + 2/\lg n$, and absolute times differing by $4.2\times$. Report both tables — the ratios test the analysis, the times show what a user experiences.
- Report what did not work. Five first-attempt failures are described in this handout, and they are more instructive than the successes.
- A negative result is a result, provided you say what follows from it.
- The defence asks about method: what would have falsified the claim, why those shapes, how you know the code is right.

✍ Review Questions

1. **(MCQ)** The most important property of a project claim is that it: (a) is true; (b) could be shown false by measurement; (c) is novel; (d) involves several algorithms.
2. **(MCQ)** Absolute running times should be reported: (a) instead of ratios; (b) alongside ratios; (c) never; (d) only for the fastest algorithm.
3. **(Short)** Give four rules of experimental method from this chapter and the failure each one prevents.
4. **(Short)** Why does the harness check correctness before timing?
5. **(Short)** Explain why a negative result can earn full marks.
6. **(Applied)** Design the experiment for one project in Section 23.2: state the claim, the input shapes, the sizes, the repetitions, what you would compute, and what result would refute you.
7. **(Applied)** A classmate reports that algorithm A is $1.3\times$ faster than B, from one run at one size on random input. List everything wrong with that, and write the two-sentence instruction that would fix it.

Appendices

The C++ Toolkit

Every measurement in this handout was produced by compiling and running a program in `code/` on one machine. This appendix is what you need to do the same, and the rules that make the result mean something.

A.1 The Machine These Numbers Came From

Item	This book
Operating system	Windows 11 Pro, 64-bit
Compiler	MSVC 19.44 (Visual Studio 2022 Build Tools, toolset 14.44)
Flags	/O2 /EHsc /std:c++17
Standard library	Microsoft STL as shipped with that toolset

⚠ Your absolute numbers will differ. Your ratios should not.

A different processor, compiler or afternoon changes every millisecond in this book. What it must not change is the *ratio* column — insertion sort's ≈ 4 on doubling, merge sort's ≈ 2.2 , trial division's ≈ 2 per two bits.

If your absolute times differ, that is expected. If your *ratios* differ, something is wrong with the measurement, and Section A.4 is the checklist.

A.2 Installing the Compiler

💡 Windows: MSVC

Install **Visual Studio Build Tools** (the free standalone download; the full IDE is not needed) and select the workload *Desktop development with C++*.

The compiler is not on the PATH by default. Open the **x64 Native Tools Command Prompt** from the Start menu, or run

```
"C:\Program Files (x86)\Microsoft Visual
Studio\2022\BuildTools\VC\Auxiliary\Build\vcvars64.bat"
```

in an ordinary prompt first. Then:

```
cl /O2 /EHsc /std:c++17 ch01_growth.cpp
ch01_growth.exe
```

💡 Linux or macOS: g++ or clang++

Every program in `code/` is portable C++17 with two exceptions, both in Chapter 18: `<intrin.h>` and `_BitScanForward64`. Replace them with

```
__builtin_ctzll(bits)
```

on GCC and Clang, or with `std::countr_zero` if you compile as C++20.

Otherwise:

```
g++ -O2 -std=c++17 -o ch01_growth ch01_growth.cpp
./ch01_growth
```

The flag correspondence is `/O2` $\leftrightarrow$ `-O2` and `/std:c++17` $\leftrightarrow$ `-std=c++17`. `/EHsc` sets the exception model and has no `g++` counterpart; it is not needed there.

A.3 Why the Flags Matter

⚠ Timing a debug build is the commonest fatal error

Without `/O2` (or `-O2`) the compiler keeps every variable in memory, inlines nothing, and leaves the loop structure as written. The result is 5 to 10 times slower — and, fatally, slower *by different factors for different code*.

A tight loop over an array loses far more to a debug build than a pointer-chasing recursion does, so a debug-build comparison can reverse the ranking of two algorithms. That is not a slower measurement; it is a measurement of something else.

Every number in this handout was produced with optimisation on, and yours must be too.

💡 Stopping the compiler from deleting your benchmark

An optimising compiler may remove a computation whose result is unused. Then you time an empty loop and report a spectacular speed-up.

Two defences are used throughout `code/`:

- assign the result to a `volatile` variable, which the compiler may not elide — `ch06_master.cpp`'s `sink`;
- fold everything into a checksum and print it at the end — `ch06_master.cpp` prints `(checksum ...)` for exactly this reason.

How to tell it happened: a time that does not grow with n at all, or a measurement at or below the clock resolution. If a sort of ten million elements reports 0.0001 ms, nothing was sorted.

A.4 Timing Correctly

💡 The rules, with the chapter that learnt each

1. **Use `std::chrono::steady_clock`**, not `system_clock`. The latter can jump backwards when the machine synchronises its time.
2. **Take the median of an odd number of runs.** A mean is dragged by one scheduler interruption; a minimum flatters the cache; one run measures the weather (Chapter 1).
3. **Generate the input before the timer starts.** Chapter 3 allocated inside the timed region and measured the allocator (Chapter 3).
4. **Measure the setup separately and subtract it**, and print the setup cost so a reader can check the subtraction was small (Chapter 3).
5. **Know what your clock can resolve.** `ch23_harness.cpp` measures it; on the book's machine it is 0.0001 ms, so nothing below about 0.01 ms is evidence.
6. **Warm up.** The first run of anything pays for cold caches, page faults and lazy loading. Taking a median of five discards it naturally.
7. **Close everything else.** A browser rebuilding a page in the background is a 30% measurement error.

💡 The timing function used everywhere in code/

```

template <class F>
static double time_ms(F&& f, int reps) {
    std::vector<double> t;
    for (int r = 0; r < reps; ++r) {
        auto t0 = std::chrono::steady_clock::now();
        f();
        auto t1 = std::chrono::steady_clock::now();
        t.push_back(
            std::chrono::duration<double, std::milli>(t1 - t0).count());
    }
    std::sort(t.begin(), t.end());
    return t[t.size() / 2];          // the MEDIAN
}

```

Seven lines, and it encodes rules 1, 2 and 6. Copy it.

A.5 Pseudocode and C++: the Index Translation

⚠️ Why the book indexes from 1 and the code from 0

The pseudocode in this book indexes arrays from 1, as the mathematical literature does; the C++ indexes from 0, as C++ does.

This is deliberate. The mathematics is cleaner from 1 — $A[1..n]$ has n elements, and sums run $\sum_{i=1}^n$ without off-by-one corrections. The code is cleaner from 0, and rewriting it otherwise would make every lab unidiomatic.

The translation:

Pseudocode	C++
$A[1..n]$	<code>a[0 .. n-1]</code>
for $j \leftarrow 2$ to n	<code>for (size_t j = 1; j < n; ++j)</code>
$A[j]$	<code>a[j]</code>
$A[i+1] \leftarrow A[i]$	<code>a[i] = a[i-1]</code>
while $i > 0$	<code>while (i > 0)</code>

Compare INSERTION-SORT in Chapter 4 with `insertion_sort` in `ch01_growth.cpp` line by line; they are the same algorithm and the indices differ by one throughout.

⚠️ One trap this causes

In C++, `size_t` is *unsigned*. The loop

```
for (size_t i = n - 1; i >= 0; --i)
```

never terminates: when i reaches 0 and is decremented it wraps to a very large number, which is still ≥ 0 . The pseudocode **downto** 0 has no such problem.

The idiom used in `code/` is

```
for (size_t i = n; i- > 0; --i)
```

which tests and then decrements, and stops correctly. Chapter 10's counting sort uses it.

A.6 Reproducing the Handout's Tables

💡 What is in code/

One program per chapter, named `chNN_topic.cpp`, each self-contained and each printing the tables that appear in that chapter's worked examples.

Every program:

- needs no input files and no arguments;
- prints its own headings, so the output can be read on its own;
- has numbered step comments `// -- n.` matching the lab listing in the handout;
- runs in under a minute, with the exceptions noted in each chapter's lab box (Chapter 19 is the slowest, at about forty minutes).

`code/README.md` records which results are machine-specific and which should reproduce anywhere.

⚠️ If your numbers look wrong

Symptom	Likely cause
All times 5–10× the book's	debug build; add <code>/O2</code>
A time that does not grow with n	the compiler deleted the computation; add a <code>volatile</code> sink
Ratios wildly noisy at small n	times near the clock resolution; raise n or the repetition count
Ratios noisy at <i>large</i> n	thermal throttling or another process; close things, take more runs
A ratio of exactly 1.00 everywhere	you are timing the setup, not the algorithm
Times vary 2× between runs	a laptop on battery; plug it in

The Mathematics This Course Assumes

Everything here was taught in *Discrete Structures* or *Calculus*. It is collected so that a chapter can point at it rather than re-derive it, and each section names the chapters that need it.

B.1 Logarithms

Notation

Throughout this book:

$$\lg n = \log_2 n, \quad \ln n = \log_e n, \quad \log n = \log_{10} n.$$

When a logarithm appears inside $O(\cdot)$ the base does not matter, since $\log_a n = \log_b n / \log_b a$ and the divisor is a constant. That is why Chapter 6 writes $\Theta(n \lg n)$ and never specifies a base for the bound.

Identities used in this book

Identity	Used in
$\log(ab) = \log a + \log b$	Chapter 10, summing $\lg(n!)$
$\log(a/b) = \log a - \log b$	Chapter 5, $\lg(n/2) = \lg n - 1$
$\log(a^b) = b \log a$	Chapter 2, $\lg(10^7) = 7 \lg 10$
$a^{\log_b n} = n^{\log_b a}$	Chapter 6 — this is where the watershed function comes from
$\log_a n = \frac{\log_b n}{\log_b a}$	change of base; why the base vanishes inside $O(\cdot)$
$\lg n \approx 3.322 \log n$	quick mental conversion

Worth memorising: $\lg 1000 \approx 10$, $\lg 10^6 \approx 20$, $\lg 10^9 \approx 30$. A thousand is ten doublings. Nearly every back-of-envelope estimate in this book uses one of these three.

B.2 Summations

The six that appear

Sum	Closed form	Where
$\sum_{k=1}^n k$	$\frac{n(n+1)}{2}$	insertion sort's worst case (Chapter 2); quicksort's (Chapter 9)
$\sum_{k=1}^n k^2$	$\frac{n(n+1)(2n+1)}{6}$	the cubic max-subarray (Chapter 7)

$\sum_{k=0}^n x^k$	$\frac{x^{n+1} - 1}{x - 1}$	geometric; case 3 of the master theorem
$\sum_{k=0}^{\infty} x^k \ (x < 1)$	$\frac{1}{1 - x}$	why table doubling is linear (Chapter 17)
$\sum_{k=0}^{\infty} \frac{k}{2^k}$	2	why BUILD-HEAP is $\Theta(n)$ (Chapter 12)
$H_n = \sum_{k=1}^n \frac{1}{k}$	$\ln n + \gamma + O(1/n)$	quicksort's expected comparisons (Chapter 9)

$\gamma \approx 0.5772$ is the Euler–Mascheroni constant. The only fact needed about H_n in this book is $H_n = \Theta(\lg n)$.

Worked Example: The Geometric Series, Where It Earns Its Place

Chapter 17 needs the total copying cost of n appends with doubling:

$$1 + 2 + 4 + \cdots + 2^{\lfloor \lg n \rfloor} = \sum_{k=0}^{\lfloor \lg n \rfloor} 2^k = 2^{\lfloor \lg n \rfloor + 1} - 1 < 2n.$$

The point is that the *last* term dominates. The sum of all earlier doublings is less than the final one, which is why amortised analysis works for any growth factor > 1 : the series is geometric and converges.

Chapter 6's case 3 is the same observation at the other end — there the *first* term dominates, because the ratio is less than 1.

B.3 Induction

The template

To prove $P(n)$ for all $n \geq n_0$:

1. **Base case.** Prove $P(n_0)$.
2. **Inductive step.** Assume $P(k)$ for some $k \geq n_0$ (or for all $n_0 \leq k < n$, for *strong* induction), and prove $P(k + 1)$.

Where it appears: Chapter 4's loop invariants are induction on the iteration count, with a third obligation added; Chapter 5's substitution method is induction on n ; and Chapter 12's heap bounds are induction on the height.

Strong induction is needed whenever the recurrence refers to more than one smaller value — $T(n) = 1 + \sum_{j < n} T(j)$ in Chapter 15, for instance.

The two mistakes

1. **Forgetting the base case.** The step alone proves nothing: “if $P(k)$ then $P(k + 1)$ ” is satisfied by statements that are false for every n .
2. **Assuming what is to be proved.** The hypothesis is $P(k)$, for a *specific* k ; concluding $P(k + 1)$ must use it rather than restate it. Chapter 5's strengthening trick is the legitimate version of “assume more”: you prove a *stronger* statement, so the hypothesis gives you more to work with.

B.4 Probability

💡 What Chapter 9 and Chapter 13 need

Expectation. For a random variable X taking value x with probability $p(x)$,

$$\mathbb{E}[X] = \sum_x x p(x).$$

Linearity of expectation, which is the single most useful fact:

$$\mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y],$$

whether or not X and Y are independent. This is why Chapter 9's proof can sum over all $\binom{n}{2}$ pairs without worrying that the comparisons are correlated — and they are heavily correlated.

Indicator variables. $X_A = 1$ if event A occurs and 0 otherwise. Then $\mathbb{E}[X_A] = \Pr[A]$, so counting an expected number of occurrences becomes summing probabilities. Chapter 2's inversion count and Chapter 9's comparison count are both this technique.

📊 Worked Example: Indicators and Linearity, in One Example

Chapter 2 needs the expected number of inversions in a uniform random permutation of n elements.

Step 1. For each pair (i, j) with $i < j$, let $X_{ij} = 1$ if the pair is inverted. The total is $X = \sum_{i < j} X_{ij}$.

Step 2. $\mathbb{E}[X_{ij}] = \Pr[\text{inverted}] = \frac{1}{2}$, since the two orderings are equally likely.

Step 3. By linearity,

$$\mathbb{E}[X] = \sum_{i < j} \frac{1}{2} = \frac{1}{2} \binom{n}{2} = \frac{n(n-1)}{4}.$$

Step 4. Note what was *not* needed: independence. The X_{ij} are strongly dependent — if $a < b$ and $b < c$ then $a < c$ is forced — and linearity does not care. Measured in Chapter 2: 0.54, 0.56, 0.54, 0.50, 0.49 of the worst case, against the predicted 0.5.

B.5 Stirling's Approximation

📖 Stirling

$$n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \left(1 + \Theta\left(\frac{1}{n}\right)\right),$$

and the two consequences this book uses:

$$n! > \left(\frac{n}{e}\right)^n, \quad \lg(n!) = n \lg n - n \lg e + \Theta(\lg n) = n \lg n - 1.4427 n + \Theta(\lg n).$$

⚠️ Use the sharp form, not $n \lg n$

Chapter 10 measured $\lg(n!)$ against both:

n	$\lg(n!)$	$n \lg n$	$n \lg n - 1.443n$
1,000	8,529	9,966	8,523
1,000,000	18,488,885	19,931,569	18,488,874

The crude $n \lg n$ is 17% high at $n = 1,000$ and still 8% high at a million. The sharp form is correct to six decimal places. Both are $\Theta(n \lg n)$, and only one is usable for arithmetic.

B.6 Floors, Ceilings and Useful Inequalities

💡 Facts used without comment in the text

Fact	Where it is used
$x - 1 < \lfloor x \rfloor \leq x \leq \lceil x \rceil < x + 1$	Chapter 5, dropping floors
$\lfloor n/2 \rfloor + \lceil n/2 \rceil = n$	merge sort's split (Chapter 8)
$1 + x \leq e^x$ for all real x	probability bounds
$\binom{n}{k} \leq \left(\frac{en}{k}\right)^k$	counting arguments
$\sum_{u \in V} \deg(u) = 2 E $	every bound in Part V (Chapter 18)
a binary tree of height h has $\leq 2^h$ leaves	the sorting lower bound (Chapter 10)
a complete binary tree on n nodes has height $\lfloor \lg n \rfloor$	heaps (Chapter 12)

B.7 Where Each Topic Is Needed

Topic	Chapters that need it
Logarithm identities	1, 2, 3, 5, 6, 10 — and $a^{\log_b n} = n^{\log_b a}$ is the key step in 6
Arithmetic series	2, 7, 9, 17
Geometric series	6, 11, 17
$\sum k/2^k = 2$	12 only, and it is the whole of BUILD-HEAP's bound
Harmonic numbers	9
Induction	4, 5, 10, 12, 14, 15, 20
Strong induction	15
Linearity of expectation	2, 9, 13
Indicator variables	2, 9
Stirling	10
L'Hôpital's rule	3, for $\lg n = o(n^\epsilon)$

⚠️ If any row is unfamiliar

Read that row's chapters' prerequisite boxes first — each names what it assumes. The three most load-bearing items in the whole book are **linearity of expectation** (Chapter 9's proof collapses without it), $a^{\log_b n} = n^{\log_b a}$ (Chapter 6 has no watershed function without it), and **the geometric series** (Chapter 17 has no result without it).

Answers to the Checkpoints

Three answers per chapter, in order. Attempt the checkpoint from memory before reading these, and argue with them where you disagree — several are judgement calls rather than facts, and they say so.

Chapter 1 — The Role of Algorithms

1. n grew by 4 and the time by 16, so the time grew as n^2 : $\Theta(n^2)$. The reasoning is the ratio test of Section 1.4 — for $T = cn^2$, $T(4n)/T(n) = 16$ independent of c , so the constant never has to be known. One check is weak evidence; three doublings in a row would be convincing.
2. A: $(10^4)^3/10^9 = 10^{12}/10^9 = 1000$ s. B: $100 \times (10^4)^2/10^7 = 10^{10}/10^7 = 1000$ s. They **tie exactly** at $n = 10^4$. Below it B wins, above it A loses — the crossover is at that point, which is the real content of the question.
3. **Harmless:** comparing insertion sort with merge sort at $n = 32,000$, where the orders of growth differ by a factor of 50 and a cache factor of a few cannot reverse the ranking. **Not harmless:** comparing two $\Theta(n \lg n)$ sorts, where the orders of growth are equal and memory behaviour is the whole difference — Chapter 8 measured `std::sort` beating merge sort by 39% for exactly this reason.

Chapter 2 — Input Size and the Three Cases

1. Yes. The input is n^2 numbers, so writing $N = n^2$ for the true input size gives $\Theta(n^3) = \Theta(N^{1.5})$ — polynomial in N . Indeed $\Theta(n^3)$ is only n times the time needed to *read* the input, which is why matrix multiplication is considered nearly efficient rather than slow.
2. Because $(n+1)/2$ and n differ by a constant factor of 2, and $\Theta(\cdot)$ discards constants. Both are $\Theta(n)$. The average case differs from the worst only in the constant, exactly as insertion sort's does (Section 2.2), and that is the common situation.
3. **“Averaged over what distribution of inputs?”** Without it the claim is unfalsifiable. The usual answer — a uniform random permutation — is very often false of real data, which arrives partly sorted; and if the answer is instead “expected over the algorithm's own randomness” then the bound is much stronger and holds for your data too.

Chapter 3 — Asymptotic Notation

1. For $n \geq 1$ we have $n^2 \leq n^3$ and $1 \leq n^3$, so $5n^3 + 200n^2 + 17 \leq (5 + 200 + 17)n^3 = 222n^3$. Take $c = 222$, $n_0 = 1$. At $n = 1$: LHS = 222, RHS = 222, so the inequality holds with equality. A sharper pair such as $c = 6$, $n_0 = 201$ is equally valid — the definition asks only that *some* pair exist.
2. **Yes.** Given any $c > 0$ we need $n^2 < cn^2 \lg n$ for all large n , i.e. $1 < c \lg n$, i.e. $n > 2^{1/c}$. So $n_0 = \lceil 2^{1/c} \rceil + 1$ works for every c , which is the definition of o . (Note $2^{1/c}$ is large when c is small, as it must be — but it is finite, and that is all that is required.)
3. (a) Both are upper bounds and need not be tight: the first algorithm might be $\Theta(n)$ and merely *described* as $O(n^3)$. (b) Even if both bounds are tight, the constants and the range of n decide which is faster in practice — $0.001n^3$ beats $1000n^2$ for every n below 10^6 .

Chapter 4 — Correctness: Loop Invariants

1. Invariant: *at the start of the iteration for i , $total = \sum_{k=1}^{i-1} A[k]$* . Initialisation: $i = 1$ and the empty sum is 0. Maintenance: adding $A[i]$ extends the sum by one term. Termination: $i = n + 1$ gives $\sum_{k=1}^n A[k]$. The measure is $n + 1 - i$, a non-negative integer strictly decreasing each iteration.
2. A program that overwrites $A[1..k]$ with zeros satisfies it: the prefix is sorted at every step. Repair: $A[1..k]$ is sorted **and** contains the k smallest elements of the original array, and $A[k + 1..n]$ holds the rest. The second clause is what forbids inventing values.
3. Initialisation and maintenance give *partial* correctness — if the loop halts, the invariant plus the exit condition give the postcondition. They say nothing about halting. What is missing is a *loop variant*: an integer-valued measure that strictly decreases and is bounded below. Section 4.6's binary search has a valid invariant and no such measure, and it ran for ever on 13,228 of 20,000 trials.

Chapter 5 — Recurrences I

1. The tree is a *path*: one node at each level, doing $n, n - 1, n - 2, \dots$ work, with n levels. The total is $\sum_{k=1}^n k = n(n + 1)/2 = \Theta(n^2)$. It is not $\Theta(n \lg n)$ because the argument shrinks by a constant *amount*, not a constant *fraction* — so the depth is n rather than $\lg n$.
2. Guess $T(n) \leq cn^2$. Then $T(n) \leq 2c(n/2)^2 + n^2 = cn^2/2 + n^2$, which is $\leq cn^2$ provided $n^2 \leq cn^2/2$, i.e. $c \geq 2$. With a base case at $n_0 = 1$ and $c = \max\{2, T(1)\}$ the induction closes. (This is case 3 of the master theorem, confirming the answer.)
3. A 1-to-99 split divides the problem by the constant factors 100 and 100/99, so the depth is $\log_{100/99} n = \Theta(\lg n)$ and every full level does n work: $\Theta(n \lg n)$, with a larger constant. A 1-to- $(n - 1)$ split removes one element per level, so the depth is n and the total is $\Theta(n^2)$. **Constant fraction, not constant amount** is the distinction.

Chapter 6 — Recurrences II: The Master Theorem

1. $a = 4, b = 2, f(n) = n$. Watershed $n^{\log_2 4} = n^2$. Since $f(n) = n = O(n^{2-\varepsilon})$ with $\varepsilon = 1$, this is **case 1**, and $T(n) = \Theta(n^2)$. The leaves dominate; the combining work is irrelevant.
2. **None**. Watershed $n^{\log_2 2} = n$, and $f(n) = n/\lg n$ is *smaller* than n — but only by a logarithmic factor, and case 1 needs $f = O(n^{1-\varepsilon})$ for a *constant* $\varepsilon > 0$. Since $n/\lg n = \omega(n^{1-\varepsilon})$ for every such ε , no case applies. (A recursion tree gives $\Theta(n \lg \lg n)$.)
3. In case 2 every level of the tree does the same total work, so the total is (work per level) $\times$ (number of levels), and the number of levels is $\lg_b n = \Theta(\lg n)$. In case 1 the work grows geometrically towards the leaves and the last level dominates; in case 3 it shrinks geometrically and the root dominates. Only a tie makes every level count, and counting levels is where a logarithm comes from.

Chapter 7 — Brute Force

1. The sum is $\sum_{p=1}^n p(p + 1)/2$. For $n = 100$ this is $\frac{1}{2}(\frac{100 \cdot 101 \cdot 201}{6} + \frac{100 \cdot 101}{2}) = \frac{1}{2}(338,350 + 5,050) = \mathbf{171,700}$. Against $n^3/6 = 166,667$ that is 3% high, as it should be — the leading term is $n^3/6$ and the next is $n^2/2$.
2. The $n^3 \rightarrow n^2$ step removed *redundant arithmetic inside* the search: the same set of $\binom{n}{2}$ candidate blocks is still examined, each now in $O(1)$ instead of $O(n)$. The $n^2 \rightarrow n$ step changed *the search itself* — Kadane's observation about blocks ending at j means pairs are never enumerated at all. The first is careful programming; the second needs an idea about the problem.

3. Ship the cubic version when n is *bounded by something real* and small — a board of at most 64 squares, a form with at most 50 fields — and the routine is off the critical path. Measured, it costs 79 μ s at $n = 100$. **The fact that would change my mind:** that n is user-supplied or unbounded, because then the bound is a prediction rather than a fact.

Chapter 8 — Divide and Conquer

1. Because merge sort's control flow does not depend on the data. The split is always into halves and MERGE always performs $\Theta(n_1 + n_2)$ work, whatever the values. Insertion sort's inner loop runs a number of times that depends on the number of inversions, which is 0, $\approx n^2/4$ or $\approx n^2/2$ depending on the input — so it has three analyses and merge sort has one.
2. The minimum $\min(n_1, n_2)$ is attained when one run is entirely smaller than the other, as on **already-sorted** input, where the loop exhausts the left run first. The maximum $n_1 + n_2 - 1$ is attained when the runs **interleave**, as on random input, so that both are nearly exhausted together. Measured: $0.496 n \lg n$ against $0.940 n \lg n$ — almost exactly the predicted factor of 2.
3. They measure different things. Chapter 1's 65 is where insertion sort beats merge sort on a *whole array*, timed from cold. The cutoff of 32 is where insertion sort beats *continued recursion* on a subarray that is already in cache and already part of a larger sort. The second has less overhead to beat, so it turns over sooner. Either is a plausible answer; the point is that they are not the same quantity.

Chapter 9 — Quicksort

1. On sorted input the pivot (the last element) is the largest, so PARTITION compares it with all $k - 1$ others and splits $(k - 1, 0)$. The subproblem sizes are therefore $n - 1, n - 2, \dots, 1$, and the comparisons are $(n - 1) + (n - 2) + \dots + 1 = n(n - 1)/2$. Measured $0.9990 \rightarrow 0.9999$ of $n^2/2$.
2. “Average case” averages over *inputs* and so needs an assumption about the data; an adversary who knows your algorithm simply supplies an input outside that distribution — and sorted data arrives by accident anyway. “Expected” averages over the algorithm's *own coin flips* with the input held fixed, so it holds for every input including the adversary's. He would need to predict your random numbers, not your data.
3. 1 MB/64 bytes = 16,384 frames, and the depth is $n - 1$, so it crashes for $n > 16,384$ — confirmed at $n = 32,000$. Doubling the stack buys $n = 32,768$: one more doubling of n , for twice the memory. It does not fix anything, because the depth grows linearly and the stack can only grow by constants. Recursing on the smaller side first bounds the depth at $O(\lg n)$ and does fix it.

Chapter 10 — The Comparison Lower Bound

1. At least $4! = 24$ leaves, so height $\geq \lceil \lg 24 \rceil = \lceil 4.585 \rceil = 5$. And $\lg(4!) = 4.585$, so the bound says at least 5 comparisons in the worst case. (Five is in fact achievable, so the bound is tight here.)
2. The step that fails is “a binary tree of height h has at most 2^h leaves” — or rather, the premise that counting sort's execution is modelled by a binary decision tree at all. Counting sort uses a key as an *array index*, an operation with K possible outcomes rather than 2, so it extracts more than one bit per step and the tree is not binary. The theorem's hypothesis, not its conclusion, is what it escapes.
3. The comparison count cannot improve by more than 1%, so essentially none of `std::sort`'s 39% comes from doing fewer comparisons. It comes from everything the comparison model

does not charge for: sorting in place and so touching half as much memory, better locality, and fewer branch mispredictions. The lower bound tells you where the remaining performance is *not*.

Chapter 11 — Selection and Order Statistics

1. Quicksort: $a = 2$, $b = 2$, watershed $n^{\log_2 2} = n = f(n)$, so **case 2**, giving $\Theta(n \lg n)$. Quickselect: $a = 1$, $b = 2$, watershed $n^{\log_2 1} = n^0 = 1$, far below $f(n) = n$, so **case 3**, giving $\Theta(n)$. The $\lg n$ was the number of *levels*; discarding one branch collapses the tree to a path whose work halves each step, and $n + n/2 + n/4 + \dots < 2n$.
2. The last-element pivot on sorted input is always the largest, so the range shrinks from the top by one each iteration. To reach the median the range must fall from n to $n/2$, costing $n + (n-1) + \dots + n/2 = \frac{(n+n/2)}{2} \cdot \frac{n}{2} = 3n^2/8$. Measured $0.7500 \times n^2/2$, which is the same number.
3. With groups of 5, the pivot exceeds at least $3 \cdot \frac{1}{2} \cdot \frac{n}{5} = \frac{3n}{10}$ elements, so the recursion is on at most $7n/10$ and the recurrence is $T(n/5) + T(7n/10) + \Theta(n)$ with $\frac{1}{5} + \frac{7}{10} = \frac{9}{10} < 1$. With groups of 3 the pivot exceeds only $2 \cdot \frac{1}{2} \cdot \frac{n}{3} = \frac{n}{3}$, so the recurrence is $T(n/3) + T(2n/3) + \Theta(n)$ and $\frac{1}{3} + \frac{2}{3} = 1$. The substitution $T(n) \leq cn$ then gives $cn + an \leq cn$, which is false for every c , and the true bound is $\Theta(n \lg n)$.

Chapter 12 — Heaps and Search Trees

1. Because the shape is enforced by the *addressing scheme*, not maintained by an algorithm. A heap is stored in an array with node i 's children at $2i+1$ and $2i+2$, and a new element goes at index n . There is no insertion order that produces an unbalanced heap, because there is no array layout in which an unbalanced heap could be represented. A BST has no such constraint, which is why it needs rotations and a heap does not.
2. The loose step is charging *every* node the cost of the *worst* node. In fact at most $\lceil n/2^{j+1} \rceil$ nodes have height j , so the total is $O(n \sum_{j \geq 0} j/2^j)$. And $\sum_{j \geq 0} j/2^j = 2$, from $\sum_{j \geq 0} jx^j = x/(1-x)^2$ at $x = 1/2$. So the true total is $O(2n) = \Theta(n)$.
3. **“In what order were the keys inserted?”** If they were shuffled, the height is about $2.3 \lg n$ and the benchmark is meaningless for sorted input — which is what real data often is. Measured, the same keys in sorted order gave height exactly n and searches $650\times$ slower.

Chapter 13 — Hashing

1. *Each key is equally likely to hash to any of the m slots, independently of the others.* It is a claim about the hash *and* the keys because the hash function is deterministic: given the keys, the slot of each is fixed, and whether they scatter depends entirely on the relationship between the function and the key set. Section 13.3's $k \bmod 2^{20}$ is a perfectly good hash for unstructured keys and uses 1.6% of the table for keys with a stride of 64.
2. Searching for x_i , the elements examined first are those inserted after it that hashed to the same slot; for $j > i$ that happens with probability $1/m$. Averaging over i : $1 + \frac{1}{nm} \sum_{i=1}^n (n-i) = 1 + \frac{\alpha}{2} - \frac{\alpha}{2n}$. At $\alpha = 4$ this is $1 + 2 = 3$ for large n ; measured **2.998**.
3. The **distribution** of chain lengths — specifically the longest chain and the mean probe count — not the mean chain length. The mean chain length is n/m *by definition*, whatever the hash does, because every key lands somewhere. Only the spread distinguishes a hash that scatters from one that piles up: measured, longest chain 8 against 62, and mean probes 1.5 against 31.0.

Chapter 14 — Greedy Algorithms

1. *Greedy-choice property*: some optimal solution contains the locally best first choice. *Optimal substructure*: an optimal solution contains optimal solutions to the subproblems remaining after that choice. **A problem with the second and not the first**: 0/1 knapsack. Its optimal solution does contain optimal sub-solutions, so dynamic programming works (Chapter 16); but taking the highest value-per-weight item first can be wrong, so greedy fails. Coin change with $\{1, 3, 4\}$ is the same story.
2. **8**. Greedy takes $4 + 3 + 1 =$ three coins; optimal is $4 + 4 =$ two. (The first failure is 6, as the chapter says; 7 is $4 + 3$ for both.) The counter-examples are small because greedy's mistake is made at the *first* step, so the smallest instance exhibiting it is the smallest amount where the largest coin is the wrong first choice — and that is bounded by the largest denomination.
3. Huffman assigns each symbol a *whole number* of bits, with a minimum of 1. A symbol of probability 0.95 carries only $-\lg 0.95 = 0.074$ bits of information, and Huffman must spend 1 on it. The algorithm is provably optimal *among prefix codes*; the restriction to integer code lengths is what costs the 0.71 bits, and arithmetic coding, which can spend fractional bits, removes it.

Chapter 15 — Dynamic Programming I

1. Optimal substructure and overlapping subproblems. **A problem with the first and not the second**: merge sort — sorting optimally requires sorting the halves, but the two halves are different subproblems and no subproblem ever recurs, so memoising it buys exactly nothing. Binary search is another.
2. $T(0) = 1$ and $T(n) = 1 + \sum_{j=0}^{n-1} T(j)$. By induction, assuming $T(j) = 2^j$ for all $j < n$: $T(n) = 1 + \sum_{j=0}^{n-1} 2^j = 1 + (2^n - 1) = 2^n$. At $n = 22$ this is 4,194,304, which is what the program counted — exactly, not approximately.
3. $\Theta(mnk)$: (number of distinct subproblems) $\times$ (work per subproblem). For rod cutting there are n subproblems and the maximum is over at most n choices, giving $\Theta(n \cdot n) = \Theta(n^2)$. Note the measured caveat: with a price table capped at 30 entries the inner loop runs at most 30 times, so the *measured* behaviour was $\Theta(30n)$ — linear. Check the parameters before quoting the bound.

Chapter 16 — Dynamic Programming II

1. $c[i][j] = c[i-1][j-1] + 1$ when $x_i = y_j$, else $\max(c[i-1][j], c[i][j-1])$, with $c[i][0] = c[0][j] = 0$. No maximum is needed in the equal case because including the matching character can never hurt: given any common subsequence of $X_{1..i}, Y_{1..j}$ not ending in it, appending it yields one at least as long. So some LCS does end in that character, and the rest is an LCS of the two shortened prefixes.
2. A running time is polynomial when polynomial in the *size* of the input, and a number's size is its bit length (Chapter 2). W is one number of $b \approx \lg W$ bits, so $\Theta(nW) = \Theta(n2^b)$ — exponential in b . Measured: eight doublings of W , each adding one bit, gave time ratios of mean 2.02. This is the same phenomenon as Chapter 2's trial division, whose eight ratios averaged 2.02 per two bits, reached from the opposite direction.
3. **Appropriate**: computing only the *length* of an LCS, or only the edit *distance*, where the answer is the final table entry. **Not appropriate**: recovering the subsequence or the alignment itself, which requires walking backwards through the whole table. What distinguishes them is whether the answer is a single value or a *path*; a path needs the table that produced it. Hirschberg's algorithm gets both, at twice the code.

Chapter 17 — Amortised Analysis

1. Copying happens at lengths $1, 2, 4, \dots$, the powers of two up to n , and the copy at length 2^k moves 2^k elements. So the total is $\sum_{k=0}^{\lceil \lg n \rceil} 2^k = 2^{\lceil \lg n \rceil + 1} - 1 < 2n$. The measured 1.024 is just over $1n$, not $2n$, because when n is a power of two the sum is exactly $n - 1$; the bound of $2n$ is the worst position within a doubling interval.
2. **Cheap:** $c = 1$, len rises by 1 so Φ rises by 2, giving $\hat{c} = 1 + 2 = 3$. **Expensive** (resize from m to $2m$ with $len = m$): $c = m + 1$; before, $\Phi = 2m - m = m$; after, $\Phi = 2(m+1) - 2m = 2$; so $\hat{c} = (m + 1) + 2 - m = 3$. They *must* come out equal, or the potential function would not be doing its job — the whole point is to choose Φ so that the expensive operation's real cost is exactly paid for by the potential the cheap ones accumulated.
3. Any **latency-bounded** context: a 16 ms frame deadline, an interrupt handler, an audio callback, a trading system. Measured, the amortised cost was 1.049 copies per append and the worst single append copied **524,288** elements — half a million times the average. And only 20 of a million appends were expensive, so a latency histogram with a thousand samples would very likely contain none of them, and the system would look fine until it was not.

Chapter 18 — Graphs

1. Each edge $\{u, v\}$ contributes 1 to $\deg(u)$ and 1 to $\deg(v)$, so summing degrees counts every edge twice. Theorem 18.1 uses it to turn the per-vertex inner loop into a bound in $|E|$: the BFS inner loop runs $\deg(u)$ times for each dequeued u , and $\sum_u \deg(u) = 2|E|$ gives the $\Theta(|V| + |E|)$.
2. $|E| = 3 \times 10^6$. **Matrix:** $|V|^2 = 10^{12}$ bytes = 1 TB — impossible. **List:** about $(10^6 \cdot 8) + (6 \times 10^6 \cdot 4) \approx 32$ MB. BFS is $\Theta(|V|^2) = 10^{12}$ steps from the matrix and $\Theta(|V| + |E|) = 7 \times 10^6$ from the list — a factor of 140,000, and the matrix cannot be built anyway.
3. That a crossover density quoted without an implementation is meaningless. The same two *data structures* gave a crossover at 0.26 with one implementation of the matrix and no crossover at all with another, and the gap between a good and a poor implementation of the matrix ($8\text{--}20\times$) was larger than the gap between the two structures. Quote the crossover for *your* code, measured.

Chapter 19 — Minimum Spanning Trees

1. Let A be a subset of some MST and let $(S, V \setminus S)$ be a cut that no edge of A crosses. Then any light edge for that cut is safe. The hypothesis that makes the exchange safe is **that no edge of A crosses the cut**: it guarantees that the edge e' removed from the cycle — which does cross the cut — is not in A , so A survives the exchange and remains inside the new MST.
2. **Prim's cut** is (vertices in the tree so far) against the rest, so the light edge is the cheapest edge leaving the tree, and the structure needed is a **priority queue**. **Kruskal's cut** is between the two components that the next edge would join, so what is needed is a test for “are these already connected?” — **disjoint-set union**. One theorem, two cuts, two data structures.
3. It is wrong because $\alpha(n)$ is not constant — it grows, and the measurement shows it growing, from 1.62 to 1.88. It does not matter because α grows so slowly that $\alpha(n) \leq 4$ for every n that can be written down; reaching 5 needs an n far larger than the number of atoms in the observable universe. It is also an *amortised* bound, so a single **find** can still cost $\Theta(\lg |V|)$.

Chapter 20 — Shortest Paths

1. The inequality $\delta(s, y) \leq \delta(s, u)$, where y lies on a shortest path to u . It says extending a path cannot make it cheaper, which requires non-negative weights. Without it, the remaining portion from y to u may have negative total weight, so a vertex finalised early can later prove reachable more cheaply — and Dijkstra never revisits it. Measured in Section 20.4: $d(3) = 4$ where the answer is 3.
2. Because a shortest path in a graph with no negative cycle uses at most $|V| - 1$ edges (more would repeat a vertex, and the cycle so formed has non-negative weight and can be removed). Each pass propagates correct distances one more edge along every path, so $|V| - 1$ passes suffice. **The $|V|$ -th pass detects a negative cycle:** if any distance still improves, some path is using $|V|$ or more edges and still getting shorter, which only a negative cycle permits.
3. That a worst-case bound describes the worst case and not the typical one, and that both facts matter. Bellman–Ford’s $O(|V||E|)$ is real — the path graph attained it, $789\times$ slower than Dijkstra — and on random graphs the early exit made it indistinguishable. A benchmark on random inputs alone would have reported them equal; the bound is what told us to go looking for the input where they are not.

Chapter 21 — String Matching

1. For $P = \text{aabaaab}$: $\pi = [0, 1, 0, 1, 2, 2, 3]$. (At $i = 5$, **aa** is a border of **aabaa**; at $i = 7$, **aab** is a border of **aabaaab**.) After a mismatch having matched 6 characters, $k \leftarrow \pi[6] = 2$ — the pattern shifts so that its first 2 characters align with the last 2 matched, a shift of $6 - 2 = 4$ positions.
2. Because the inner loop only ever *decreases* k , and k is increased by at most 1 per outer iteration. Take $\Phi = k$: it starts at 0, never goes negative, and rises by at most n in total across the run, so it can fall at most n times in total. The inner loop therefore executes at most n times *over the whole run*, not per iteration — the same amortised argument as BUILD-HEAP and the dynamic array.
3. **Naive:** every alignment matches the $m - 1$ leading **a**’s and fails on the final **b**, so each of the n shifts costs m comparisons — the worst case, attained exactly. **Rabin–Karp:** it never compares characters at all unless the hashes agree. Every window of $\mathbf{a}^n$ has the same hash, and $\mathbf{a}^{m-1}\mathbf{b}$ has a different one, so every window is rejected in one integer comparison. The two algorithms fail on different things: naive on *long partial matches*, Rabin–Karp on *hash collisions*.

Chapter 22 — Complexity Classes

1. **P** is the class of decision problems solvable in polynomial time. **NP** is the class whose yes-instances have a certificate verifiable in polynomial time. $P \subseteq NP$ because if you can *solve* a problem in polynomial time you can verify any proposed certificate in polynomial time by ignoring it and solving the instance yourself.
2. From a **known hard problem** to X : $\text{SAT} \leq_P X$. That shows X is at least as hard as SAT, since a fast algorithm for X would give one for SAT. The other direction, $X \leq_P \text{SAT}$, proves only that X is no *harder* than SAT — which is true of every problem in NP and says nothing about X ’s difficulty.
3. $\Theta(nW)$ is polynomial in the *value* W and exponential in its *length* $\lg W$, so it is pseudo-polynomial, not polynomial, and does not place knapsack in P. Chapter 16 measured this: with n fixed at 200, eight doublings of W — each adding exactly one bit of input — multiplied

the running time by a mean factor of 2.02. An algorithm whose time doubles when the input grows by one bit is exponential in the input size.

Chapter 23 — The Semester Project

1. For example: “*The best insertion-sort cutoff for 64-byte records is larger than the best cutoff for 4-byte integers, on the same machine.*” It predicts a direction, it names the quantity to measure, and the result “it is smaller” or “there is no difference” would refute it. Compare with “I will compare sorting algorithms”, which no measurement could contradict.
2. **Your noise level and your clock resolution.** If repeated runs of the *same* measurement vary by $\pm 15\%$, then 3.6–4.4 is consistent with 4 and the answer is yes. If they vary by $\pm 1\%$, a spread of 3.6 to 4.4 is a real effect and something else is going on. Also check that the smallest time is well above the clock resolution. A ratio without a noise estimate is not evidence either way.
3. For example: **(a)** uniformly random keys, probing the ideal case the analysis assumes; **(b)** keys with a fixed stride, such as multiples of 64, probing whether the hash uses the whole key — Chapter 13 measured 31 probes instead of 1.5 here; **(c)** keys deliberately chosen to collide under your hash, probing the adversarial worst case. Several other answers are good: a realistic sample of your own data is often the most informative of all.

Glossary

adjacency list. A graph representation storing each vertex's neighbours. $\Theta(|V| + |E|)$ space. (Chapter 18)

adjacency matrix. A $|V| \times |V|$ array of edge indicators. $\Theta(|V|^2)$ space, $\Theta(1)$ edge test. (Chapter 18)

amortised cost. The total cost of a worst-case sequence of n operations, divided by n . A worst-case quantity involving no probability — not an average case. (Chapter 17)

approximation algorithm. One returning a solution provably within a stated factor of optimal. (Chapter 22)

asymptotic notation. O , Ω , Θ , o , ω : statements about how a function grows, ignoring constants and small inputs. (Chapter 3)

average case. Expected cost over a *stated distribution of inputs*. Meaningless until the distribution is named. (Chapter 2)

best case. The minimum cost over inputs of a given size. Promises nothing. (Chapter 2)

Bellman–Ford. Single-source shortest paths by relaxing every edge $|V| - 1$ times. $O(|V||E|)$; handles negative edges and detects negative cycles. (Chapter 20)

big-O. $f = O(g)$ if $\exists c, n_0$ with $f(n) \leq cg(n)$ for all $n \geq n_0$. An upper bound, not necessarily tight. (Chapter 3)

big-Omega. $f = \Omega(g)$: a lower bound. Desirable about a problem, undesirable about your algorithm. (Chapter 3)

big-Theta. $f = \Theta(g)$: both bounds. Equivalently $O(g) \cap \Omega(g)$. (Chapter 3)

branch misprediction. A processor's penalty for guessing a conditional's outcome wrongly. Invisible to the RAM model; measured at a factor of 2.0 in Chapter 8.

brute force. Generating and testing every candidate. The baseline, the oracle, and sometimes the answer. (Chapter 7)

certificate. A short proof that an instance is a yes-instance, checkable in polynomial time. What defines NP. (Chapter 22)

comparison sort. A sort learning about its input only by comparing pairs. Subject to the $\Omega(n \lg n)$ bound. (Chapter 10)

counting sort. A linear-time sort for integer keys in a small known range. Outside the comparison bound because it never compares. (Chapter 10)

cut property. The lightest edge crossing any cut uncrossed by the current partial tree is safe to add. The basis of both MST algorithms. (Chapter 19)

decision problem. A problem with a yes/no answer. The form complexity theory is built on. (Chapter 22)

decision tree. A model of a comparison sort's execution; its leaves are permutations and its height is the worst-case comparison count. (Chapter 10)

dense. $|E| = \Theta(|V|^2)$. (Chapter 18)

Dijkstra's algorithm. Single-source shortest paths, greedy, requiring non-negative weights. (Chapter 20)

divide and conquer. Divide into smaller instances of the same problem, conquer recursively, combine. (Chapter 8)

dynamic programming. Solving each distinct subproblem once and storing the answer. Needs optimal substructure *and* overlapping subproblems. (Chapter 15)

edit distance. The minimum number of insertions, deletions and substitutions turning one string into another. (Chapter 16)

exchange argument. A proof that an optimal solution can be modified to contain a greedy choice without getting worse. (Chapter 14)

expected running time. The average over an algorithm's *own random choices*, for a fixed input. Assumes nothing about the data. (Chapter 2, Chapter 9)

Floyd–Warshall. All-pairs shortest paths in $\Theta(|V|^3)$, with the subproblem index outermost. (Chapter 20)

greedy algorithm. Commits to the locally best choice and never reconsiders. Optimal when the greedy-choice property and optimal substructure both hold. (Chapter 14)

greedy-choice property. Some optimal solution contains the greedy first choice. (Chapter 14)

hash flooding. An attack supplying keys chosen to collide, turning a hash table into a linked list. Measured at $324\times$ in Chapter 13.

heap. A complete binary tree with the heap property, stored in an array. Height exactly $\lfloor \lg n \rfloor$, guaranteed by the layout. (Chapter 12)

Huffman code. An optimal prefix code built by repeatedly merging the two least frequent symbols. (Chapter 14)

input size. How much input there is: elements for an array, two parameters for a graph, *bits* for a number. (Chapter 2)

inverse Ackermann, $\alpha(n)$. A function below 5 for every writable n ; the amortised cost of a union–find operation. (Chapter 19)

KMP. Knuth–Morris–Pratt string matching, $\Theta(n + m)$ always, because the text pointer never moves backwards. (Chapter 21)

load factor, α . n/m in a hash table: the average chain length. (Chapter 13)

loop invariant. An assertion true before every iteration. Proved by initialisation, maintenance and termination. (Chapter 4)

loop variant. An integer-valued measure that strictly decreases, proving termination. (Chapter 4)

master theorem. Solves $T(n) = aT(n/b) + f(n)$ by comparing $f(n)$ with $n^{\log_b a}$. (Chapter 6)

median of medians. A deterministic worst-case linear selection algorithm, with a constant poor enough that it is used as a fallback. (Chapter 11)

memoisation. Caching a recursive function’s results. Top-down dynamic programming. (Chapter 15)

minimum spanning tree. A spanning tree of least total weight. (Chapter 19)

NP. Decision problems whose yes-instances have polynomial-time verifiable certificates. *Non deterministic* polynomial, not “not polynomial”. (Chapter 22)

NP-complete. In NP and NP-hard. A polynomial algorithm for any one of them would give one for all of NP. (Chapter 22)

NP-hard. At least as hard as every problem in NP. Need not be in NP. (Chapter 22)

optimal substructure. An optimal solution contains optimal solutions to subproblems. Needed by both greedy and dynamic programming. (Chapter 14, Chapter 15)

order statistic. The i -th smallest element. (Chapter 11)

overlapping subproblems. A recursion revisits the same subproblems repeatedly. What distinguishes dynamic programming from divide-and-conquer. (Chapter 15)

P. Decision problems solvable in polynomial time. (Chapter 22)

partition. Rearranging an array about a pivot. Quicksort’s divide step. (Chapter 9)

path compression. Flattening the path to the root during a union–find *find*. (Chapter 19)

potential method. Amortised analysis via a function Φ of the state, with $\hat{c} = c + \Delta\Phi$. (Chapter 17)

prefix code. A code in which no codeword is a prefix of another. (Chapter 14)

prefix function. For KMP: the longest proper prefix of $P[1..i]$ that is also a suffix. (Chapter 21)

Prim’s algorithm. MST by growing one tree, taking the lightest edge leaving it. (Chapter 19)

pseudo-polynomial. Polynomial in the numeric *value* of an input number, exponential in its bit *length*. Knapsack’s $\Theta(nW)$. (Chapter 16, Chapter 22)

quickselect. Quicksort with one recursive call deleted. $\Theta(n)$ expected. (Chapter 11)

Rabin–Karp. String matching by rolling hash. $O(n + m)$ expected; inherits every caveat of hashing. (Chapter 21)

radix sort. Counting sort applied digit by digit. Requires a stable inner sort. (Chapter 10)

RAM model. Unit-cost random-access machine: one operation, one time unit, uniform memory. All three assumptions are false and all three are constant factors. (Chapter 1)

randomised algorithm. One making its own random choices, so its guarantee holds for every input. (Chapter 9)

recurrence. An equation defining a function in terms of itself at smaller arguments. (Chapter 5)

recursion tree. A device for *guessing* a recurrence’s solution by summing level by level. Not a proof. (Chapter 5)

reduction. $A \leq_P B$: a polynomial-time transformation preserving yes-instances. Means B is at least as hard as A . (Chapter 22)

regularity condition. $af(n/b) \leq cf(n)$ for some $c < 1$; required by case 3 of the master theorem. (Chapter 6)

simple uniform hashing. The assumption that keys scatter evenly — about the hash *and* the keys together. (Chapter 13)

sparse. $|E| = O(|V|)$. (Chapter 18)

stability. A sort is stable if equal keys keep their input order. Required by radix sort’s inner pass. (Chapter 8, Chapter 10)

strengthening the hypothesis. Proving a stronger statement so the induction has more to work with — subtracting a lower-order term. (Chapter 5)

substitution method. Guess the solution to a recurrence, then prove it by induction. (Chapter 5)

union by rank. Attaching the shorter union–find tree under the taller. (Chapter 19)

watershed function. $n^{\log_b a}$: the leaf count of the recursion tree, against which $f(n)$ is compared. (Chapter 6)

worst case. The maximum cost over inputs of a given size. A guarantee, and the default unless stated otherwise. (Chapter 2)

Further Reading

E.1 The Books the Outline Names

⚠ Two things to say about the published reading list

The HEC 2017 and 2023 outlines give the same three books, and both editions contain the same two defects, recorded here rather than inherited:

1. “**Corman**” is a misspelling of **Cormen**, throughout both documents.
2. **The 3rd edition of CLRS (2009) is cited.** The **4th edition (2022)** has been current since before this course was written. It is cited below, with the correspondence noted, because a student buying the recommended book today will buy the 4th.

💡 The three prescribed texts

Cormen, Leiserson, Rivest and Stein, *Introduction to Algorithms*, 4th edition, MIT Press, 2022. (The outline cites the 3rd edition, 2009.)

The standard reference, and the one this handout’s structure most nearly follows. It is a reference rather than a narrative: thorough, rigorous, and heavy going read front to back. Use it for the chapter you are currently on.

What changed in the 4th edition: machine learning and matrix applications added; the chapter on dynamic multithreading rewritten; a new chapter on online algorithms; Fibonacci heaps moved to the website. Chapter numbering shifted, so a reference to “CLRS chapter 15” means different things in the two editions — this handout cites by *topic*, not by number.

Kleinberg and Tardos, *Algorithm Design*, Pearson, 2006.

Better than CLRS at one thing and worth owning for it: **how to find** an algorithm, rather than how to verify one you have been given. Its treatment of greedy algorithms — the exchange-argument style used in Chapter 14 and Chapter 19 — is the clearest in print. Weaker on data structures.

Sedgewick and Wayne, *Algorithms*, 4th edition, Addison-Wesley, 2011.

The implementation-first book, in Java. Its strength is exactly this handout’s measured-reality strand: it reports real running times and discusses constants honestly. If you want to see what a professional implementation of the algorithms in Part II looks like, this is the book.

E.2 Beyond the List

💡 Worth knowing about

Skiena, *The Algorithm Design Manual*, 3rd edition, Springer, 2020.

Two halves. The first is a brisk textbook; the second is a catalogue of problems with advice on which algorithm to reach for. The “war stories” — accounts of real problems and how they were solved or not — are the best answer in print to a student asking whether any of this is used.

Knuth, *The Art of Computer Programming*, volumes 1–4A, Addison-Wesley.

The deepest analysis anywhere, in a hypothetical assembly language. Not a course text. Volume 3 (sorting and searching) contains analyses of the algorithms in Parts II and III to a precision no other source attempts.

Dasgupta, Papadimitriou and Vazirani, *Algorithms*, McGraw-Hill, 2006.

Short — about 300 pages — and free from the authors' pages. If CLRS is too much, this covers most of the same ground in a quarter of the length. Its chapter on NP-completeness is a good companion to Chapter 22.

Bentley, *Programming Pearls*, 2nd edition, Addison-Wesley, 1999.

Old and not superseded. Column 8 is the maximum-subarray problem of Chapter 7, including the history of how the linear algorithm was found. The book's subject is precisely this handout's: the difference between an idea and an implementation.

E.3 The Original Papers

💡 Short, readable, and worth an afternoon

Several results in this book come from papers a student can actually read.

Chapter	Paper	Note
Chapter 14	Huffman, "A Method for the Construction of Minimum-Redundancy Codes", <i>Proc. IRE</i> , 1952	Three pages. Written while Huffman was a student, to avoid a final exam.
Chapter 20	Dijkstra, "A Note on Two Problems in Connexion with Graphs", <i>Numerische Mathematik</i> , 1959	Two and a half pages, covering both MST and shortest paths.
Chapter 21	Knuth, Morris and Pratt, "Fast Pattern Matching in Strings", <i>SIAM J. Computing</i> , 1977	Longer, and the prefix-function argument repays the effort.
Chapter 22	Cook, "The Complexity of Theorem-Proving Procedures", <i>STOC</i> , 1971	The paper that created the subject of Chapter 22.
Chapter 22	Karp, "Reducibility Among Combinatorial Problems", 1972	The 21 problems. Readable, and the source of the reduction technique.
Chapter 19	Tarjan, "Efficiency of a Good But Not Linear Set Union Algorithm", <i>JACM</i> , 1975	Where the inverse-Ackermann bound of Chapter 19 is proved.
Chapter 12	Reed, "The Height of a Random Binary Search Tree", <i>JACM</i> , 2003	The source of the $4.311 \ln n - 1.953 \ln \ln n$ that Chapter 12 measures against.

Huffman's and Dijkstra's are the two to start with. Both are shorter than the chapters that describe them.

E.4 Online

💡 Used with care

MIT 6.006 and 6.046, OpenCourseWare. Full lecture video from the authors of CLRS. 6.006 maps onto Parts I–III of this handout and 6.046 onto Parts IV–VI.

Visualgo (visualgo.net). Animations of most algorithms in this book. Genuinely useful for Chapter 12's trees and Chapter 19's algorithms, where the shape is the point.

Competitive-programming references (cp-algorithms.com and similar). Excellent implementations with the constant factors already tuned. Treat their *analyses* with more caution than their code: they are written for contest conditions, where the input distribution is known and a worst case that cannot occur is often ignored — which is the opposite of this course's habit.

Large language models. They will produce a plausible analysis for any algorithm you name, including a wrong one, and they are particularly unreliable about *which* case a bound refers to and about amortised versus worst-case cost. Use the measurement harness of Chapter 23 to check anything you are told. That is what it is for.

E.5 A Reading Path

💡 If you read one more thing

If you want	Read
the exam answered	CLRS, the chapter matching the week
to design algorithms	Kleinberg & Tardos, chapters 4 and 6
to make them fast	Sedgewick & Wayne, and Bentley
to know what is used in practice	Skiena, part II
the subject's history	the Huffman and Dijkstra papers, in one sitting
a shorter CLRS	Dasgupta, Papadimitriou & Vazirani
to go further than this course	the deleted postgraduate syllabus in Appendix F names the topics; CLRS covers flow and approximation

The Boundary With Other Courses

About a third of the HEC outline for this course is also in the outline for *Data Structures*, which BSIT students take three or four semesters earlier. This appendix records what belongs where, so that a lecturer can see what to assume and a student can see why a familiar topic is being covered again.

HEC's own ACM mapping (2023, p. 10) groups *Data Structures*, *Analysis of Algorithms* and *Theory of Automata* as jointly discharging the Algorithms and Complexity knowledge area. The split below is this handout's reading of that grouping.

F.1 Data Structures

💡 The rule

Data Structures owns the structures and their operations. This course owns their analysis.

A student arriving here should already be able to insert into a heap, walk a tree and resolve a hash collision. What this course adds is *why the heights and the costs are what they are*, and what happens when the assumptions behind them fail.

Topic	Data Structures covers	This course adds
Complexity, Big-Oh	informal use; “this is $O(n^2)$ ”	Ω , Θ , o , ω defined formally and proved from the definitions (Chapter 3)
Recursion	writing recursive procedures	costing them: recurrences, recursion trees, the master theorem (Chapter 5, Chapter 6)
Merge and quick sort	implementation	why the bounds hold in every case or only in expectation, and what randomisation buys (Chapter 8, Chapter 9)
Heaps	build, insert, extract	why the height is <i>exactly</i> $\lfloor \lg n \rfloor$, and why BUILD-HEAP is $\Theta(n)$ not $\Theta(n \lg n)$ (Chapter 12)
Search trees	insert, delete, traverse	expected height $2.99 \lg n$ for random insertion, exactly n for sorted (Chapter 12)
Hashing	chaining, open addressing	what $O(1)$ assumes, and the two ways it fails (Chapter 13)
Graphs	representation, BFS, DFS	the two-parameter bound and the sparse/dense crossover (Chapter 18)
Dynamic arrays	vector and its growth	why n appends cost $\Theta(n)$ — amortised analysis (Chapter 17)

⚠ What this course does *not* re-teach

Pointer manipulation, rotations in a balanced tree, iterator design, memory management, the implementation of a linked list.

Chapter 12 represents a binary search tree as three parallel arrays precisely so that no part of it is about pointers: the measurement is about *shape*, and pointers would be a distraction.

F.2 Theory of Automata

💡 The line

Theory of Automata owns computability. This course owns complexity.

Question	Course	Answer type
Can this be computed at all?	Theory of Automata	decidable / undecidable
What is a Turing machine?	Theory of Automata	a model of computation
Is the halting problem solvable?	Theory of Automata	no, provably
Among computable problems, which are <i>efficient</i> ?	this course	P, NP, NP-complete (Chapter 22)
Is $P = NP$?	this course	open

The distinction is worth making to students explicitly, because the two sound alike and are not. The halting problem has *no algorithm*. NP-complete problems all have algorithms — Chapter 22 runs one — and the algorithms are merely too slow. Confusing “undecidable” with “intractable” is the commonest error in this area, and Chapter 22’s pitfall list names it.

Chapter 22 therefore states the Cook–Levin theorem and uses it, but does not prove it: the proof is a simulation of a Turing machine, and Turing machines are the other course’s subject.

F.3 Discrete Structures

💡 Assumed throughout, re-derived nowhere

Appendix B collects what this course uses: logarithm identities, arithmetic and geometric series, induction, elementary probability, and the handful of inequalities in Section B.6.

All of it is *Discrete Structures* material. Appendix B exists so that a chapter can point at a result rather than re-prove it, and so that a student who has forgotten one can find it in two pages rather than in a textbook.

The three items that carry the most weight, and that are worth revising before Week 4: **linearity of expectation**, $a^{\log_b n} = n^{\log_b a}$, and **the geometric series**.

F.4 Programming Fundamentals and OOP

💡 What is assumed about C++

The labs assume you can write and compile a C++ program with loops, arrays, functions and `std::vector`. They do not assume templates, inheritance, operator overloading or the finer points of move semantics.

Where a modern facility is used — a lambda passed to the timing helper, a structured binding in a range-for — the lab’s commentary says what it does. Appendix A gives the compiler setup and the one index-translation trap that catches people.

F.5 A Note on the 2017 Postgraduate Course

💡 *Advanced Algorithm Analysis*, deleted

The HEC 2017 curriculum listed a postgraduate elective, *Advanced Algorithm Analysis*, covering amortised analysis in depth, advanced data structures (Fibonacci heaps, van Emde Boas trees), network flow, linear programming, approximation algorithms and randomised algorithms.

The 2023 and 2025 editions do not list it, and no replacement appears. So the topics below are now covered nowhere in the BSIT programme unless an individual department adds them:

- network flow and max-flow/min-cut;
- linear programming and duality;
- approximation algorithms with proved ratios — Chapter 22 names them and does not develop them;
- advanced randomised algorithms beyond Chapter 9’s pivot and Chapter 13’s universal families.

This handout touches three of the four in passing, deliberately: Chapter 17 covers amortised analysis properly (the one topic the deleted course had that this one genuinely needs), Chapter 19 and Chapter 20 mention Fibonacci heaps without developing them, and Chapter 22 states the three strategies for NP-complete problems.

A department wanting to restore the deleted material would find Chapter 17, Chapter 22 and the project of Chapter 23 the natural places to extend.

F.6 Summary: What to Assume on Day One

⚠️ For the lecturer

Assume the student can: write and compile C++; implement a linked list, stack, queue, heap, BST and hash table; write a recursive function; say informally that a nested loop is “ $O(n^2)$ ”.

Do not assume the student can: state the definition of $O(\cdot)$; prove anything from it; solve a recurrence; distinguish worst case from expected case; or say what $O(1)$ amortised means.

Revise briefly, do not re-teach. Chapter 12 spends one chapter on heaps, trees and the structures beneath, and spends it entirely on analysis. Chapter 13 does the same for hash tables. Neither re-implements anything, and the front matter’s prerequisite self-check is there to tell a student whether that assumption holds for them.

The Sixteen-Week Schedule

This appendix is the **source of record** for the teaching schedule. The one-page `AoA_Fall2026_Lecture_Schedule.pdf` distributed to students is generated from it, and the slide pipeline parses the table below. Change this first; regenerate the one-page version afterwards.

Twenty-three chapters are taught across sixteen weeks, so seven weeks carry two chapters. Those pairings are not arbitrary: in every case the second chapter is the one that cannot be stated honestly without the first — O and Ω with Θ , merge sort with the divide-and-conquer pattern that produces it, graphs with the representation that decides their cost.

G.1 The Schedule

Wk	Lecture topic	Chapters	Lab	Assessment
1	What an algorithm is; input size and the three cases	1, 2	Growth measured: the ratio test, and the crossover at $n \approx 65$	—
2	Asymptotic notation, formally	3	Fit a curve; exclude the exponents either side	—
3	Correctness and loop invariants	4	A bug that is never wrong and never finishes	—
4	Recurrences; the master theorem	5, 6	Recursion trees against measured depth and cost	—
5	Brute force; divide and conquer	7, 8	Where n^2 beats $n \lg n$, and by how much	—
6	Quicksort and randomisation	9	The adversarial input, and what randomising costs	Quiz 1 (Ch 1–6)
7	The comparison lower bound; linear-time sorting	10	Counting and radix sort against <code>std::sort</code>	Assignment 1 set
8	Selection and order statistics	11	Midterm, then quickselect against a full sort	Midterm (Ch 1–9)
9	Heaps, search trees and hashing, analysed	12, 13	A hash table degrading under a poor hash	Assignment 1 due
10	Greedy algorithms and exchange arguments	14	Huffman coding on real text; measure the compression	—
11	Dynamic programming: elements and classics	15, 16	Memoised against bottom-up, timed and counted	—

Wk	Lecture topic	Chapters	Lab	Assessment
12	Amortised analysis	17	A growing array: aggregate cost against worst-case cost	Quiz 2 (Ch 10–16)
13	Graph representation, traversal and spanning trees	18, 19	The matrix/list crossover as density falls	Assignment 2 set; project proposal
14	Shortest paths	20	Dijkstra with and without a priority queue	—
15	String matching; P, NP and NP-completeness	21, 22	Naive, Rabin–Karp and KMP on the same corpus	Assignment 2 due
16	The semester project	23	Implement, measure, defend	Project due; Final (Ch 1–22)

G.2 How the Weeks Map to the HEC Outline

Every clause of the published HEC 2023 outline is reached, and the Course Specification in the front matter traces each clause to its chapter. The table below is the same trace read the other way round — by week, for a course file that asks when a topic is delivered rather than where it is written.

Outline clause	Weeks	Note
Introduction; role of algorithms in computing	1	Ch 1
Analysis on nature of input and size of input	1	Ch 2; CLO-1 and CLO-2
Asymptotic notations; O , Ω , Θ , o , ω	2	Ch 3; CLO-5
Sorting algorithm analysis	5–8	Ch 8–11, including the lower bound the outline does not name
Loop invariants	3	Ch 4
Recursion and recurrence relations	4	Ch 5, 6
Algorithm design techniques; brute force	5	Ch 7
Divide-and-conquer; merge, quick sort	5, 6	Ch 8, 9; CLO-6
Greedy approach	10	Ch 14
Dynamic programming; elements of DP	11	Ch 15, 16
Search trees; heaps	9	Ch 12 — analysed, not reimplemented
Hashing	9	Ch 13

Outline clause	Weeks	Note
Graph algorithms; shortest paths; sparse graphs	13, 14	Ch 18–20; CLO-7
String matching	15	Ch 21; CLO-8
Introduction to complexity classes	15	Ch 22; CLO-4

💡 Two weeks the outline does not ask for

Week 7 (Chapter 10), the comparison lower bound. The outline says “sorting algorithm analysis” and lists the sorts. It does not ask for the $\Omega(n \lg n)$ decision-tree proof. It is taught anyway because it is the first result in the course about a *problem* rather than an algorithm, and because without it “merge sort is optimal” is an assertion rather than a theorem.

Week 12 (Chapter 17), amortised analysis. Also unnamed. Chapter 13’s claim about hash tables and Chapter 19’s union-find both depend on it, and CLO-3 asks for the time complexity of simple algorithms — which, for a sequence of operations on a growing structure, is an amortised question. The front matter states this justification in full.

⚠️ If the semester is shorter than sixteen weeks

Cut in this order, and say so in the course file:

1. **Chapter 16** (dynamic programming problems) down to two problems rather than five. Chapter 15 carries the examinable content.
2. **Chapter 21** (string matching) down to naive and Rabin–Karp, leaving KMP’s prefix function as reading. CLO-8 asks only that a string-matching algorithm be traced or implemented.
3. **Chapter 12** to a single lecture, since Data Structures has already built these structures and this chapter only re-analyses them.

Do *not* cut Chapters 3, 4, 6, 10 or 15. Each is a prerequisite for something later, and Chapter 10 is the only place the course proves a lower bound at all.